

MikroC İLE PIC18F4550 UYGULAMALARI

EDİTÖR

Öğr. Gör. Hakan TERZİOĞLU
Serkan SATUK

YAZARLAR

Fatih Alpaslan KAZAN
Abdullah Cem AĞAÇAYAK
Cemile ARSLAN
Murat SELEK

İÇİNDEKİLER

Sayfa Numarası

1. MİKRODENETLEYİCİ SİSTEMLERİ	1
1.1. MİKROİŞLEMCİ NEDİR?	1
1.2. MİKRODENETLEYİCİ NEDİR?	1
1.3. MİKROİŞLEMCİ İLE MİKRODENETLEYİCİ ARASINDAKİ FARKLAR	3
2. PIC18F4550 ÖZELLİKLERİ VE YAPISI	7
2.1. NEDEN PIC18F4550?	7
2.2. GENEL ÖZELLİKLERİYLE PIC18F4550	8
2.2.1. USB Özellikleri	11
2.2.2. Güç Tasarruf Modları	11
2.2.3. Esnek Osilatör Yapısı	12
2.2.4. Çevrebirim Özellikleri	12
2.2.5. Özel Mikrodenetleyici Özellikleri	13
2.3. OSİLATÖR DEVRELERİ	13
2.3.1. RC Osilatör	14
2.3.2. Rezonatör Osilatör	14
2.3.3. Kristal Osilatör	14
2.4. PIC18F4550 OSİLATÖR AYARLARI	15
2.4.1. PLL (Frekans Çarpanı)	17
2.4.2. USB İçin Osilatör Ayarları	17
2.5. PIC18F4550'DE RESET (SIFIRLAMA) İŞLEMİ	18
2.5.1. MCLR İle Sıfırlama	18
2.5.2. Enerji Verildiğinde Sıfırlama (POR, Power-On Reset)	18

2.5.3. Gerilim Düşmesi Sıfırlaması (BOR, Brown-Out Reset).....	19
2.6. PIC18F4550'NİN HAFIZA BİRİMİ	19
3. MİKROC	21
3.1. MİKROC NEDİR?.....	21
3.2. NEDEN MİKROC?	21
3.3. MİKROC'YE GİRİŞ	22
3.4. MİKROC'NİN BİLGİSAYARA KURULUMU	24
3.5. MİKROC YAZIM KURALLARI	31
3.6. TANIMLAYICI KELİMELE (TÜR İSİMLERİ)	33
3.7. TAMSAYILAR	34
3.8. ONDALIKLI SAYILAR.....	35
3.9. KARAKTERLER	35
3.10. YAZILAR (STRING İFADELER).....	35
3.11. KAÇIŞ DÜZENLERİ	36
3.12. SATIR DEVAMI VE TERS EĞİK ÇİZGİ KARAKTERİ	36
3.13. AYIRICI İŞARETLER	37
3.13.1. Köşeli Parantez ([]).....	38
3.13.2. Parantez (())	38
3.13.3. Küme Parantezi (Süslü Parantez) ({ })	38
3.13.4. Virgül (,).....	38
3.13.5. Noktalı Virgül (;).....	38
3.13.6. İki Nokta (:).....	39
3.13.7. Nokta (.)	39
3.13.8. Yıldız İşareti (*).....	39
3.13.9. Eşittir İşareti (=).....	39

3.13.10.	Diyez İşareti (#)	39
3.14.	TİP NİTELEYİCİLERİ.....	40
3.14.1.	Const.....	40
3.14.2.	Volatile.....	40
3.15.	ASM TANIMLAYICISI	40
3.16.	BAŞLANGIÇ DEĞERİ ATAMA.....	40
3.17.	MİKROC'DE VERİ TÜRLERİ	41
3.17.1.	Aritmetik Türler	41
3.17.2.	Void Türü	43
3.17.3.	İşaretçiler	43
3.17.4.	Yapılar	43
3.17.5.	Diziler	44
3.18.	FONKSİYONLAR	45
3.19.	OPERATÖRLER.....	46
3.19.1.	Aritmetik Operatörler	47
3.19.2.	Karşılaştırma Operatörleri	49
3.19.3.	Mantıksal Operatörler	50
3.19.4.	Diğer Operatörler	50
3.20.	KOŞUL İFADELERİ VE DÖNGÜLER.....	50
3.20.1.	If-Else Deyimi	51
3.20.2.	Switch-Case Deyimi	52
3.20.3.	FOR Döngüsü	53
3.20.4.	While Döngüsü.....	53
3.20.5.	Do-While Döngüsü.....	54
4.	MİKROC DERLEYİCİSİ	55

4.1. MİKROC MENÜLERİ	57
4.1.1. File Menüsü	57
4.1.2. Edit Menüsü	58
4.1.3. View Menüsü	59
4.1.4. Project Menüsü	60
4.1.5. Build Menüsü	61
4.1.6. Run Menüsü	61
4.1.7. Tools Menüsü	62
4.1.8. Help Menüsü	63
4.2. MİKROC İLE İLK UYGULAMA	64
4.2.1. Donanım Şeması.....	65
4.2.2. Devre Yazılımının Yapılması.....	69
4.2.3. Devrenin Çalıştırılması.....	79
4.2.4. Mikrodenetleyicinin Programlanması	81
4.2.5. ICSP Üzerinden Programlama	84
5. LED UYGULAMALARI	85
UYGULAMA_01.....	85
UYGULAMA_02.....	89
UYGULAMA_03.....	93
UYGULAMA_04.....	95
UYGULAMA_05.....	99
UYGULAMA_06.....	103
UYGULAMA_07.....	105
UYGULAMA_08.....	109
UYGULAMA_09.....	113

UYGULAMA_10.....	115
6. BUTON UYGULAMALARI.....	119
UYGULAMA_01.....	119
UYGULAMA_02.....	123
UYGULAMA_03.....	127
UYGULAMA_04.....	131
UYGULAMA_05.....	135
7. LCD UYGULAMALARI	139
UYGULAMA_01.....	139
UYGULAMA_02.....	143
UYGULAMA_03.....	147
UYGULAMA_04.....	153
8. DISPLAY UYGULAMALARI	159
UYGULAMA_01.....	159
UYGULAMA_02.....	163
UYGULAMA_03.....	167
9. TIMER UYGULAMALARI	171
UYGULAMA_01.....	171
UYGULAMA_02.....	179
UYGULAMA_03.....	185
UYGULAMA_04.....	191
10. ADC UYGULAMALARI.....	195
UYGULAMA_01.....	195
UYGULAMA_02.....	203
UYGULAMA_03.....	207

11. PWM UYGULAMALARI	211
UYGULAMA_01.....	211
UYGULAMA_02.....	221
12. DEVİR SAYACI (FREKANS SAYICI) UYGULAMASI	227
13. SERİ HABERLEŞME (EUSART)	235
13.1. SENKRON SERİ İLETİŞİM	237
13.2. ASENKRON SERİ İLETİŞİM	237
ÖRNEK UYGULAMA	243
14. USB HABERLEŞME	251
14.1. USB HIZLARI.....	251
14.2. USB'NİN SAĞLADIĞI AVANTAJLAR	252
14.3. PIC18F4550 USB MODÜLÜ.....	253
14.3.1. USB Durum Kaydedicisi (UCON)	254
14.3.2. USB Konfigürasyon Kaydedicisi (UCFG)	256
14.3.3. USB Durum Kaydedicisi (USTAT).....	257
USB UYGULAMASI.....	259
KAYNAKLAR	281

MESLEKİ
AKADEMİ



ISBN

978-605-84867-3-7

Yazır Mah. Sultan Cad. Revlon İş Merkezi Kat: 4 No: 34/404 Selçuklu/ONYA
www.meslekiakademi.com.tr info@meslekiakademi@gmail.com
Tel.: 0332-235-89-97 0543-235-89-97

MİKRODENETLEYİCİ SİSTEMLERİ

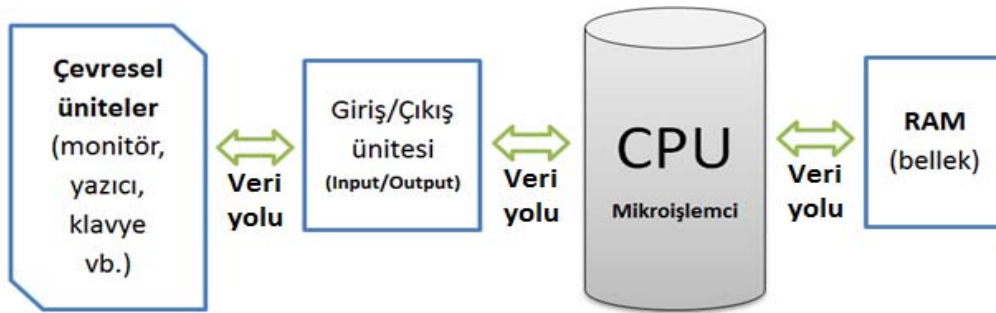
BÖLÜM 1

1. MİKRODENETLEYİCİ SİSTEMLERİ

1.1. MİKROİŞLEMCİ NEDİR?

Mikroişlemci, temelde bilgisayarın tüm işlemleri yapmasını sağlayan, bilgisayarın beyni olarak nitelendirilebilecek, hesaplama, karar verme ve yönetim mekanizmasıdır. Bir mikroişlemci, işlevini yerine getirebilmek için aşağıdaki yardımcı elemanlara ihtiyaç duyar.

1. Giriş (input) ünitesi
2. Çıkış (output) ünitesi
3. Bellek (memory) ünitesi

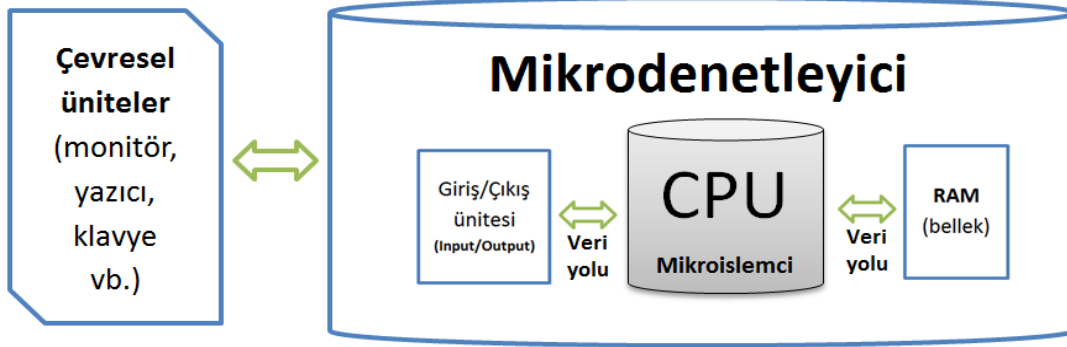


Şekil 1.1. Mikroişlemci sisteminin temel blok diyagramı

1.2. MİKRODENETLEYİCİ NEDİR?

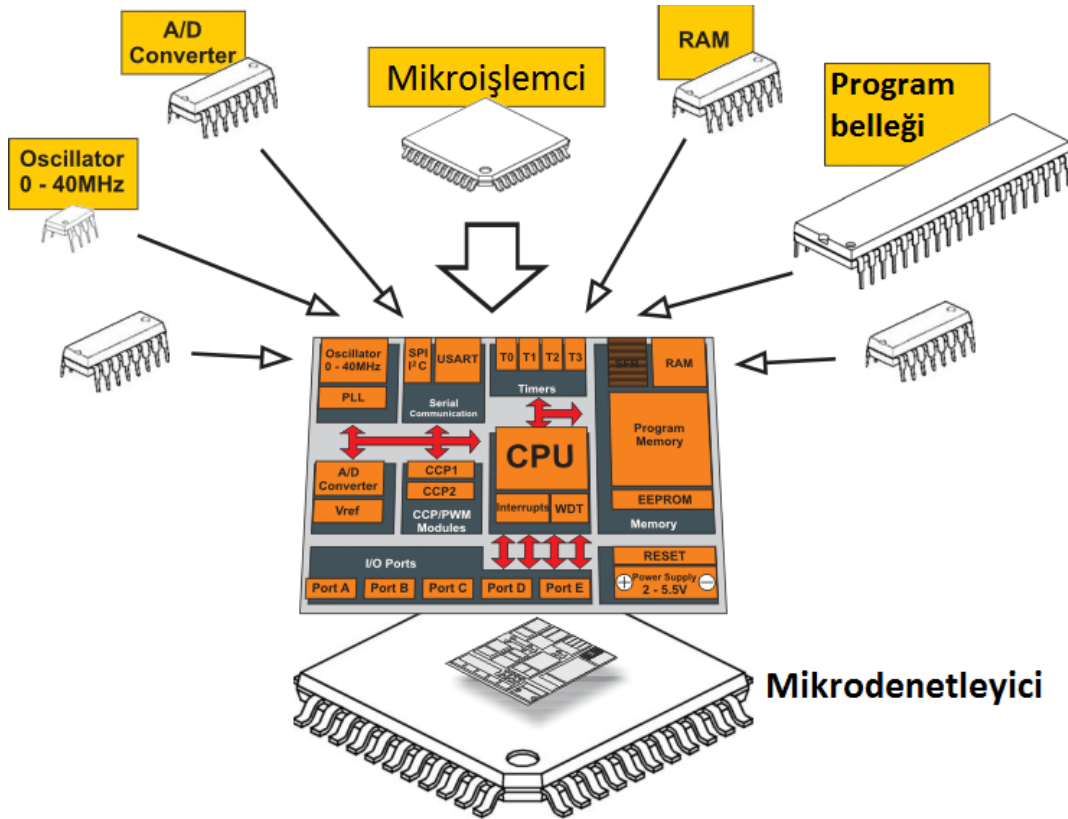
Bir mikro işlemcili sistemi meydana getiren temel bileşenlerden mikro işlemci, bellek ve Giriş/Çıkış birimlerinin, bazı özellikleri kırılarak (azaltılarak) tek bir entegre içerisinde üretilmiş biçimine mikrodenetleyici (microcontroller) denir.

MİKRODENETLEYİCİ SİSTEMLERİ



Şekil 1.2. Mikrodenetleyici sisteminin temel blok diyagramı

Genel olarak bir mikrodenetleyicinin, kendi içerisinde bir mikroislemciyi barındırdığı söylenebilir.

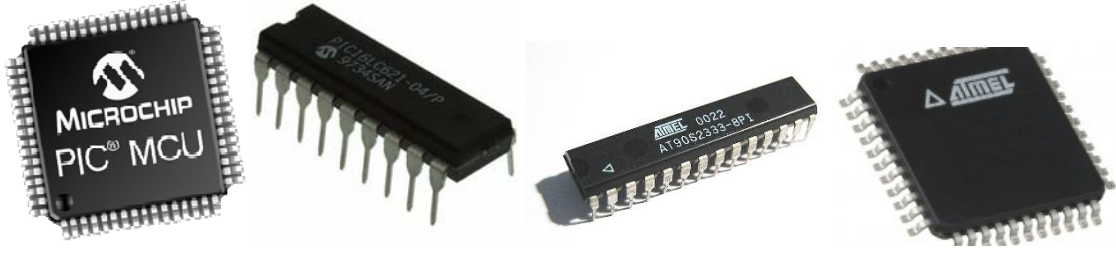


Şekil 1.3. Bir mikrodenetleyicinin yapısı

Mikrodenetleyiciler otomatik olarak kontrol edilen ürünler ve otomobil, motor kontrol sistemleri, tıbbi cihazlar, uzaktan kumandalar, ofis makineleri, elektrikli el aletleri, robotlar ve diğer gömülü sistemler gibi cihazlarda kullanılırlar.

Aşağıdaki resimde değişik kılıflarda üretilen mikrodenetleyiciler yer almaktadır.

MİKRODENETLEYİCİ SİSTEMLERİ



Şekil 1.4. Mikrodenetleyici dış kılıfları

1.3. MİKROİŞLEMCİ İLE MİKRODENETLEYİCİ ARASINDAKİ FARKLAR

- Mikroişlemci, merkezi işlem birimidir. Yani bilgisayarın ana işlemcisi gibidir. Sadece işlemleri yönetir ve karar verir, ancak işlemleri tek başına gerçekleştiremez. İşlemleri gerçekleştirebilmesi için bellek (RAM), program belleği (sabit disk) ve bunları birbirine bağlayan veri yollarına ihtiyaç duyar. Mikrodenetleyici ise tüm bu gereksinimlerin birleşmiş hali gibidir. Kendi içerisinde işlemcisini, hafızasını, giriş/çıkış, RAM, ROM gibi birimleri taşıyan küçük yapıda bir bilgisayardır. Bu sebeple işlemleri tek başına denetleyip gerçekleştirebilir.
- Mikroişlemci ile karmaşık sistemler kontrol edilebilirken, mikrodenetleyici buna göre daha basit devrelerde tercih edilir.
- Mikroişlemcilerin programlanması mikrodenetleyicilere göre daha zor ve karmaşıktır.
- Mikroişlemcilerin fiyatları pahalıdır, ayrıca bellek, giriş/çıkış ünitesi gibi elemanlara da ihtiyaç duyduğundan basit devre tasarımlarında bile maliyeti artırmaktadır. Buna karşın mikrodenetleyicilerin fiyatları çok daha uygundur. Mikrodenetleyicili bir sistemin çalışması için mikrodenetleyicinin kendisi ve bir osilatör kaynağının olması yeterlidir.

Piyasada **Microchip**, **Atmel**, **Motorola**, **Intel**, **Zilog** gibi firmaların ürettiği mikrodenetleyiciler vardır. Bu firmaların arasındaki **Microchip**'in ürettiği **PIC** mikrodenetleyicileri, en çok kullanılan mikrodenetleyicilerin başında gelir. Uygun fiyatlı oluşu ve yüksek performansa sahip olmasıyla beraber kolayca çalıştırılabilir olması, bu

MİKRODENETLEYİCİ SİSTEMLERİ

mikrodenetleyicilerin çok kullanılması ve tercih edilmesinin sebeplerindendir.

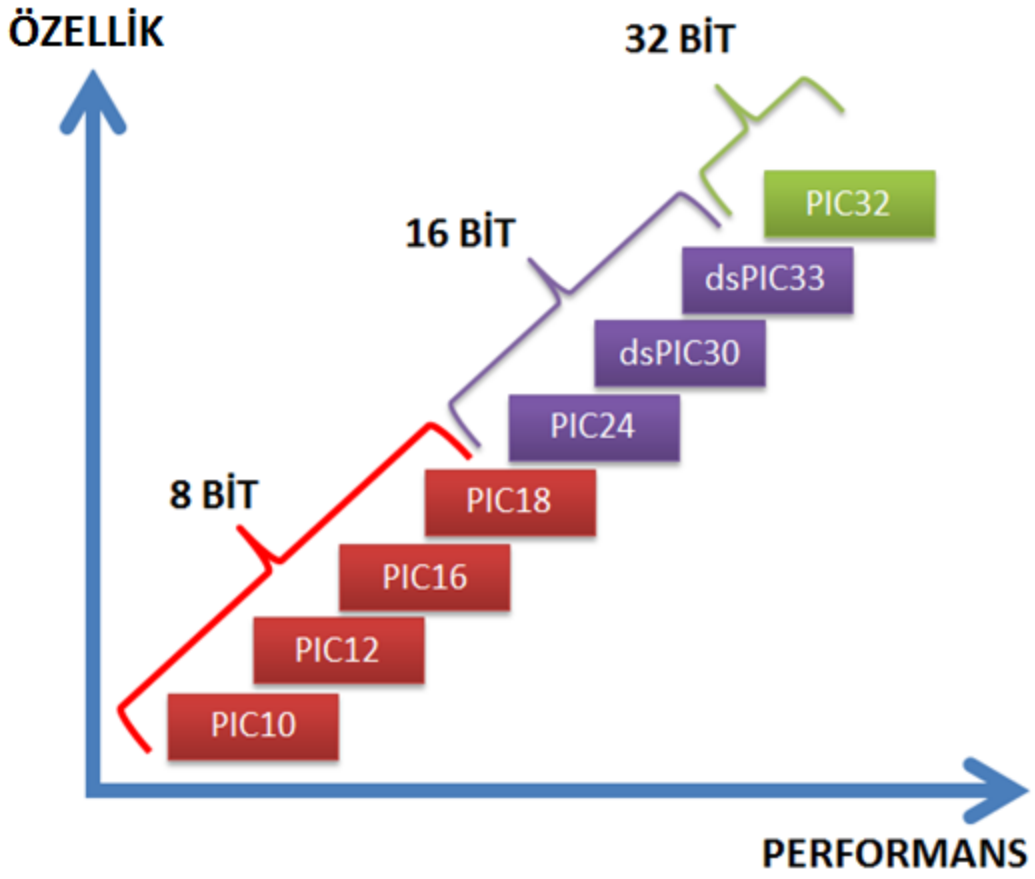
Mikrodenetleyicilerden bahsederken genellikle bu uzun isim yerine **PIC** terimi kullanılır. Orijinal kısaltması **Peripheral Interface Controller** olan **PIC** terimi, zaman içinde, **Programmable Interface Controller** deyimine, daha sonra ise **Programmable Intelligent Computer**'a dönüşmüştür.

Uygun PIC seçimi için aşağıdaki kriterlere ve özelliklere dikkat etmek gerekmektedir.

- Çalışma frekansı
- Osilatör tipi (RC, XT, HF gibi)
- Analog-sayısal dönüştürücü modülü
- Giriş-çıkış arabirim kapasitesi
- Analog ve sayısal olarak ayarlanabilen giriş/çıkış arabirimi ve uç sayısı
- Desteklenen haberleşme protokolleri (I²C, SPI, USART, PSP...)
- Darbe sinyali çıkışı
- Harici kesme
- Zamanlayıcı tabanlı kesme
- Dâhili programlama belleği kapasitesi
- Harici bellek arabirimi
- Dâhili bellek tipi seçenekleri (ROM, EPROM, PROM ve EEPROM)
- Dâhili RAM seçeneği ve kapasitesi
- Dâhili zamanlayıcı sayısı ve büyüklükleri
- USB, CAN, Ethernet bağlantı kontrolcüsü, kızılötesi modülü, v.b.

Microchip tarafından üretilen PIC'ler, 8 bitlik, 16 bitlik ve 32 bitlik entegreler olarak üretilmektedir. PIC10XXXX, PIC12XXXX, PIC16XXXX, PIC18XXXX serisi 8 bitlik, PIC24XXXX, dsPIC30XXXX, dsPIC33XXXX serisi 16 bitlik ve PIC32XXXX serisi 32 bitlik PIC'leri ifade etmektedir. Şekil 1.5'te PIC ailesine ait özellik-performans grafiği gösterilmektedir.

MİKRODENETLEYİCİ SİSTEMLERİ



Şekil 1.5. PIC ailesi, özellik - performans grafiği

MikroC

BÖLÜM 2

2. PIC18F4550 ÖZELLİKLERİ VE YAPISI

2.1. NEDEN PIC18F4550?

PIC18F4550, Microchip firması tarafından üretilen, PIC18 ailesi içerisindeki en gelişmiş mikrodenetleyicilerden biridir.

Fiyatının ucuz olması yine tercih sebebidir.

Bu PIC aşağıdaki modülleri bünyesinde barındırmasından dolayı endüstride, otomotiv uygulamalarında, medikal uygulamalarda ve tüketici elektroniğinde çokça tercih edilmektedir. Çünkü üzerindeki donanımlar birçok uygulamada oldukça yeterlidir. Sahip olduğu donanımlar şunlardır:

- Donanımsal USB iletişim birimi, (USB 2.0 ile tam uyumludur)
- 3 harici kesme ve 4 zamanlayıcı modülü
- 2 adet Yakalama (Capture) / Karşılaştırma (Compare) / Darbe Genişlik Modülasyonu (PWM) modülü (CCP)
- Geliştirilmiş Yakalama/Karşılaştırma/Darbe Genişlik Modülasyonu modülü (ECCP)
- Geliştirilmiş USART (Universal Synchronous Asynchronous Receiver Transmitter) Evrensel Senkron Asenkron Alıcı Verici modülü
- MSSP (Master Synchronous Serial Port/Ana Senkron Seri Port) modülü
- SPI (Serial Peripheral Interface/Seri Çevreirim Arabirimi) ve I2C (Inter-Integrated Circuit) ana-uydu modları
- 13 adet 10-bitlik Analog/Dijital çevirici modülü
- 256 Byte'lık EEPROM arabirimi
- 2 KB RAM kapasitesi

MikroC

2.2. GENEL ÖZELLİKLERİYLE PIC18F4550

PIC18F4550, ailedeki diğer PIC mikrodenetleyicilerde olduğu gibi ekonomik fiyata yüksek hesaplama performansı ile birlikte yüksek dayanıklılığa ve geliştirilmiş Flash program hafızasına sahiptir. Bu mikrodenetleyici, PIC18 mikrodenetleyici ailesinin bütün avantajlarını sunar.

Şekil 2.1’de, PIC18F4550’nin dış kılıfı görünmektedir.

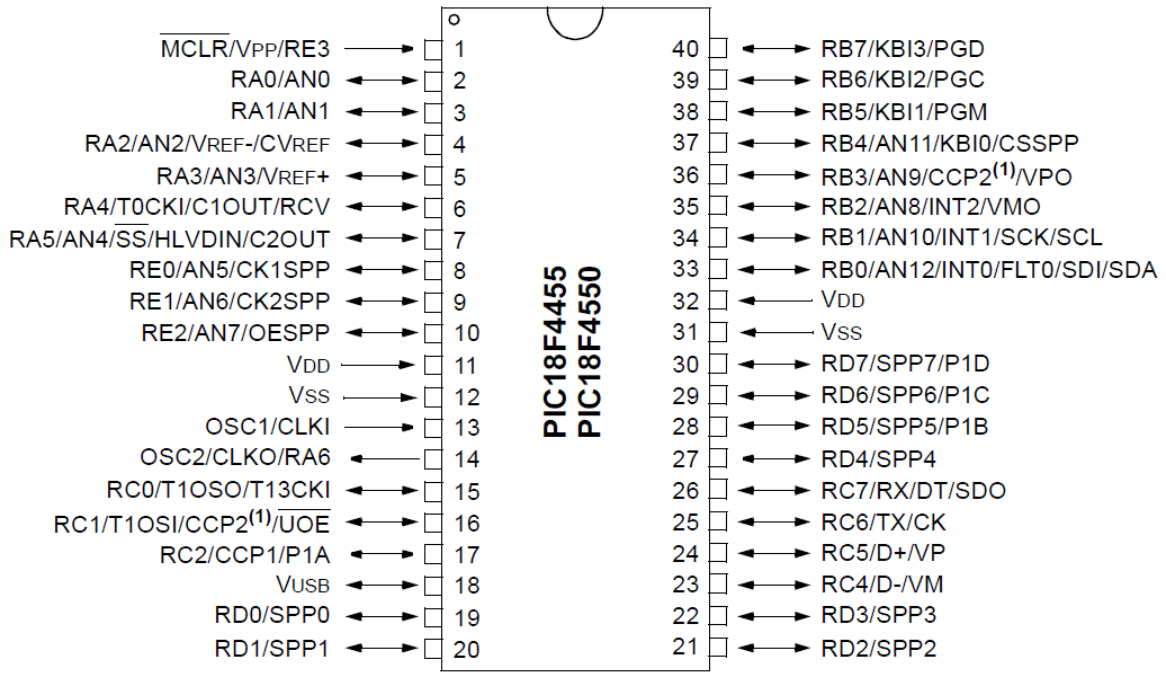


Şekil 2.1. PIC18F4550'nin dış kılıfları

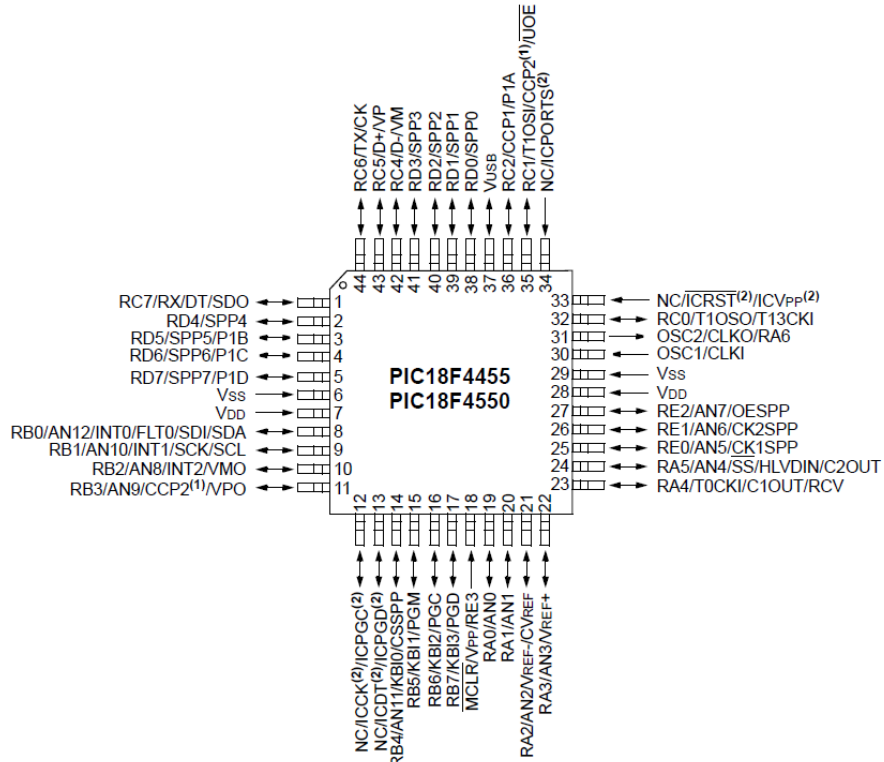
PIC18F4550, besleme gerilimi, osilatör, reset (MCLR) pinleri gibi mikrodenetleyicinin çalışması için gerekli olan pinlerin dışında 32 adet giriş/çıkış pinine sahiptir. Toplam bacak sayısı PDIP kılıfta 40 adet, TQFP ve QFN kılıflarında 44 adettir.

Şekil 2.2, Şekil 2.3 ve Şekil 2.4’te her üç paketin de pin görünüş şeması verilmiştir.

MikroC

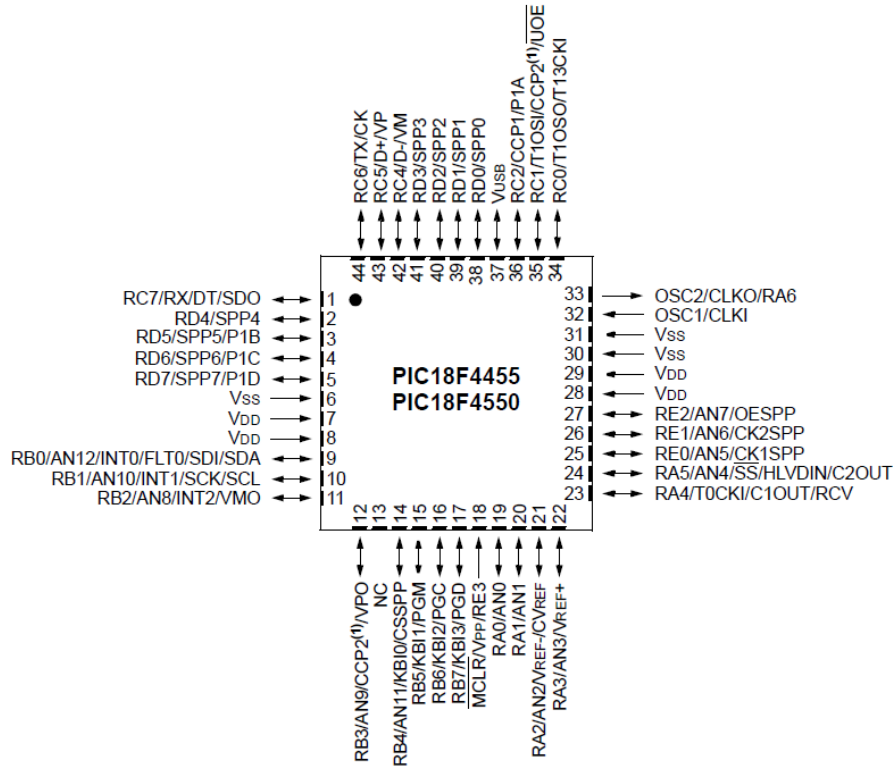


Şekil 2.2. PIC18F4550'nin PDIP kılıftaki pin görünüş şeması



Şekil 2.3. PIC18F4550'nin TQFP kılıftaki pin görünüş şeması

MikroC



Şekil 2.4. PIC18F4550'nin QFN kılıftaki pin görünüş şeması

PIC18F4550'nin genel özellikleri Tablo 2.1'de görülmektedir. Bu PIC, 48 MHZ hıza kadar çalışabilmektedir.

Tablo 2.1 PIC18F4550'nin genel özellikleri tablosu

Device	Program Memory		Data Memory		I/O	10-Bit A/D (ch)	CCP/ECCP (PWM)	SPP	MSSP		EUSART	Comparators	Timers 8/16-Bit
	Flash (bytes)	# Single-Word Instructions	SRAM (bytes)	EEPROM (bytes)					SPI	Master I ² C™			
PIC18F2455	24K	12288	2048	256	24	10	2/0	No	Y	Y	1	2	1/3
PIC18F2550	32K	16384	2048	256	24	10	2/0	No	Y	Y	1	2	1/3
PIC18F4455	24K	12288	2048	256	35	13	1/1	Yes	Y	Y	1	2	1/3
PIC18F4550	32K	16384	2048	256	35	13	1/1	Yes	Y	Y	1	2	1/3

PIC18F4550'nin özellikleri kısaca şunlardır:

- **32 Kbyte** 'lık flash program belleğine sahiptir. Bu flash belleğe 100.000 kez yazılıp silinebilir.
- Değişkenlerin bulunduğu veri belleği (SRAM) **2048 byte**'tır. Bu, 2048 adete kadar değişken tanımlayabileceğimiz anlamına gelmektedir.
- **256 byte**'lık EEPROM belleğe sahiptir. (EEPROM belleğin SRAM'den farkı kalıcı veri

MikroC

saklamak için kullanılan hafıza olmasıdır. Enerji kesilse bile veriler hafızada kalır ve istenildiği zaman bu veriler değiştirilebilir. Bu belleğe 1.000.000 defa yazma silme yapılabilir.)

- **13 adet 10 bit** çözünürlüğe sahip Analog/Dijital Dönüştürücüye sahiptir
- **İki adet CCP** (Capture / Compare / PWM) yakalama, karşılaştırma ve darbe genişlik modülasyon modülüne sahiptir.
- **1 adet ECCP** (Enhanced Capture / Compare / PWM) modülüne sahiptir.
- **SPI (master)** ve **I²C (master/slave)** protokollerinde seri iletişimi özelliğine sahip **MSSP** arabirimi vardır.
- **Asenkron ve Senkron Seri İletişimi** destekleyen **EUSART** modülüne sahiptir.
- İki adet karşılaştırıcıya sahiptir.
- 4 adet zamanlayıcı/sayıcısı vardır. (8 bitlik ve 16 bitlik)

2.2.1. USB ÖZELLİKLERİ:

- USB 2.0 ile tam uyumludur.
- Düşük hız (1.5 Mb/s) ve Tam Hız (12 Mb/s) desteklidir.
- Kontrol, kesme, eşzamanlı ve yığın tipi veri transferlerini desteklemektedir.
- 32 uç noktasına kadar destek verir. (16 iki-yönlü)
- USB için çift erişimli 1 Kbayt'lık RAM'e sahiptir.
- Kendi üzerinde USB alıcı-vericisi ve voltaj regülatörüne sahiptir.
- Harici USB alıcı-vericileri için arabirime sahiptir.
- Donanımsal Paralel Port'a (SPP) sahiptir.

2.2.2. GÜÇ TASARRUF MODLARI

- Çalışma modu (Run): CPU açık, çevrebirimler açık
- Boşta modu (Idle): CPU kapalı, çevrebirimler kapalı.
- Uyku modu (Sleep): CPU kapalı, çevrebirimler kapalı.
- Boşta modunda çekilen akım tipik olarak 5,8 μ A seviyelerindedir.
- Uyku modunda çekilen akım tipik olarak 0,1 μ A seviyelerindedir.
- Timer1 osilatörü: 1,1 μ A tipik, 32 kHz, 2 V

MikroC

- Watchdog Timer: 2.1 μ A tipik
- İki farklı hızda osilatör ile çalışabilme

2.2.3. ESNEK OSİLATÖR YAPISI:

- Yüksek Duyarlıklı PLL içeren 4 kristal modu.
- 48 MHz'e kadar iki farklı Harici saat modu
- Dahili osilatör donanım özelliği:
 - o 31 kHz – 8 MHz arası kullanıcının seçebileceği frekanslar
 - o Frekans sapmasını önlemek için kullanıcı tarafından ayarlanabilme özelliği
- Timer1 kullanılarak oluşturulan 32 kHz'lik ikinci osilatör
- Çift osilatör seçeneği, mikrodenetleyici ve USB modülünün farklı saat hızlarında çalışabilmesini sağlar.
- Arıza-güvenli saat izleyici, herhangi bir saatin durması halinde güvenli kapanmayı sağlar.

2.2.4. ÇEVREBİRİM ÖZELLİKLERİ:

- Yüksek alıcı/kaynak akımı; 25 mA/25 mA
- Üç harici kesme
- Dört Zamanlayıcı Modülü: Timer0 dan Timer3'e kadar
- 2 adet Capture/Compare/PWM (Yakalama/Karşılaştırma/Darbe Genişlik Modülasyonu) modülü (CCP)
 - o 16 bit yakalama, maksimum 5.2 ns çözünürlük
 - o 16 bit karşılaştırma, maksimum 83.3 ns çözünürlük
 - o PWM çıkışı: PWM çözünürlüğü 1-10 bit arası
- Geliştirilmiş Yakalama/Karşılaştırma/Darbe Genişlik Modülasyonu modülü (ECCP)
 - o Çoklu çıkış modları
 - o Seçilebilir polarite
 - o Programlanabilen ölü zaman (dead time)
 - o Otomatik kapanma ve otomatik yeniden başlama
- Geliştirilmiş USART (Universal Synchronous Asynchronous Receiver)

MikroC

- o LIN desteği (Local Interconnect Network / Yerel Bağlantılı Ağ)
- MSSP (Master Synchronous Serial Port/Ana Senkron Seri Port) modülü 3–tel SPI
 - o (Serial Peripheral Interface/Seri Çevreirim Arabirimi) (dört modun tamamı) ve I2C master ve slave modları
- 13 adet 10–bitlik Analog/Dijital çevirici modülüne sahiptir.
- Giriş çoklayıcılı çift analog karşılaştırıcıya sahiptir.

2.2.5. ÖZEL MİKRODENETLEYİCİ ÖZELLİKLERİ:

- Genişletilmiş komut setiyle C derleyicileri için mükemmel bir mimariye ulaştırılmıştır.
- 100.000 kez yazma/silme kapasitesine sahip geliştirilmiş Flash program hafızasına sahiptir.
- 1.000.000 kez yazma/silme kapasitesine sahip Data EEPROM hafızasına sahiptir.
- Flash/Data EEPROM üzerindeki bilgiler 40 yılın üzerinde saklanabilir.
- Yazılım kontrolü ile kendi kendisini programlayabilme özelliğine sahiptir.
- Kescmeler için öncelik seviyelerine sahiptir.
- 8 x 8 tek–döngülü donanım çarpanına sahiptir.
- İki bacağından verilecek 5 Voltluk tek bir kaynakla ICSP (In-Circuit Serial Programming) sağlanır.
- İki pin ile devre çalışırken hata ayıklayabilme (In-Circuit Debugging) özelliğine sahiptir.
- Geniş bir çalışma voltaj aralığına sahiptir. (2,0 V–5,5 V)

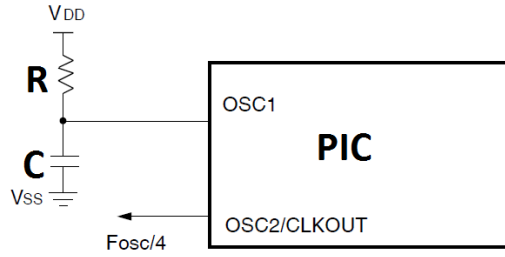
2.3. OSİLATÖR DEVRELERİ

Mikrodenetleyici devrelerinin çalışması için sürekli bir saat darbesi gerekmektedir. Bu saat darbesi, hassas zamanlama gerektiren veya zamanlamanın önemli olduğu durumlarda dışarıdan bir kristal bağlayarak sağlanır. 18F4550 içerisinde olduğu gibi bazı PIC’lerde dahili saat darbesi üreten devreler bulunmaktadır. Hassas zaman istemeyen uygulamalarda bu dahili saat devreleri tercih edilebilir.

MikroC

2.3.1. RC OSİLATÖR

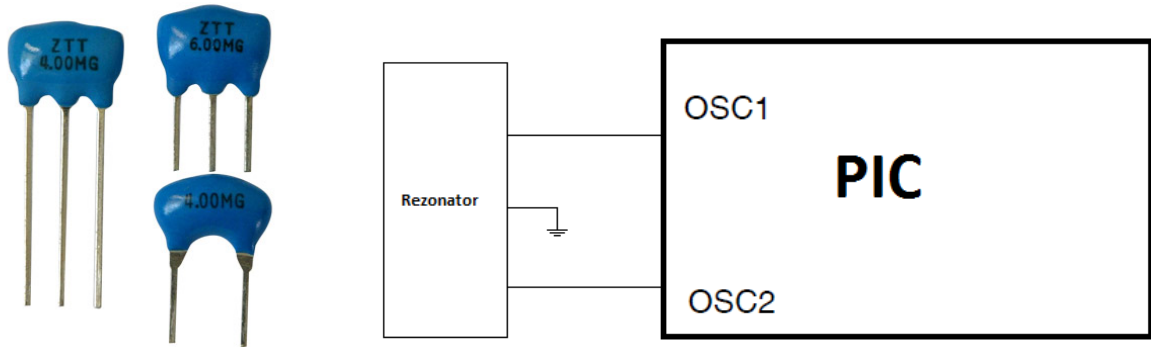
Hassas zaman gerektirmeyen uygulamalarda kullanılabilecek bir osilatör türüdür. Genellikle daha ucuza mal olması istenen devrelerde kristal yerine mikrokontrolöre bir direnç ve kondansatör bağlayarak saat darbesi sağlanır (Şekil 2.5). Direnç olarak 5 ile 100 K Ω arasında bir değer, kapasitör olarak da genellikle 20-22 pF lık bir değer seçilir.



Şekil 2.5 RC osilatör devresi

2.3.2. REZONATÖR OSİLATÖR

Kristal osilatör kadar hassas olmayan bu rezonatör osilatörler ucuz olması istenen devrelerde tercih edilebilir. Genellikle 4MHz ile 20MHz arasında 2 bacaklı ya da 3 bacaklı olarak üretilirler. 3 bacaklılarda orta bacak toprağa (şaseye) bağlanır (Şekil 2.6).



Şekil 2.6 Rezonatör ve rezonatör osilatör

2.3.3. KRİSTAL OSİLATÖR

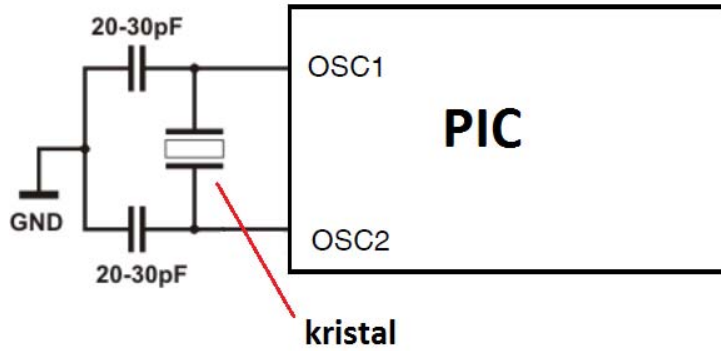
Hassas saat darbesi gereken uygulamalarda tercih edilen osilatör türüdür. Bir kristal ve iki kondansatörden oluşan bir devre şeklindedir. Kristaller Şekil 2.7 deki gibi değişik kılıflarda ve değişik frekanslarda üretilir. Frekans değerleri kılıfların üzerinde yazılıdır.

MikroC



Şekil 2.7 Kristaller

Şekil 2.8’de kristal osilatörün PIC’e bağlantısı görülmektedir.



Şekil 2.8 Kristal osilatör

PIC mikrodeneleyicilerde aşağıdaki gibi değişik şekillerde saat darbeleri elde edilebilir.

- LP modu (düşük güç) düşük frekanslı kristal için kullanılır. Bu mod, sadece 32.768 kHz kristaller, genellikle saatler içinde gömülü sürücüler için tasarlanmıştır. Bu kristaller küçük boyutlu ve silindir şeklinde olduğu için tanımak kolaydır.
- Kristal ve rezonatör ile kullanılan XT modu 8 MHz’e kadar olan frekans üretiminde kullanılır.
- HS modu (yüksek hız) yüksek frekans kristalleri ile birlikte 8 MHz üzerinde frekans üretmek için kullanılır.

2.4. PIC18F4550 OSİLATÖR AYARLARI

PIC18F2455/2550/4455/4550 ailesindeki mikrodeneleyiciler, daha önceki PIC18F tipi cihazlardan farklı bir osilatör ve mikrodeneleyici saat sistemine sahiptirler. USB modülünün

MikroC

eklenmesiyle, USB düşük-hız ve tam-hız için kararlı bir saat kaynağına ihtiyaç duyulması ve bu gereksinimleri karşılamak üzere, düşük-hız ve yüksek-hıza uyum sağlayabilecek ayrı bir saat kaynağı eklenmesini gerekli hale getirmiştir. Bu gereksinimleri karşılamak için PIC18F2455/2550/4455/4550 ailesindeki tüm mikrodeneleyicilere, USB tam-hız işlemini gerçekleştirebilecek 48 MHz bir saat kısmı eklenmiştir. Diğer tüm osilatör özellikleri PIC18 geliştirilmiş mikrodeneleyicilerdekilerle aynı bırakılmıştır.

PIC18F4550'nin osilatörü iki konfigürasyon kaydedicisi (CONFIG1L ve CONFIG1H) ile iki kontrol kaydedicisi (OSCCON ve OSCTUNE) tarafından kontrol edilir. PIC18F4550 oniki farklı osilatör modunda çalıştırılabilir. FOSC3:FOSC0 konfigürasyon bitleri ayarlanarak bu 12 adet osilatör tipinden biri seçilebilir.

- | | |
|-------------------|---|
| 1. XT | Kristal/Rezonatör osilatör |
| 2. XTPLL | Kristal/Rezonatör osilatör (PLL seçili) |
| 3. HS | Yüksek – Hız Kristal/Rezonatör osilatör |
| 4. HSPLL | Yüksek – Hız Kristal/Rezonatör osilatör (PLL seçili) |
| 5. EC | Harici Saat $F_{osc}/4$ çıkışı ile osilatör |
| 6. ECIO | Harici Saat RA6 üzerindeki I/O ile osilatör |
| 7. ECPLL | Harici Saat (PLL seçili) ve RA6 üzerindeki $F_{osc}/4$ ile osilatör |
| 8. ECPIO | Harici Saat (PLL seçili) RA6 üzerindeki I/O ile osilatör |
| 9. INTHS | Dahili osilatör (mikrodeneleyici saat kaynağı olarak kullanılır, HS osilatör USB saat kaynağı olarak kullanılır.) |
| 10. INTXT | Dahili osilatör (mikrodeneleyici saat kaynağı olarak kullanılır, XT osilatör USB saat kaynağı olarak kullanılır.) |
| 11. INTIO | Dahili osilatör (mikrodeneleyici saat kaynağı olarak kullanılır, EC osilatör USB saat kaynağı olarak kullanılır, RA6 üzerindeki dijital I/O ile.) |
| 12. INTCKO | Dahili osilatör (mikrodeneleyici saat kaynağı olarak kullanılır, EC osilatör USB saat kaynağı olarak kullanılır, RA6 üzerindeki $F_{osc}/4$ ile.) |

MikroC

2.4.1. PLL (FREKANS ÇARPANI)

PLL genellikle girişe uygulanan kristal frekansını çarparak mikrodnetleyicideki komutların daha hızlı işlemesi amacıyla kullanılır. Örneğin girişe 4 MHz'lik bir kristal bağlanarak PLL x4 olarak ayarlandığında mikrodnetleyici 16 MHz'lik frekans verilmiş gibi işlem yapar.

Tablo 2.2'de örnek çarpan durumları (PLL Multiplier), girişe uygulanan kristal frekansı (F_{IN}) ile çıkışta elde edilen frekans değerleri (F_{OUT}) görülmektedir.

Tablo2.2 PLL, giriş ve çıkış frekans değerleri tablosu

PLL Frequency Range

F_{IN}	PLL Multiplier	F_{OUT}
4 MHz-10 MHz	x4	16 MHz-40 MHz
4 MHz-10 MHz	x8	32 MHz-80 MHz
4 MHz-7.5 MHz	x16	64 MHz-120 MHz

PLL; HSPLL, XTPLL, ECPLL ve ECPIO Osilatör modlarında aktiftir. Sabitlenmiş 4 MHz'lik girişten sabitlenmiş 96 MHz'lik referans saati üretmek için dizayn edilmiştir. Çıkış daha sonra hem USB hem de mikrodnetleyici çekirdek saati tarafından bölünerek kullanılır.

2.4.2. USB İÇİN OSİLATÖR AYARLARI

PIC18F4550'nin kullanıldığı bir USB devresinde, alçak-hız (low-speed) için 6 MHz, yüksek-hız (full-speed) için 48 MHz'lik bir saat frekansı gerekmektedir. Bu nedenle osilatör frekansının seçimine dikkat edilmelidir. Alçak-Hız modu için gereken USB saati, PLL modülünden değil birincil osilatör kaynağından elde edilir ve 4'e bölünerek 6 MHz'lik saat darbesi üretilir. Bu yüzden, USB modül aktifken ve denetleyici saat kaynağı birincil osilatör modlarından (XT, HS veya EC, PLL ile birlikte) biri iken, mikrodnetleyici sadece 24 MHz'lik saat frekansını kullanabilir. Bu kısıtlama mikrodnetleyici ikincil osilatör veya dahili osilatör bloğunu saat kaynağı olarak kullanıyorsa uygulanamaz.

MikroC

2.5. PIC18F4550'DE RESET (SIFIRLAMA) İŞLEMİ

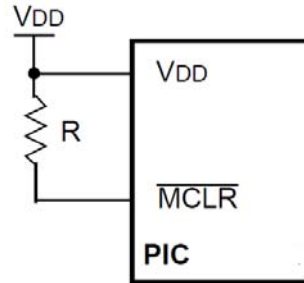
PIC18F4550 birçok Reset (Sıfırlama) özelliğini bünyesinde barındırır. RCON kaydedicisi PIC18F4550 için sıfırlama işlemlerini yürüten kaydedicidir. Sıfırlama işlemleri arasında en çok kullanılanlar MCLR ile sıfırlama ve POR ile sıfırlamadır (Power-On Reset/Enerji Verildiğinde Sıfırlama). Diğer tüm sıfırlama durumları bu iki sıfırlamadan türetilmiştir.

2.5.1. MCLR İLE SIFIRLAMA

PIC'in MCLR girişine 5 Volt uygulandığı sürece program normal akışında çalışmaya devam eder. MCLR bacağına 0 Volt uygulandığı anda sıfırlama gerçekleşir.

2.5.2. ENERJİ VERİLDİĞİNDE SIFIRLAMA (POR, POWER-ON RESET)

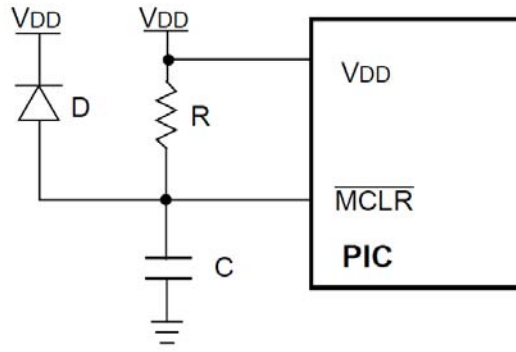
PIC, VDD geriliminde bir artış tespit ettiği anda kaydedicileri hazır hale getirerek kendisini sıfırlar. Power On Reset avantajından yararlanmak için MCLR pini doğrudan veya bir direnç üzerinden VDD gerilimine bağlanır (Şekil 2.9).



Şekil 2.9 MCLR bacağına bir direnç üzerinden VDD'ye bağlantısı

Şekil 2.10'daki gibi harici bir POR bağlantısı da kullanılabilir. Bu devredeki diyot, VDD enerjisi kesildiğinde kodansatörün hızlıca boşalması için kullanılmıştır. PIC'in elektriksel karakteristiklerinin bozulmaması açısından R direncinin 40 kΩ'dan düşük bir değer olması önerilmektedir.

MikroC



Şekil 2.10 Harici POR bağlantısı

2.5.3. GERİLİM DÜŞMESİ SIFIRLAMASI (BOR, BROWN-OUT RESET)

Ani gerilim düşmesi esnasında PIC'in içindeki kaydedicilerin ve diğer değişkenlerin hatalı değer almasının önlenmesi açısından programın kontrolünün sağlanabilmesi için PIC'in sıfırlanması gerekmektedir. Bu da Brown Out Reset kaydedicileri ayarlanarak sağlanabilir.

2.6. PIC18F4550'NİN HAFIZA BİRİMİ

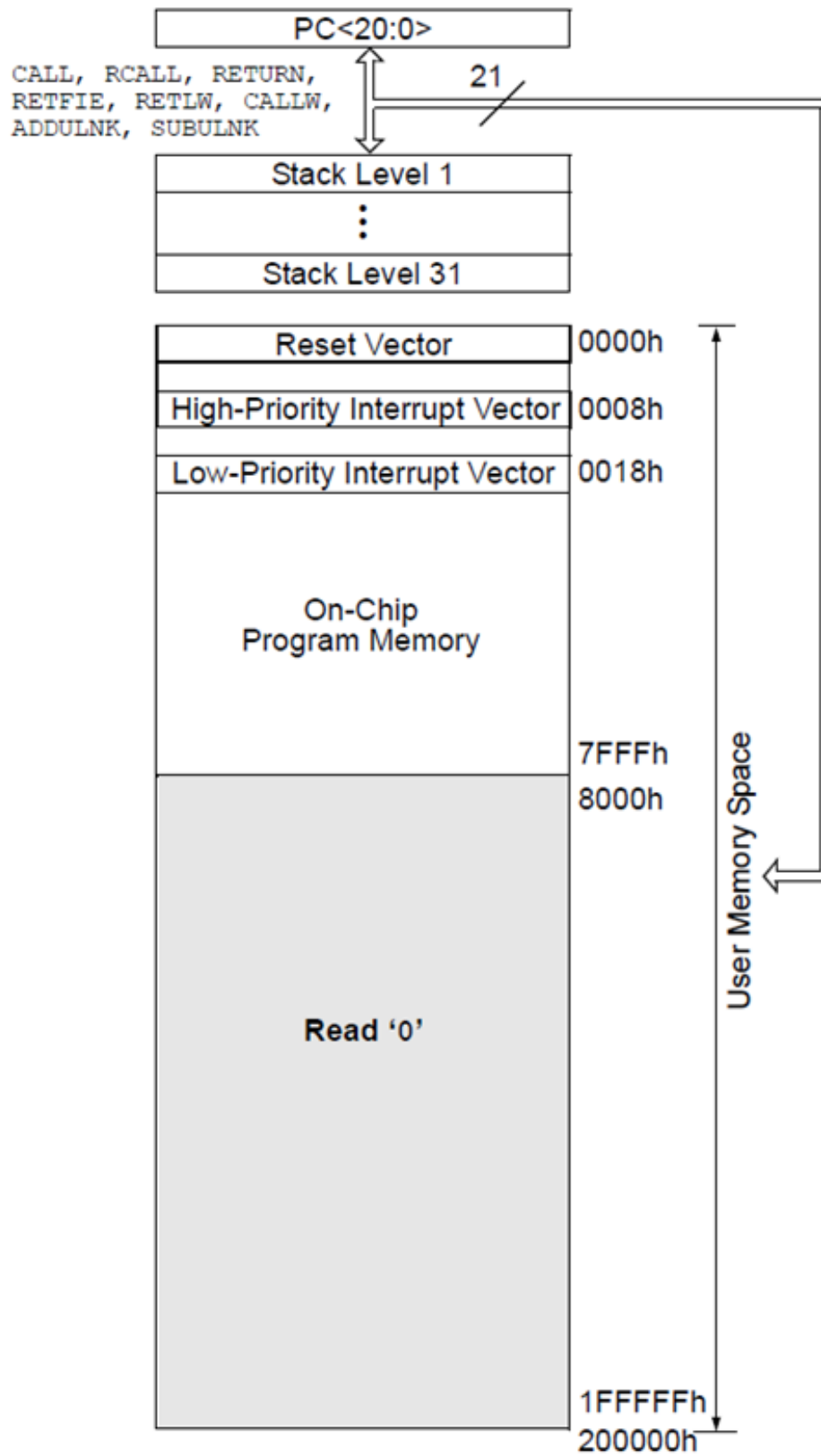
PIC18F4550, PIC18F ailesi içerisindeki diğer mikrodeneleyicilerde olduğu gibi 3 çeşit hafızaya sahiptir. Bunlar;

- Program Hafızası
- Veri Hafızası (RAM)
- Kalıcı veri hafızası (EEPROM)

PIC18F4550 2 Megabyte'lık program hafıza alanını adresleyebilecek 21-bitlik bir sayıcı içerir. Bu PIC, 32 KBayt'lık bir Flash belleğe sahiptir. Bu Flash Bellek sayesinde 16.384 adet tek-kelimelik komutu hafızasında saklayabilir. PIC18F4550 iki adet kesme vektörüne sahiptir. Sıfırlama vektör adresi 0000h ve kesme vektörleri adresleri 0008h ve 0018h'dır.

PIC18F4550'nin program hafıza diyagramı Şekil 2.11'de gösterilmiştir.

MikroC



Şekil 2.11 PIC18F4550'nin hafıza diyagramı

MikroC

BÖLÜM 3

3. MİKROC

3.1. MİKROC NEDİR?

MikroC, Mikroelektronika firmasının geliştirdiği bir derleyicidir. C dilini kullanması sebebiyle C programcılığı ile uğraşmış olan kişiler, PIC mantığını kavradıktan sonra rahatlıkla MikroC ile program yazabilirler. Ancak C dilini kullanmamış olanlar hiç endişe etmesin. Çünkü bu kitap sayesinde hem C dilinin mantığını hem de MikroC ile program yazmayı öğrenmiş olacaksınız. Üstelik kitabın ilerleyen bölümlerinde Microsoft Visual C# ile yapılmış uygulamaları incelediğinizde, aynı zamanda Visual C#'ı a öğrenmiş olacaksınız.

3.2. NEDEN MİKROC?

Piyasada CCS-C, HI-TECH C, MPLAB® C gibi değişik derleyiciler bulunmasına rağmen, MikroC'nin tercih edilmesinin başlıca sebeplerini aşağıdaki gibi sıralayabiliriz.

- ▶ C tabanlı bir yazım dili olması, bununla birlikte komut yapısının kolay ve kullanışlı olması
- ▶ Tek bir programda bir mikrodenetleyici programcısı için gerekli olan her şeyin olması, yani hiçbir yardımcı programa gerek kalmaması
- ▶ Düşünülen bir proje için kütüphanesinin oldukça yeterli olması ve hiçbir eklentiye ihtiyaç duymaması
- ▶ Mikroelektronika firmasının derleyici ve kütüphanelerini, diğer firmalara oranla daha sık güncellemesi ve yenilemesi, teknolojiye daha çabuk ayak uydurması
- ▶ Tek bir dil ile PIC, PIC24/dsPIC, PIC32, ARM, AVR, 8051 platformlarında program yazılabilmesi
- ▶ MikroC içerisinde gelen örnek devre şemaları ve program kodlarının her birinin hemen hemen birer proje yapısında olması
- ▶ İnternet üzerinden forum aracılığıyla veya doğrudan iletişim yöntemiyle Mikroe team tarafından yardım olanağı

MikroC

- ▶ Paralel, seri ve USB veri iletişimine destek vermesi ve bu iletişim türlerini kullanıcıya çok kolay bir şekilde kullandıran yapısının bulunması
- ▶ Libstock sitesi üzerinden örnek projelere ve örnek kodlara ulaşım imkanının bulunması
- ▶ Derleyici içerisinde yer almayan ve sonradan geliştirilen kütüphanelerin programa rahatlıkla entegre edilebilmesi ve kütüphane desteğinin artırılması
- ▶ Mikroelektronika firmasının ses, iletişim, depolama vs... konularında yaklaşık 300 adet hazır modül kartının bulunması ve bu kartların her birinin kullanımı konusunda açıklayıcı örneklerin bulunması

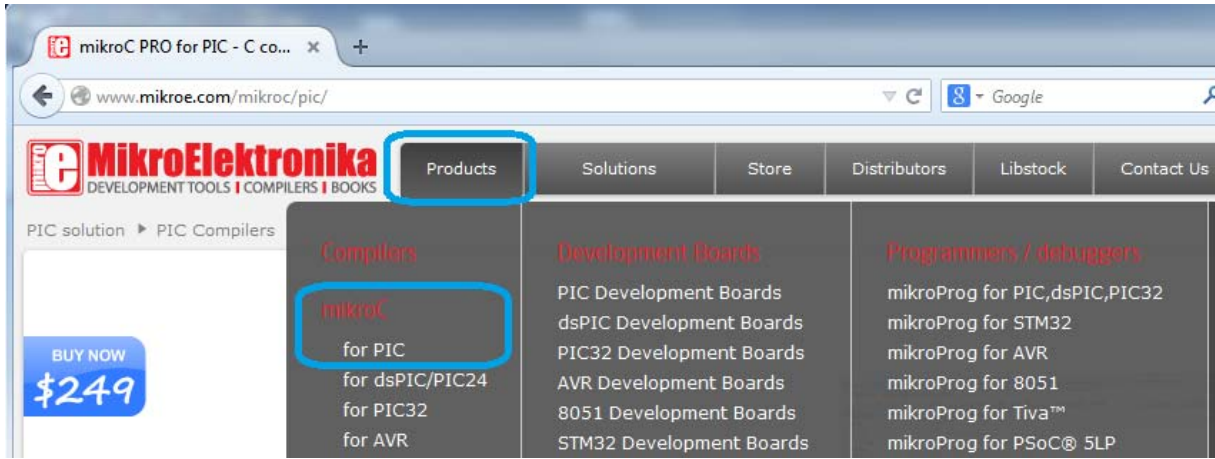
3.3. MİKROC'YE GİRİŞ

MikroC proje mantığıyla çalışan bir derleyicidir. MikroC'nin PIC, ARM, PIC32, dsPIC gibi birçok mikrodenetleyiciye destek vermesi sebebiyle bu kitaptaki anlatımlar için ve PIC18F4550 için kullanacağımız derleyici **MikroC for PIC**'dir. Bu derleyici PIC12, PIC16 ve PIC18 ailelerindeki hemen hemen bütün mikrodenetleyicilere destek vermektedir. Kitabın yazıldığı tarih itibarıyla 565 adet mikrodenetleyici desteklenmektedir (Derleyici sürekli güncellenmekte ve desteklenen PIC sayısı her geçen gün artmaktadır).

MikroC for PIC'in demo versiyonunu www.mikroe.com adresinden indirerek bilgisayarınıza kurabilirsiniz. Demo versiyonundaki tek kısıtlama 2 Kbyte boyutuna kadar program yazımına izin vermesidir. Bu da başlangıç aşamasındaki birçok projenin sorunsuzca yazılıp derlenebilmesi anlamına gelmektedir. Yazım limiti haricinde bütün kütüphaneler ve bütün kodlar sorunsuzca kullanılabilir. Lisansı satın aldığınızda ise yazım limiti kalkmakla birlikte ömür boyu teknik desteğe ve güncelleme hakkına sahip oluyorsunuz.

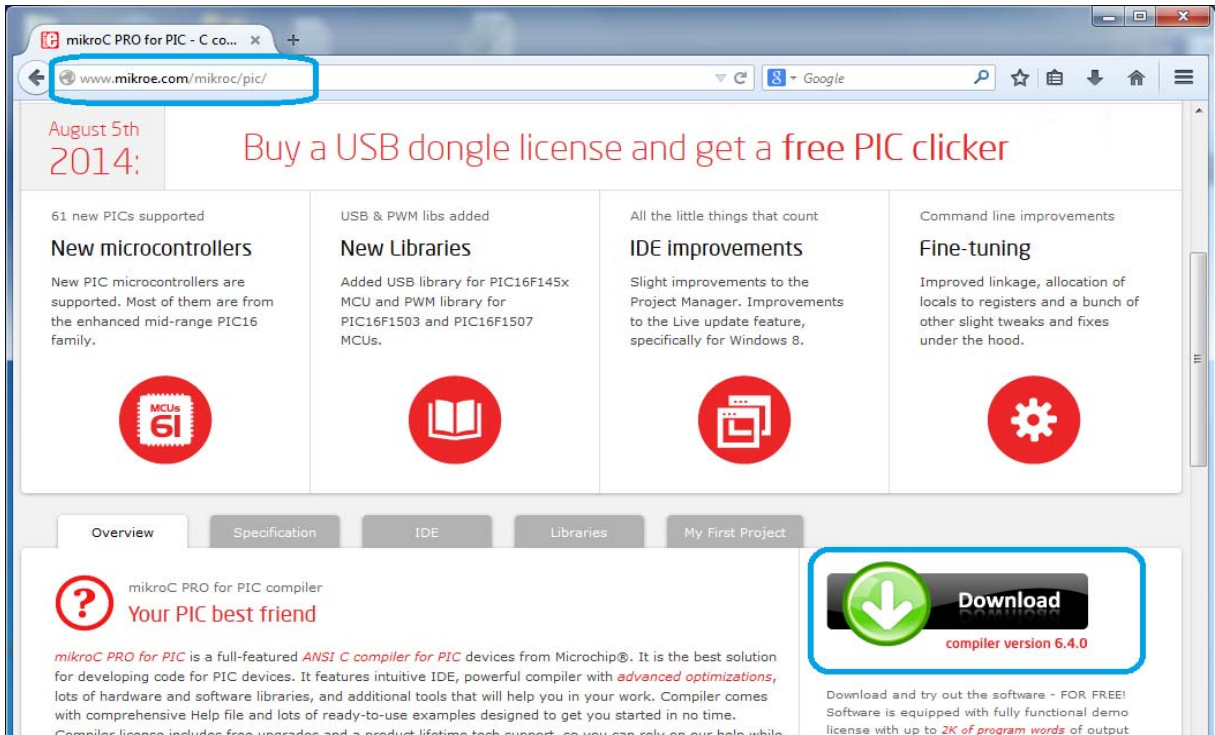
MikroC'nin son versiyonunu indirmek için ilgili web sayfası açıldıktan sonra üst menüden sırasıyla Products → MikroC for PIC bağlantısı tıklanır (Şekil 3.1).

MikroC



Şekil 3.1 MikroElektronika ana sayfası

Açılan sayfada biraz aşağıda bulunan **Download** düğmesi üzerine tıklanarak son versiyonu indirilir. Bu kitabın yazıldığı sıradaki versiyonu 6.4.0'dır (Şekil 3.2).



Şekil 3.2 MikroC indirme sayfası

Program **zip** dosyası şeklinde indiği için bilgisayarınıza kurabilmeniz için öncelikle zip içerisinden çıkarılması gerekmektedir. Bunun için Windows'un kendi yerleşik arşivcisi kullanılabileceği gibi Winzip ya da Winrar benzeri bir programla da çıkartma işlemi yapılabilir.

MikroC

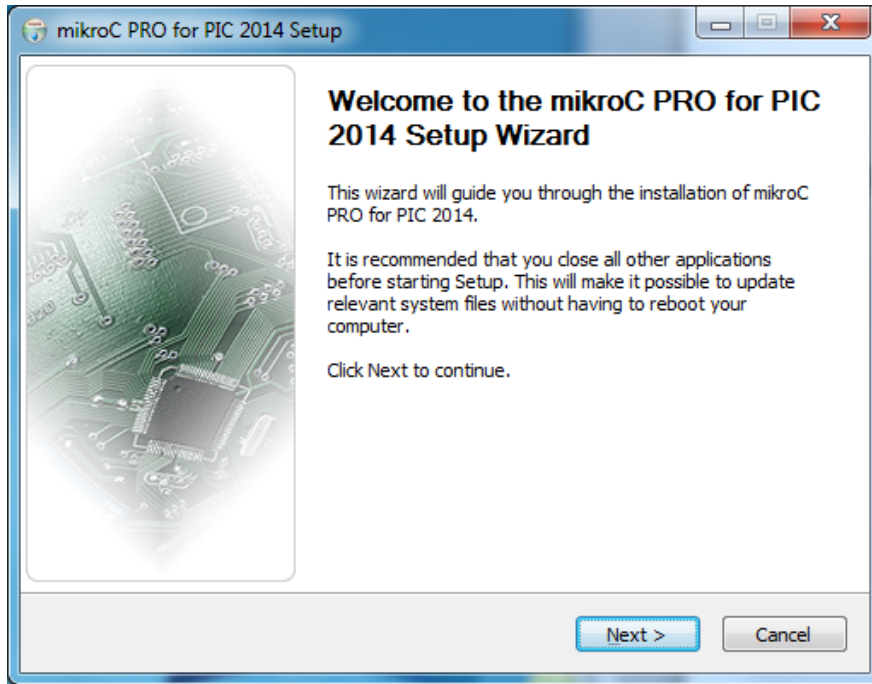
3.4. MİKROC'NİN BİLGİSAYARA KURULUMU

MikroC'yi zip içerisinde çıkarttıktan sonra kurulumu geçilebilir. Bunun için dosya üzerine çift tıklanır (Şekil 3.3).



Şekil 3.3 MikroC Kurulum Dosyası

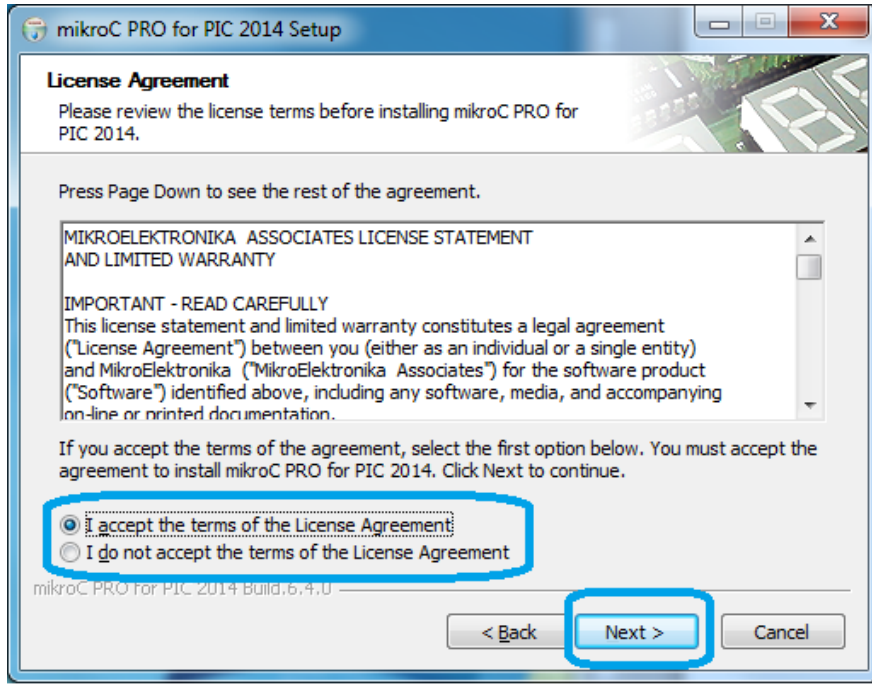
Kurulumun başlangıcında bir Hoşgeldiniz penceresi görünür. Next düğmesi tıklanır (Şekil 3.4).



Şekil 3.4 MikroC kurulum penceresi 1

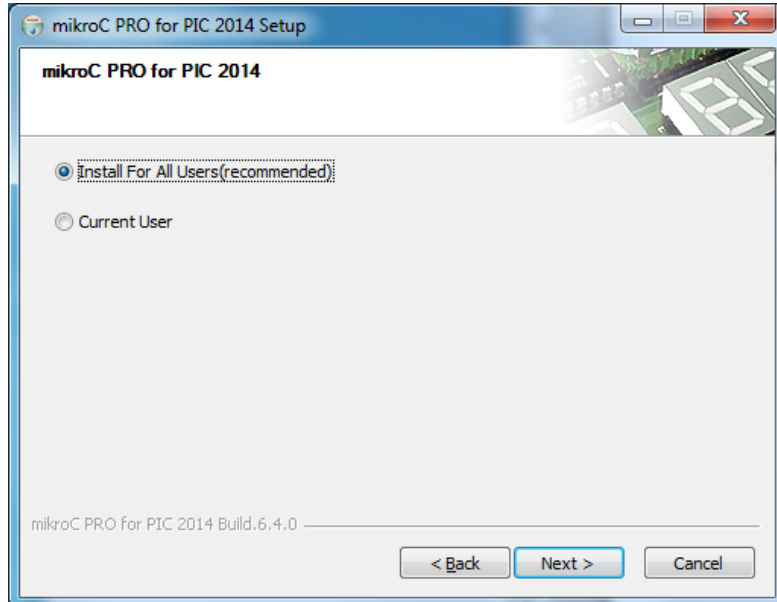
Ardından lisans sözleşmesinin olduğu pencere gelir. Lisans şartları kabul edilerek kurulumu devam edilir (Şekil 3.5).

MikroC



Şekil 3.5 MikroC Kurulum Penceresi 2

Diğer sayfada kurulumun tüm kullanıcılar için mi yoksa sadece Windows'ta oturum açmış olan kullanıcı için mi kurulumun yapılacağını sorar. Tüm kullanıcılar'dan yana tercihimizi kullanarak devam ediyoruz (Şekil 3.6).

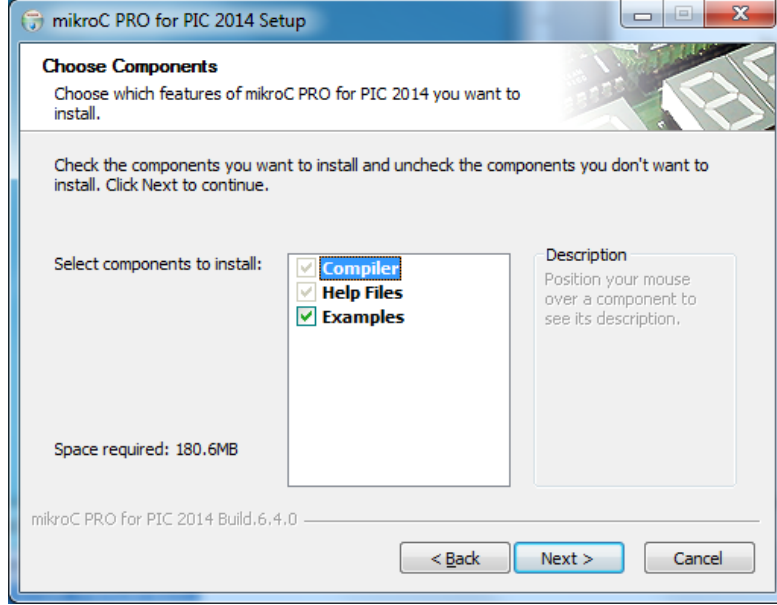


Şekil 3.6 MikroC Kurulum Penceresi 3

Sonraki sayfada kurulum sırasında hangi bileşenlerin kurulacağı sorulmaktadır.

MikroC

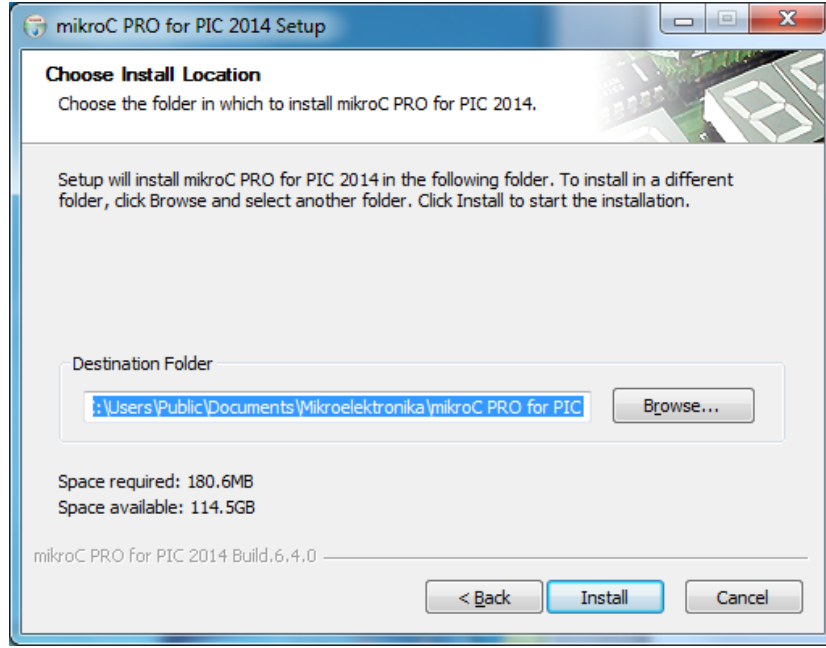
Burada derleyicinin kendisi (Compiler), yardım dosyaları (Help Files) zorunlu olarak kurulmakta, kullanıcıya örnek dosyaların ve projelerin (Examples) kurulup kurulmayacağı onayı istenmektedir. Değişiklik yapmadan devam ediyoruz (Şekil 3.7).



Şekil 3.7 MikroC Kurulum Penceresi 4

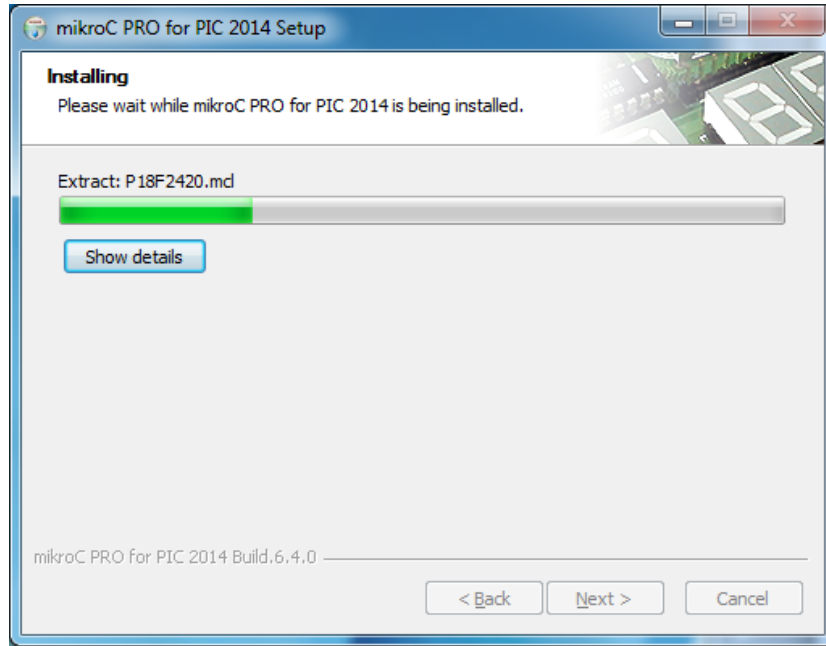
Kurulumun bilgisayarda hangi klasör altına yapılacağı sorulmaktadır. Bu sayfada değişiklik yapmazsak, program varsayılan olarak tüm kullanıcıların ortak belgeler klasörü altında Mikroelektronika klasörüne kurulmaktadır. Değişiklik kişisel tercihe bağlıdır. Değiştirmeden devam ediyoruz (Şekil 3.8).

MikroC



Şekil 3.8 MikroC Kurulum Penceresi 5

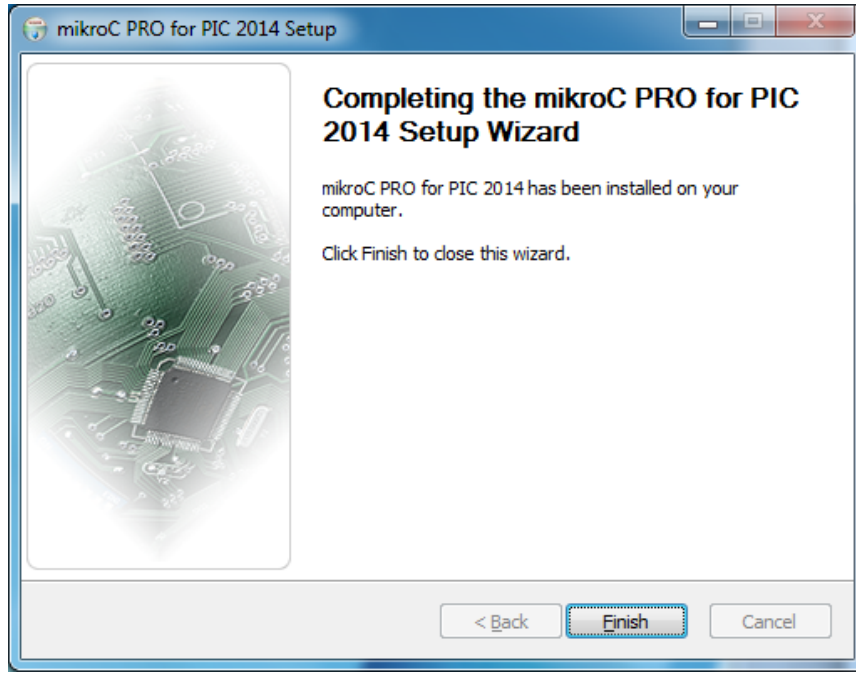
Kurulum başlıyor ve program bilgisayara kuruluyor (Şekil 3.9).



Şekil 3.9 MikroC Kurulum Penceresi 6

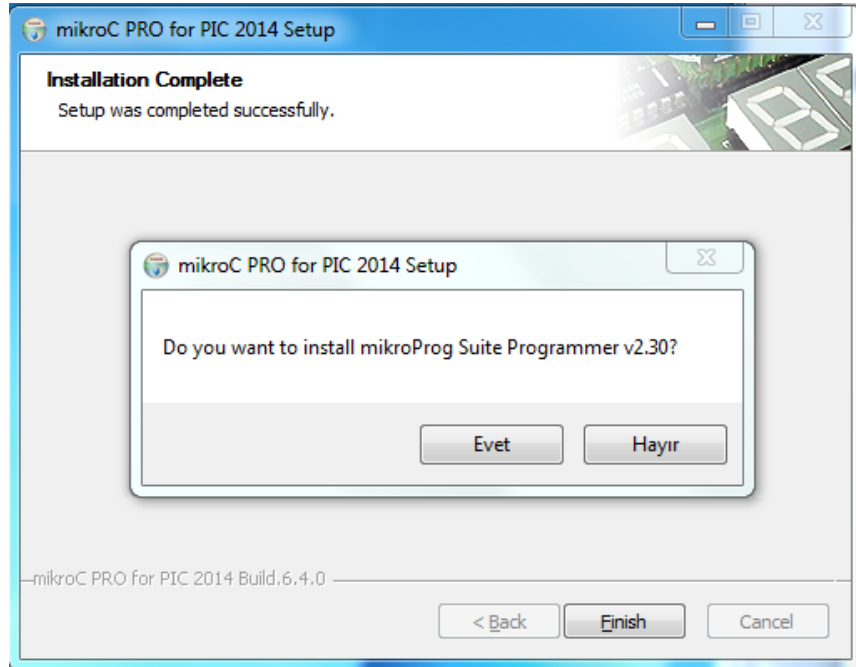
Kurulum bittiğinde **Finish** düğmesine tıklayabileceğimiz bir pencere geliyor ve finish'e tıklıyoruz (Şekil 3.10).

MikroC



Şekil 3.10 MikroC Kurulum Penceresi 7

Karşımıza bir mesaj penceresi geliyor ve bizden mikroProg Suite Programmer'ı kurmak isteyip istemediğimizi soruyor (Şekil 3.11).



Şekil 3.11 MikroC Kurulum Penceresi 8

MikroC

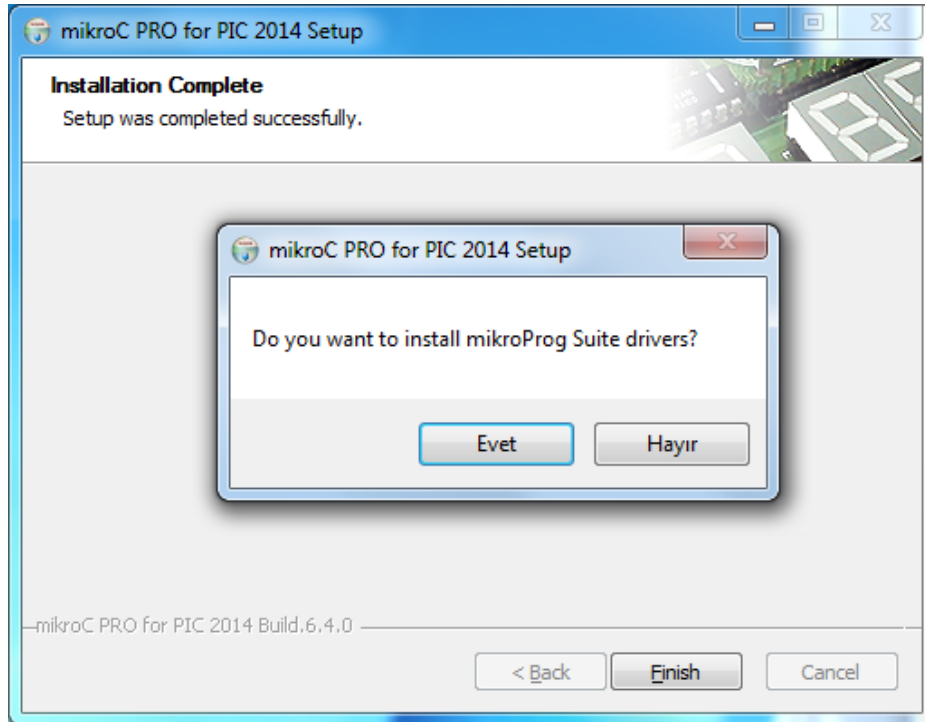
Yeri gelmişken bu programın ne olduğundan bahsedelim. mikroProg Suite Programmer, mikroelektronika firmasının kendi geliştirme kartlarıyla beraber çalıştığınızda, PIC'in doğrudan devre üzerinden sökölmeden programlanmasını ve böcek ayıklamasını sağlayan bir yazılımdır. Gerekli donanım elinizde mevcutsa bu programı kurabilirsiniz. Bu kitaptaki örneklerin birçoğu Mikroelektronika firmasının **EasyPIC 7** geliştirme kartıyla rahatlıkla uygulanabilir (Şekil 3.12).



Şekil 3.12 Easy PIC 7 Geliştirme Kartı

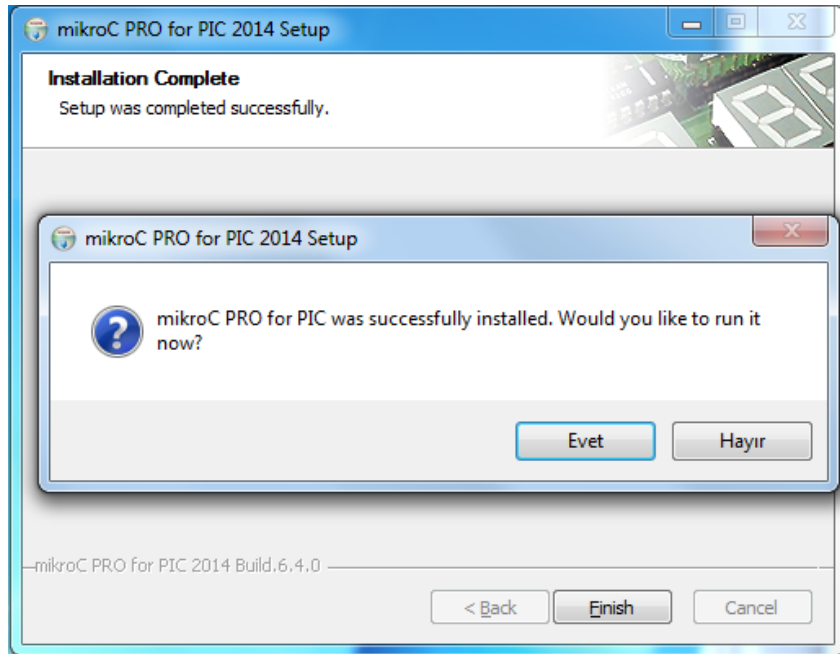
Konuyu bölmeden devam edelim. mikroProg Suite Programmer sorusuna verdiğiniz cevaptan sonra, kurulum dosyası bu kez de gerekli sürücüler bilgisayarınıza kurmak isteyip istemediğinizi soruyor. Bu soruya da donanımınız mevcutsa Evet diyerek devam edebilirsiniz (Şekil 1.13).

MikroC



Şekil 3.13 MikroC Kurulum Penceresi 9

Kurulumun en son soracağı soru, derleyiciyi hemen çalıştırmak isteyip istemediğiniz hususundadır. Çalıştırarak derleyici arayüzünü görebiliriz (Şekil 3.14).

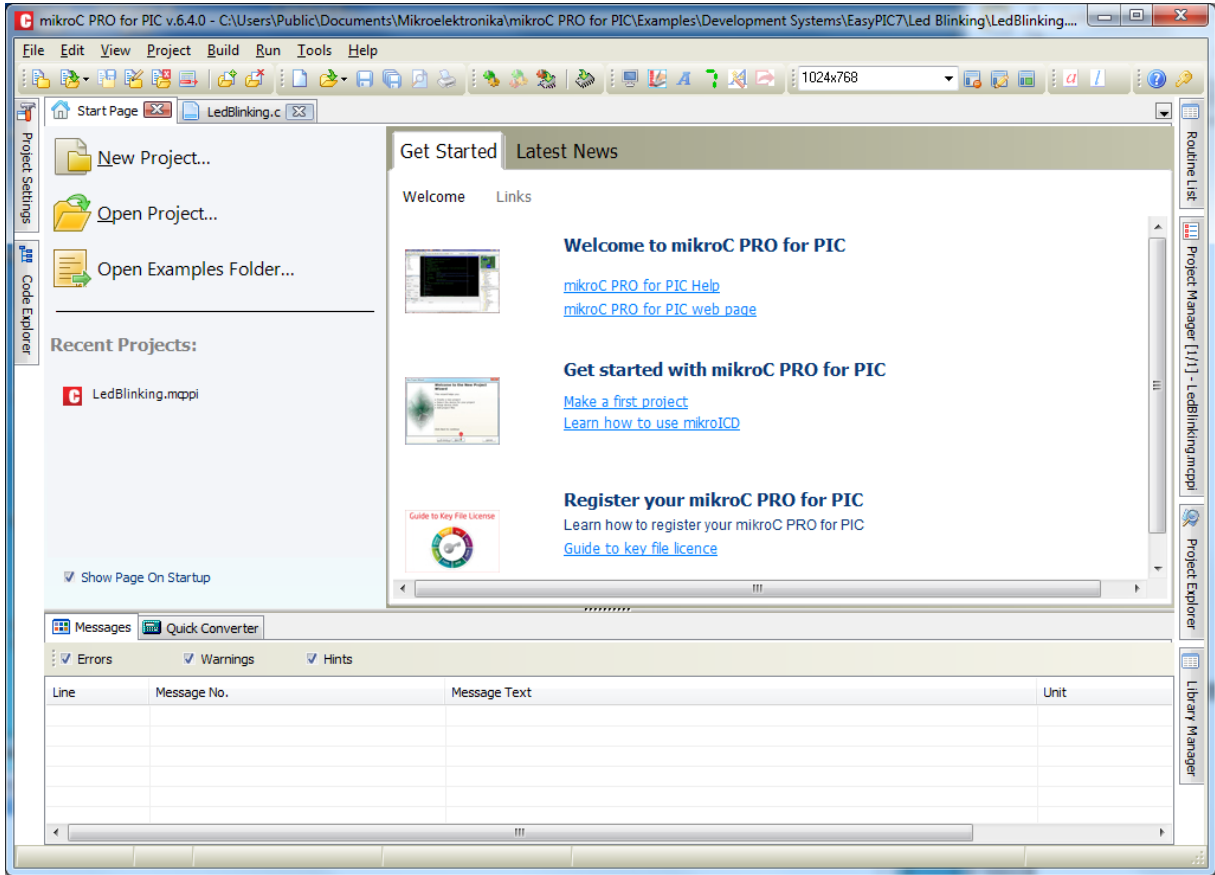


Şekil 3.14 MikroC Kurulum Penceresi 10

Şekil 3.15'te görüldüğü gibi derleyici açıldı. Proje mantığıyla çalışan bir derleyici

MikroC

olması sebebiyle karşımıza gelen pencerede sol tarafta, yeni proje oluşturabileceğimiz veya var olan projelerden birini açabileceğimiz bir bölüm, bu bölüm içerisinde en son açılan projelerin isimlerinin listelendiği bir kısım (Recent Projects), orta kısımda yeni başlayanlar için yardım dosyalarına erişilebilecek bir alan (Get Started) ve üst konumda menüler görünmektedir.



Şekil 3.15 MikroC Derleyici Arayüzü

Başlangıç sayfasının her açılışta karşınıza gelmesini istemezseniz **Show Page On Startup** kutucuğundaki işareti kaldırabilirsiniz.

3.5. MİKROC YAZIM KURALLARI

Her programlama dilinde olduğu gibi MikroC'nin de kendine has yazım kuralları vardır. Bu kuralların dışına çıkıldığında derleyici karşımıza hatalar çıkaracaktır. Bu tarz hatalarla karşılaştıkça, hatanın nereden olduğunu ilerleyen zamanlarda daha kolay tespit edebilecek ve sorunun çözümüne yönelik girişimlerde bulunabileceksiniz. Bu hatalarla

MikroC

karşılaşmamak için yazım kurallarını iyi bilmeniz gerekmektedir.

- C programcılığı kÜÇÜK/BÜYÜK harflere duyarlı iken, MikroC derleyicisi özel komutlar hariç kÜÇÜK/BÜYÜK harflere duyarlı değildir.

Örneğin; şu üç değişken ismi de MikroC için aynıdır: **sicaklik, SICAKLIK, Sicaklik**

- Her komut bitiminde (bazı özel komutlar hariç) noktalı virgül ' ; ' komutu sonlandırmak için kullanılır. (Noktalı virgül aynı zamanda assembly kod bloklarında açıklama satırı olarak kullanılır.)
- Değişkenler veya alt program/fonksiyon tanımlanırken ayrılmış kelimeler değişken ya da fonksiyon adı olarak **kullanılamaz**. Bu ayrılmış kelimeler Tablo 3.1'de verilmiştir.

Tablo 3.1 Değişken yada fonksiyon adı olarak kullanılamayan ayrılmış kelimeler tablosu

__asm	far*	static	catch **
_asm	fdata*	struct	cdecl **
absolute	float	switch	inline **
asm	for	typedef	throw **
at	goto	union	true **
auto	idata*	unsigned	try **
bdata*	if	using	class **
bit	ilevel*	void	explicit **
break	int	volatile	friend **
case	io*	while	mutable **
cdata*	large*	xdata*	namespace **
char	long	ydata*	operator **
code*	ndata*	__automated **	private **
compact*	near*	__cdecl **	protected **
const	org	__export **	public **
continue	pascal	__finally **	template **
data*	pdata*	__import **	this **
default	register	__pascal **	typeid **
delete	return	__stdcall **	typename **
dma*	rx*	__try **	virtual **
do	sbit	__cdecl **	
double	sfr*	__export **	* architecture
else	short	__import **	dependent
enum	signed	__pascal **	
extern	sizeof	__stdcall **	** reserved for
false	small*	bool **	future upgrade

- Program kodları arasına açıklama satırları konulabilir. Bu açıklama satırları programın okunabilirliğini artırmak için kullanılan ve programcı tarafından yazılıp okunabilen ancak derleyici tarafından işleme sokulmayan ifadelerdir. Açıklama satırlarının kullanılmasının amacı programcıya özellikle uzun soluklu projelerde ve kod sayısının çok fazla olduğu programlarda hangi kısımlarda ne iş yaptığını hatırlatmaktır (Şekil 3.16).

○ Eğer açıklama tek satırdan oluşuyorsa açıklamanın başına // işareti yazılır.

MikroC

- o Açıklama birden fazla satırı kapsıyorsa /* */ işaretlerinin arasına açıklama yazılır.

```
/* Program her 1000 ms de bir
   PORTB'nin durumunu değiştirir
   ve PORTB ye bağlı LED'ler her saniye
   yanıp söner */

while(1) {
    LATB = ~LATB;          // PORTB değerini tersle
    Delay_ms(1000);
}
```

Şekil 3.16 Açıklama Satırları

3.6. TANIMLAYICI KELİMELE (TÜR İSİMLERİ)

Değişken, sabit veya fonksiyon tanımlanırken bazı kurallara uyulması gerekir. Bunlara isim verilirken uyulacak kurallar şu şekildedir:

- İngiliz alfabesindeki bütün harfler büyük ve küçük olarak kullanılabilir
- İlk karakterden sonra harfler, rakamlar veya “_” karakteri gelebilir.
- Tür ismi içerisinde boşluk **bırakılamaz**.
- Türkçe karakter **kullanılamaz** (ç, ğ, ı, ö, ş, ü gibi)
- Tür isimleri, MikroC diline ait bir komutun adı **olamaz**.
- Alt çizgi «_» karakteri hariç özel karakterler **kullanılamaz** (% , + , @ gibi)
- Tanımlanan ismin ilk karakteri ya bir harf olmalı ya da alt çizgi karakteri «_» olmalı
- İsimler rakamla **başlayamaz**.

MikroC

Örnekler:

- Sicaklik4 ← Geçerli
- Nem ← Geçerli
- _toplam ← Geçerli
- Ort%deger ← Geçersiz (% işareti kullanılamaz)
- Çevresi ← Geçersiz (Ç harfi kullanılamaz)
- While ← Geçersiz (Anahtar kelimeler kullanılamaz)
- 3RPM ← Geçersiz (Değişken isimleri rakamla başlayamaz)
- Hava basinci ← Geçersiz (boşluk karakteri kullanılamaz)

3.7. TAMSAYILAR

Tamsayılar, hexadecimal (16'lık sayı tabanında), decimal (10'luk sayı tabanında), octal (8'lik sayı tabanında) ve binary (2'lik sayı tabanında) yazılabilir. Hangi tamsayının hangi tabanda yazıldığıнын bir önemi yoktur. MikroC derleyicisi her yazım şeklini kabul etmektedir.

- 0x ile başlayan sabitler hexadecimal sayıları ifade eder. Örneğin; 0x8F.
- Decimal tipi sayılar + veya – işareti ile başlayarak vürgül, boşluk ya da nokta karakteri içermeyen yazılırlar. Önüne + işareti koyulmamış olan sayılar pozitif kabul edilir.
Örnek: 6258
- 0 ile başlayan sayılar octal sayı tabanında kabul edilir. Örnek: 0357
- 0b ile başlayan sayılar ikilik sayı düzeninde (binary) olarak kabul edilir. Örnek: 0b10100101.

Aşağıda birkaç örnek verilmiştir:

0x15	→	Hexadecimal sayı onluk tabanda 21'e eşittir.
125	→	Decimal sayıdır
024	→	Octal sayıdır. Onluk tabanda 20'ye eşittir.
0b101	→	Binary sayıdır. Onluk tabanda 5'e eşittir.

MikroC

3.8. ONDALIKLI SAYILAR

Ondalıkli sayılar aşağıdaki şekillerde yazılabilirler.

0. $\rightarrow$ = 0.0

-1.23 $\rightarrow$ = -1.23

23.45e6 $\rightarrow$ = 23.45×10^6

2e-5 $\rightarrow$ = 2.0×10^{-5}

3E+10 $\rightarrow$ = 3.0×10^{10}

.09E34 $\rightarrow$ = 0.09×10^{34}

3.9. KARAKTERLER

Karakter, tek tırnaklar arasında belirtilen bir ASCII karakter bilgisidir.

Örn: 'A', '2', 'Y'

3.10. YAZILAR (STRING İFADELER)

Yazılar, çift tırnaklar arasına alınmış ASCII karakter dizisini temsil eder. Karakter dizisi boşluk karakteri de içerebilir. Her dizinin en sonunda bir boş (null) karakter vardır ve bu karakter (ASCII 0 karakteri) sonlandırma karakteri olup dizinin toplam boyutuna dahil edilmemektedir.

Aşağıda string ifadelere örnekler verilmiştir.

"merhaba dünya!"	$\rightarrow$	mesaj, 14 karakter uzunluğunda
"bugün hava çok güzel"	$\rightarrow$	mesaj, 20 karakter uzunluğunda
" "	$\rightarrow$	2 boşluk karakteri, 2 karakter uzunluğunda
"C"	$\rightarrow$	harf, 1 karakter uzunluğunda
""	$\rightarrow$	boş (null) yazı, 0 karakter uzunluğunda

MikroC

3.11. KAÇIŞ DÜZENLERİ

Ters eğik çizgi karakteri ' \ ' gerçekte yazılı olarak görünmeyen ancak işlevsel olarak işimize yarayan işlemleri yapmamızı sağlar. Örneğin \n olarak kullanıldığında, bu karakterlerden sonra gelen karakter dizisinin yeni bir satırdan itibaren yazılmasını sağlar. \ karakteri ile birlikte özel anlam taşıyan kaçış düzenleri Tablo 3.2’de verilmiştir.

Tablo 3.2 Kaçış Düzenleri tablosu

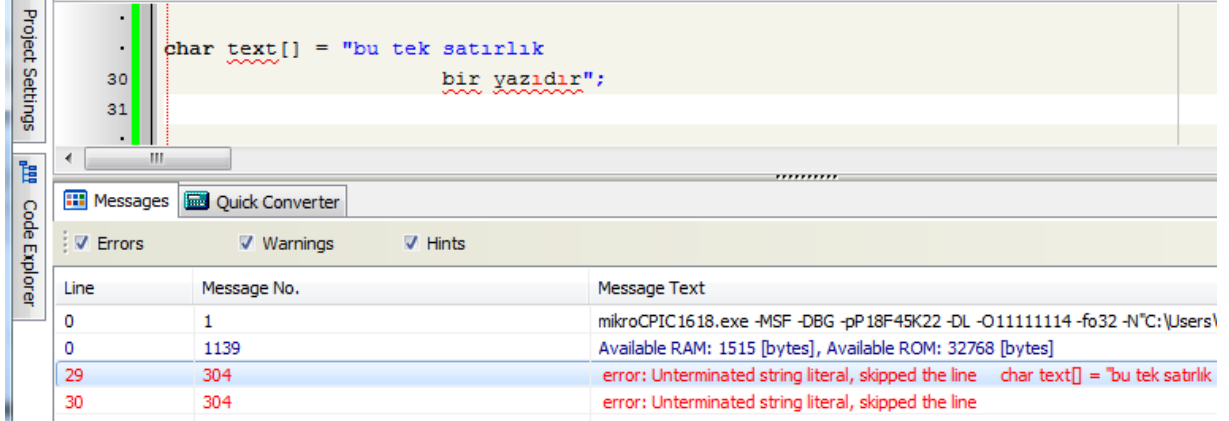
Karakter	Anlamı
\a	Ses üretir(alert)
\b	İmleci bir sola kaydırır (backspace)
\f	Sayfa atlar. Bir sonraki sayfanın başına geçer (formfeed)
\n	Bir alt satıra geçer (newline)
\r	Satır başı yapar (carriage return)
\t	Yatay TAB (Horizontal TAB)
\v	Dikey TAB (vertical TAB)
\"	Çift tırnak karakterini ekrana yazar
\'	Tek tırnak karakterini ekrana yazar
\\	\ karakterini ekrana yazar
\%	% karakterini ekrana yazar

3.12. SATIR DEVAMI VE TERS EĞİK ÇİZGİ KARAKTERİ

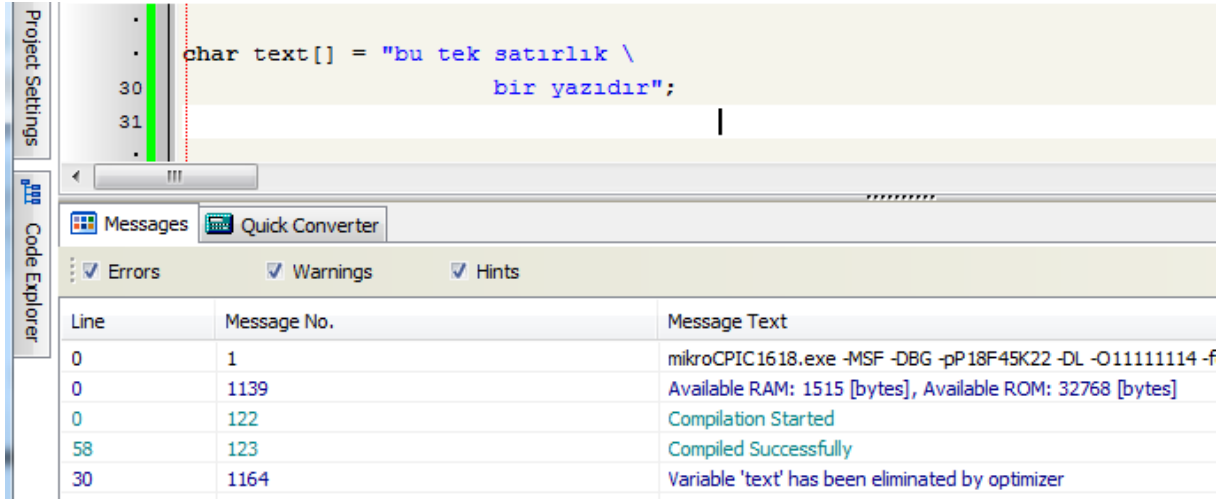
Ters eğik çizgi karakteri satıra sığmayan yazıların devamı için kullanılır. Bu işaret kullanılmadan satır bölünürse MikroC derleyicisi bunu hata olarak algılar.

MikroC

Şekil 3.17 ve Şekil 3.18’de ters eğik çizgi kullanılmadan ve kullanıldığı anki durum farkları görülmektedir.



Şekil 3.17 Ters eğik çizgi kullanılmadan satır bölünmesi



Şekil 3.18 Ters eğik çizgi kullanılarak satır bölünmesi

3.13. AYIRICI İŞARETLER

MikroC’de aşağıdaki ayırıcı işaretler kullanılır.

[] – köşeli parantez	: - iki nokta
() - parantez;	. - nokta
{ } – küme parantezi (süslü parantez)	* - yıldız işareti
, - virgül;	= - eşittir işareti
; - noktalı virgül;	# - diyez işareti

MikroC

3.13.1. KÖŞELİ PARANTEZ ([])

Köşeli parantez, tek ve çok boyutlu dizilerin indekslerini göstermek için kullanılır.

```
char ch, str[] = "mikro";  
int mat[3][4]; /* 3 x 4 matrix */  
ch = str[3]; /* 4th element */
```

3.13.2. PARANTEZ (())

Parantez, grup ifadeleri, koşullu deyimleri yalıtmak ve belirtmek için kullanılır.

```
d = c * (a + b); /* önceliği değiştirmek için kullanılır */  
if (d == z) ++x; /* temel koşul deyimi ile kullanılır */  
func(); /* işlev çağırarak için kullanılır */  
void func2(int n); /* işlev parametreleri tanımlamak için kullanılır */
```

3.13.3. KÜME PARANTEZİ (SÜSLÜ PARANTEZ) ({ })

Küme parantezi, bir kod bloğunun başlangıç ve bitişini belirtir. Küme parantezi açıldıktan sonra kod bloğunun sonuna gelindiğinde mutlaka parantez kapatılmalıdır.

```
if (d == z)  
{  
    ++x;  
    func();  
}
```

3.13.4. VİRGÜL (,)

Virgül, çağrılan fonksiyonlarda parametreler arasına, aynı türden tanımlanan değişkenlerin arasına ya da dizi elemanlarının arasına koyulur.

```
Lcd_Out(1, 1, txt);
```

Parametreler

```
char i, j, k;
```

Dizi elemanları

```
const char AYLAR[12] = (31,28,31,30,31,30,31,31,30,31,30,31);
```

3.13.5. NOKTALI VİRGÜL (;)

Noktalı virgül, komut satırlarını sonlandırmak için kullanılır. Herhangi bir noktalı virgül ile sonlandırılmış ifade (boş olan satırlar da dahil olmak üzere) geçerli bir C komut ifadesi olarak algılanır. Noktalı virgül ayrıca assembly komut satırlarında açıklama satırı olarak

MikroC

kullanılır.

```
a + b;  
++a;
```

3.13.6. İKİ NOKTA (:)

İki nokta, etiket tanımlamalarında kullanılır.

```
start: x = 0;  
...
```

```
goto start;
```

3.13.7. NOKTA (.)

Nokta, bir yapı alanına ulaşmak için kullanılır. Aynı zamanda nokta, MikroC'de kaydedicilerin bitlerine ulaşmak için kullanılır.

```
ogrenci.adi = "Mehmet";  
PORTC.RC5 = 1;
```

3.13.8. YILDIZ İŞARETİ (*)

Yıldız işareti, işaretçileri (pointer) tanımlamak için kullanılır. İşaretçilerle ilgili bilgi ilerleyen konularda verilecektir.

```
char *char_ptr;  
int *sicaklik;
```

3.13.9. EŞİTTİR İŞARETİ (=)

Eşittir işareti, değişkenlere ya da sabitlere bir değer atamak için kullanılır.

```
int test[5] = { 1, 2, 3, 4, 5 };  
int x = 5;
```

3.13.10. DİYEZ İŞARETİ (#)

işareti önışlemci komutlarını tanımlamak için kullanılır ve bu komutlar, program derlenmeden önce MikroC önışlemcisi tarafından işletilir. **#include** ve **#define** en çok kullanılan önışlemci komutlarıdır.

MikroC

```
#include "test.h" // test.h kütüphanesini programa dahil et
#define LED PORTB.F1 // LED ismini kullandığımızda
// PORTB nin 1. biti ile işlem yap
```

3.14. TİP NİTELEYİCİLERİ

3.14.1. CONST

Const, değişkenin değerinin sabit olacağını ve değiştirilemeyeceğini belirtir. Değeri başlangıçta ne ise öyle kalır.

3.14.2. VOLATİLE

Volatile, değişkenin değerinin, kullanıcı müdahalesiyle ya da kullanıcı müdahalesi olmadan değiştirilebileceğini belirtir.

3.15. ASM TANIMLAYICISI

MikroC, program kodları arasında assembly komutlarının kullanımına izin verir. Bu komutlar asm tanımlaması ile örnekteki gibi yapılır:

```
asm {
    // assembly komutları
}
```

3.16. BAŞLANGIÇ DEĞERİ ATAMA

Oluşturulan değişkenlere başlangıçta bir değer atanmak istendiğinde, bu işlem, örnekteki gibi yapılır.

```
int i = 5;
char *s = "merhaba";
struct complex c = {0.1, -0.2};
// 'complex' iki ondalık sayıyı tutan bir yapıdır.
```

MikroC

3.17. MİKROC'DE VERİ TÜRLERİ

3.17.1. ARİTMETİK TÜRLER

Aritmetik türler; *short*, *long*, *signed* ve *unsigned* kelimelerinin önüne geldiği, *void*, *char*, *int*, *float* ve *double* kelimelerini içeren ifadelerdir. Bu ifadeler tamsayı ya da ondalıklı değişkenlerdir.

3.17.1.1. TAMSAYI DEĞİŞKENLER

MikroC'de tamsayı değişkenler aşağıdaki tablodaki gibidir. Bu değişkenlerin alabileceği minimum ve maksimum değerler ile hafızada kapladığı alanlar Tablo 3.3'te verilmiştir. Değişken tanımlanırken parantez içindeki ifadelerin yazılması zorunlu değildir.

Tablo 3.3 Tamsayı Değişkenler tablosu

Tür	Boyut (byte)	Sınırlar
bit	1-bit	0 veya 1
sbit	1-bit	0 veya 1
(unsigned) char	1	0 .. 255
signed char	1	- 128 .. 127
(signed) short (int)	1	- 128 .. 127
unsigned short (int)	1	0 .. 255
(signed) int	2	-32768 .. 32767
unsigned (int)	2	0 .. 65535
(signed) long (int)	4	-2147483648 .. 2147483647
unsigned long (int)	4	0 .. 4294967295

MikroC

3.17.1.2. ONDALIKLI SAYI DEĞİŞKENLER

Float, double veya Long Double isimleri ile tanımlanan değişkenler, MikroC tarafından ondalıklı sayı olarak algılanır. Bu değişkenlere ait boyut ve sınırlar Tablo 3.4'te görüldüğü gibidir.

Tablo 3.4 Ondalıklı Sayı Değişkenler tablosu

Tür	Boyut (byte)	Sınırlar
float	4	$-1.5 * 10^{45} .. +3.4 * 10^{38}$
double	4	$-1.5 * 10^{45} .. +3.4 * 10^{38}$
long double	4	$-1.5 * 10^{45} .. +3.4 * 10^{38}$

3.17.1.3. ENUM

Enum; değişkenin alabileceği değerlerin belirli (sabit) olduğu durumlarda, programın okunurluğunu artırabilmek amacıyla kullanılır. Sondaki değişken adı kullanılmazsa, daha sonra tanımlanması gerekir. MikroC, enum içerisinde tanımlanan değerlerin yerine 0'dan başlayarak karşılık olan sayı değerini aktarır.

```
enum renkler{kirmizi,mavi,sari} renk;
enum bolumler{programcilik, donanim, muhasebe, elektrik};
enum aylar{ocak,subat,mart,nisan,mayis,haziran,temmuz,agustos,eylul};

enum bolumler bolum;
//yukarıda "bolum" değişkeni tanımlanmadığı için şimdi tanımlanıyor
enum aylar ay;
// yukarıda "ay" değişkeni tanımlanmadığı için şimdi tanımlanıyor

//renk değişkeni enum içerisinde tanımlandığı için tekrar tanımlanmadı

bolum = programcilik; /* bolum = 0 anlamında */
ay    = eylul;      /* ay = 8 anlamında */
renk  = mavi; /* renk = 1 anlamında */
```

MikroC

3.17.2. VOID TÜRÜ

Void, herhangi bir değerin olmadığını söyleyen özel bir ifadedir. Özellikle geriye hiçbir değer döndürmeyecek olan fonksiyon tanımlamalarında kullanılır.

```
void EkranaYaz(char deger)
{
    Lcd_Cmd(_LCD_CLEAR);
    Lcd_Out(1,1, "Okunan Sicaklik: ");
    Lcd_Out_CP(deger);
}
```

3.17.3. İŞARETÇİLER

İşaretçiler, mikrodenetleyicinin belleğindeki belli bir adresi gösteren ve gösterdiği bellek adresine erişimi sağlayan değişkenlerdir. İşaretçilerin diğer değişkenlerden tek farkı başka bir değişkenin bellek adresini içeriyor olmasıdır. İşaretçilerin tanımlanması da değişkenlerin tanımlanması ile aynıdır ancak tek fark, işaretçiler tanımlanırken * işareti kullanılır.

```
unsigned int *p; //işaretçi tanımlandı

//...

*p = 5; //işaretçinin adresine 5 sayısı atandı
```

3.17.4. YAPILAR

Yapılar sayesinde, aynı türden veya farklı türlerden oluşan değişkenleri bir araya getirip kendi türlerimizi oluşturabiliriz. Örneğin; iki adet daire tanımlanacak, bu dairelerin yarıçap uzunluğu ile merkez noktasının x ve y koordinatları girilecek. Bunları bir yapı halinde tanımlarsak ilgili veriye ulaşma ve ilgili veriyi gerektiğinde değiştirmemiz çok daha kolay olacaktır.

```
/* x ve y deki koordinat değeri ile bir nokta yapısı tanımlanıyor: */
struct Dot {float x, y};

/* Bir daire yapısı tanımlanıyor: */
struct Circle {
    float r;
    struct Dot center;
} o1, o2;
/* bu yapıdan o1 ve o2 isimli iki adet daire oluşturuluyor */
```

MikroC

Şimdi tanımlanan bu yapıdan yeni daireler oluşturarak bu dairelerin değerlerini atıyoruz.

```
/* Üstte tanımlanan yapı ve değişkenlere göre: */  
  
/* yukarıdaki yapıdan p ve q noktalarını oluştur: */  
struct Dot p = {1., 1.}, q = {3.7, -0.5};  
  
/* o1 dairesini tanımla ve oluştur: */  
struct Circle o1 = {1., {0., 0.}};  
// yarıçapı 1, daire merkezi 0,0 koordinatlarında olan bir daire
```

3.17.5. DİZİLER

Diziler C dilinde (veya diğer dillerde) aynı türdeki birden fazla veriyi bir değişken dizisinin içinde depolamak için en çok kullanılan en basit anlamdaki yapı tipleridir. Dizideki herhangi bir elemana ulaşmak için o dizideki indis numarası belirtilir. Dizideki ilk elemanın indis numarası 0 dır. Dizinin son elemanının indis numarası ise “dizideki eleman sayısı – 1” dir.

Diziyi tanımlamak için dizinin türü ve boyutu belirtilmelidir. Örneğin int tipinde 10 değişkenli bir dizi tanımlanacaksa aşağıdaki gibi tanımlama yapılmalıdır.

```
int Sayilar[10];
```

Bu dizinin sıfıncı, ikinci ve yedinci elemanlarının değerlerini değiştirmek için yazılacak komutlar aşağıdaki gibidir.

```
Sayilar[0] = 5;  
Sayilar[2] = 12;  
Sayilar[7] = 15;
```

Bu işlemlerden sonra dizideki elemanların son durumu aşağıdaki gibi olur.

Sayılar	{...}
[0]	5
[1]	0
[2]	12
[3]	0
[4]	0
[5]	0
[6]	0
[7]	15
[8]	0
[9]	0

MikroC

3.18. FONKSİYONLAR

Fonksiyonlar, özel görevler üstlenen program parçacıklarıdır. MikroC’de uygulamaları geliştirmek için çok sayıda hazır fonksiyon bulunduğu gibi, kendimize özel fonksiyonlar da tanımlayabiliriz. Fonksiyonlar, geriye bir değer göndermediği zaman **Alt Program** adını alırlar.

Fonksiyonlar tanımlanırken fonksiyonun adı, giriş parametreleri, -gerekliyse değişkenler-, geriye dönecek değerin tipi belirtilir.

Örnek: Bir sayının karesini alacak bir fonksiyon yazalım:

```
//Kare isminde bir fonksiyon tanımlanıyor
//Giriş parametresi 1 adet int tipinde sayi isminde bir değişken
long Kare(int sayi)
{
    return sayi * sayi;    //return ile hesaplanan değer geriye döndürülüyor.
}

long sonuc;

void main()
{
    sonuc = Kare(15); // burada oluşturduğumuz fonksiyon çağrılarak
    //15 in karesi hesaplanıyor ve sonuc değişkenine aktarılıyor
}
```

Yukarıdaki örnekte, programın çalışma mantığı şu şekildedir. Derleyiciler program kodlarını her zaman yukarıdan aşağıya doğru derlerler. Program çalışmaya başladığında long tipinde tanımlanan değişken hafızaya alınır. Daha sonra program **void main** kısmından itibaren çalışmaya başlar. Sonuc isimli değişkene bir değer atanacak ancak bu değerın Kare isimli fonksiyondan hesaplanması istendiği için program, Kare isimli fonksiyona dallanır. Giriş değeri olarak da sayi değişkeninin yerine belirttiğimiz 15 değerini koyar. Fonksiyon içinde gerekli hesaplamalar yapıldıktan sonra program return ifadesini gördüğü anda hesaplanan değeri hafızaya alarak fonksiyondan çıkar ve çağrıldığı yere geri döner. Böylelikle hesaplanarak hafızaya alınmış olan değer, sonuc değişkenine aktarılır.

Bu basit örnekte, “ne gerek var bu kadar kod yazmaya” demiş olabilirsiniz. Ancak giriş parametreleri arttıkça ve aynı işlemleri sürekli yapma ihtiyacı duyduğunuzda her seferinde formülleri yazmak yerine bir defa fonksiyonu tanımlayarak sadece giriş değerlerini

MikroC

değiştirmek çok daha mantıklı olacaktır.

Fonksiyonlarda birden fazla giriş parametresi olabilir. Böyle durumlarda tanımlama aşağıdaki örnekteki gibi olur.

Örnek: Dik kenar uzunlukları bilinen bir dik üçgenin, hipotenüs uzunluğunu hesaplayacak bir fonksiyon yazalım.

```
//dikkenar uzunlukları dışarıdan girileceği için iki adet giriş parametresi tanımlandı
long Hipotenus(int dikkenar1, int dikkenar2)
{
    int kareleritoplami;
    kareleritoplami = (dikkenar1 * dikkenar1) + (dikkenar2 * dikkenar2);
    return sqrt(kareleritoplami);    //return ile hesaplanan değer geriye döndürülüyor
}

long sonuc;
void main()
{
    sonuc = Hipotenus(3, 4);
}
```

Yukarıdaki örnekte dik kenar uzunlukları 3 ve 4 birim olan bir dik üçgenin hipotenüs uzunluğunu hesaplayarak sonuç değişkenine aktaran bir fonksiyon örneği verilmiştir.

3.19. OPERATÖRLER

Operatörler, değişken veya sabitler üzerinde belirli işlemlerin yapılmasını sağlarlar. Parantez içerisinde bir işlem yoksa, operatörlerin kendi aralarında öncelik sıraları vardır.

C dilinde kullanılan operatörler şunlardır:

- Aritmetik operatörler
- Karşılaştırma operatörleri
- Atama operatörleri
- Mantıksal operatörler
- Diğer operatörler

MikroC’de operatörlerin 15 öncelik sırası vardır. Aynı kategorideki operatörlerin öncelik sıraları aynıdır. Aynı kategorideki operatörler, şayet aynı satırda işleme girerlerse bunlar arasında da belirli bir öncelik sırası vardır. Bu sıralamalar Tablo 3.5’te verilmiştir.

MikroC

Tablo 3.5 Operatörler ve öncelik sıraları

Öncelik	Operatörler	Aynı satırdaki öncelik
15	() [] . ->	→
14	! ~ ++ -- + - * & (type) sizeof	←
13	* / %	→
12	+ -	→
11	<< >>	→
10	< <= > >=	→
9	== !=	→
8	&	→
7	^	→
6		→
5	&&	→
4		→
3	?:	←
2	= *= /= %= += -= &= ^= = <<= >>=	←
1	,	→

3.19.1. ARİTMETİK OPERATÖRLER

Toplama, çıkarma, çarpma, bölme gibi işlemleri gerçekleştiren operatörlerdir. Bütün aritmetik operatörlerde parantez olmadığı müddetçe işlem sırası soldan sağa doğrudur. Matematikte olduğu gibi çarpma ve bölmenin toplama ve çıkarmaya göre önceliği vardır.

Aritmetik operatörlerin aynı satırda kullanılması durumundaki öncelik sırasına ilişkin aşağıdaki örnek verilmiştir.

```
y=(7+(6+4)*5)*2;  
//bu işlemin sonucu hesaplanırken öncelik sıraları  
//dikkate alındığında işlem aşağıdaki sırada yapılır
```

1. $6 + 4 = 10$;
2. $10 * 5 = 50$;
3. $50 + 7 = 57$;
4. $57 * 2 = 114$;
5. $y=114$;

MikroC

Tablo 3.6 Aritmetik Operatörler

Operatör	İşlem	Öncelik
İkili İşlemler		
+	toplama	12
-	çıkarma	12
*	çarpma	13
/	bölme	13
%	mod alma	13
Tekli İşlemler		
+	Artı işareti olarak kullanılır. Değişkenin ya da işlemin işaretini değiştirmez	14
-	Eksi işareti olarak kullanılır. Değişkenin ya da işlemin işaretini değiştirir.	14
++	Değişkenin değerini 1 artırır	14
--	Değişkenin değerini 1 azaltır.	14

++ ve – operatörlerinin ifadenin sağında veya solunda kullanılması işlemisel açıdan sonuçları değiştirir. Şöyle ki;

Tablo 3.7 Operatörlerin kullanım şekline göre alacağı sonuçlar

Operatör	İşlem
A++	Önce A'yı kullan, sonra A'yı 1 artır
++A	Önce A'yı 1 artır, sonra bu artırılmış değeri kullan
A--	Önce A'yı kullan, sonra A'yı 1 azalt
--A	Önce A'yı 1 azalt, sonra bu azaltılmış değeri kullan

Örnek:

a = 4 ve b = 5 olsun.

1. c = a++ * b

işleminde işlemin gerçekleşme sırası;

MikroC

$$c = a * b = 4 * 5 = 20$$

$$a = a + 1 = 4 + 1 = 5$$

Bu işlemler sonucunda değişkenlerin son değerleri $a = 5$, $b = 5$ ve $c = 20$ olur.

2. $c = ++a - b$

işleminde işlemin gerçekleşme sırası;

$$a = a + 1 = 4 + 1 = 5$$

$$c = a - b = 5 - 5 = 0$$

Bu işlemler sonucunda değişkenlerin son değerleri $a = 5$, $b = 5$ ve $c = 0$ olur.

3. $c = --a + b--$

işleminde işlemin gerçekleşme sırası;

$$a = a - 1 = 4 - 1 = 3$$

$$c = a + b = 3 + 5 = 8$$

$$b = b - 1 = 5 - 1 = 4$$

Bu işlemler sonucunda değişkenlerin son değerleri $a = 3$, $b = 4$ ve $c = 8$ olur.

3.19.2. KARŞILAŞTIRMA OPERATÖRLERİ

Karşılaştırma operatörleri, mantıksal işlemlerde kullanılır. Bu işlemlerin sonucunda sağ taraftan sol tarafa doğru, geriye 1 (doğru – true) veya 0 (yanlış – false) değeri döner.

Tablo 3.8 Karşılaştırma operatörleri

Operatör	İşlem	Öncelik
==	Eşittir	9
!=	Eşit değildir	9
<	Küçüktür	10
>	Büyüktür	10
<=	Küçük eşittir	10
>=	Büyük eşittir	10

MikroC

3.19.3. MANTIKSAL OPERATÖRLER

Mantıksal operatörler, karşılaştırma yaparak, geriye 1 (true) veya 0 (yanlış) olarak değer döndürür. Karşılaştırma sonucu 1 veya 0 dan farklı bir değer olamaz.

Tablo 3.9 Mantıksal Operatörler

Operatör	İşlem	Öncelik
&&	Mantıksal VE işlemi	5
	Mantıksal VEYA işlemi	4
!	Mantıksal DEĞİL işlemi	14

3.19.4. DİĞER OPERATÖRLER

() operatörü, fonksiyon çağırma operatörü olup en yüksek önceliğe sahiptir. [] karakterleri, dizi operatörüdür. ? operatörü de Excel'deki EĞER koşulu gibidir. Kullanım şekli aşağıdaki gibidir.

İşlem1 ? işlem2 : işlem3;

Bu ifadede öncelikle işlem1 e bakılır. Eğer işlem1 doğru ise, işlem2 uygulanır ve işlem3 görmezden gelinir. Ancak işlem1 yanlış ise, bu sefer işlem3 uygulanır ve işlem2 görmezden gelinir.

Bunlardan başka bir de sağa kaydırma >> ve sola kaydırma << operatörleri vardır. Bu operatörler önüne geldiği değişkenin bit değerlerini sola veya sağa kaydırır ve bu operatörün öncelik sırası 11 dir.

3.20. KOŞUL İFADELERİ VE DÖNGÜLER

Programlar içerisinde bazı durumlarda program akışının değişmesi ya da farklı işlemlerin yapılması gerekebilir. Bazen de seçimlerin yapılması ya da belirli işlemlerin belli bir sayıda tekrar edilmesi gerekebilir. Bu gibi işlemleri gerçekleştirmek için koşul ifadeleri ve

MikroC

döngü yapıları kullanılır.

C dilinde, koşula bağlı işlemleri gerçekleştirmek için iki farklı deyim kullanılır. Bunlar;

- **if-else** deyimi
- **Switch-case** deyimi

C dilinde belirli işlerin tekrar tekrar yapılmasını sağlamak için de döngüler kullanılır.

MikroC dilinin desteklediği döngüler şunlardır;

- **For** döngüsü
- **while** döngüsü
- **do while** döngüsü

3.20.1. IF-ELSE DEYİMİ

If-Else deyimi, koşulun sağlanıp sağlanmama durumuna göre işlem yapar. Kullanım şekli aşağıdaki gibidir:

```
if (koşul)
    işlem1 //koşul sağlanıyorsa işlem1 çalıştırılır
else
    işlem2 //koşul sağlanmıyorsa işlem2 çalıştırılır
```

If deyimi tek başına da kullanılabilir. Else'den itibaren olan satırlar yazılmazsa, if deyimi koşulu kontrol eder, koşul sağlanıyorsa alt satırındaki komutu çalıştırır, koşul sağlanmıyorsa if deyimi içinden çıkar ve program akışına devam eder.

Programın belirli yerlerinde iç içe if koşullarını kullanmamız gerekebilir. Aşağıdaki kullanım şekli buna örnektir:

```
if (kosul1) işlem1
else if (kosul2)
    if (kosul3) işlem2
    else işlem3      /* kosul3 sağlanmazsa işlem3 ü çalıştır */
else işlem4          /* kosul2 sağlanmazsa işlem4 ü çalıştır */
```

Yukarıdaki örnekte, **kosul1** sağlanırsa **işlem1** çalıştırılır. **Kosul1** sağlanmazsa, program **kosul2** yi kontrol eder. **Kosul2**'nin sağlanmaması durumunda program **işlem4**'ü çalıştırır. **Kosul2** sağlanırsa, program **kosul3**'ü kontrol eder. **Kosul3** sağlanırsa **işlem2**, sağlanmazsa

MikroC

islem3 çalıştırılır.

NOT: İf deyiminden sonra birden fazla satırlık işlem yapılması isteniyorsa bu kodlar süslü parantezler ({}) arasına yazılmalıdır.

```
if (kosul)
{
    // koşul sağlanırsa buradaki
    // işlemler yapılır
}
else
{
    // koşul sağlanmazsa buradaki
    // işlemler yapılır
}
```

3.20.2. SWITCH-CASE DEYİMİ

Switch – case deyimi, bir sabitin ya da değişkenin bir sabit ile eşleşip eşleşmediğini kontrol eden ve eşlemenin olduğu kısımdaki kodları çalıştıran bir deyimdir. Kullanım şekli aşağıdaki gibidir:

```
switch (ifade)
{
    case sabit1: //ifade sabit1 e eşitse işlemler1'i yap
        işlemler1;
        break;

    case sabit2: //ifade sabit2 ye eşitse işlemler2'yi yap
        işlemler2;
        break;

    default:
        işlemler3; //ifade şartların hiçbirine eşit değilse işlemler3'ü yap
        break;
}
```

Switch deyimi kullanılırken dikkat edilecek kurallar şunlardır:

- case sözcüğünden sonra gelen ifadeler sabit olmak zorundadır.
- case ifadeleri tamsayı, karakter ya da string sabitler olabilir.
- default ve case ifadeleri istenilen sırada yazılabilir.
- Aynı switch bloğu içerisinde birden fazla aynı case ifadesi bulunamaz.
- default kullanmak zorunlu değildir.

MikroC

3.20.3. FOR DÖNGÜSÜ

MikroC’de en çok kullanılan döngüdür. For döngüsünün yazım şekli aşağıdaki gibidir.

```
for (DöngüBaşlangıçDeğeri; DöngüBitişDeğeri; ArtışVeyaAzalışMiktarı)
{
    çalıştırılacak işlemler;
}
```

For döngüsü içerisinde iki adet noktalı virgül (;) ile ayrılmış üç ifade bulunur. Bunlardan bazıları boş olabilir fakat mutlaka noktalı virgül kullanılmalıdır. İlk ifade bir defaya mahsus olmak üzere çalıştırılır. Genellikle ilk değer ataması için kullanılır. İkinci ifade, döngünün kontrol edildiği kısımdır. Buradaki ifade “true” değer ürettiği sürece döngü devam eder. Üçüncü ifade ise genellikle döngü değişkeninin değerinin değiştirildiği kısımdır.

For(;;) ifadesi sonsuz bir döngüdür. for döngüsü de iç içe kullanılabilir.

Örnek:

```
int i;
int x = 10;

void main()
{
    for (i = 0; i < 3; i++)
    {
        x = x * 2;
    }
}
```

Yukarıdaki işlemde, i ve x isimlerinde iki adet tamsayı tipinde değişken tanımlandı ve i değişkeni for döngüsü içinde sayaç olarak kullanıldı. Bu döngünün anlamı şu şekildedir. i’yi 0 (sıfır) değerinden başlat, i’nin değerini her defasında bir artır ve i 3’ten küçük olduğu müddetçe dönmeye devam et. Her dönüşte x değerinin 2 katını al. Böylelikle döngü tamamlanıp döngüden çıkıldığında x değeri 80, i değeri 3 olacaktır.

3.20.4. WHILE DÖNGÜSÜ

While döngüsü, belirtilen koşul sağlandığı müddetçe döngünün devam ettiği bir döngü çeşididir. Bu döngüde eğer koşul sağlanmıyorsa döngü hiç çalıştırılmaz. Döngünün kullanım şekli aşağıdaki gibidir:

MikroC

```
while (koşul)
{
    //çalıştırılacak
    //işlemler
}
```

Örnek:

```
int i = 10;

void main()
{
    while (i < 1000)
    {
        i += 5; //i 1000 den küçük olduğu müddetçe dön ve
        //i nin değerini her döngüde 5 artır
    }
}
```

3.20.5. DO-WHILE DÖNGÜSÜ

Do-while döngüsü de while döngüsü gibi şart sağlandığı müddetçe işlemleri çalıştırmaya devam eder. Aradaki tek fark, do-while döngüsünde ilk kontrolde şart sağlanmasa bile döngü içindeki komutlar en az bir defa çalıştırılır. Kullanım şekli aşağıdaki gibidir:

```
do{

    //çalıştırılacak komutlar

}while(koşul);
```

Örnek:

```
int i = 10;

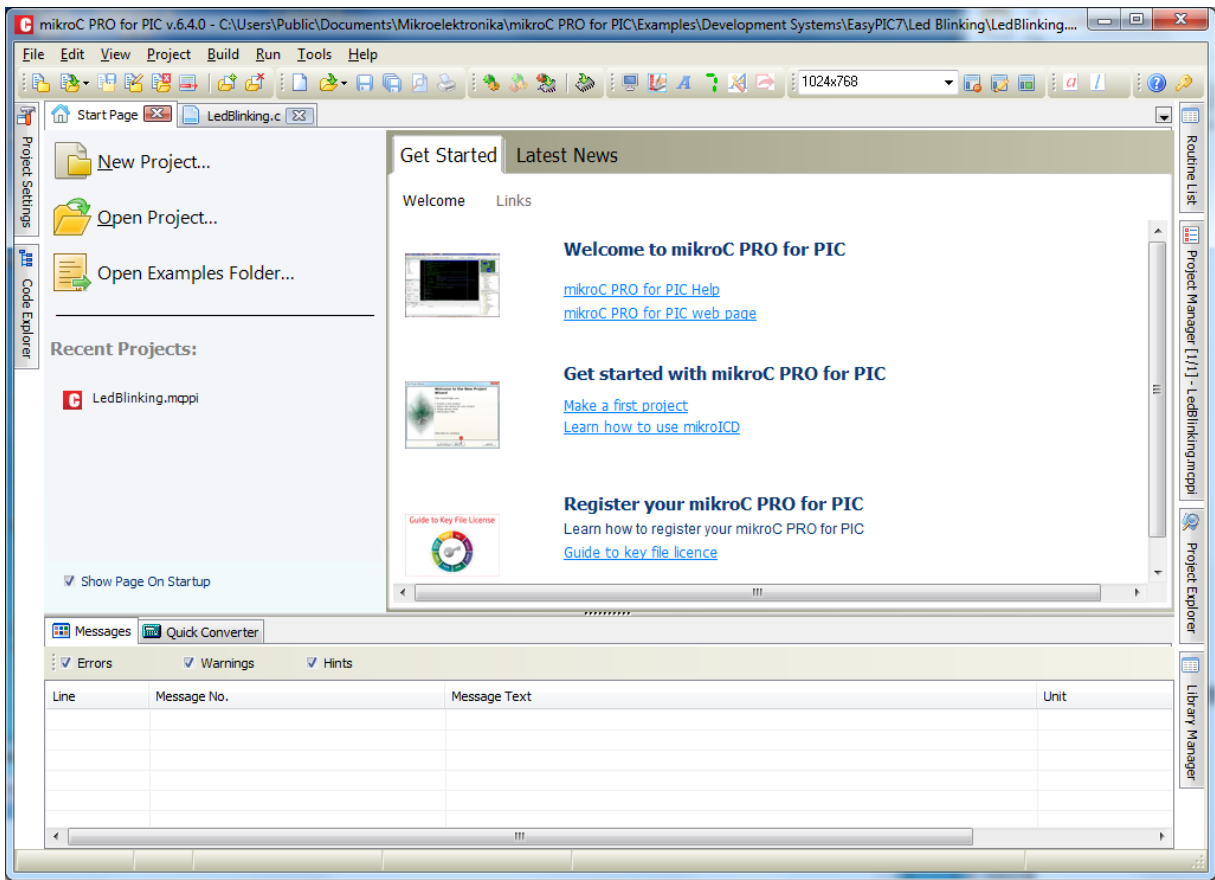
void main()
{
    do
    {
        i += 5; //i 1000 den küçük olduğu müddetçe dön ve
        //i nin değerini her döngüde 5 artır
    } while (i < 1000);
}
```

MicroC DERLEYİCİSİ

BÖLÜM 4

4. MICROC DERLEYİCİSİ

MikroC derleyicisi Mikroelektronika firması tarafından üretilen ve sürekli kendini güncelleyen bir derleyici olmakla birlikte, program yazımlarında ihtiyaç duyabileceğiniz her türlü yardımcı aracı da (7 segment editör, grafik LCD resim düzenleyici, Seri Haberleşme Arabirimi... gibi) bünyesinde barındıran bir derleyicidir.



Şekil 4.1 MikroC Derleyici Arayüzü

MikroC programında da Windows'ta çalışan diğer programlarda olduğu gibi menüler bulunmaktadır. Microsoft Visual Studio'da olduğu gibi MikroC derleyicimiz de proje mantığıyla çalışmaktadır.

MikroC bilgisayarınıza DEMO olarak kurulmaktadır ancak kurduktan sonra tam fonksiyonlu bir şekilde, bütün kütüphanelerini kullanabilirsiniz. Kurulum esnasında gelen

MicroC DERLEYİCİSİ

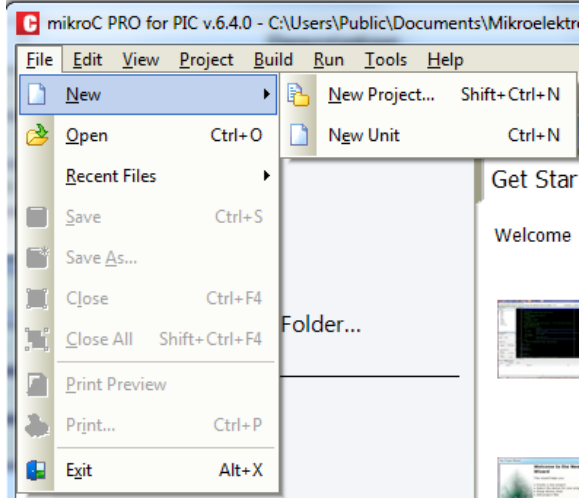
örnekler zaten birer proje mahiyetinde olduğu için, örnekleri inceleyerek kendi oluşturacağınız programlarınız için onları yardımcı/referans olarak kullanabilirsiniz. Programın DEMO modundaki tek kısıtlaması, oluşturacağı HEX dosyasının 2 KB'tan büyük olamayacak olmasıdır. Bu da başlangıç seviyesindeki birçok projenin derlenip mikrodenetleyicide çalıştırılabilmesi anlamına gelir. Eğer, MikroC ile daha profesyonel ve kapsamlı projelerde çalışmayı düşünüyorsanız lisansını satın almanız gerekmektedir. Mikroelektronika firması iki çeşit lisans imkanı sunmaktadır. Birincisi klasik seri numarası ve bilgisayar numarasına bağlı aktivasyon kodu alma yöntemi, ikincisi ise usb dongle seçeneğidir. Aktivasyon keyi mantığında programı sadece bir bilgisayarda çalıştırabilirsiniz, ikinci bir bilgisayar için ikinci lisansın satın alınması gerekmektedir. Gönderilen lisans anahtarı ve dosyası, bilgisayarınız formatlanıncaya kadar geçerlidir. Olası bir format durumunda, yeni aktivasyon kodu talep edilmesi gerekmektedir. İkinci yöntemde ise, firma size bir USB dongle göndermekte ve bu dongle'ın takılı olduğu bilgisayarda MikroC'yi çalıştırabilmektesiniz. Dongle'ın avantajı, hangi bilgisayarda takılı ise o bilgisayarda programı sorunsuzca ve tam versiyon olarak kullanabilmenizdir. Böylelikle olası bilgisayar çökmeleri ve formatlanması durumunda lisansla ilgili sıkıntı yaşamamış olursunuz.

MikroC'nin en üstünde File, New, View, Project, Build, Run, Tools ve Help menüleri vardır. Kısaca bu menülerin ne işe yaradığına göz atalım.

MicroC DERLEYİCİSİ

4.1. MİKROC MENÜLERİ:

4.1.1. FILE MENÜSÜ:



MENÜ SEÇENEĞİ	GÖREVİ
New →New Project	Yeni bir proje oluşturur.
New →New Unit	Proje içinde yeni bir dosya oluşturur.
Open	Daha önceden kaydedilmiş bir projeyi veya dosyayı açmak için kullanılır.
Recent Files	Derleyici tarafından daha önce oluşturduğunuz veya açmış olduğunuz (en son eriştiğiniz) projeleri tekrar açmak için kullanılır.
Save	Kaydet.
Save As	Farklı Kaydet.
Close	Açık olan dosyayı kapat.
Close All	Açık olan projeye ait ne varsa hepsini kapat.
Print Preview	Program kodlarını yazıcıdan çıktı almadan önce önizleme yaptırır.
Exit	Derleyici arayüzünü kapatarak programdan çıkar.

MicroC DERLEYİCİSİ

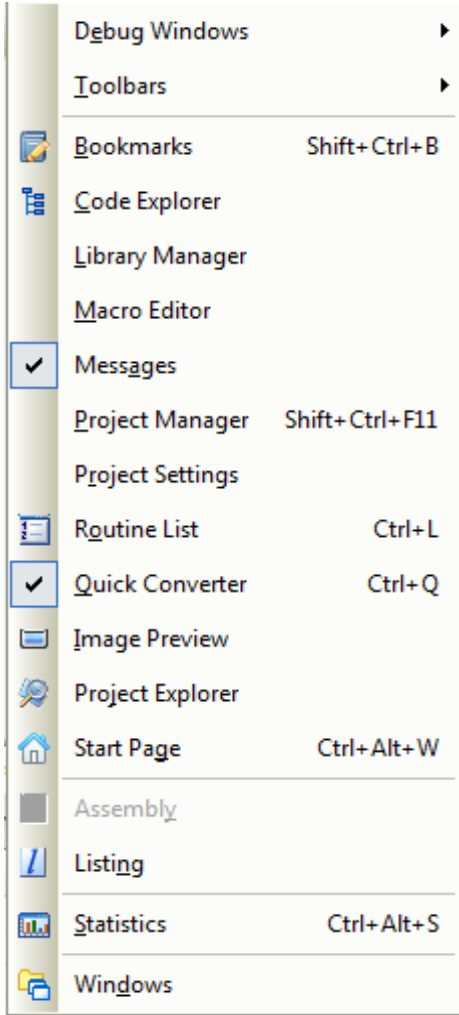
4.1.2. EDIT MENÜSÜ

Edit Menüsü	Görevi
 <u>Undo</u> Ctrl+Z	Yapılan son değişikliği geri alır
 <u>Redo</u> Shift+Ctrl+Z	Yapılan son değişikliği tekrar uygular
 <u>Cut</u> Ctrl+X	Seçilen yazıyı keserek panoya kopyalar
 <u>Copy</u> Ctrl+C	Seçilen yazıyı panoya kopyalar
 <u>Paste</u> Ctrl+V	Panodaki yazıyı yapıştırır
 <u>Delete</u>	Seçilen yazıyı siler
<u>Select All</u> Ctrl+A	Aktif sayfadaki tüm yazıları seçer
 <u>Find...</u> Ctrl+F	Aktif sayfada belirtilen bir yazıyı arar
 <u>Find Next</u> F3	Arama işlemini devam ettirerek sonraki yazıyı bulur
 <u>Find Previous</u> Shift+F3	Arama işlemini devam ettirerek bir önceki yazıyı bulur
 <u>Replace...</u> Ctrl+R	Belirlenen bir yazıyı belirlenen başka bir yazı ile değiştirir
 <u>Find In Files...</u> Alt+F3	Açık olan dosyada / projedeki tüm dosyalarda veya belirlenen bir klasör içindeki tüm dosyalarda belirlenen bir yazıyı arar
 <u>Goto Line...</u> Ctrl+G	Belirlenen bir satıra gider
<u>Advanced</u> ►	Gelişmiş editör seçenekleri

Advanced Menüsü »	Görevi
 <u>Comment</u> Shift+Ctrl+.	Seçilen kodu açıklama satırına dönüştür, eğer seçilen bir kod yoksa iki tane // işareti koy
 <u>Uncomment</u> Shift+Ctrl+,	Seçilen açıklama satırındaki açıklama işaretlerini kaldır
 <u>Indent</u> Shift+Ctrl+I	Seçilen kodu bir adım içe kaydır
 <u>Outdent</u> Shift+Ctrl+U	Seçilen kodu bir adım dışa kaydır
 <u>Lowercase</u> Ctrl+Alt+L	Seçilen yazıyı küçük harfe dönüştür
 <u>Uppercase</u> Ctrl+Alt+U	Seçilen yazıyı büyük harfe dönüştür
 <u>Titlecase</u> Ctrl+Alt+T	Seçilen yazının ilk harfini büyüt

MicroC DERLEYİCİSİ

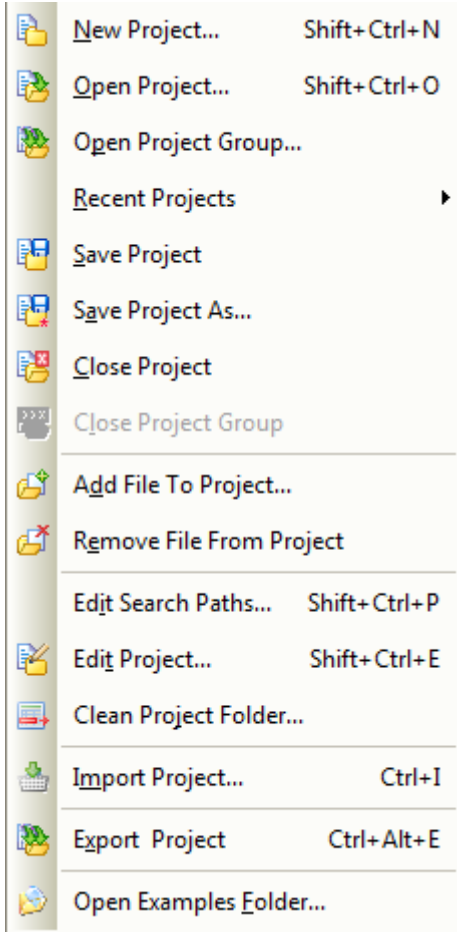
4.1.3. VIEW MENÜSÜ



- **Debug Windows:** Böcek ayıklama ile ilgili alt menüyü açar.
- **Toolbars:** Çeşitli hızlı araç çubuklarını gösterip gizlemek için alt menüyü açar.
- **Bookmarks:** Yer imleri penceresini göster/gizle.
- **Code Explorer:** Kod tarayıcı penceresini göster/gizle.
- **Library Manager:** Kütüphaneyi göster/gizle.
- **Macro Editor:** Makro düzenleyicisini göster/gizle.
- **Messages:** Mesaj penceresini göster/gizle.
- **Project Manager:** Proje yöneticisini göster/gizle.
- **Project Settings:** Proje ayarlarıyla ilgili paneli göster/gizle.
- **Routine List:** Projede kullanılan alt program/fonksiyon penceresini göster/gizle.
- **Quick Converter:** İkili,onlu ve onaltılı sayı düzenleri arasında dönüşümü sağlayan hızlı dönüştürücü penceresini göster/gizle.
- **Image Preview:** Resim önizleyiciyi göster/gizle.
- **Project Explorer:** Proje tarayıcısını göster/gizle.
- **Start Page:** Başlangıçta açılış sayfasını göster/gizle.
- **Assembly:** Assembly kod sayfasını göster/gizle.
- **Listing:** .lst uzantılı dosyadaki kodları göster/gizle.
- **Statistics:** İstatistik penceresini göster/gizle.
- **Windows:** Pencere listesini göster/gizle.

MicroC DERLEYİCİSİ

4.1.4. PROJECT MENÜSÜ



- **New Project:** Yeni bir proje oluşturur.
- **Open Project:** Kayıtlı olan projeyi açar.
- **Open Project Group:** Kayıtlı olan proje grubunu açar.
- **Recent Projects:** Kayıtlı olup en son kullanılmış olan projeleri listeleterek hızlıca açılmasını sağlar.
- **Save Project:** Projeyi kaydeder.
- **Save Project As:** Projeyi farklı bir isimle kaydeder.
- **Close Project:** Projeyi kapatır.
- **Close Project Group:** Proje grubunu kapatır.
- **Add File To Project:** Açık olan projeye bir dosya eklemek için kullanılır.
- **Remove File From Project:** Projeden bir dosyayı kaldırmak/silmek için kullanılır.
- **Edit Search Paths:** Derleyicinin kütüphane ve kaynak dosyalarla ilgili arama klasörlerini düzenleme/ekleme/silme penceresini açar.
- **Edit Project:** Proje ayarlarının yapıldığı pencereyi açar.
- **Clean Project Folder:** Derleyici tarafından oluşturulmuş ve yeniden oluşturulabilecek olan dosyaları seçerek temizleme işlemi yapan pencereyi açar.
- **Import Project:** Eski versiyon MikroC'de yazılmış bir projeyi yeni versiyon (MikroC Pro) derleyiciye uyarlar.
- **Export Project:** Projeyi farklı bir klasöre ayrıyeten kaydeder (daha sonra başka bir proje olarak kullanılmak üzere).
- **Open Examples Folder:** MikroC ile birlikte gelen örneklerin bulunduğu klasörü açar.

MicroC DERLEYİCİSİ

4.1.5. BUILD MENÜSÜ

	B uild	Ctrl+F9
	R ebuild All Sources	Alt+F9
	B uild All Projects	Shift+F9
	S top Build All	Ctrl+F12
	B uild + Program	Ctrl+F11

- **Build:** Program kodlarının derlenerek hex dosyasına (mikrodenetleyiciye yüklenecek) dönüştürülmesini sağlar.
- **Rebuild All Sources:** Projedeki program kodlarıyla birlikte kaynak dosyaların da yeniden derlenmesini sağlar.
- **Build All Projects:** Bir proje grubu ile çalışılıyorsa, gruptaki tüm projelerin yeniden derlenmesini sağlar.
- **Stop Build All:** Proje derlemesini durdurur.
- **Build + Program:** Proje kodlarını derleyerek, donanımın bilgisayara bağlı olması halinde hex dosyasını mikrodenetleyiciye gönderir ve mikrodenetleyiciyi programlar.

4.1.6. RUN MENÜSÜ:

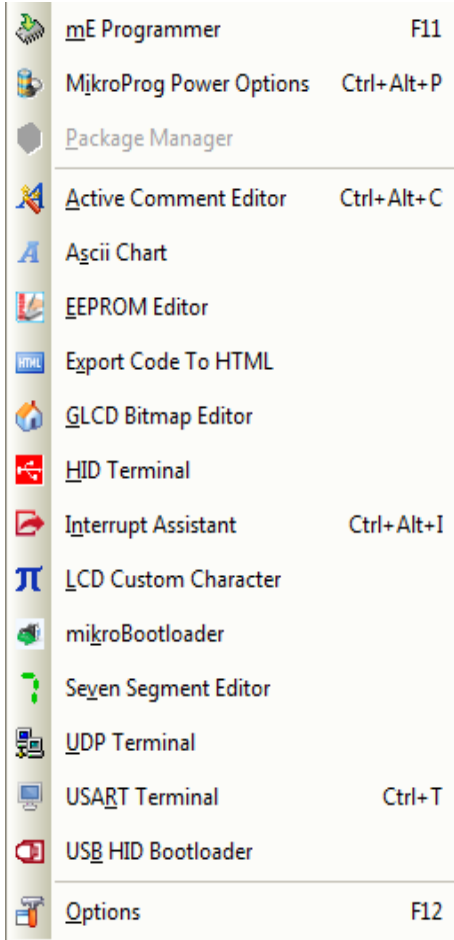
	S tart Debugger	F9
	S top Debugger	Ctrl+F2
	R un/Pause Debugger	F6
	S tep Into	F7
	S tep Over	F8
	S tep Out	Ctrl+F8
	R un To Cursor	F4
	J ump To Interrupt	F2
	T oggle Breakpoint	F5
	C lear Breakpoints	Shift+Ctrl+F5
	D isassembly mode	Alt+D

- **Start Debugger:** Böcek ayıklayıcıyı başlatır.
- **Stop Debugger:** Beöcek ayıklama işlemini durdurur.
- **Run/Pause Debugger:** Böcek ayıklama işlemini duraklatır/devam ettirir.
- **Step Into:** Böcek ayıklama işlemi sırasında alt programların / fonksiyonların içine girerek analiz yapılır
- **Step Over:** Böcek ayıklama işlemi sırasında alt programların / fonksiyonların içine girmeden analiz yapılır
- **Step Out:** Böcek ayıklama sırasında eğer bir alt program veya fonksiyona girilmişse, hızlıca bu kısımdan çıkılarak programın normal akışına dönülür
- **Run To Cursor:** Programda böcek ayıklama sırasında kursorün bulunduğu konuma hızlıca dallanmasını sağlar

MicroC DERLEYİCİSİ

- **Jump To Interrupt:** Programda böcek ayıklama sırasında kesme işlemine dallanır
- **Toggle Breakpoint:** Yer imi aktif/pasif yapılır
- **Clear Breakpoints:** Yer imlerini temizler
- **Disassembly mode:** Kaynak kod ile assembly kodları arasında geçiş yapar

4.1.7. TOOLS MENÜSÜ

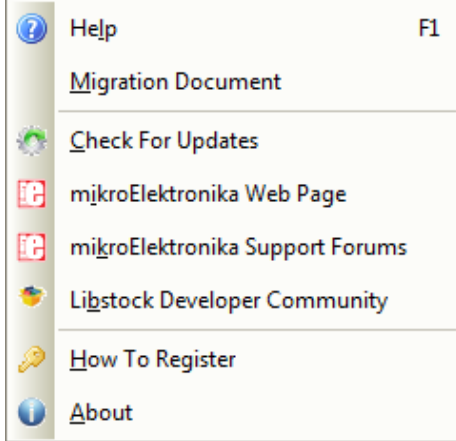


- **mE Programmer:** mikroC ile kurulum sırasında bilgisayara yüklenen mikroProg Suite uygulamasını çalıştırır.
- **mikroProg Power Options:** mikroProg Suite programı için gerekli voltaj ayarının yapılmasını sağlayan pencereyi açar
- **Package Manager:** Mikroelektronika'nın ücretsiz bir programı olan Package Manager'ı çalıştırır. Bu program projede kullanılan mikroElektronika'ya ait veya 3. parti kütüphanelerin paket haline getirilerek dağıtılmasını sağlar.
- **Active Comment Editor:** Projelere aktif açıklama satırları/resimleri eklemek için kullanılır.
- **Ascii Chart:** Programcılar, zaman zaman özel karakterler için ASCII karakter tablosuna ihtiyaç duyarlar. Bu tablo Tools menüsünde mevcuttur.
- **EEPROM Editor:** Mikrodenetleyicinin kalıcı veri hafızasını düzenlemek için kullanılır.
- **Export Code To HTML:** Program kodlarını HTML koduna dönüştürür.
- **GLCD Bitmap Editor:** KS0108, T6963 ve Nokia 3310 tabanlı grafik LCD'ler için özel resim dosyasını derleyicide kullanılabilecek şekilde koda dönüştüren bir araçtır.

MicroC DERLEYİCİSİ

- **HID Terminal:** USB tabanlı projeler için USB den gelen verileri kontrol etme, USB bağlantısı için gerekli olan kodları hazırlama gibi işlemleri yapan araçtır.
- **Interrupt Assistant:** İstenen kesme için program kodu alanında alt programı oluşturur.
- **LCD Custom Character:** LCD ekranda özel karakterler kullanabilmek için özel karakterlerin, program içerisinde kullanılabilecek kodunu üreten araçtır.
- **mikroBootloader:** Bootloader destekli mikrodenetleyicilere USB bağlantısı üzerinden hex dosyasını göndererek programlamayı sağlayan araçtır.
- **Seven Segment Editor:** 7 segment displaylerde istediğimiz karakteri gösterebilmek amacıyla program içerisinde kullanılabilecek kodu üreten araçtır.
- **UDP Terminal:** Network tabanlı projeler için kullanılabilecek yardımcı bir araçtır.
- **USART Terminal:** Seri iletişim (comm port) tabanlı projelerde karşılıklı veri alış verişini test etmek amacıyla kullanılabilecek yardımcı bir araçtır.
- **USB HID Bootloader:** Mikrobootloader'ı çalıştırır.
- **Options:** Kod editörüyle ilgili çeşitli ayarların yapıldığı pencereyi açar.

4.1.8. HELP MENÜSÜ



- **Help:** Derleyicinin yardım dosyasını açar
- **Migration Document:** Eski versiyon MikroC derleyici kodlarını yeni versiyon MikroC Pro kodlarına dönüştürebilmek için gerekli yardımları içeren dökümanı açar.
- **Check For Updates:** Derleyicinin yeni bir versiyonu olup olmadığını online olarak kontrol eder.
- **mikroElektronika Web Page:** Tarayıcıda mikroElektronika firmasının web sayfasını açar.
- **mikroElektronika Support Forums:** mikroElektronika'nın forum destek sayfasını açar.
- **Libstock Developer Community:** Libstock, mikroElektronika kullanıcılarının kendi kodlarını paylaşımına açtığı bir internet ortamıdır. Herhangi bir konuda daha önce yapılmış (GSM ile

MicroC DERLEYİCİSİ

haberleşme, PID kontrol gibi) mikroC, mikroBasic veya mikroPascal örnek projelerinden ve kodlarından faydalanabileceğiniz bir ortamdır.

- **How To Register:** Derleyiciyi satın aldıysanız çevrimiçi veya çevrimdışı olarak aktivasyonu yapabileceğiniz bölümdür.
- **About:** Derleyici ile ilgili versiyon, lisans gibi bilgilerin görüntülediği pencereyi açar.

4.2. MİKROC İLE İLK UYGULAMA

MikroC PRO for PIC, uygulamaları bir veya daha fazla sayıdan oluşan kaynak dosyaları ile birlikte tek bir proje dosyası şeklinde kaydeder (".mcp" uzantılı). Derleyici, birden fazla projeyi aynı anda yönetmenize izin verir. Kaynak dosyalarının derlenebilmesi için onların, proje dosyalarının bir parçası olması gerekmektedir.

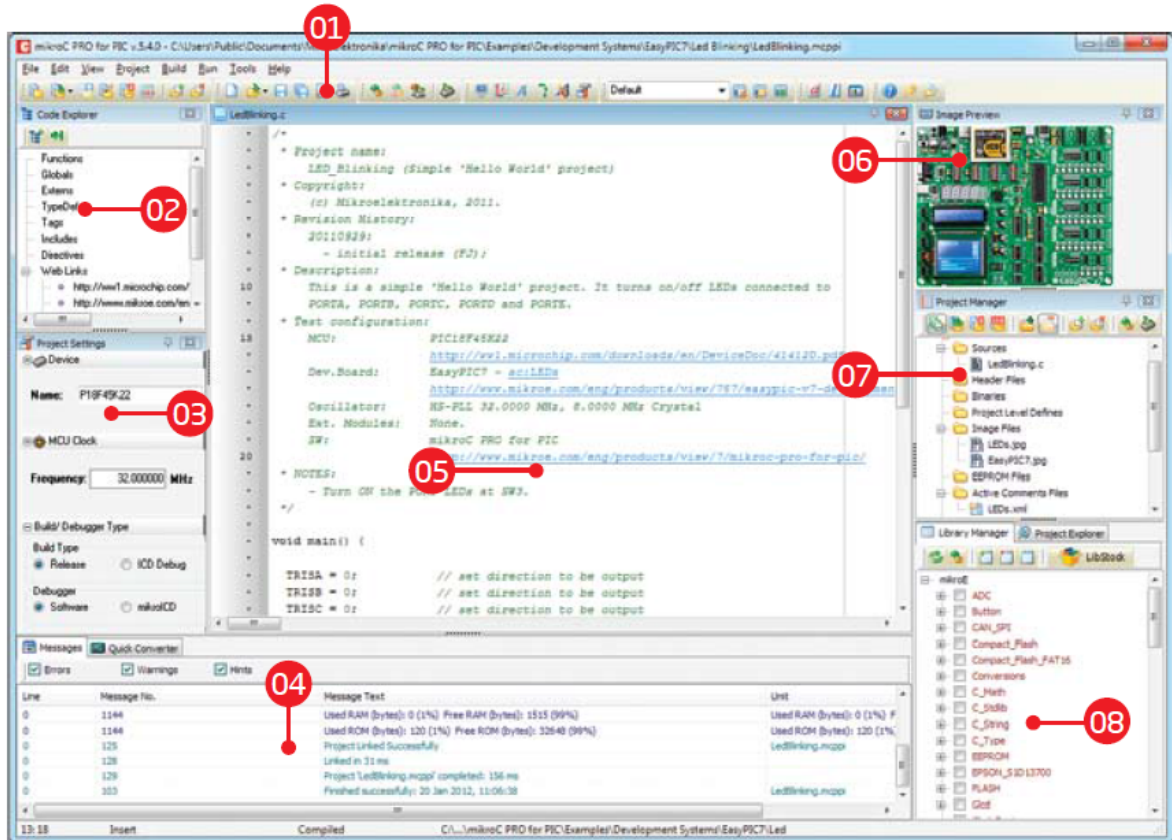
Bir proje dosyası aşağıdakileri içerir:

- Proje adı ve opsiyonel tanımlama
- Kullanılacak Mikrodenetleyicinin modeli
- Saat frekansı
- Proje kaynak dosyalarının listesi
- Binary dosyaları (*.mcl) ve
- Diğer dosyalar

Bu başlık altında yeni bir proje oluşturup, kod yazacak, ardından proje kodlarını derleyip sonuçlarını göreceğiz.

Şekil 4.2’de MikroC derleyicisinin arayüzü görülmektedir.

MicroC DERLEYİCİSİ



Şekil 4.2 MikroC Arayüzü

Bu arayüzdeki bölümler aşağıdaki gibidir:

01 – Araç Çubukları

04 – Mesaj Penceresi

07 – Proje Yöneticisi

02 – Kod tarayıcı penceresi

05 – Kod Düzenleyicisi

08 – Kütüphane Yöneticisi

03 – Proje Ayarları

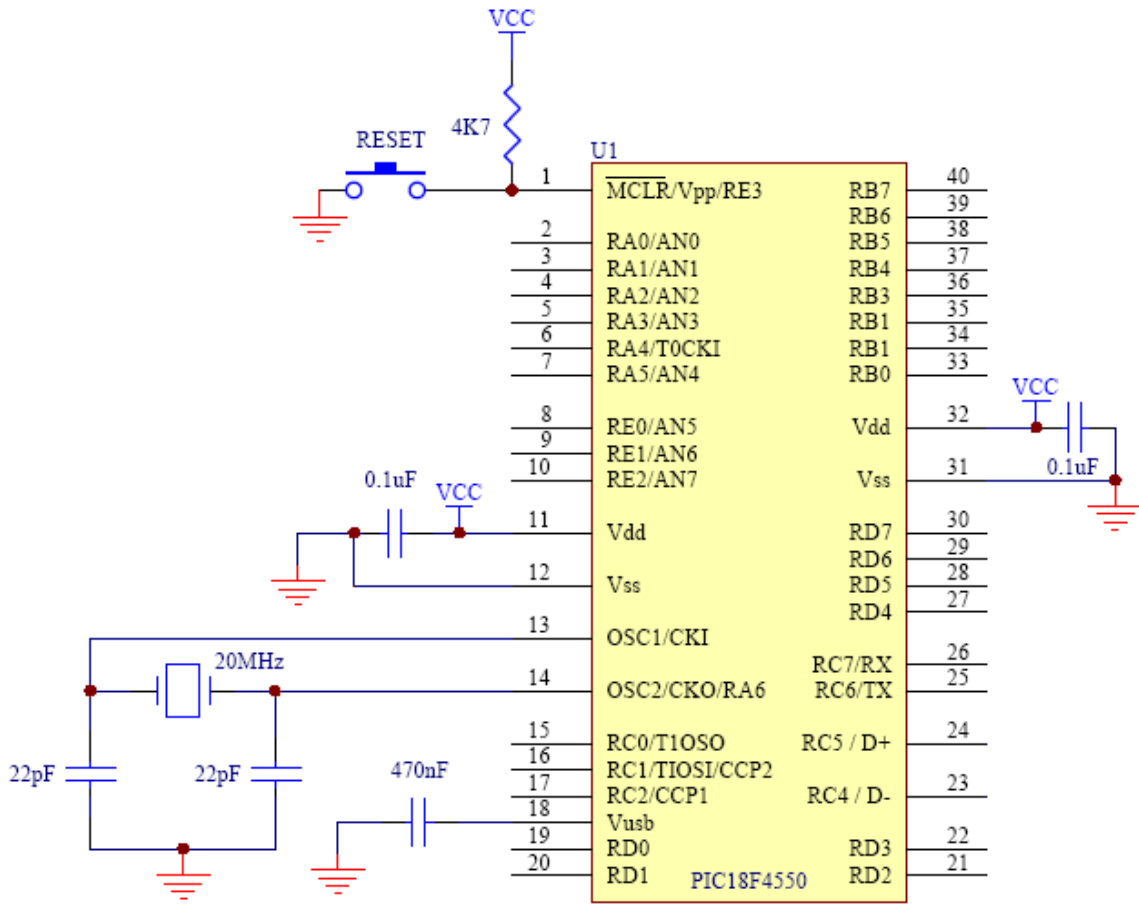
06 – Resim Gösterici

4.2.1. DONANIM ŞEMASI

PIC mikrodeneleyiciler çalışabilmeleri için sürekli bir saat darbesine (clock signal) ihtiyaç duyarlar. Genellikle bu saat darbesi, hassas zamanlama gerektiren uygulamalarda dışarıdan bir kristal bağlanarak sağlanır. 18F4550 8 MHZ'lik ($\pm 2\%$ doğruluk) dahili osilatöre sahiptir. Zamanlamanın hassas olması gerekmeyen durumlarda bu dahili osilatör kullanılabilir. Ayrıca mikrodeneleyicilerin stabil çalışabilmesi için reset (MCLR) bacağına 4.7K veya 10K'lık bir direnç üzerinden 5V uygulanması gerekir. Bu bacağına bir buton yardımıyla GND (0 volt) uygulanırsa PIC resetlenir. Şekil 4.3'te 18F4550'nin çalışabilmesi için en basit

MicroC DERLEYİCİSİ

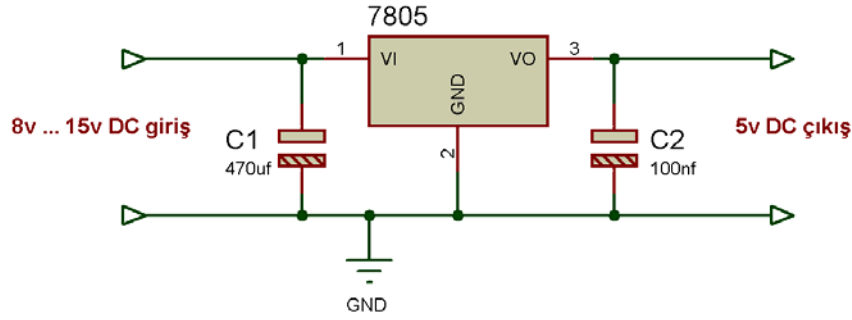
haliyle donanım bağlantıları görülmektedir.



Şekil 4.3 PIC18F4550'nin minimum konfigürasyonlu bağlantı şeması

Vcc besleme gerilimi 5V'tur. 5 Volt'u elde edebilmek için piyasada mevcut bulunan LM78XX serisi (5 volt için LM7805, 12 volt için LM7512 gibi) gerilim regülatörleri veya LM2576 gibi bir gerilim regülatörü kullanılabilir. Biz projelerimizde daha az ek malzeme gerektirmesi sebebiyle LM7805'i kullanacağız. Bu entegrenin girişine 8V – 15V arasında DC gerilim uyguladığımızda çıkış sabit 5V olarak elde edilir. LM7805'in bağlantı şeması da Şekil 4.4'te gösterilmiştir.

MicroC DERLEYİCİSİ



Şekil 4.4 LM7805 bağlantı şeması

Bir cihaz geliştirmek istendiğinde, proje için neler gerekiyor öncelikle onların tespit edilmesi ve buna uygun mikrodnetleyicilerin seçilmesi gerekmektedir. Daha düşük donanımlı bir mikrodnetleyici projemizde işimizi görebilecekken, daha donanımlı bir mikrodnetleyici seçmek, maliyeti yükseltir. Gelişmiş mikrodnetleyiciler, analog dijital dönüştürücüden, seri haberleşme, USB haberleşmesi arabirimlerine kadar birçok modülü bünyesinde barındırmaktadır. Örneğin bir proje yapacağımızı düşünelim. Bu proje için şuna benzer adımlar izlememiz gerekir:

- Devremiz analog ölçüm (sıcaklık, gerilim gibi) yapacak mı? (Analog/Dijital dönüştürücü modülü gerekir)
 - Devrede seri haberleşmeye ihtiyaç duyulacak mı? (UART modülü gerekir)
 - Devre USB bağlantısı kullanacak mı? (USB modülü gerekir)
 - Devrede LCD veya Grafik LCD gibi donanımlar bulunacak mı?
 - Devre kalıcı veri kaydedecek mi? (EEPROM gerekir)
 - Devrede sinyal yakalama, karşılaştırma yapılacak mı? (CCP modülü gerektirir)
- Yapılacaksa kaç adet sinyal girişine ihtiyaç vardır?

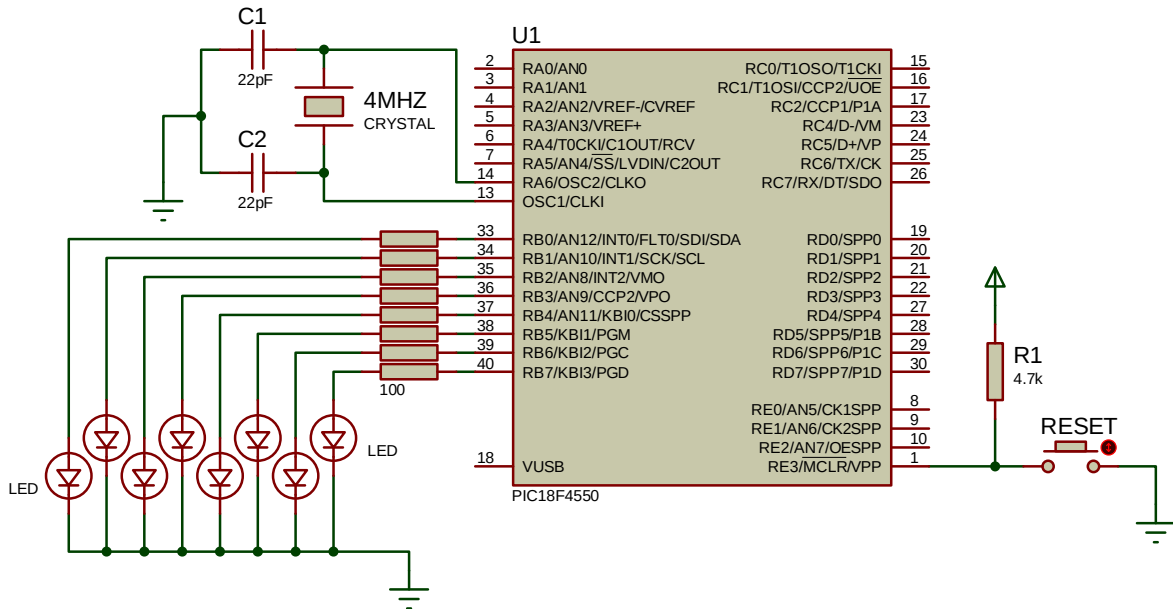
Yukarıdaki özellikler artırılabilir.

Devreler tasarlanırken mikrodnetleyicilerin özel bacakları hariç (seri haberleşme, analog ölçüm gibi) diğer bacakları (besleme bacakları hariç) istediğimiz gibi kullanabiliriz. Yani bir bacağı hem giriş hem de çıkış olarak kullanabiliriz. Giriş olarak kullanmak demek, en basitinden bu bacağı bir buton bağlayarak kullanıcıdan müdahalede bulunmasını beklemek demektir. Bacağı çıkış olarak kullanmak ise, bu çıkışa bir LED veya röle gibi bir çıkış aygıtı

MicroC DERLEYİCİSİ

bağlamak anlamına gelmektedir. Özel bacakları zorda kaldığımızda veya o bacağı kendi görevinde kullanmadığımız durumlarda yine istediğimiz gibi kullanma hakkına sahibiz. Ama bunlara devrenin tasarımı esnasında dikkat etmek gerekir. Sonuç olarak, bir devre tasarlarken giriş çıkış bacakları ve donanım bacak bağlantıları sağlandıktan sonra program kodunuzu buna göre düzenlemeniz gerekmektedir. (Önce yazılımı yapmışsanız, devre bağlantınızı bu sefer yazılıma göre yapmalısınız).

MikroC ile ilk projemizi yapalım. Bu projede mikrodenetleyicideki PORTB'nin 8 bacağına bağlı 8 adet LED'i yakıp söndüren bir devre tasarlayıp gerekli kodları yazalım. Devrenin bağlantı şeması Şekil 4.5'teki gibi olacaktır.



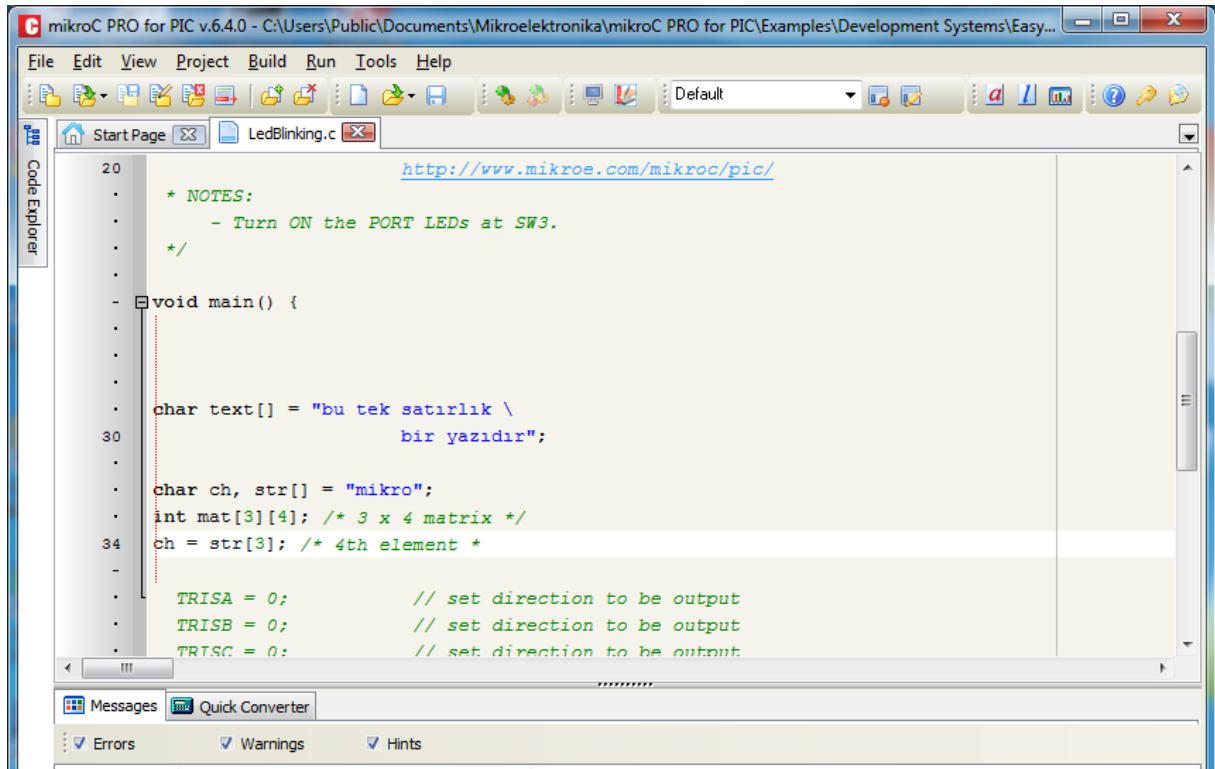
Şekil 4.5 Devre şeması

Devrede PORTB deki 8 adet bacağın her birine 100'er ohm'luk birer direnç ve birer adet LED bağlanılarak LED'lerin birer bacağı şaseye verilmiştir. PIC bacaklarında enerji olduğu zaman enerji değeri her zaman 5 voltur. Yani PIC'ler de 0-1 mantığı gibi dijital olarak çalışır. Bacak gerilimi ya 0 volt, ya da 5 voltur. Bu nedenle, PIC bacaklarında enerji olduğunda LED'lerin üzerinden aşırı akım geçmesini önlemek amacıyla (LED'lerin 1.5v - 2.5v aralığındaki gerilimlerde çalışması sebebiyle) dirençler bağlanmıştır.

MicroC DERLEYİCİSİ

4.2.2. DEVRE YAZILIMININ YAPILMASI

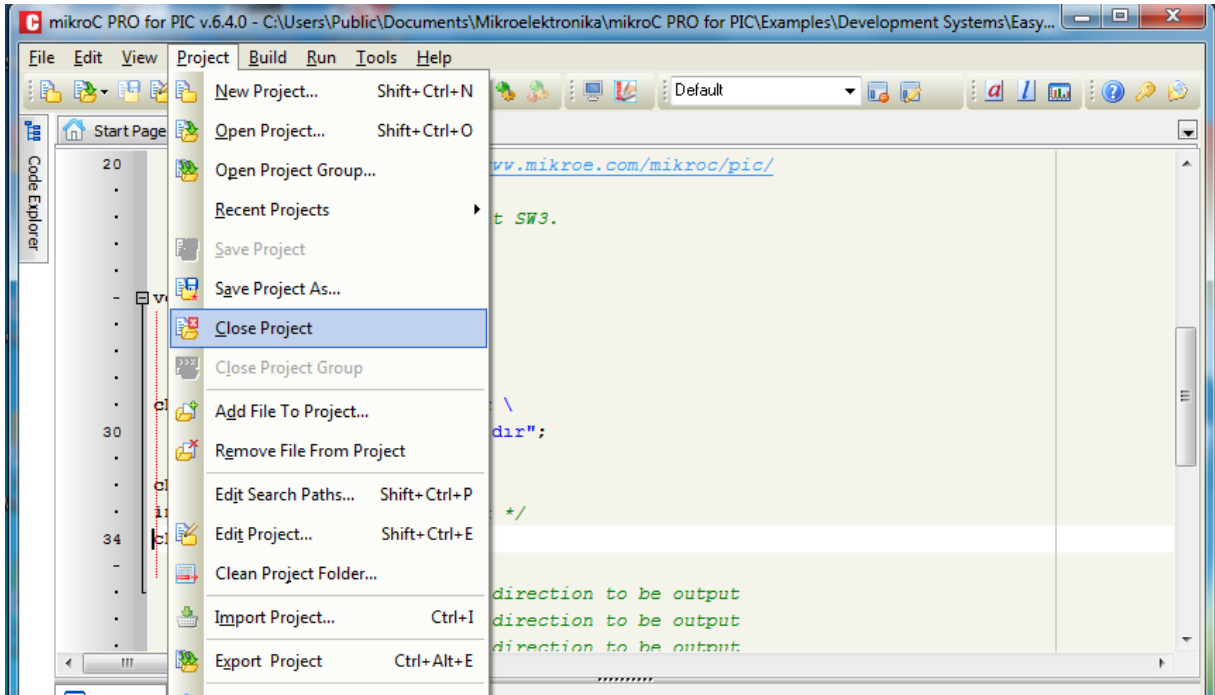
Devrenin yazılımına geçelim. MikroC Pro for PIC derleyicisini çalıştırıyoruz. MikroC her açılışta en son çalışılan projeyi otomatik olarak açmaktadır. Proje karmaşasına girmemek ve proje dosyalarının birbiriyle karışmasını önlemek amacıyla öncelikle Project menüsünden Close Project komutunu verelim. (Programı ilk defa çalıştırıyorsanız örnek proje de açılmış olabilir. Onu da kapatıyoruz)



Şekil 4.6 MikroC'nin ilk çalışması

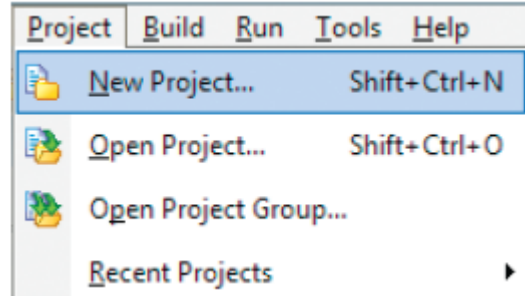
Şekil 4.6'da görüldüğü gibi derleyici, mikroC ile birlikte gelen proje ile birlikte açıldı. Project menüsünden Close Project diyoruz (Şekil 4.7).

MicroC DERLEYİCİSİ

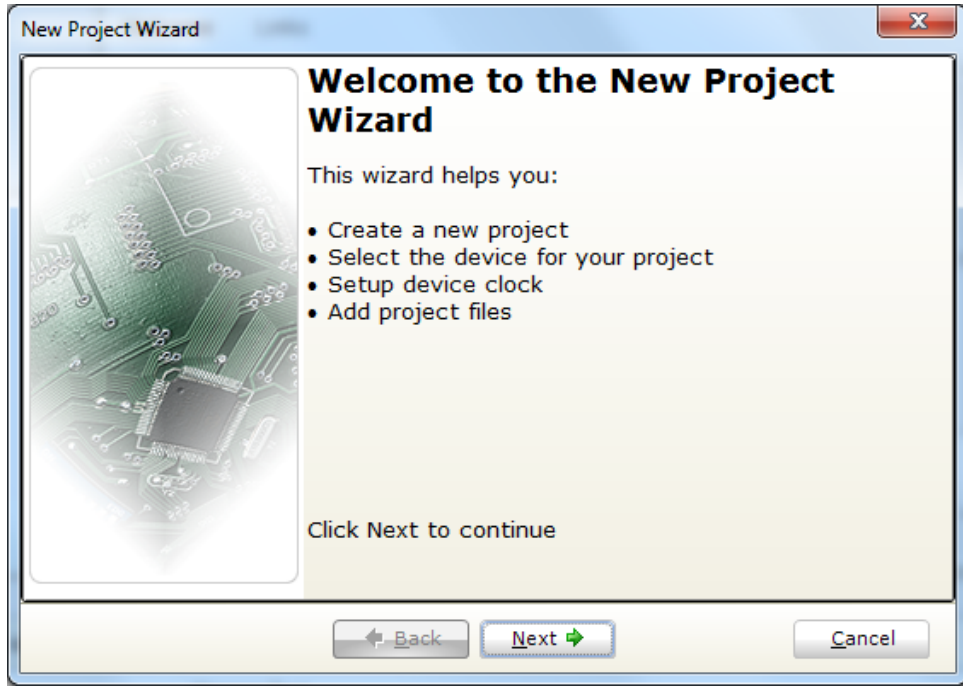


Şekil 4.7 Açılıştaki gelen projenin kapatılması

Şimdi Project menüsünden New Project komutuna tıklıyoruz. Karşımıza, proje oluştururken bize yardımcı olacak bir proje sihirbazı gelecektir (Şekil 4.8).

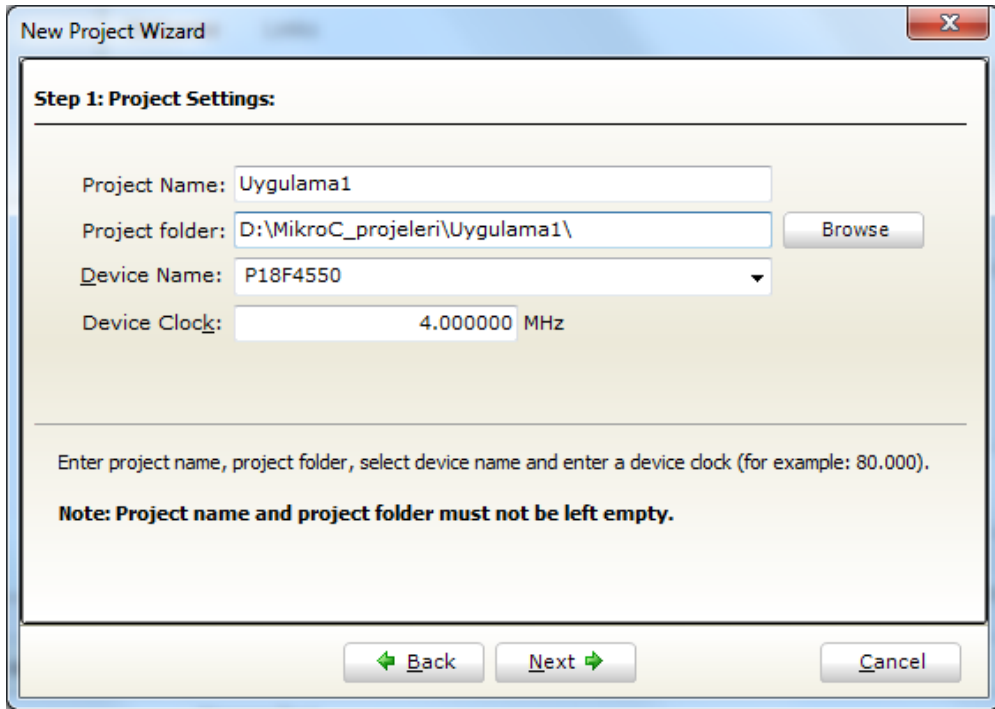


MicroC DERLEYİCİSİ



Şekil 4.8 Yeni bir proje oluşturulması

Next düğmesine tıklayarak devam ediyoruz.



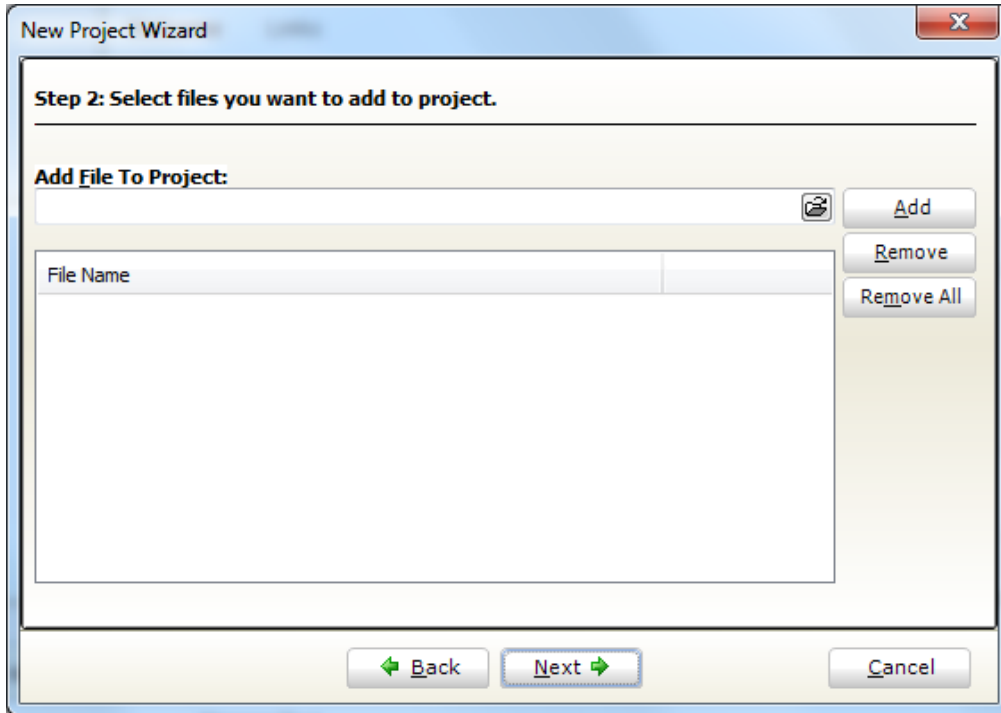
Şekil 4.9 Proje Sihirbazı – Proje ayarları

Karşımıza Şekil 4.9'daki gibi Proje ayarlarını soran bir pencere gelecektir. Bu kısımları kendimize göre uyarlıyoruz.

MicroC DERLEYİCİSİ

- **Project Name:** Projemize bir isim veriyoruz. (İsimde Türkçe karakter ve boşluk kullanmayınız)
- **Project Folder:** Projemizi, kaynak dosyalarıyla beraber hangi klasörün altına kaydetmek istiyorsak Browse düğmesine tıklayarak gerekli konumu seçiyoruz. ***Burada önemli olan, her proje için yeni bir klasör oluşturmaktır. Aksi takdirde birden fazla projeyi aynı klasöre kaydederseniz, proje dosyaları ve kaynak dosyaları birbiriyle karışır ve hangi dosyanın hangi projeye ait olduğunu bulmak zorlaşır.***
- **Device Name:** Projemizde kullanacağımız mikrodenetleyici modeli seçilir. Biz PIC18F4550 kullanıyoruz.
- **Device clock:** PIC'in çalışması istenen frekansı buraya yazıyoruz. Biz harici 4 MHZ'lik bir kristal bağlayacağımız için buraya 4 MHZ yazdık. Buraya yazılan kristal frekansı tek başına yeterli değildir. Donanımsal çarpan PLL kullanmadığınız zaman donanımınıza da ona göre aynı frekanstaki bir kristal bağlamanız gerekir. Örneğin PLL x8 gibi donanımsal çarpan kullanmışsanız buraya yazılan frekansın 8'de biri oranında bir kristal bağlanması gerekir.

Next düğmesine tıklayarak devam ediyoruz.

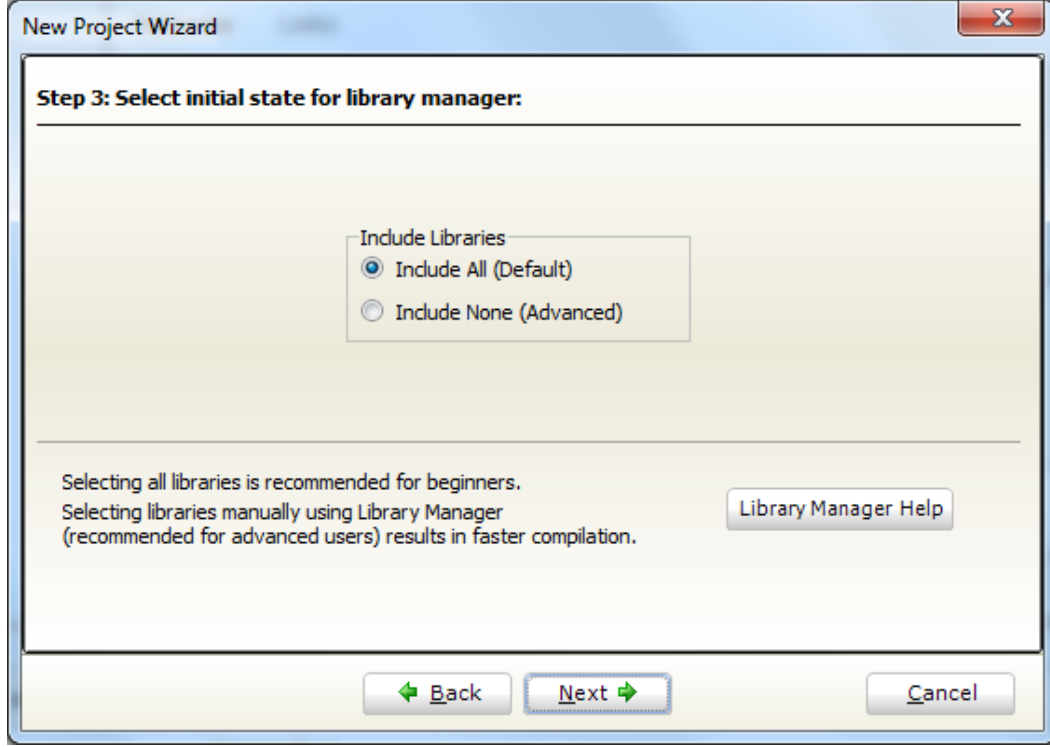


Şekil 4.10 Proje Sihirbazı - ilave kaynak dosyaları

MicroC DERLEYİCİSİ

Projemize dahil etmek istediğimiz dosyalar veya kendimize özel kaynak kodları varsa Şekil 4.10'daki sayfada bunları dahil edebiliriz.

Next düğmesine tıklayarak devam ediyoruz.

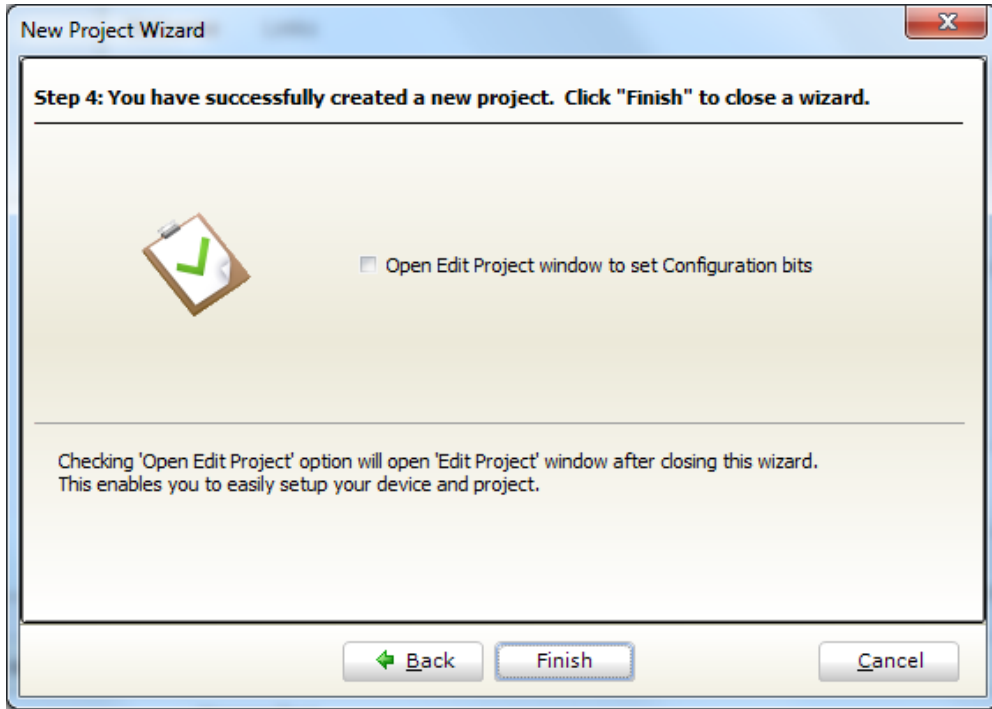


Şekil 4.11 Proje Sihirbazı - Kütüphane seçimi

Kütüphane seçim penceresinde projemizde kullanmak istediğimiz kütüphaneleri seçmemiz gerekiyor (Şekil 4.11). Burada, başlangıç seviyesindeki kullanıcılar için Include All seçeneği önerilmektedir. İlerleyen aşamalarda, kendinizi geliştirdikçe sadece ihtiyaç duyacağınız kütüphaneleri seçebilirsiniz. Burada ilave edip de projenizde kullanmadığınız kütüphaneler, sadece derleme süresini artırmakta, onun haricinde bellekte herhangi bir yer kaplamamaktadır. Yine ihtiyacınız olan fakat seçmediğiniz bir kütüphane olursa, mikroC denetleyicisi o kütüphaneye ait yazacağınız kodu tanımayacak ve hata verecektir. Unutulan kütüphane olursa derleyici arayüzündeki Kütüphane Yöneticisinden (Şekil 4.2) ilgili kütüphane dahil edilebilir.

Next düğmesine tıklayarak devam ediyoruz.

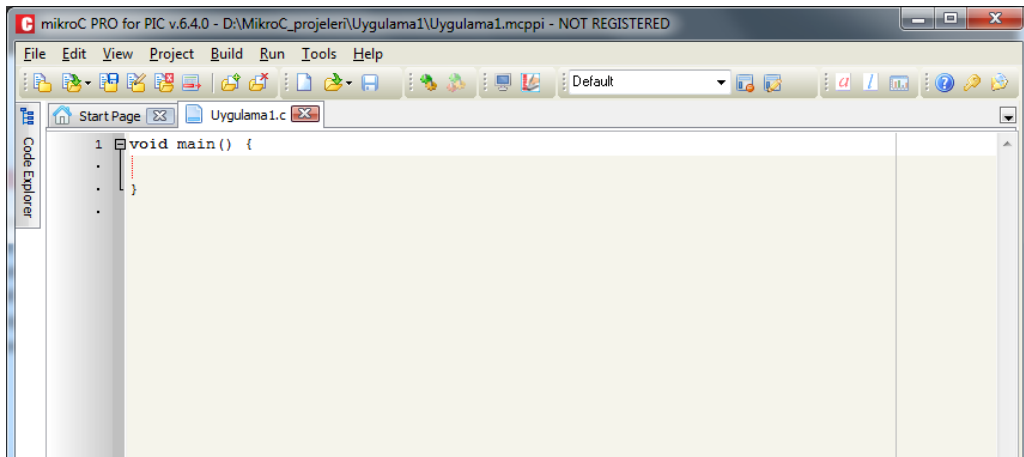
MicroC DERLEYİCİSİ



Şekil 4.12 Proje Sihirbazı – Son Adım

Son adımda, ***Open Edit Project Windows to set Configuration bits*** kısmını işaretleyerek sihirbazı sonlandırırsak, Proje ayarlarının ve mikrodenetleyici ayar bitlerinin ayarlandığı bir pencere karşımıza gelir. Biz işaretlemeden Finish düğmesine tıklayarak sihirbazı sonlandırıyoruz (Şekil 4.12).

Projemiz oluştu ve karşımıza ekranda Şekil 4.13'teki gibi sadece **void main()** yazılı bir pencere geldi.



Şekil 4.13 Proje ekranı

MikroC programlarında kodlar derlenirken ve çalışırken yukarıdan aşağıya doğru

MicroC DERLEYİCİSİ

işlenir ve ana program her zaman void main kısmından başlayarak çalışır. Dolayısıyla değişkenlerimizi, kesme rutinlerimizi, alt program ve fonksiyonlarımızı void main bloğunun üst tarafına yazmalıyız. Böylelikle kodlarımız işlenirken, önce değişken veya fonksiyonlarımızı tanıyacak, daha sonra ana programdan onları çağırınca kullanabilecek. Biz programlarımızı yazarken aşağıdaki mantıkla yazmalıyız ki derleyicimiz hata vermesin.

```
//varsa ilave kütüphaneler
// .
// .
// .

// değişkenler
// .
// .
// .
// .

// kesme alt programları
// .
// .
// .
// .

//alt programlar / fonksiyonlar
// .
// .
// .
// .

void main(){

    //ANA PROGRAM KODLARI BURAYA
    //YAZILACAK

}
```

PORT'deki 8 adet LED'i yakıp söndüren programı artık yazalım. Programları yazarken ilk yapmamız gereken, portlardaki pinlerin giriş olarak mı yoksa çıkış olarak mı kullanılacağını belirtmesi olacaktır. Eğer yazılımda bunu belirtmezsek PIC, bu bacaklarla ilgili nasıl davranacağını bilemez. Projede şuan için kullanmaya ihtiyaç duyduğumuz bir değişken olmadığından kodlarımızı doğrudan void main() bloğunun içine (süslü parantezler arasına) yazıyoruz.

PortB'deki pinlerin tamamının çıkış pini (LED bağlanacak) olarak kullanılmasını sağlayacak komutumuz aşağıdaki gibidir.

MicroC DERLEYİCİSİ

TRISB = 0;

TRISB, PORTB'nin dijital olarak çıkış veya giriş yönlendirmesini yapan yazmaçtır. (Ayrıntılı bilgi için PIC18F4550'nin datasheet'ine bakabilirsiniz.) Aynı mantıkla düşünürsek, TRISA, A portunun, TRISC, C portunun yönlendirme kaydedicisidir.

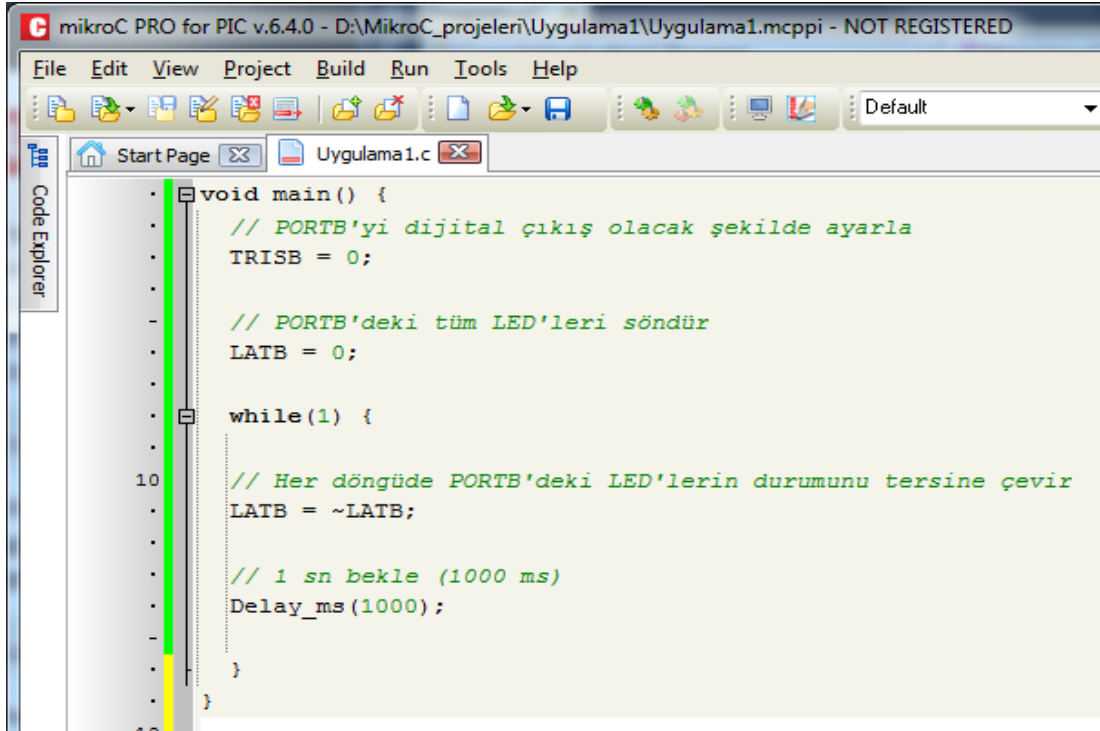
Mikrodenetleyicilerde bir pinin değeri ne ise biz onu değiştirtinceye kadar konumunu korur. Yani değeri 0 volt ise 0 olarak, 5 Volt ise 5 Volt olarak kalır. Mikrodenetleyicinin ilk çalışması anında, biz PORTB'nin bütün pinlerini sıfırlayarak tüm ledlerin sönmük olarak başlamasını sağlıyoruz. Bunu da LATB = 0 komutu ile yapıyoruz.

LATB = 0xFF komutu ile istersek bütün ledleri yakabiliriz. Biz projede ledleri yakıp söndürme işlemini sonsuz bir döngünün içine koymazsak, program ilk çalıştığında, mikrodenetleyiciye bağlı ledler bir kez yanar ve o şekilde kalır. Sürekli yanıp sönmelerini istediğimiz için bir while döngüsü kurmamız gerekmektedir. PORTB'deki pinlerin durumunu da her seferinde mantıksal olarak DEĞİL'leyerek terslersek istediğimizi yapmış oluruz. Bunu da LATB = ~LATB komutu ile sağlarız.

Son olarak her yanıp sönmeye arasında 1 sn kadar bir bekleme süresi koymak istersek, bunu Delay_ms(1000) komutu ile yapabiliriz. 1 sn yerine 500 ms isteseydik Delay_ms(500) şeklinde yazmamız gerekirdi.

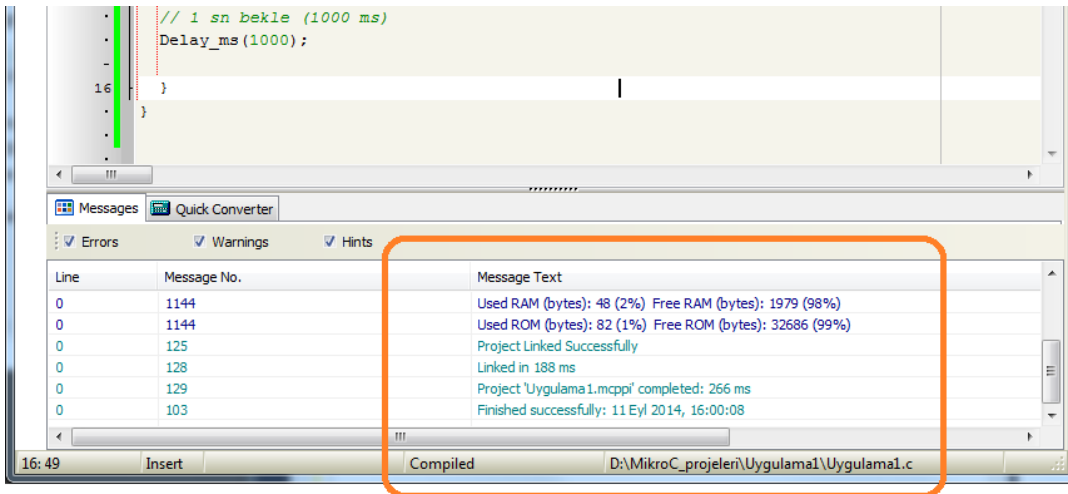
Bu aşamadan sonra proje kodlarımızın son hali Şekil 4.14'teki gibi olacaktır.

MicroC DERLEYİCİSİ



Şekil 4.14 Proje kodları

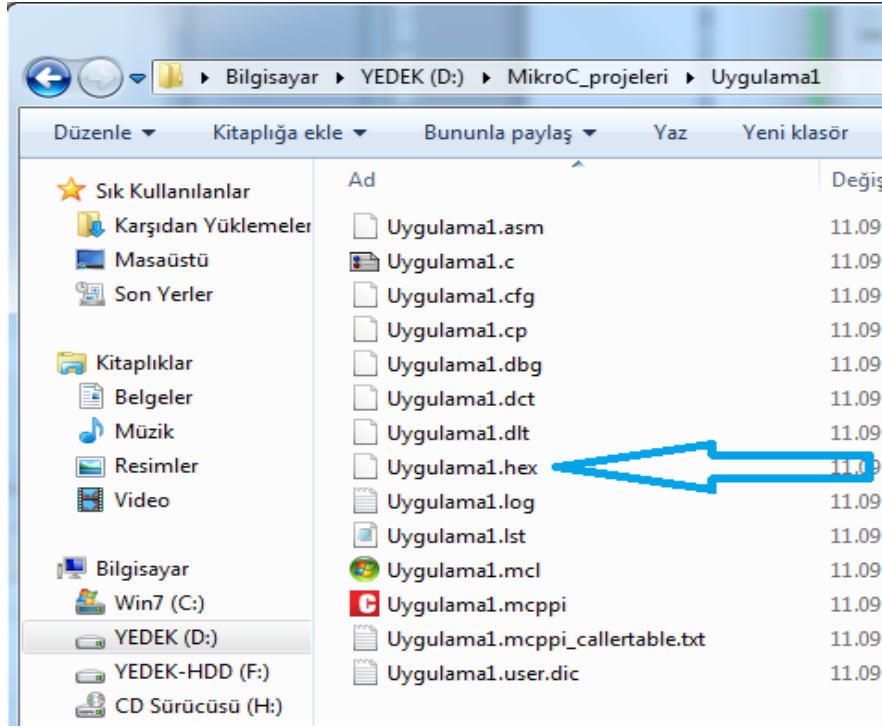
Kodlarımızı yazdıktan sonra bunu mikrodnetleyicide çalıştırabilmemiz için hex dosyası haline çevirmemiz gerekir. Bu işleme derleme denir. Kodlar derlendikten sonra mikrodnetleyici içerisine hex dosyası yazılır. Derleme işlemi için Build menüsünden Build komutu verilebilir ya da klavyeden Ctrl + F9 kısayol tuşlarına basılabilir. Eğer kodumuzda bir hata yoksa, derleyicimiz kodumuzu başarılı bir şekilde derleyip bize mesaj verecektir (Şekil 4.15).



Şekil 4.15 Projenin derlenmesi

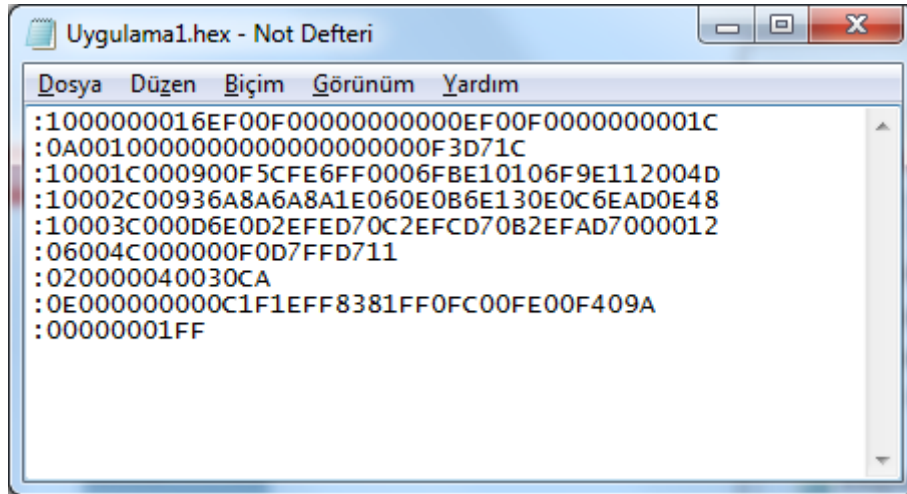
MicroC DERLEYİCİSİ

Oluşturulan .hex uzantılı dosya proje klasörünün içindedir (Şekil 4.16).



Şekil 4.16 Oluşturulan hex dosyası

Oluşturulan hex dosyasını not defteri ile açıp içine bakıyoruz (Şekil 4.17).



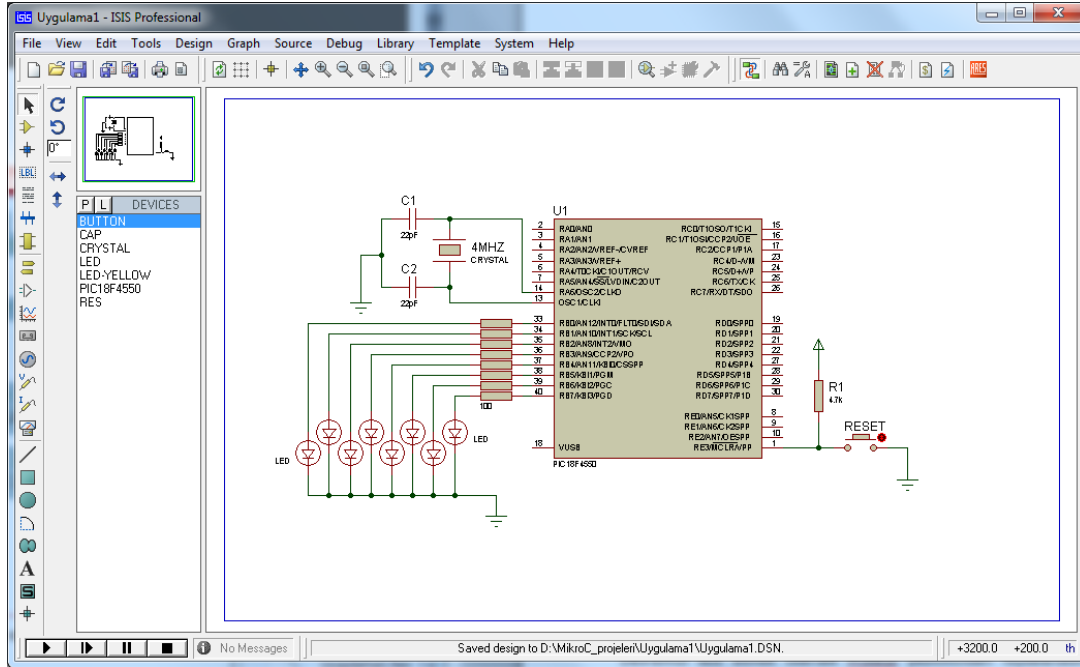
Şekil 4.17 Hex dosyası içeriği

Görüleceği üzere, hex dosyası, Mikrodenetleyicinin algılayabileceği makine kodlarından oluşmaktadır.

MicroC DERLEYİCİSİ

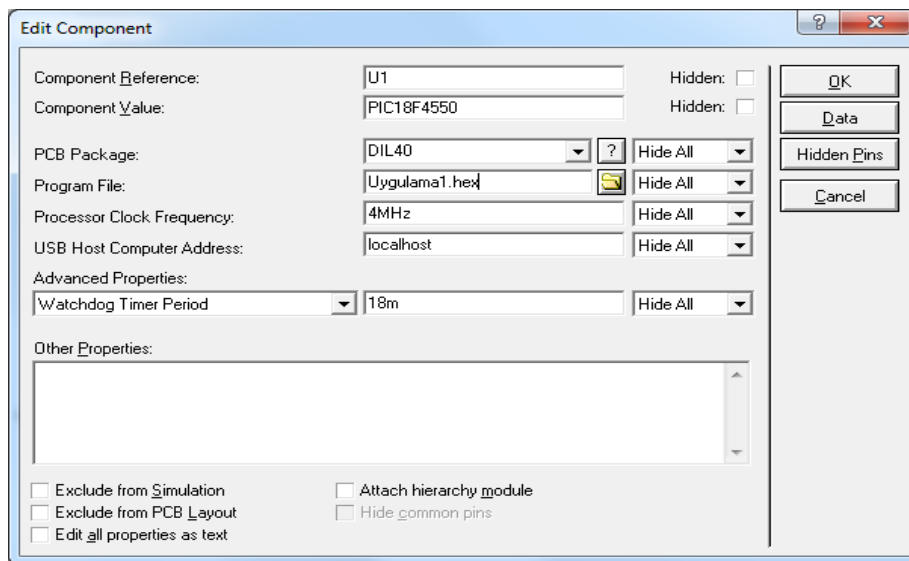
4.2.3. DEVRENİN ÇALIŞTIRILMASI

Devremizi test etmek istersek Proteus yazılımından faydalanabiliriz. Bu amaçla Proteus'ta Şekil 4.18'deki devre çizilmiştir.



Şekil 4.18 Proteus'ta devrenin kurulması

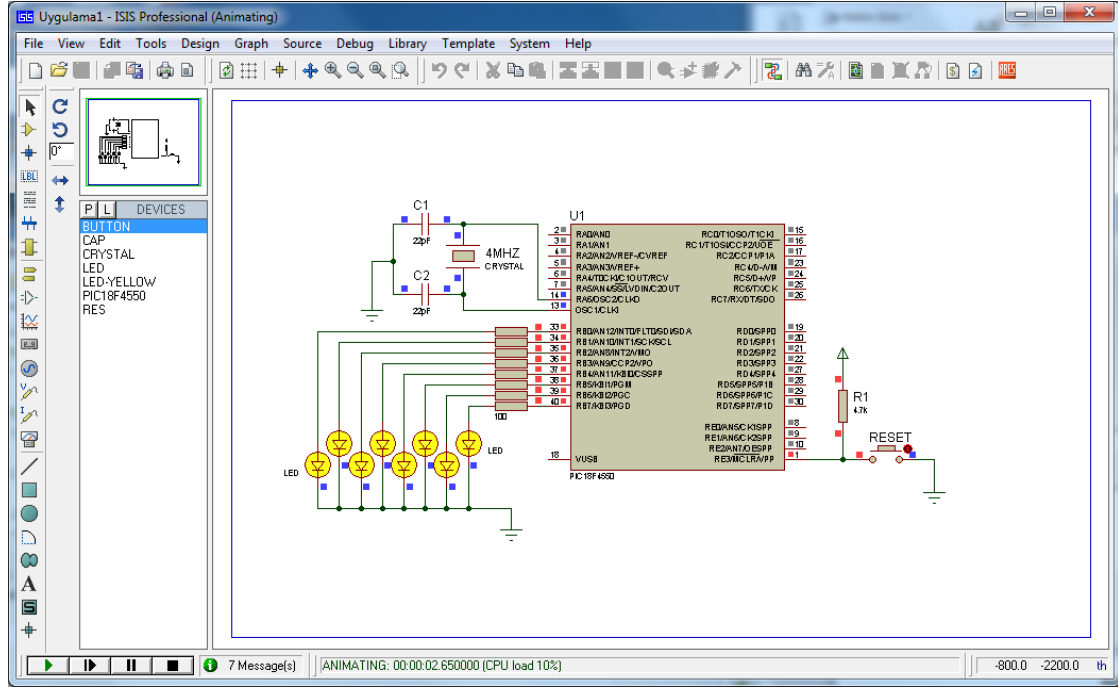
Proteus'ta hex dosyasını mikrodenetleyiciye yükleyebilmek için mikrodenetleyici üzerine çift tıklanır. Açılan pencerede Program File kısmından dosya seçilir ve ardından OK düğmesine tıklanır (Şekil 4.19).



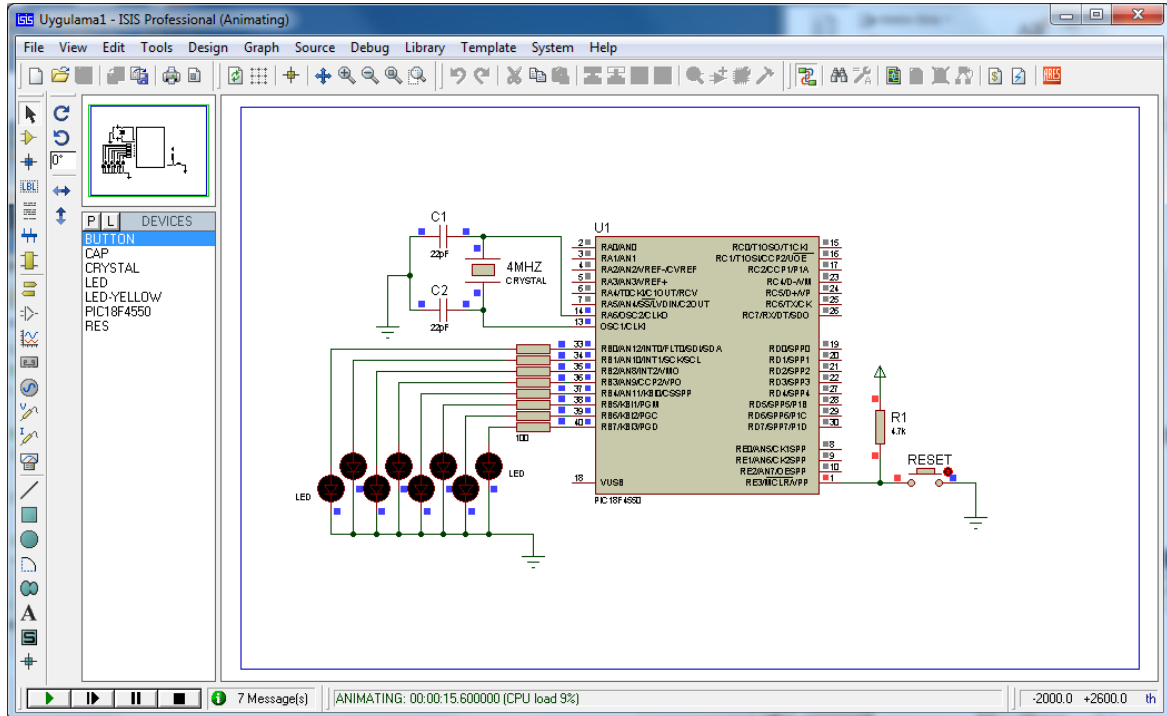
Şekil 4.19 Proteus'ta hex dosyasının yüklenmesi

MicroC DERLEYİCİSİ

Programı çalıştırdığımızda birer saniye aralıklarla LED'lerin yanıp sönüğünü görebiliriz (Şekil 4.20 – Şekil 4.21).



Şekil 4.20 LED'ler yanıyor



Şekil 4.21 LED'ler sönük durumda

Proteus yazılımı, birçok konuda devre ve elektronik malzeme masrafına girmeden

MicroC DERLEYİCİSİ

devre simülasyonlarını yapabilmemizi sağlamakla birlikte, gerçek hayatta uygulamamız gereken durumlarda maalesef yetersiz kalmaktadır. Örneğin Proteus'ta düzgün çalışan bir PIC yazılımı, gerçek board üzerinde denendiğinde çalışmayabilmekte veya daha farklı şekilde çalışabilmektedir. Bu nedenle, imkanlar doğrultusunda en güzeli gerçek board üzerinde denemeleri yapmaktır. Bir sonraki adımda, hex dosyasının gerçek ortamda mikrodnetleyiciye nasıl atılabileceği anlatılmıştır.

4.2.4. MİKRODNETLEYİCİNİN PROGRAMLANMASI

Hex dosyasının mikrodnetleyiciye yüklenebilmesi için özel bir donanım gerekmektedir. Bu özel donanımlar, microchip firması tarafından veya piyasada bazı firmalar tarafından tasarlanmakta ve satılmaktadır. Bu amaçla Microchip firmasının PICKIT2 veya PICKIT3 donanımlarından biri tercih edilebilir. Şekil 4.22'de Microchip'in PICKIT2 ve PICKIT3 donanımları görülmektedir.



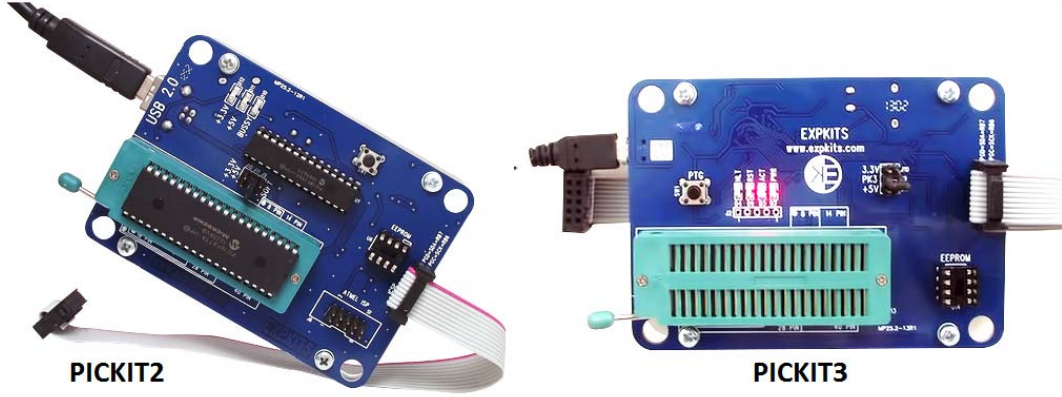
Şekil 4.22 Microchip PICKIT2 ve PICKIT3

PICKIT2 ve PICKIT3, programlama ve debug yapma özelliklerine sahiptir. ICSP üzerinden tüm PIC'leri programlayabilir. Eeprom programlayabilme ve Keeloq gibi bir takım programlanabilir entegreleri de programlayabilme kabiliyetine sahiptir. USB üzerinden ek besleme ihtiyacı duymadan çalışırlar. Laptop ya da masaüstü bilgisayarlar için ideal programlayıcılarıdır. PICKIT3'ün PICKIT2'ye göre farkı yeni nesil 32 bit bazı PIC32XXX serisi mikrodnetleyicileri desteklemesidir.

Microchip PICKIT2 ve PICKIT3 programlayıcıların kopya versiyonları da piyasada

MicroC DERLEYİCİSİ

mevcuttur. Buna örnek olarak Expkits firmasının Pickit2 ve Pickit3 programlayıcıları verilebilir. Şekil 4.23'te Expkits firmasının programlayıcıları görülmektedir.



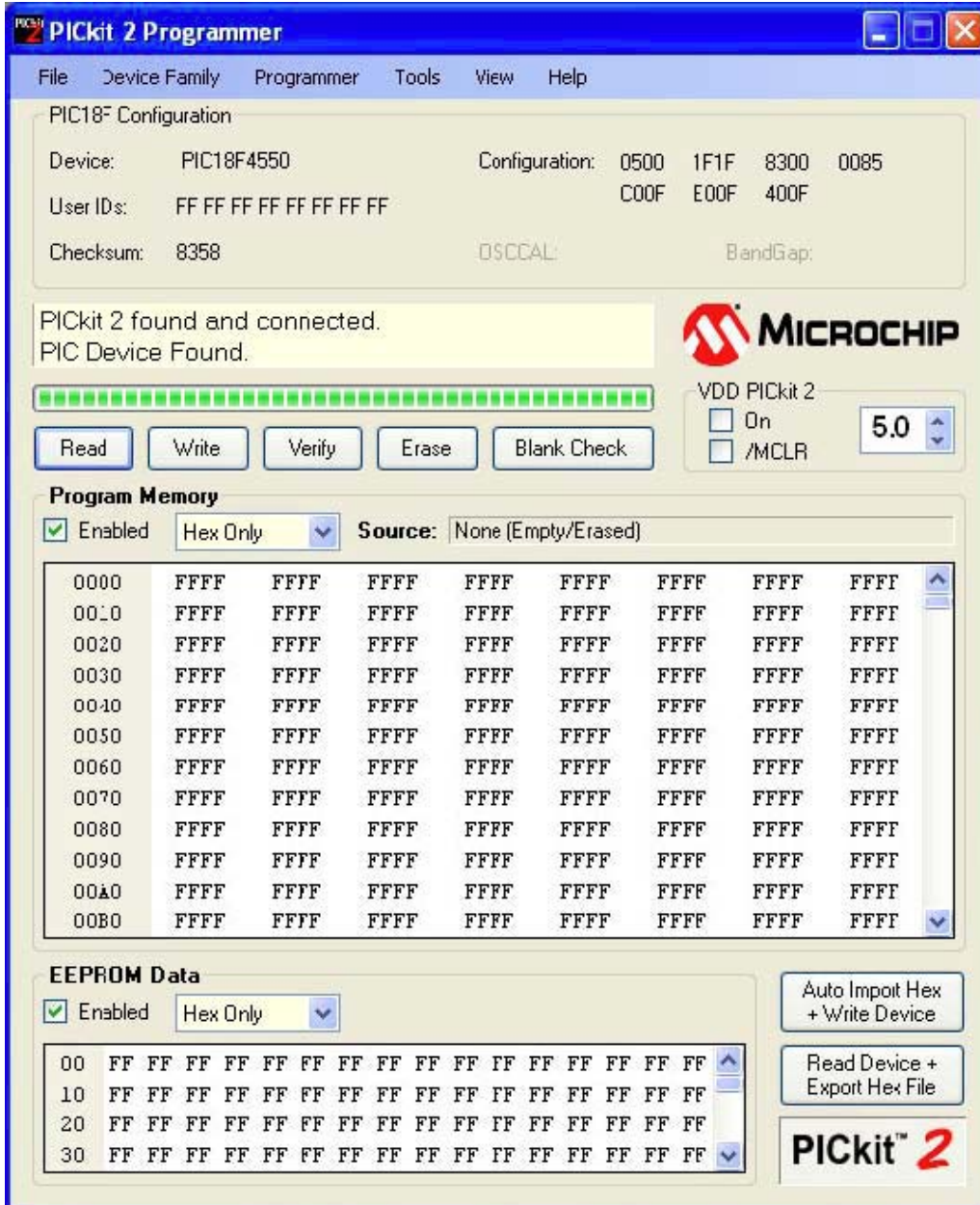
Şekil 4.23 Expkits PICKIT2 ve PICKIT3

Expkits PICKIT2 ve PICKIT3 devreleri orijinal devrelerin klonudur. Orijinal yazılım ve firmware kullanır. Orijinal devrenin yapabildikleri dışında ZIF soket sayesinde hızlı program yükleyebilme özelliği kazandırılmıştır. ICSP programlama soketi sayesinde harici devrelerinizdeki PIC'leri rahatlıkla programlayabilirsiniz.

Burada örnek olarak PICKIT2 ve bunun için PICKIT2 Programmer yazılımı anlatılmıştır.

PICKIT2 Programmer yazılımını çalıştırın. PICKIT2 yazılımını çalıştırmadan önce PICKIT2 programlayıcısını usb kablosu vasıtası ile PC'ye bağlayınız. Bağlı programlayıcı var ise programlayıcı penceresinde **“PICKit2 found and connected”** yazısı çıkacaktır. Bu programlayıcının bulunduğu ve programlayıcıya bağlanıldığı anlamına gelir. Eğer ZIF sokette bir mikrodenetleyici doğru şekilde takılı ise **“PIC Device Found”** yazısı ile bu PIC'in de bulunduğu programda belirtilir (Şekil 4.24).

MicroC DERLEYİCİSİ



Şekil 4.24 PICKIT2 Programmer

Device kısmında, donanıma bağlı bulunan mikrodenetleyicinin modeli görünür.

“Program Memory” penceresi, kullanılan hex dosyasının gösterildiği penceredir.

“EEPROM Data” penceresi, Eeprom verilerinin gösterildiği penceredir.

Yazdığımız programın hex dosyasını PIC18F4550’ye aktarmak için, **File** menüsünden **“Import”** komutu ile hex dosyasını seçerek program hafızasına yükleriz. **Programmer** menüsünden **“Write”** komutu ile programı mikrodenetleyiciye yazdırırız.

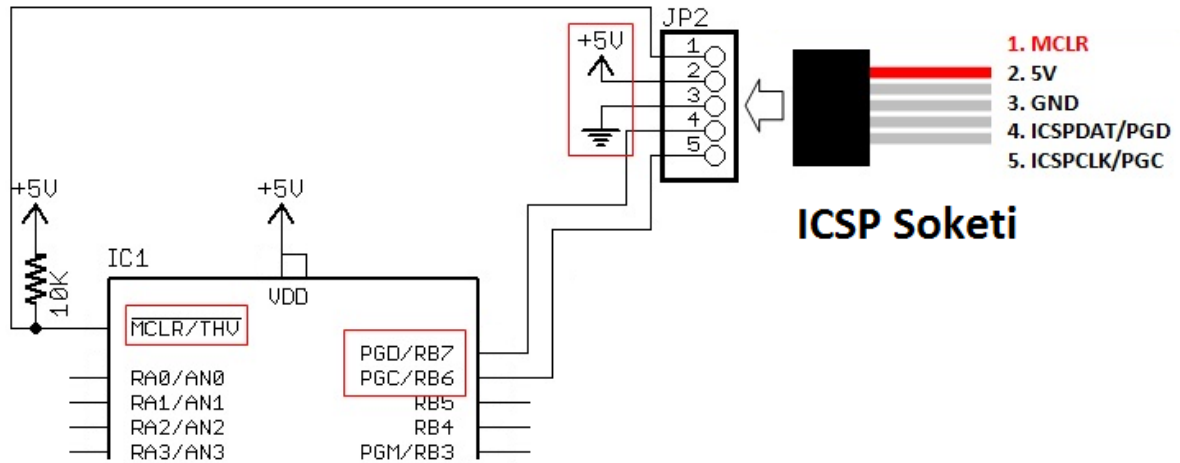
MicroC DERLEYİCİSİ

4.2.5. ICSP ÜZERİNDEN PROGRAMLAMA

ICSP, In Circuits Serial Programming kelimelerinin kısaltılmışıdır. Bazı durumlarda devre üzerindeki mikrodenetleyicileri sökmek çok zahmetli olmakta ya da sökmek gerektiğinde başka sıkıntılara neden olabilmektedir. Mikrodenetleyicileri programlamak için mikrodenetleyiciyi devreden sökmek yerine, devre üzerinde yer alan bir ICSP soketi kullanılabilir. Genellikle büyük çaplı projelerde ve pin sayısının çok fazla olduğu mikrodenetleyicilerde firmalar, kolay programlayabilmek için devre üzerine ICSP bağlantı soketini yerleştirir.

Microchip'in programlamak için hazırlamış olduğu PICKIT2 ve PICKIT3 gibi programlayıcıların üzerinde entegreler için zif soket bulunmamaktadır. Bu nedenle ICSP bağlantısını bilmemiz ve bu bağlantıyı kendimiz oluşturarak PIC'e yazılımı atmamız gerekir.

ICSP üzerinden programlama yapabilmek için 5 adet bacak lazımdır. ICSP ile programlamada her mikrodenetleyicinin PGC(RB6), PGD(RB7) ve MCLR pinleri kullanılır. Program atılırken mikro denetleyicinin beslemesinin verilmesi gereklidir. Bağlantı şeması Şekil 4.25'teki gibi olmalıdır.



Şekil 4.25 ICSP Bağlantı Şeması

BÖLÜM 5

5. LED UYGULAMALARI

UYGULAMA_01

Bu uygulama, PIC'e enerji verildiğinde B portunun 5. bitine bağlı olan bir LED'in yakılmasını sağlayan bir programdır. PIC enerjili kaldığı sürece bu LED yanmaya devam edecektir. Programın yazımı sırasında ADCON1 yazmacı sayesinde pinlerin analog pin /dijital pin olarak kullanımının ayarlanması ve CMCON kaydedici kullanılarak da karşılaştırma (COMPARATOR) modülünün nasıl iptal edileceği gösterilecektir. Bu işlem, bu bölümdeki LED uygulamalarının tümünde pinler dijital çıkış olarak kullanılacağından rutin olarak yapılacaktır. Kullanılan osilatör aksi belirtilmedikçe 8MHz'lik yüksek hızlı (HS) osilatördür.

PROGRAM KODLARI:

*/*Buraya programa ilişkin detaylar yazılabilir. Örneğin programın neye ilişkin yazıldığı, ne zaman ve kim tarafından yazıldığı, kullanılan osilatör ve frekansı gibi*/*

//===== Program başlangıcı =====

```
void main() {           //Ana program her zaman bu ifadeyle başlar ve bu noktadan itibaren
                        // iki süslü //parantez içinde yazılır.

    ADCON1 |= 0x0F;    // ADCON1 yazmacının ilk 4 biti lojik 1 yapılarak yani F değeri
                        // yüklenerek PIC'in tüm pinleri dijital giriş-çıkış olarak ayarlandı.

    CMCON  |=7;        // CMCON yazmacına 7 değeri yüklenerek karşılaştırma
                        // (COMPARATOR) modülü iptal edildi. Dolayısıyla bu modülün
                        //kullanacağı pinler dijital giriş/çıkış için ayrıldı.

    trisb.rb5=0;       // TrisB yazmacının 5.biti 0 yapılarak B portunun 5. biti çıkış olarak
                        // ayarlandı.

    portb.rb5=1;       // B portunun 5 biti lojik 1 yapılarak RB5'e bağlı olan LED yakıldı

    while(1);          // Yapılmak istenen gerçekleştirildiği için program burada bekletildi.
                        // PIC'in enerjisi //kesilene kadar program burada bekler. Noktalı
```

```
// virgülün varlığına dikkat ediniz!  
}  
//Ana programın bittiği, süslü parantez ile gösterildi.  
//===== Program sonu =====
```

Yukarıda verilen program MikroC’de yazıldığında Şekil 5.1’deki ekran görüntüsü elde edilecektir. Ekranda “//” işaretinden sonra yazılan ve otomatikman yeşil renge bürünen yazılar açıklamalar olup program tarafından derlenmezler. Bu satırlar, o satırda yapılan işlem hakkında programı yazana veya bu programı inceleyen kişiye bilgi verir. Dolayısıyla yazılması zorunlu olmayan ancak yazıldığında programcıya gelecek çalışmalarda programı hatırlatması açısından kolaylık sağlayan ifadelerdir. Belki yukarıdaki gibi çok küçük programcıklarda sizler için anlamsız gelebilir. Ancak ilerleyen aşamalarda birkaç bin satırlık program yazdığınızda hangi satırı neden yazdığınızı hatırlamanız ilk bakışta kolay olmayacaktır. O nedenle bunu alışkanlık haline getirmek için açıklama satırları yazmanızı tavsiye ediyoruz.

Program bu şekilde yazılıp derlendiğinde kullanıcıya Şekil 5.2’deki gibi bir mesaj gelecektir. Bu mesajda mevcut ve kullanılabilir RAM ve ROM bilgisi, derleme süresi ve derlemenin başarılı bir şekilde bitirilip bitirilmediğine dair çeşitli bilgiler bulunmaktadır. Bundan sonraki uygulamalarda da derleme işlemi sonucunda bu tür mesajlar gelecektir. Şayet derleme sırasında bir hatayla karşılaşırsa hatanın bulunduğu satır ve hata kullanıcıya sunulacaktır. O nedenle bundan sonraki uygulamalarda bu mesaj görüntüsü bulunmayacaktır.

Şekil 5.1’deki ekran görüntüsü incelendiğinde aslında programın 10. satırdan başlayıp 23. satırda bittiği görülecektir. Ancak buradaki açıklamalar da yazılmasa programın tamamı toplam 7 satırdan oluşacaktır.

```

Start Page LED_01.c
/* Bu uygulama, PIC'e enerji verildiğinde B portunun 5. bitine bağlı olan bir
LED'in yakılmasını sağlayan bir programdır.
Programın yazımı sırasında da pinlerin analog/dijital pin olarak kullanımının
ayarılanması ve CMCON kaydedici kullanılarak COMPARATOR modülünün nasıl iptal
edileceği gösterilecektir.
Kullanılan PIC: 18F4550
Osilatör: 8MHz HS
*/
//===== Program başlangıcı =====
10 void main() { //Ana program bu noktadan itibaren yazılır
    ADCON1 |= 0x0F; //ADCON1 yazmacının ilk 4 biti lojik 1 yapılarak yani
    //F değeri yüklenerek PIC'in tüm pinleri dijital
    //giriş/çıkış olarak ayarlandı
    CMCON |= 7; //CMCON yazmacına 7 değeri yüklenerek karşılaştırma
    //(COMPARATOR) modülü iptal edildi. Dolayısıyla
    // bu modülün kullanacağı pinler dijital giriş/çıkış
    // için ayarlandı
    trisb.rb5=0; //TrisB yazmacının 5.biti 0 yapılarak B portunun 5.
    //biti çıkış olarak ayarlandı
20 portb.rb5=1; //RB5'e bağlı LED yakıldı
    while(1); //Yapılmak istenen gerçekleştirildiği için program
    //burada bekletildi
}
//===== Program sonu =====

```

Şekil 5.1 Programın MikroC'de yazılmasına ilişkin ekran görüntüsü

Messages Quick Converter		
Errors Warnings Hints		
Line	Message No.	Message Text
0	1	mikroC PIC1618.exe -MSF -DBG -p18F4550 -DL -O11111114 -f
0	1139	Available RAM: 2027 [bytes], Available ROM: 32768 [bytes]
0	122	Compilation Started
23	123	Compiled Successfully
0	127	All files Compiled in 47 ms
0	1144	Used RAM (bytes): 48 (2%) Free RAM (bytes): 1979 (98%)
0	1144	Used ROM (bytes): 60 (1%) Free ROM (bytes): 32708 (99%)
0	125	Project Linked Successfully
0	128	Linked in 1029 ms
0	129	Project 'LED_01.mcppi' completed: 1279 ms
0	103	Finished successfully: 29 Ağu 2014, 21:30:40

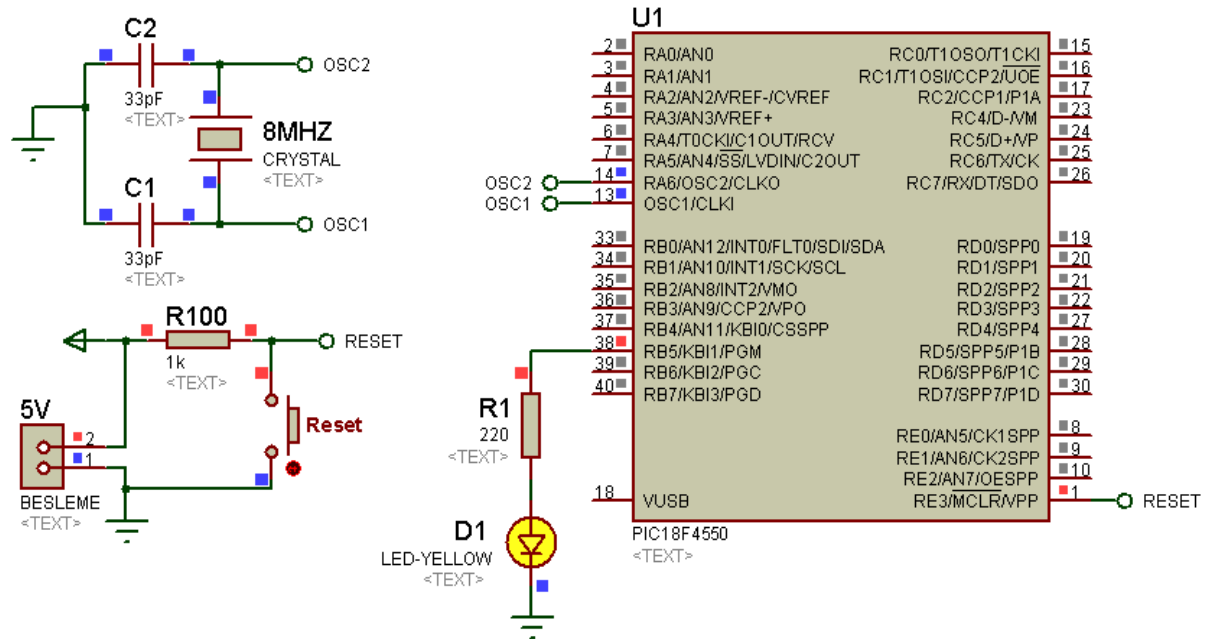
Şekil 5.2 Yazılan programın derlenmesi sonucunda çıkan mesajlar

Program yazılıp derlendiğinde ve Şekil 5.3'teki devre kurulduğunda, PIC'e enerji verildiği takdirde ilgili pine bağlı LED yanacaktır. PIC'in enerjisi kesilmediği sürece de LED yanık kalmaya devam edecektir. Şayet RESET butonuna basılırsa, butona basıldığı süre boyunca ilgili LED sönmük kalacaktır.

Bu uygulamada olduğu gibi bundan sonraki uygulamalarda da PIC'e bağlı LED'lerin seri direnç değerleri 220Ω veya 330Ω olacaktır.

AD SOYAD:

LED UYGULAMALARI



Şekil 5.3 PIC'e enerji verildiğinde PIC'e bağlı bir LED'in yakılması

UYGULAMA_02

Bu uygulamanın bir önceki uygulamadan farkı; kullanılan portun B yerine D olması ve birden fazla LED'in yakılmasıdır. Dolayısıyla bu uygulamada port değiştiğinde TRIS yazmaçlarının değişeceği ve birden fazla giriş-çıkış ayarlaması yapılacaksa bunu farklı şekilde ayarlamanın mümkün olacağı vurgulanacaktır.

Bu örneğimizde PIC'e enerji verildiğinde D portunun 2. ve 3. bitine bağlı LED'ler yanacaktır. PIC enerjili kaldığı sürece bu LED'ler yanmaya devam edecektir.

PROGRAM KODLARI:

```
/* Bu uygulama, PIC'e enerji verildiğinde D portunun 2. ve 3. bitlerine bağlı LED'lerin yakılmasını sağlayan programdır.*/  
//===== Program başlangıcı =====  
void main() {      // Ana program her zaman bu ifadeyle başlar ve bu noktadan itibaren  
                  // iki süslü parantez içinde yazılır.  
    ADCON1 |= 0x0F; // ADCON1 yazmacının ilk 4 biti lojik 1 yapılarak yani F değeri  
                  // yüklenerek PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı  
    CMCON  |= 7;    // CMCON yazmacına 7 değeri yüklenerek karşılaştırma  
                  // (COMPARATOR) modülü iptal edildi. Dolayısıyla bu modülün  
                  //kullanacağı pinler dijital giriş/çıkış için ayrıldı.  
    trisd.rd2=0;     //TrisD yazmacının 2.biti 0 yapılarak D portunun 2. biti çıkış olarak  
                  //ayarlandı  
    trisd.rd3=0;     //TrisD yazmacının 3.biti 0 yapılarak D portunun 3. biti çıkış olarak  
                  //ayarlandı  
    portd.rd2=1;     // D portunun 2. biti lojik 1 yapılarak RD2'ye bağlı olan LED yakıldı  
    portd.rd3=1;     // D portunun 3. biti lojik 1 yapılarak RD3'e bağlı olan LED yakıldı  
    while(1);        //Yapılmak istenen gerçekleştirildiği için program burada bekletildi  
}                    //Ana programın bittiği, süslü parantez ile gösterildi.  
//===== Program sonu =====
```

Yukarıda verilen program MikroC'de yazıldığında Şekil 5.4'teki ekran görüntüsü elde edilecektir. Derlenen programın hex kodları Şekil 5.5'deki devreye yüklenip simülasyon başlatılacak olursa, sadece istenen çıkışların aktif olduğu ve ilgili LED'lerin yandığı görülecektir. PIC'in enerjisi kesilmediği sürece de LED'ler yanık kalmaya devam edecektir. Şayet RESET butonuna basılırsa, butona basıldığı süre boyunca ilgili LED'ler sönük kalacaktır.

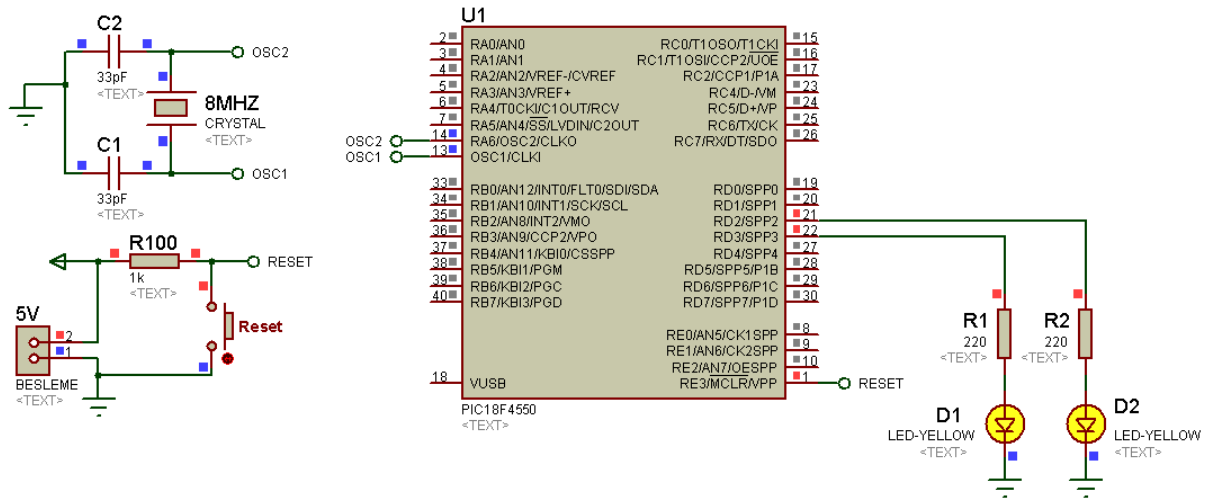
```

/* Bu uygulama, PIC'e enerji verildiğinde D portunun 2. ve 3. bitlerine bağlı
LED'lerin yakılmasını sağlayan bir programdır.
Kullanılan PIC: 18F4550
Osilatör: 8MHz HS */
//===== Program başlangıcı =====
void main() {
    // Ana program her zaman bu ifadeyle başlar ve bu
    //noktadan itibaren iki süslü parantez içinde yazılır.
    ADCON1 |= 0x0F; //ADCON1 yazmacının ilk 4 biti lojik 1 yapılarak yani
    //F değeri yüklenerek PIC'in tüm pinleri dijital
    //giriş/çıkış olarak ayarlandı
    CMCON |= 7; //CMCON yazmacına 7 değeri yüklenerek karşılaştırma
    //(COMPARATOR) modülü iptal edildi. Dolayısıyla
    // bu modülün kullanacağı pinler dijital giriş/çıkış
    // için ayarlandı
    trisd.rd2=0; //TrisD yazmacının 2.biti 0 yapılarak D portunun 2.
    //biti çıkış olarak ayarlandı
    trisd.rd3=0; //TrisD yazmacının 3.biti 0 yapılarak D portunun 3.
    //biti çıkış olarak ayarlandı
    portd.rd2=1; //RD2'ye bağlı LED yakıldı
    portd.rd3=1; //RD3'e bağlı LED yakıldı
    while(1); //Yapılmak istenen gerçekleştirildiği için program
    //burada bekletildi
    //Ana programın bittiği, süslü parantez ile gösterildi.
//===== Program sonu =====

```

Şekil 5.4 Aynı porta bağlı iki LED'in yakılmasına ilişkin MikroC'de yazılan programın ekran görüntüsü

Burada **trisd.rd2=0;** ve **trisd.rd3=0;** satırları yerine sadece **trisd=0;** diyerek D portunun tüm pinleri çıkış olarak ayarlanabilir. Ancak bu port hem giriş hem de çıkış olarak kullanılacaksa, bu durumda TrisD yazmacındaki ilgili bitlere **çıkışlar için 0, girişler için 1** verilmek suretiyle sadece ilgili bitlerin giriş veya çıkış olarak ayarlanması sağlanabilir. Bu uygulamada sadece 2. ve 3. bitler için bu yapılacak olursa, bu durumda binary formatlı **trisd=0b11110011;** veya hexadecimal formatlı **trisd=0xF3;** yazmak yeterli olacaktır.



Şekil 5.5 PIC'e enerji verildiğinde PIC'e bağlı iki adet LED'in yakılması

Aynı işlem ilgili LED'lerin tek satırda yakılmasını sağlamak için de kullanılabilir. Yani ***portd.rd2=1;*** ve ***portd.rd3=1;*** satırları yerine sadece ***portd=0xFF;*** yazılabilir. Bu kod yazıldığında D portuna ait tüm pinler lojik 1 yapılmış olur. Ancak bizim bu uygulamamızda sadece 2 ve 3. pinlere LED bağlı olduğu için diğer pinlerin durumunun ne olacağı bir anlam ifade etmez, biz amacımıza ulaşmış oluruz. Şayet sadece bu iki pin aktif edilmek istenirse bu durumda binary formatlı ***portd=0b00001100;*** veya hexadecimal formatlı ***portd=0x0C;*** yazmak yeterli olacaktır.

Aynı şekilde programın hiçbir şey yapmadan beklemesini sağlamak için ***while(1);*** kullanılabileceği gibi ***while(1) {}*** de yazılabilir. Burada yapılan şey, programı iki süslü parantez içerisinde sonsuz döngüye sokmaktır. Ancak iki süslü parantez içerisinde herhangi bir kod bulunmadığı için, PIC'in enerjisi kesilmediği sürece program bu süslü parantezlerin içerisinde herhangi bir işlem yapmadan bekleyecektir.

Bu açıklamalara ilişkin örnek bir sonraki uygulamada yapılmıştır.

UYGULAMA_03

Uygulama_02’de PIC’e enerji verildiğinde D portunun 2. ve 3. bitine bağlı LED’leri yakan program üzerinde durulmuştu. Bu uygulamada ise D portunun tüm pinlerine bağlı olan 8 adet LED, PIC’e enerji verildiğinde yakılacaktır. PIC enerjili kaldığı sürece bu LED’ler yanmaya devam edecektir.

İlgili program kodları aşağıda verilmiştir. Programın MikroC programındaki görüntüsü ve devre şeması sırasıyla Şekil 5.6 ve Şekil 5.7’de görülmektedir.

PROGRAM KODLARI:

```
/* Bu uygulama, PIC'e enerji verildiğinde D portunun tüm bitlerine bağlı LED'lerin yakılmasını
sağlayan programdır.*/

//===== Program başlangıcı =====

void main() {           // Ana program başlangıcı
    ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |= 7;     //Karşılaştırma (COMPARATOR) modülü iptal edildi.
    trisD=0;         //D portunun tüm pinleri çıkış olarak ayarlandı
    portD=0xFF;      //D portunun tüm pinleri lojik 1 yapıldı (tüm LED'ler yakıldı)
    while(1)         //Program sonsuz döngüye sokuldu (noktalı virgül yok!)
    {
        //Hiçbir kod yazılmadığına dikkat ediniz!
    }
}                       //Ana program sonlandırıldı.

//===== Program sonu =====
```

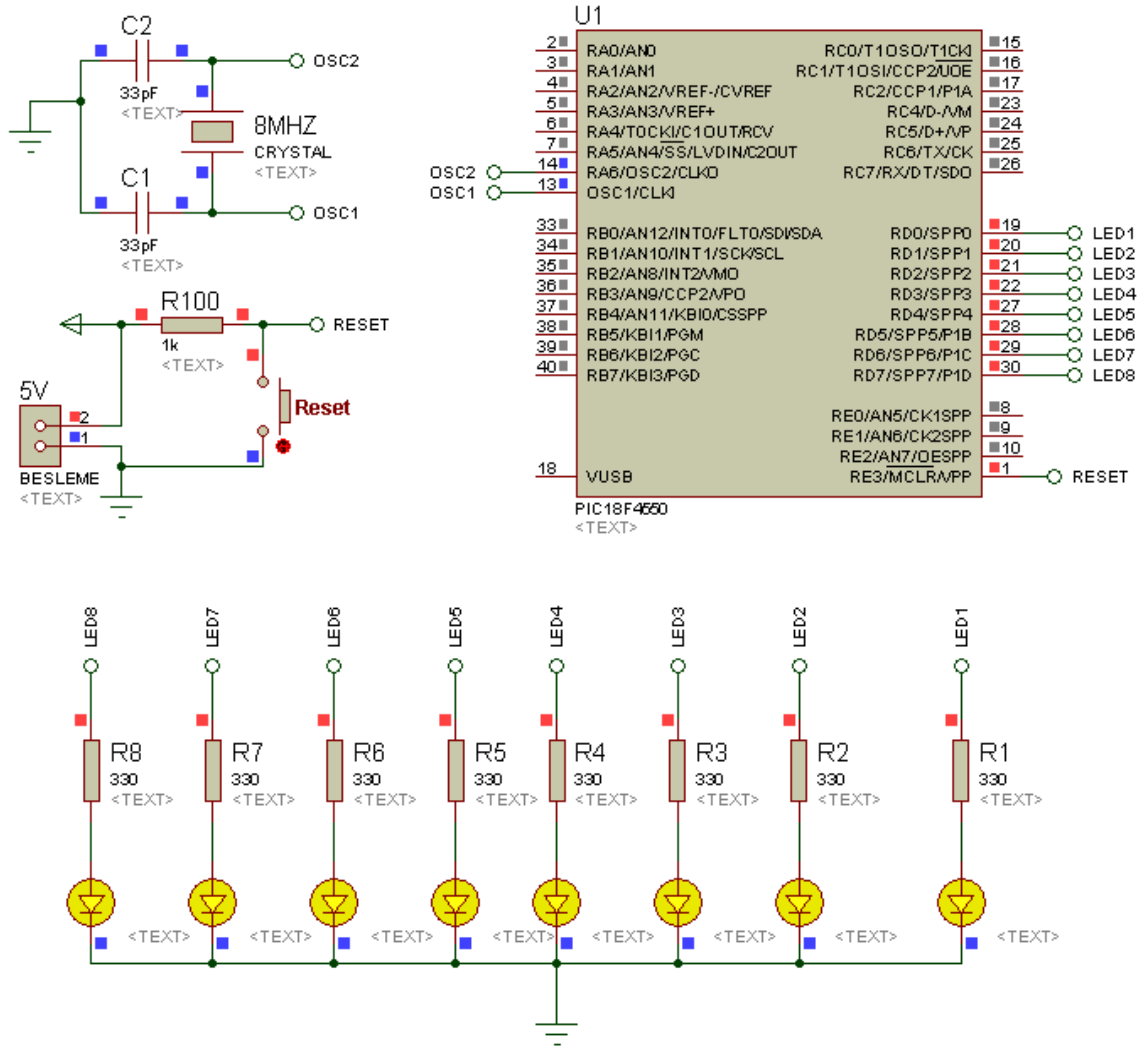
Şekil 5.7’deki simülasyon devresinde PIC’in enerjisi kesilmediği sürece LED’ler yanık kalmaya devam edecektir. Şayet RESET butonuna basılırsa, butona basıldığı süre boyunca ilgili LED’ler sönmük kalacaktır.

```

Start Page LED_03.c
/* Bu uygulama, PIC'e enerji verildiğinde D portuna bağlı tüm LED'lerin
yakılmasını sağlayan programdır.
Kullanılan PIC: 18F4550
Osilatör: 8MHz HS */
//===== Program başlangıcı =====
void main() { // Ana program başlangıcı
    ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON |= 7; //Karşılaştırma (COMPARATOR) modülü iptal edildi.
    trisD=0; //D portunun tüm pinleri çıkış olarak ayarlandı
    portD=0xFF; //D portunun tüm pinleri lojik 1 yapıldı (LED'ler yakıldı)
    while(1) //Program sonsuz döngüye sokuldu (noktalı virgül yok!)
    {
        //Hiçbir kod yazılmadığına dikkat ediniz!
    }
} //Ana program sonlandırıldı.
//===== Program sonu =====

```

Şekil 5.6 D portundaki tüm LED'lerin yakılmasına ilişkin MikroC programının ekran görüntüsü



Şekil 5.7 D portundaki tüm LED'lerin yakılmasına ilişkin ISIS devresi

UYGULAMA_04

Bu uygulamanın bir önceki uygulamadan farkı; B portunun kullanılması ve bu porta bağlı tüm LED'lerin belirli aralıklarla yakılıp söndürülmesidir. Uygulamada ihtiyaç duyulan zaman gecikmesi, istenilen zaman gecikmesinin ms cinsinden Delay_ms() komutunda yazılmasıyla sağlanabilir.

Buradaki zaman gecikmeleri LED'lerin yanık ve sönük kalma sürelerini ayarlamak için kullanılır. LED'lerin belirli bir süre yanık kalması isteniyorsa bu durumda LED'ler yakıldıktan sonra zaman gecikmesi konulur. Konulan bu zaman gecikmesi boyunca program o noktada bekler. Dolayısıyla bir önceki satırda LED'ler yakıldığı için, konulan gecikme süresince LED'ler yanık kalmaya devam eder. Ardından LED'ler söndürülür. Bu defa da LED'lerin sönük kalma süresi ayarlanır. Bu amaçla konulan zaman gecikmesi bir öncekinden bağımsız ayarlanabilir. Dolayısıyla yanık ve sönük kalma süreleri değiştirilerek farklı animasyonlar gerçekleştirilebilir.

Bu uygulamada LED'ler sonsuz döngü içinde 1 saniye yanık, 300 ms sönük olarak ayarlanmıştır.

PROGRAM KODLARI:

```
/* B portunun tüm bitlerine bağlı LED'lerin belirli aralıklarla yakılıp söndürülmesine ilişkin program.*/
```

```
//===== Program başlangıcı =====
```

```
void main() {           // Ana program başlangıcı
    ADCON1 |= 0x0F;      //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |= 7;         //Karşılaştırma (COMPARATOR) modülü iptal edildi.
    trisB=0;             //B portunun tüm pinleri çıkış olarak ayarlandı
    portB=0;             //B portunun tüm pinleri lojik 0 yapıldı (tüm LED'ler sönük)
    while(1) {           //Program sonsuz döngüye sokuldu (noktalı virgül yok!)
        portB=0xFF;      //B portuna bağlı tüm LED'ler yakıldı
        Delay_ms(1000);   //LED'lerin 1 saniye( =1000 ms) yanık kalması için gecikme konuldu
        portB=0;         //B portuna bağlı tüm LED'ler söndürüldü
        Delay_ms(300);    //LED'lerin 300 ms sönük kalması için gecikme konuldu
    }
}
```

```

    }
}
//Ana program sonlandırıldı.
//===== Program sonu =====

```

Yukarıdaki uygulama kodları Şekil 5.8'deki gibi yazıldıktan sonra derlenir. Bu derleme sonunda elde edilen hex kodu, Şekil 5.9'daki devreye yüklendiğinde ilgili LED'lerin belirlenen aralıklarda yanıp söndüğü görülecektir. Şayet RESET butonuna basılırsa, butona basılıp bırakıldığında program ilk noktaya geri dönecektir.

```

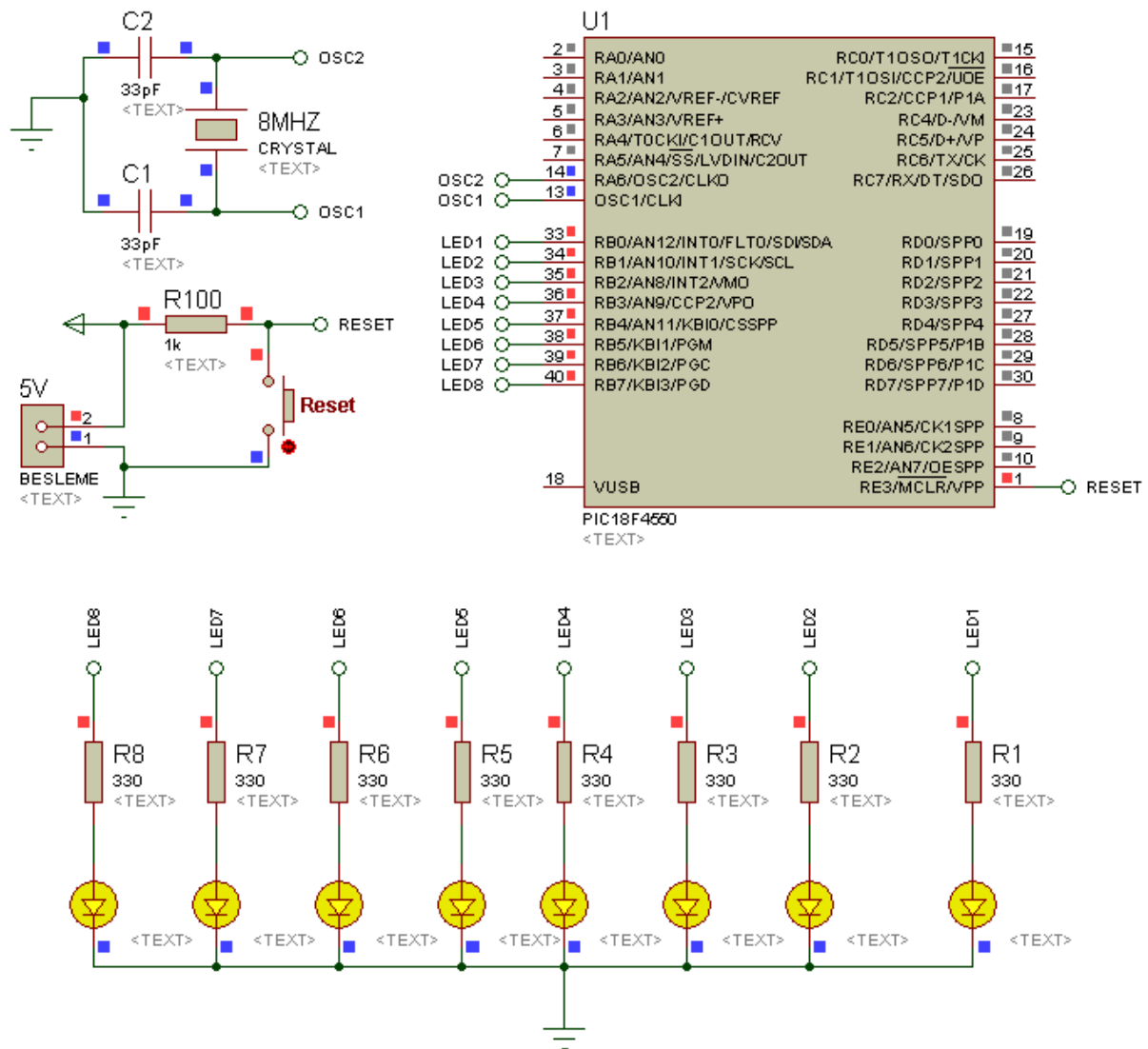
//===== Program başlangıcı =====
void main() {
    //Ana program başlangıcı
    ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |= 7;    //Karşılaştırma (COMPARATOR) modülü iptal edildi.
    trisB=0;        //B portunun tüm pinleri çıkış olarak ayarlandı
    portB=0;        //B portunun tüm pinleri lojik 0 yapıldı (tüm LED'ler sönmük)
    while(1){
        //Program sonsuz döngüye sokuldu (noktalı virgül yok!)
        portB=0xFF; //B portuna bağlı tüm LED'ler yakıldı
        Delay_ms(1000); //LED'lerin 1 saniye( =1000 ms) yanık kalması için gecikme konuldu
        portB=0;      //B portuna bağlı tüm LED'ler söndürüldü
        Delay_ms(300); //LED'lerin 300 ms sönmük kalması için gecikme konuldu
    }
    //Ana program sonlandırıldı.
//===== Program sonu =====

```

Şekil 5.8 B portundaki tüm LED'lerin belirli aralıklarla yakılıp söndürülmesine ilişkin MikroC kodu ekran görüntüsü

AD SOYAD:

LED UYGULAMALARI



Şekil 5.9 B portundaki tüm LED'lerin belirli aralıklarla yakılıp söndürülmesine ilişkin ISIS devresi

UYGULAMA_05

Bu uygulamada da B portuna bağlı 8 LED'in belirli aralıklarla yakılıp söndürülmesi gerçekleştirilecektir. Ancak bir önceki uygulamadan farklı olarak LED'ler 4'erli grup hâlinde yakılıp söndürülecektir. Yani 4'lü gruplar hâlindeki flaşör uygulaması yapılacaktır.

LED'lerin yanık ve sönük kalma sürelerinin eşit olduğu bu örnekte, daha önceki uygulamalarda kullanmadığımız bir komut olan tersleme komutu kullanılacaktır. Bu komut kullanıldığında ilgili port veya bitin durumu terslenecektir.

PROGRAM KODLARI:

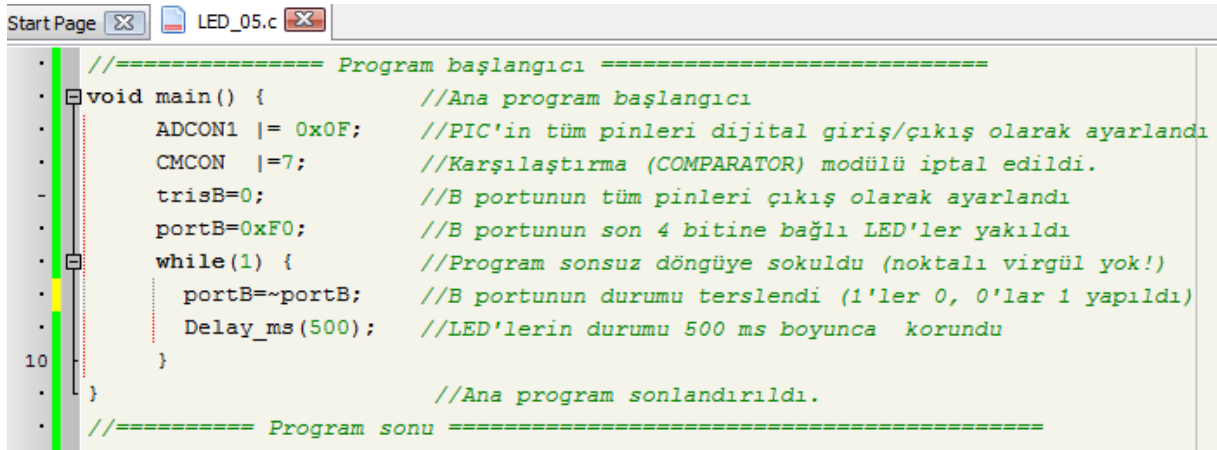
```
/* B portuna bağlı LED'lerin 4'erli gruplar halinde belirli aralıklarla yakılıp söndürülmesine ilişkin program.*/

//===== Program başlangıcı =====

void main() {           // Ana program başlangıcı
    ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |= 7;    //Karşılaştırma (COMPARATOR) modülü iptal edildi.
    trisB=0;        //B portunun tüm pinleri çıkış olarak ayarlandı
    portB=0xF0;     //B portunun son 4 bitine bağlı LED'ler yakıldı
    while(1) {      //Program sonsuz döngüye sokuldu (noktalı virgül yok!)
        portB=~portB; //B portunun durumu terslendi (1'ler 0, 0'lar 1 yapıldı)
        Delay_ms(500); //LED'lerin durumu 500 ms boyunca korundu
    }
}                       //Ana program sonlandırıldı.

//===== Program sonu =====
```

Yukarıdaki uygulama kodları Şekil 5.10'daki gibi yazıldıktan sonra derlenir. Bu derleme sonucunda elde edilen hex kodu, Şekil 5.11'deki devreye yüklendiğinde, B portuna bağlı LED'lerin 4'erli gruplar halinde flaşör şeklinde yanıp söndüğü görülecektir.



```

//===== Program başlangıcı =====
void main() { //Ana program başlangıcı
    ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON |= 7; //Karşılaştırma (COMPARATOR) modülü iptal edildi.
    trisB=0; //B portunun tüm pinleri çıkış olarak ayarlandı
    portB=0xF0; //B portunun son 4 bitine bağlı LED'ler yakıldı
    while(1) { //Program sonsuz döngüye sokuldu (noktalı virgül yok!)
        portB=~portB; //B portunun durumu terslendi (1'ler 0, 0'lar 1 yapıldı)
        Delay_ms(500); //LED'lerin durumu 500 ms boyunca korundu
    }
} //Ana program sonlandırıldı.
//===== Program sonu =====

```

Şekil 5.10 B portundaki tüm LED'lerle yapılan flaşör uygulamasına ilişkin MikroC kodu ekran görüntüsü

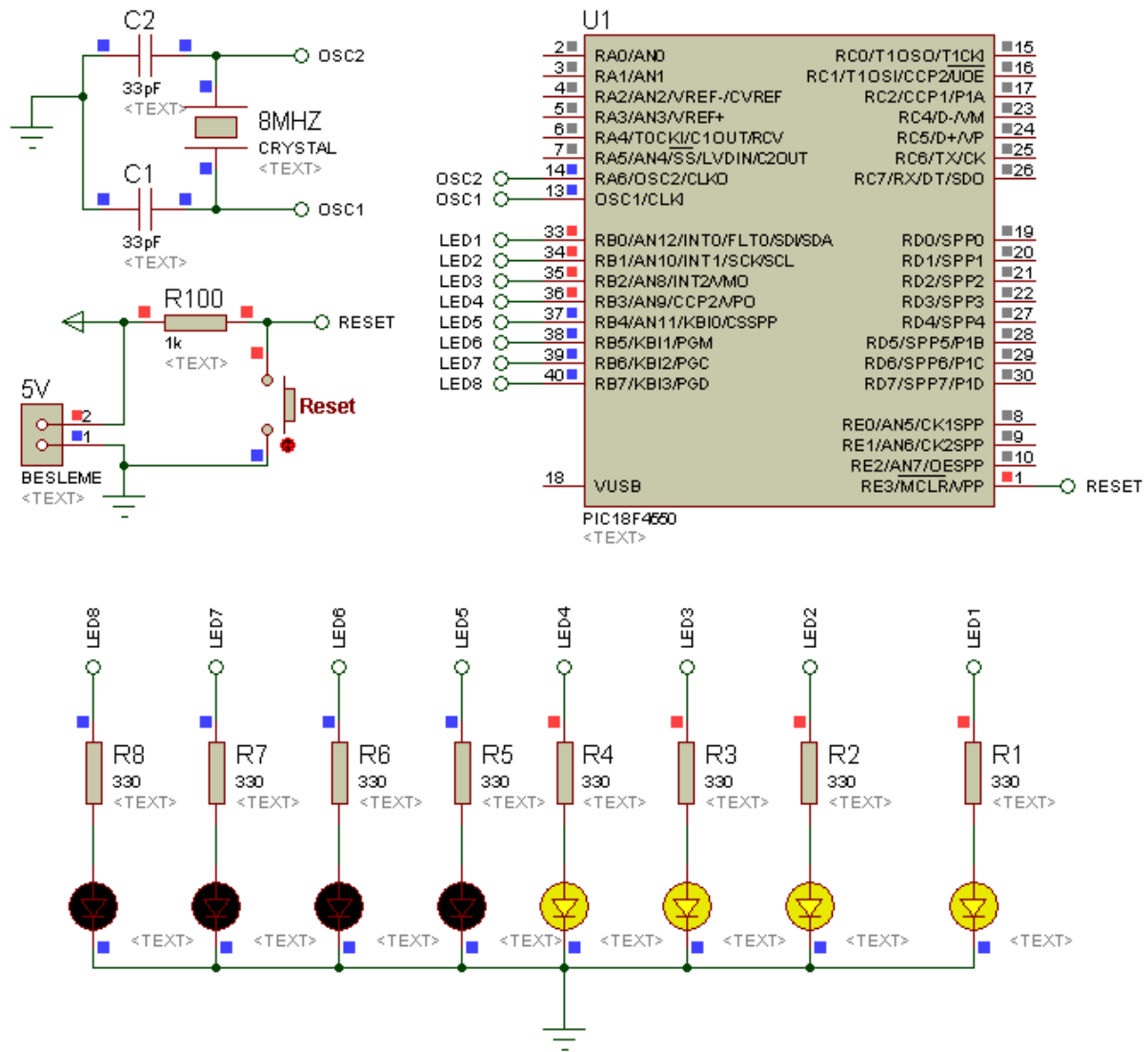
Programda sonsuz döngüye girilmeden önce LED'lerin son 4'ü (RB4-RB7) yakılmıştır. Sonsuz döngüye girildiğinde ise bu durum terslenmiştir. Yani sönmüş olan LED'ler yanmış, yanan LED'ler sönmüştür. Bu işlem sonsuz döngü içinde PIC'in enerjisi kesilene kadar devam edecektir.

Ancak Şekil 5.11'deki ISIS devresi kurulup çalıştırıldığında ilk aşamada yanması gereken RB4, RB5, RB6 ve RB7'ye bağlı son 4 LED'in yanmadığı, bunun yerine ilk önce RB0, RB1, RB2 ve RB3'e bağlı ilk 4 LED'in yandığı görülecektir. Aslında bu bir göz yanılmasıdır. Gerçekte ilk aşamada yanması gereken RB4, RB5, RB6 ve RB7'ye bağlı son 4 LED yanmaktadır. Ancak μ s'ler mertebesindeki bir süre sonra bu LED'lerin durumu terslendiği için gözümüz bu LED'lerin yanıp söndüğünü görememektedir. Dolayısıyla ortada herhangi bir yanlıklık yoktur.

6. satırdaki **portB=0xF0;** kodunun yerine **portB=0x0F;** yazılırsa bu göz yanılmaları ortadan kalkacaktır. Yani devrede ilk önce ilk 4 bite bağlı LED'lerin yanık son 4 bite bağlı LED'lerin sönmüş, 500ms sonra da ilk 4 bite bağlı LED'lerin sönmüş son 4 bite bağlı LED'lerin yanık olduğu görülecektir.

AD SOYAD:

LED UYGULAMALARI



Şekil 5.11 B portundaki tüm LED'lerle yapılan flaşör uygulamasına ilişkin ISIS şeması

UYGULAMA_06

Bu programda yukarı binary sayıcı uygulaması yapılacaktır. D portunun kullanıldığı bu uygulamada öncelikle D portuna 0 verisi yüklenecek, ardından 500ms aralıklarla D portunun değeri 1 artırılarak binary sayma **portd=0b11111111** (decimal 255) oluncaya kadar devam edecektir. Bu noktadan sonra D portunun değeri decimal 0 olacak ve program tekrar baştan başlayacaktır. Bu işlem PIC'in enerjisi kesilene veya RESET butonuna basılıncaya kadar devam edecektir.

İlgili program kodları aşağıda verilmiştir. Programın MikroC programındaki ekran görüntüsü ve devre şeması sırasıyla Şekil 5.12 ve Şekil 5.13'te görülmektedir.

PROGRAM KODLARI:

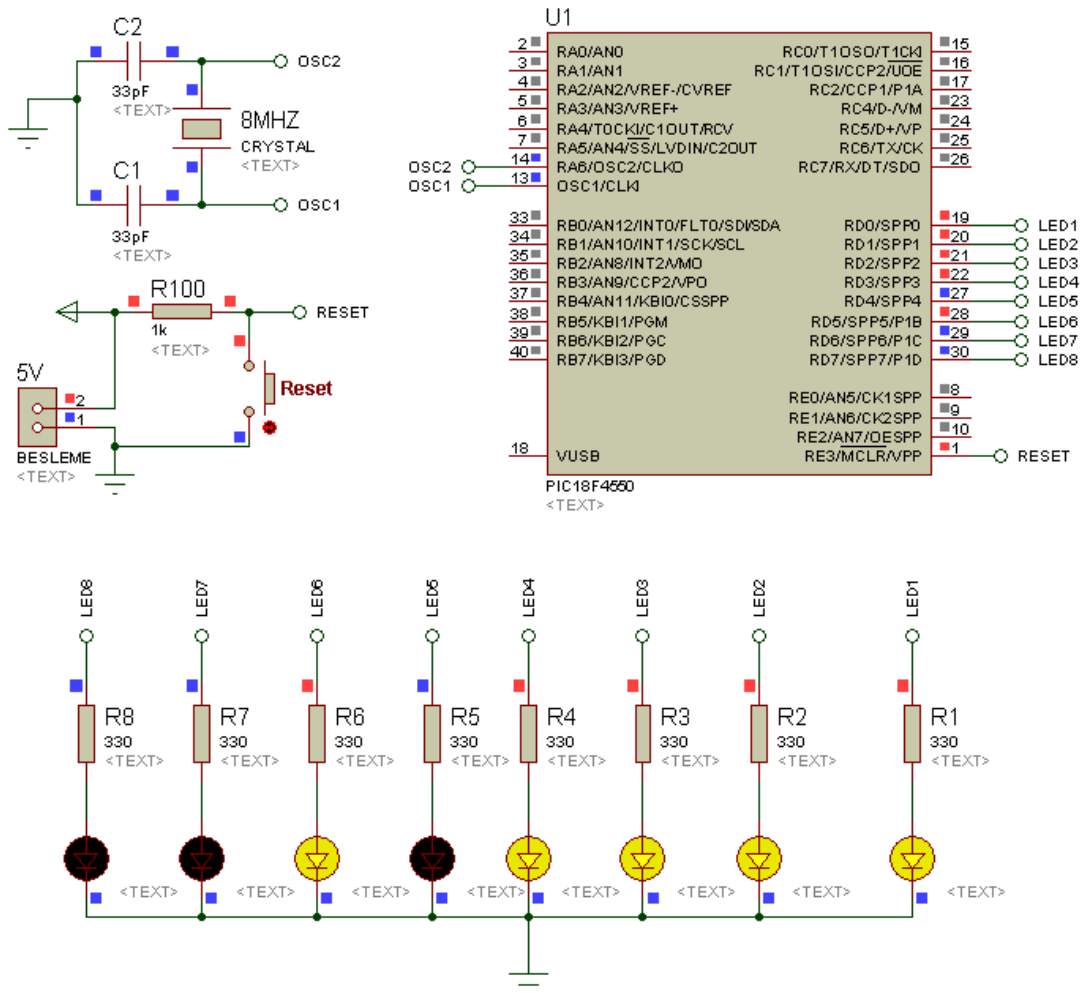
```
/* D portuna bağlı LED'lerle yapılan yukarı binary sayıcı uygulamasına ilişkin program. */  
//===== Program başlangıcı =====  
void main() {           // Ana program başlangıcı  
    ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı  
    CMCON  |= 7;     //Karşılaştırma (COMPARATOR) modülü iptal edildi.  
    trisD=0;         //D portunun tüm pinleri çıkış olarak ayarlandı  
    portD=0;         //D portunun tüm bitleri sıfırlandı  
    while(1) {       //Program sonsuz döngüye sokuldu (noktalı virgül yok!)  
        portd++;      //PortD'ye 1 eklendi (portd=portd+1 ile aynı işlemdir)  
        Delay_ms(500); //LED'lerin durumu 500 ms boyunca korundu  
    }  
}                       //Ana program sonlandırıldı.  
//===== Program sonu =====
```

```

//===== Program başlangıcı =====
void main() { //Ana program başlangıcı
    ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON |= 7; //Karşılaştırma (COMPARATOR) modülü iptal edildi.
    trisD=0; //D portunun tüm pinleri çıkış olarak ayarlandı
    portD=0; //D portunun tüm bitleri sıfırlandı
    while(1) { //Program sonsuz döngüye sokuldu (noktalı virgül yok!)
        portD++; //PortD'ye 1 eklendi (portD=portD+1 ile aynı işlemdir)
        Delay_ms(500); //LED'lerin durumu 500 ms boyunca korundu
    }
} //===== Program sonu =====

```

Şekil 5.12 D portuna bağlı LED'lerle yapılan yukarı binary sayıcı uygulamasına ilişkin MikroC kodu ekran görüntüsü



Şekil 5.13 D portuna bağlı LED'lerle yapılan yukarı binary sayıcı uygulamasına ilişkin ISIS şeması (LED'lerin mevcut değeri decimal 47'dir)

UYGULAMA_07

Bu uygulamada bir önceki uygulamaya benzer bir uygulama yapılacaktır. Ancak bu defa yukarı sayma işlemi 255'e ulaşıldığında belli bir süre beklenerek ve ardından 0'a kadar aşağı sayma işlemi gerçekleştirilecektir. Decimal 0 değerine ulaşıldığında ise program tekrar yukarı sayma işlemine geçecektir. Bu işlem PIC'in enerjisi kesilene veya RESET butonuna basılana kadar devam edecektir.

İlgili program kodları aşağıda verilmiştir. Programın MikroC programındaki görüntüsü ve devre şeması sırasıyla Şekil 5.14 ve Şekil 5.15'de görülmektedir.

PROGRAM KODLARI:

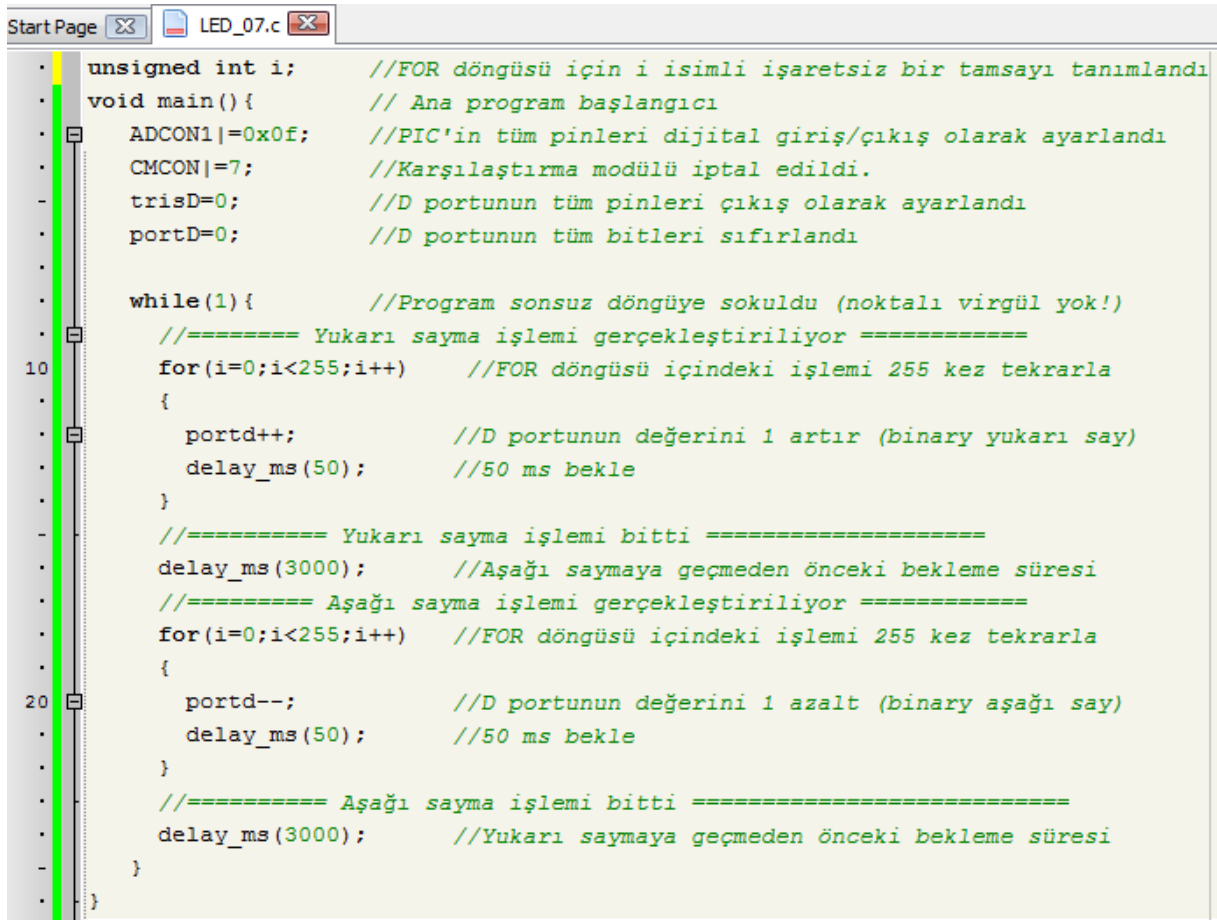
```
//===== Program başlangıcı =====
/* D portuna bağlı LED'lerle yapılan yukarı-aşağı binary sayıcı uygulamasına ilişkin
program.*/
unsigned int i;          //FOR döngüsü için i isimli işaretli bir tamsayı tanımlandı
void main(){             // Ana program başlangıcı
    ADCON1|=0x0f;        //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON|=7;            //Karşılaştırma modülü iptal edildi.
    trisD=0;             //D portunun tüm pinleri çıkış olarak ayarlandı
    portD=0;             //D portunun tüm bitleri sıfırlandı
    while(1){            //Program sonsuz döngüye sokuldu (noktalı virgül yok!)
//===== Yukarı sayma işlemi gerçekleştiriliyor =====
        for(i=0;i<255;i++) //FOR döngüsü içindeki işlemi 255 kez tekrarla
        {
            portD++;       //D portunun değerini 1 artır (binary yukarı say)
            delay_ms(50);   //50 ms bekle
        }
//===== Yukarı sayma işlemi bitti =====
        delay_ms(3000);     //Aşağı saymaya geçmeden önceki bekleme süresi
//===== Aşağı sayma işlemi gerçekleştiriliyor =====
```

```
for(i=0;i<255;i++) //FOR döngüsü içindeki işlemi 255 kez tekrarla
{
    portd--; //D portunun değerini 1 azalt (binary aşağı say)
    delay_ms(50); //50 ms bekle
}
//===== Aşağı sayma işlemi bitti =====
delay_ms(3000); //Yukarı saymaya geçmeden önceki bekleme süresi
}
}
//===== Program sonu =====
```

Program kodlarında sonsuz döngüye girildiğinde yukarı ve aşağı sayma işlemlerinde FOR komutunun kullanıldığı görülecektir. Bu komutun kullanımında ilk önce döngü değişkeninin ilk değeri atanır. Ardından döngünün şartı belirlenir. Son olarak da şartın sağlanması durumunda döngü değişkeninin artış/azalış miktarı belirtilir ve yapılacak işleme geçilir. Buradaki döngü değişkeni “*i*” olarak tayin edilmiştir. Değişken türü ise programın ilk satırında işaretli tam sayı olarak belirtilmiştir.

for(i=0;i<255;i++) yazıldığında *i* değişkeni 0’dan başlayıp değeri 255’ulaşınca kadar her döngüde 1 artırılacaktır. Döngüye her girdiğinde de ilk FOR döngüsünde D portunun değerini 1 artırarak yukarı saymayı, ikinci FOR döngüsünde ise D portunun değerini 1 azaltarak aşağı saymayı gerçekleştirecektir.

FOR döngülerinin içindeki gecikmeler sayma işleminin hızını belirler. Burada 50ms alınmak suretiyle saymanın makul bir hızda gerçekleşmesi hedeflenmiştir. Yukarıdan aşağıya ve aşağıdan yukarıya dönme işleminde 3’er saniyelik gecikme konularak decimal 255 ve 0 değerlerinin rahatça gözlenmesi sağlanmıştır. Bu gecikmelerin değerleri ile oynanarak farklı hız ve bekleme süreleri elde edilebilir.



```

StartPage  LED_07.c
• unsigned int i;      //FOR döngüsü için i isimli işaretli bir tamsayı tanımlandı
• void main(){         // Ana program başlangıcı
•   ADCON1|=0x0f;      //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
•   CMCON|=7;          //Karşılaştırma modülü iptal edildi.
•   trisD=0;           //D portunun tüm pinleri çıkış olarak ayarlandı
•   portD=0;           //D portunun tüm bitleri sıfırlandı
•
•   while(1){          //Program sonsuz döngüye sokuldu (noktalı virgül yok!)
•       //===== Yukarı sayma işlemi gerçekleştiriliyor =====
10      for(i=0;i<255;i++) //FOR döngüsü içindeki işlemi 255 kez tekrarla
•       {
•           portD++;      //D portunun değerini 1 artır (binary yukarı say)
•           delay_ms(50); //50 ms bekle
•       }
•       //===== Yukarı sayma işlemi bitti =====
•       delay_ms(3000);   //Aşağı saymaya geçmeden önceki bekleme süresi
•       //===== Aşağı sayma işlemi gerçekleştiriliyor =====
•       for(i=0;i<255;i++) //FOR döngüsü içindeki işlemi 255 kez tekrarla
20      {
•           portD--;      //D portunun değerini 1 azalt (binary aşağı say)
•           delay_ms(50); //50 ms bekle
•       }
•       //===== Aşağı sayma işlemi bitti =====
•       delay_ms(3000);   //Yukarı saymaya geçmeden önceki bekleme süresi
•   }
• }

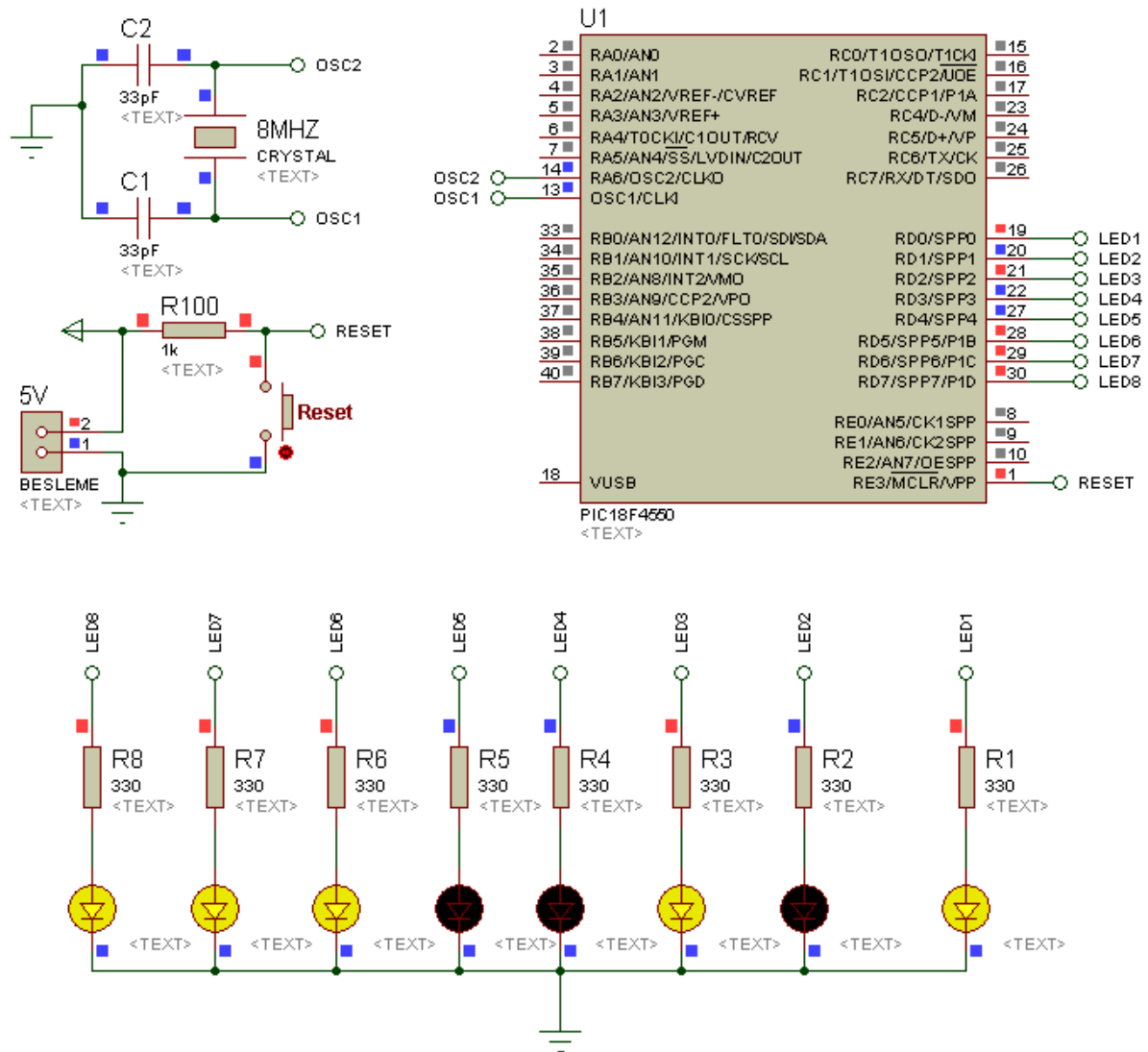
```

Şekil 5.14 D portuna bağlı LED'lerle yapılan yukarı-aşağı binary sayıcı uygulamasına ilişkin

MikroC kodu ekran görüntüsü

AD SOYAD:

LED UYGULAMALARI



Şekil 5.15 D portuna bağlı LED'lerle yapılan yukarı-aşağı binary sayıcı uygulamasına ilişkin
ISIS şeması (LED'lerin mevcut değeri decimal 229'dur)

UYGULAMA_08

Bu uygulamada LED'lerin sırayla sola ve sağa doğru sadece bir tanesinin yanarak kaymasına dair program üzerinde durulacaktır. "Karaşımşek" olarak da adlandırılan bu uygulamada 8 adet LED D portuna bağlanmıştır. Bu LED'lerden ilk önce D portunun 0. bitine bağlı olan LED yakılacaktır. Ardından önce sola sonra da sağa doğru en sondaki LED yanıncaya kadar sırayla tek LED yanarak kaymaya devam edecektir.

İlgili program kodları aşağıda verilmiştir. Programın MikroC programındaki ekran görüntüsü ve devre şeması sırasıyla Şekil 5.16 ve Şekil 5.17'de görülmektedir.

PROGRAM KODLARI:

```
/* D portuna bağlı LED'lerin sırayla sola ve sağa doğru sadece bir tanesinin yanarak
kaymasına dair program (Karaşımşek uygulaması) */
//===== Program başlangıcı =====
void main() {
    // Ana program başlangıcı
    ADCON1 |= 0x0F; //Tüm pinler dijital giriş/çıkış olarak ayarlandı
    CMCON |= 7; //Karşılaştırma modülü iptal edildi.
    TrisD=0; //D portu çıkış olarak atandı
    PortD=1; //D portunun ilk bitini (0. bitini) 1 yap
    while(1){ //Programı sonsuz döngüye sok
        //===== Sola kaydırma işlemi yaptırılacak =====
        while (portd < 0b10000000) { //PortD'nin değeri 0b10000000'dan küçükken döngüye gir
            delay_ms(100); //100ms bekle
            portd = portd << 1; //D portundaki veriyi bir sola kaydır
        }
        //===== Sağa kaydırma işlemi yaptırılacak =====
        while (portd > 0b00000001) { //PortD'nin değeri 0b00000001'den büyükken döngüye gir
            delay_ms(100); //100ms bekle
            portd = portd >> 1; //D portundaki veriyi bir sağa kaydır
        }
    }
}
```

```

}
}

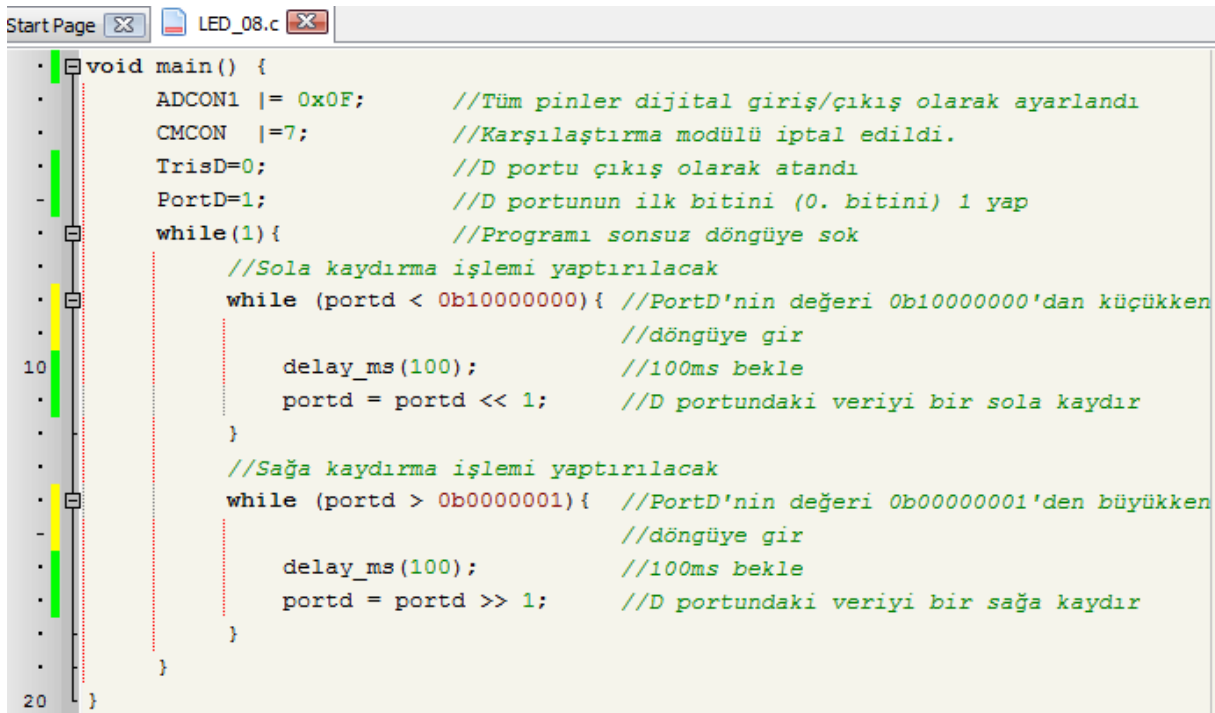
```

```

//===== Program sonu =====

```

Yukarıdaki program kodları incelendiğinde şimdiye kadarki uygulamalarımızda hep sonsuz döngü oluşturmak için kullandığımız **while** döngüsünün, burada farklı bir şekilde kullanıldığı görülecektir. **while (portd < 0b10000000)** yazıldığında PortD'nin değeri 0b10000000'dan küçükken yani **RD7=1** oluncaya kadar program akışı döngüye girecek ve iki süslü parantez arasındaki işlemler bu şart bozuluncaya kadar devam edecektir. Bu döngü içerisinde kullanılan << ifadesi ise D portundaki verinin 1 bit sola kaymasını sağlayacaktır. Yani bu komut her işletildiğinde yanan LED'in bir soldaki olması sağlanacaktır. Diğer while döngüsündeki mantık da aynıdır. Ancak buradaki şart **RD0=1** oluncaya kadar program akışının döngüye girmesidir. Sağa kaydırma işlemi için de >> komutu kullanılmaktadır.



```

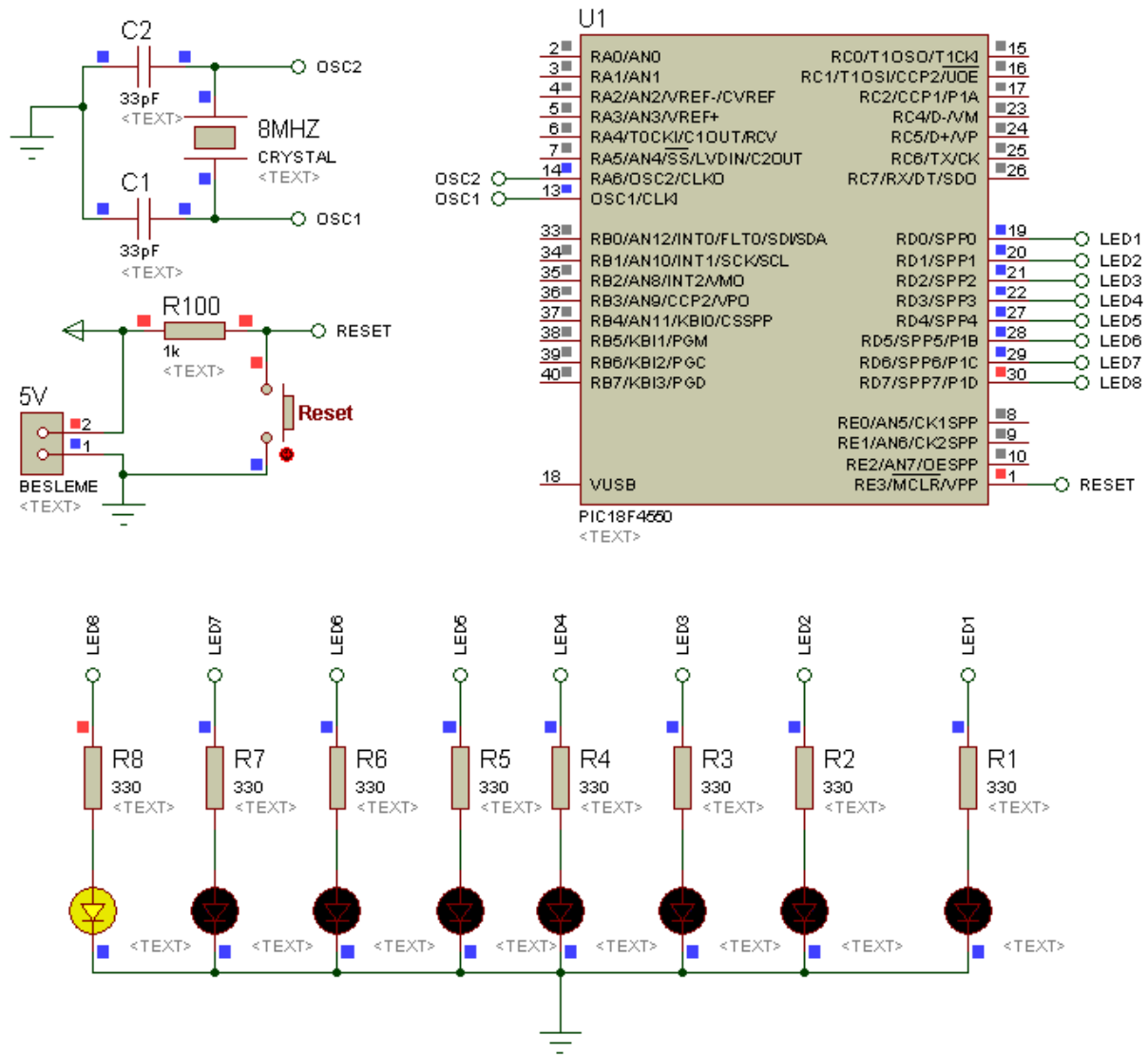
void main() {
    ADCON1 |= 0x0F;    //Tüm pinler dijital giriş/çıkış olarak ayarlandı
    CMCON  |= 7;       //Karşılaştırma modülü iptal edildi.
    TrisD=0;           //D portu çıkış olarak atandı
    PortD=1;           //D portunun ilk bitini (0. bitini) 1 yap
    while(1){          //Programı sonsuz döngüye sok
        //Sola kaydırma işlemi yaptırılacak
        while (portd < 0b10000000){ //PortD'nin değeri 0b10000000'dan küçükken
            //döngüye gir
            delay_ms(100);          //100ms bekle
            portd = portd << 1;     //D portundaki veriyi bir sola kaydır
        }
        //Sağa kaydırma işlemi yaptırılacak
        while (portd > 0b00000001){ //PortD'nin değeri 0b00000001'den büyükken
            //döngüye gir
            delay_ms(100);          //100ms bekle
            portd = portd >> 1;     //D portundaki veriyi bir sağa kaydır
        }
    }
}

```

Şekil 5.16 Sırayla sola ve sağa kayan LED uygulamasına ilişkin MikroC kodu ekran görüntüsü

AD SOYAD:

LED UYGULAMALARI



Şekil 5.17 Sırayla sola ve sağa kayan LED uygulamasına ilişkin ISIS şeması

UYGULAMA_09

Şimdiye kadar yaptığımız uygulamaların tümünde PIC'in her zaman tek portunu kullandık. Ancak bu uygulamada PIC'in tüm portları aynı program içerisinde kullanılacaktır. O nedenle PIC'e enerji verildiğinde A portunda 3. bite, B portunda 4. bite, C portunda 2. bite, D portunda ise 7. bite bağlı olan LED'leri hep birlikte yakan program yazılarak tüm portlar aynı program içerisinde çıkış olarak kullanılmış olacaktır.

İlgili program kodları aşağıda verilmiştir. Programın MikroC programındaki ekran görüntüsü ve devre şeması sırasıyla Şekil 5.18 ve Şekil 5.19'da görülmektedir. PIC'in enerjisi kesilene veya RESET butonuna basılınca kadar LED'lerin tamamı yanık kalmaya devam edecektir.

PROGRAM KODLARI:

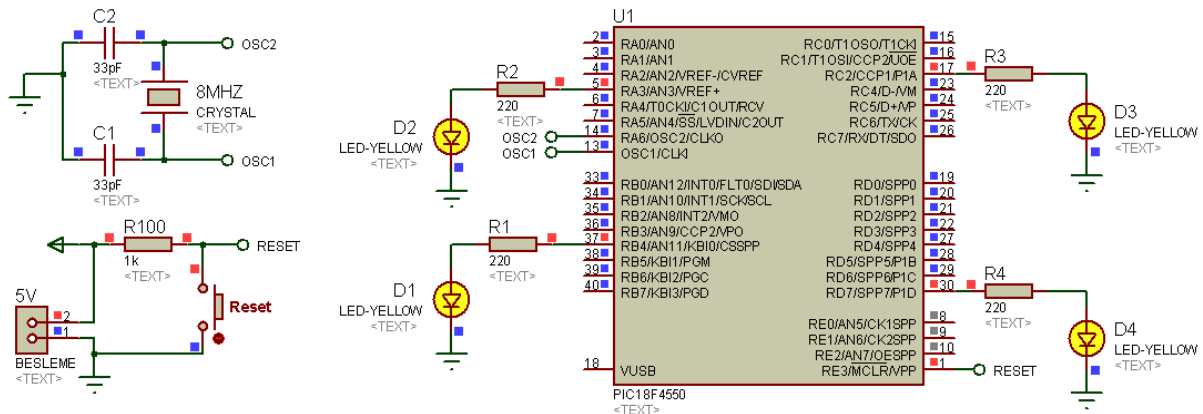
```
/* Tüm portların kullanıldığı aynı program içerisinde kullanımına ilişkin uygulama */  
//===== Program başlangıcı =====  
void main() {           //Ana program başlangıcı  
    ADCON1 |= 0x0F;     // PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı  
    CMCON  |= 7;        //Karşılaştırma (COMPARATOR) modülü iptal edildi.  
    trisA=0;            //A portunun tüm pinleri çıkış olarak ayarlandı  
    trisB=0;            //B portunun tüm pinleri çıkış olarak ayarlandı  
    trisC=0;            //C portunun tüm pinleri çıkış olarak ayarlandı  
    trisD=0;            //D portunun tüm pinleri çıkış olarak ayarlandı  
    porta.ra3=1;        // Sadece RA3'e bağlı LED yakıldı  
    portb.rb4=1;        // Sadece RB4'e bağlı LED yakıldı  
    portc.rc2=1;        // Sadece RC2'ye bağlı LED yakıldı  
    portd.rd7=1;        // Sadece RD7'ye bağlı LED yakıldı  
    while(1);           // Program akışı burada bekletildi (noktalı virgüle dikkat!)  
}                        //Ana programın bittiği, süslü parantez ile gösterildi.  
//===== Program sonu =====
```

```

StartPage LED_09.c
//===== Program başlangıcı =====
void main() { // Ana program başlangıcı
    ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON |= 7; //Karşılaştırma (COMPARATOR) modülü iptal edildi.
    trisA=0; //A portunun tüm pinleri çıkış olarak ayarlandı
    trisB=0; //B portunun tüm pinleri çıkış olarak ayarlandı
    trisC=0; //C portunun tüm pinleri çıkış olarak ayarlandı
    trisD=0; //D portunun tüm pinleri çıkış olarak ayarlandı
    porta.ra3=1; //Sadece RA3'e bağlı LED yakıldı
    portb.rb4=1; //Sadece RB4'e bağlı LED yakıldı
    portc.rc2=1; //Sadece RC2'ye bağlı LED yakıldı
    portd.rd7=1; //Sadece RD7'ye bağlı LED yakıldı
    while(1); //Program akışı burada bekletildi (noktalı virgüle dikkat!)
} //===== Program sonu =====

```

Şekil 5.18 Tüm portların aynı program içerisinde kullanılmasına ilişkin yapılan programın MikroC kodu ekran görüntüsü



Şekil 5.19 Tüm portların aynı program içerisinde kullanılmasına ilişkin yapılan uygulamanın ISIS şeması

UYGULAMA_10

Bu uygulamaya kadar öğrenilen bilgiler bu örnekte kullanılarak her porta bağlı LED'lerle yapılmış küçük bir animasyon programı sunulacaktır.

Programda ilk önce ayrı portlara bağlı LED'lerin tamamı yakılacaktır. Ardından program sonsuz döngüye girip FOR döngüsü kullanılarak LED'ler belli sayıda sırayla söndürülüp tekrar yakılacaktır. Nihai olarak tüm LED'ler belirli aralıklarla hep birlikte yakılıp söndürülecektir. Bu işlem de yine FOR döngüsü kullanılarak belli sayıda tekrarlanacaktır. Bu işlem sırası PIC'in enerjisi kesilene veya RESET butonuna basılıncaya kadar devam edecektir.

Programa ilişkin kodlar aşağıda verilmiştir. Uygulama devresi ise Şekil 5.20'de görülmektedir.

PROGRAM KODLARI:

```
/* Tüm portlara birer LED bağlanarak yapılan küçük bir animasyon uygulaması */
unsigned int i=0;      //For döngüsü için tanımlanan değişken
//===== Program başlangıcı =====
void main() {          //Ana program başlangıcı
    ADCON1 |= 0x0F;    //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |=7;        //Karşılaştırma (COMPARATOR) modülü iptal edildi.
    trisA=0;           //A portunun tüm pinleri çıkış olarak ayarlandı
    trisB=0;           //B portunun tüm pinleri çıkış olarak ayarlandı
    trisC=0;           //C portunun tüm pinleri çıkış olarak ayarlandı
    trisD=0;           //D portunun tüm pinleri çıkış olarak ayarlandı
    porta.ra3=1;       //Sadece RA3'e bağlı LED yakıldı
    portb.rb4=1;       //Sadece RB4'e bağlı LED yakıldı
    portc.rc2=1;       //Sadece RC2'ye bağlı LED yakıldı
    portd.rd7=1;       //Sadece RD7'ye bağlı LED yakıldı
    Delay_ms (500);    //500ms bekle
    while(1){          //Program sonsuz döngüye sokuldu
        for (i=0;i<3;i++) {
```

AD SOYAD:

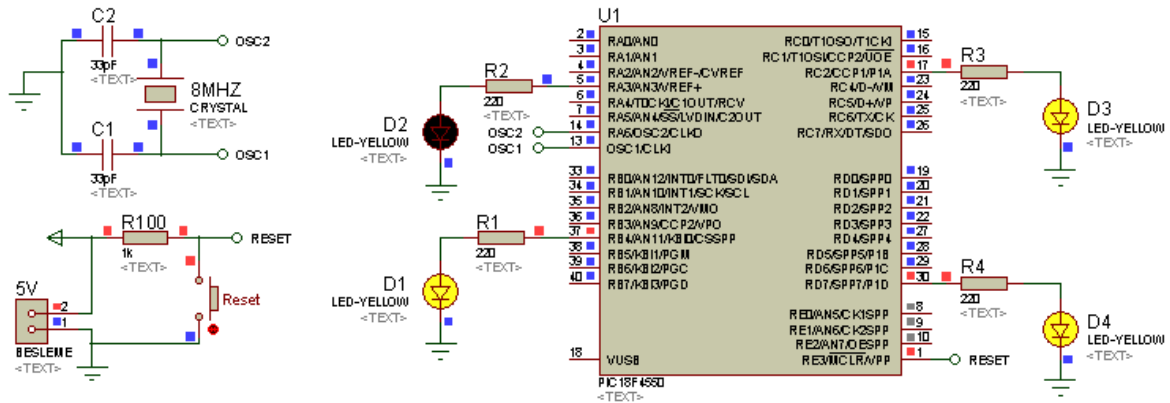
LED UYGULAMALARI

```
    porta.ra3=0;           //Sadece RA3'e bağlı LED söndürüldü
    Delay_ms(250);         // 250ms bekle
    portb.rb4=0;           //Sadece RB4'e bağlı LED söndürüldü
    Delay_ms(250);         //250ms bekle
    portc.rc2=0;           //Sadece RC2'ye bağlı LED söndürüldü
    Delay_ms(250);         // 250ms bekle
    portd.rd7=0;           //Sadece RD7'ye bağlı LED söndürüldü
    Delay_ms(250);         // 250ms bekle
    portd.rd7=1;           //Sadece RD7'ye bağlı LED yakıldı
    Delay_ms(250);         // 250ms bekle
    portc.rc2=1;           //Sadece RC2'ye bağlı LED yakıldı
    Delay_ms(250);         // 250ms bekle
    portb.rb4=1;           //Sadece RB4'e bağlı LED yakıldı
    Delay_ms(250);         // 250ms bekle
    porta.ra3=1;           //Sadece RA3'e bağlı LED yakıldı
    Delay_ms(250);         // 250ms bekle
}
for (i=0;i<4;i++)
{
    porta.ra3=0;           //Sadece RA3'e bağlı LED söndürüldü
    portb.rb4=0;           //Sadece RB4'e bağlı LED söndürüldü
    portc.rc2=0;           //Sadece RC2'ye bağlı LED söndürüldü
    portd.rd7=0;           //Sadece RD7'ye bağlı LED söndürüldü
    Delay_ms (250);        //250ms bekle
    porta.ra3=1;           //Sadece RA3'e bağlı LED yakıldı
    portb.rb4=1;           //Sadece RB4'e bağlı LED yakıldı
    portc.rc2=1;           //Sadece RC2'ye bağlı LED yakıldı
    portd.rd7=1;           //Sadece RD7'ye bağlı LED yakıldı
    Delay_ms (250);        //250ms bekle
}
```


AD SOYAD:

LED UYGULAMALARI

```
} //Sonsuz döngüye ait parantez  
}  
//===== Program sonu =====
```



Şekil 5.20 Yapılan animasyon programının kullanılacağı ISIS şeması

BÖLÜM 6

6. BUTON UYGULAMALARI

UYGULAMA_01

LED uygulamalarında ADCON1 ve CMCON yazmaçlarının kullanımı üzerinde durulduğu için bu bölümde tekrar değinilmeyecektir. Dolayısıyla bu bölümün temel hedefleri; buton bağlantısı durumunda TRIS yazmaçlarında nasıl bir değişikliğin olacağı, aynı porta buton ve LED bağlanması durumunda TRIS yazmacının nasıl yönlendirileceği ve farklı mikroC kodu kullanımlarının öğretilmesi olacaktır.

LED uygulamalarının yapıldığı bölümde portlara bağlanan elemanlar sadece LED'ler olduğu için TRIS yazmaçlarının yönlendirilmesinde ilgili bitlere hep lojik "0" (sıfır) yüklenmişti. Örneğin B portunun 1. bitine bir LED bağlanacaksa buna ilişkin yönlendirme **TrisB.RB1=0;** şeklinde yapılmıştı. Veya d portunun tümüne LED bağlanmışsa **Trisd=0xFF;** yapılmıştı. Çünkü portlardan aldığı bilgiye göre çalışacak olan LED veya transistör gibi elemanlar çıkış elemanı olarak adlandırılır ve TRIS yazmaçlarının yönlendirilmesinde portlara bağlanacak bu elemanların varlığı dikkate alınarak yönlendirme yapılır.

Ancak sensör, enkoder veya buton gibi elemanlar PIC'e bağlandıklarında bu elemanlar giriş elemanı olarak adlandırılır ve bunların PIC'e bağlanmaları durumunda ilgili TRIS yazmaçlarının ilgili bitlerine lojik "1" (bir) yüklenir.

Özetlenecek olursa; eğer PIC, kendisine bağlanan elemandan gelen bilgiye göre hareket edecekse bu elemanın (buton, sensör, enkoder vb) bağlandığı ilgili bit giriş olarak ayarlanır. Dolayısıyla ilgili TRIS yazmacının ilgili bitine lojik "1" (bir) yüklenir. Şayet PIC'e bağlı eleman PIC'den gelecek bilgiye göre çalışacaksa bu durumda o elemanın (LED, transistör, tristör vb) PIC'e bağlandığı bit çıkış olarak yönlendirilir. Dolayısıyla ilgili TRIS yazmacının ilgili bitine lojik "0" (sıfır) yüklenir.

Bu uygulamada D portunun 1. bitine bağlı butona basıldığında C portunun 2. bitine bağlı olan LED'i yakan, butona basılmadığında ise söndüren program üzerinde durulacaktır.

İlgili program kodları aşağıda verilmiştir. Programın MikroC programındaki görüntüsü ve devre şeması sırasıyla Şekil 6.1 ve Şekil 6.2’de görülmektedir.

PROGRAM KODLARI:

*/*Bu uygulama D portunun 1. bitine bağlı butona basıldığında C portunun 2. bitine bağlı olan LED’i yakan, butona basılmadığında ise söndüren programdır.*/*

//===== Program başlangıcı =====

```
void main() {           // Ana program başlangıcı
    ADCON1 |= 0x0F;      //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |=7;          //Karşılaştırma (COMPARATOR) modülü iptal edildi.
    TRISD1_bit=1;        //D portunun 1. biti GİRİŞ yapıldı. Çünkü BUTON bağlanacak.
    TRISC2_bit=0;        //C portunun 2. biti ÇIKIŞ yapıldı. Çünkü LED bağlanacak.
    RD1_bit= 0;          //RD1=0 yapıldı ("Henüz butona basılmadı" yapıldı)
    RC2_bit=0;           //RC2'ye bağlı LED sönmük ayarlandı (LED yanmıyor yapıldı)
    while(1){           //Programı sonsuz döngüye sok
        if (RD1_bit==1)  //Eğer RD1=1 ise yani butona basılmışsa aşağıdaki işlemleri yap
        {
            RC2_bit =1;   //RC2'ye bağlı LED'i yak
            while(RD1_bit=1); //RD1=1 iken yani butona basılıyken bekle*/
        }
        RC2_bit=0;        //Butona basılmadığı için LED'i söndür
    }                     //Sonsuz döngüye ait parantez
}                         //Ana program sonlandırıldı.
```

//===== Program sonu =====

```

//===== Program başlangıcı =====
void main() { // Ana program başlangıcı
    ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON |= 7; //Karşılaştırma (COMPARATOR) modülü iptal edildi.
    TRISD1_bit=1; //D portunun 1. biti GİRİŞ yapıldı
    TRISC2_bit=0; //C portunun 2. biti ÇIKIŞ yapıldı
    RD1_bit= 0; //RD1=0 yapıldı ("Henüz butona basılmadı" yapıldı)
    RC2_bit=0; //RC2'ye bağlı LED sönmük ayarlandı (LED yanmıyor yapıldı)
    while(1){ //Programı sonsuz döngüye sok
        if (RD1_bit==1) //Eğer RD1=1 ise yani butona basılmışsa aşağıdaki işlemleri yap
        {
            RC2_bit =1; //RC2'ye bağlı LED'i yak
            while(RD1_bit=1); //RD1=1 iken yani butona basılıyken bekle*/
        }
        RC2_bit=0; //Butona basılmadığı için LED'i söndür
    } //Sonsuz döngüye ait parantez
} //Ana program sonlandırıldı.
//===== Program sonu =====

```

Şekil 6.1 Butona basılıyken bir LED'in yakılmasına ilişkin MikroC programının ekran görüntüsü

Program kodları incelendiğinde daha önceki bölümde hiç kullanmadığımız iki kullanım göze çarpacaktır. Önceki uygulamalarımızda tek bitin çıkış olarak yönlendirilmesini **trisC.rC5=0;** yapısını kullanarak yapmıştık. Ancak bu uygulamada **TRISC2_bit=0;** yazımı tercih edilmiştir.

Aynı şekilde herhangi bir portun bir tek bitinin durumunu ifade ederken örneğin **portD.RD1=1;** şeklinde komut yazmıştık. Ancak onun yerine bu uygulamada **RD1_bit= 0;** ifadesi kullanılmıştır. Dolayısıyla bunlar MikroC'nin kullanıcıya sunduğu çeşitli yazım kolaylıklarıdır.

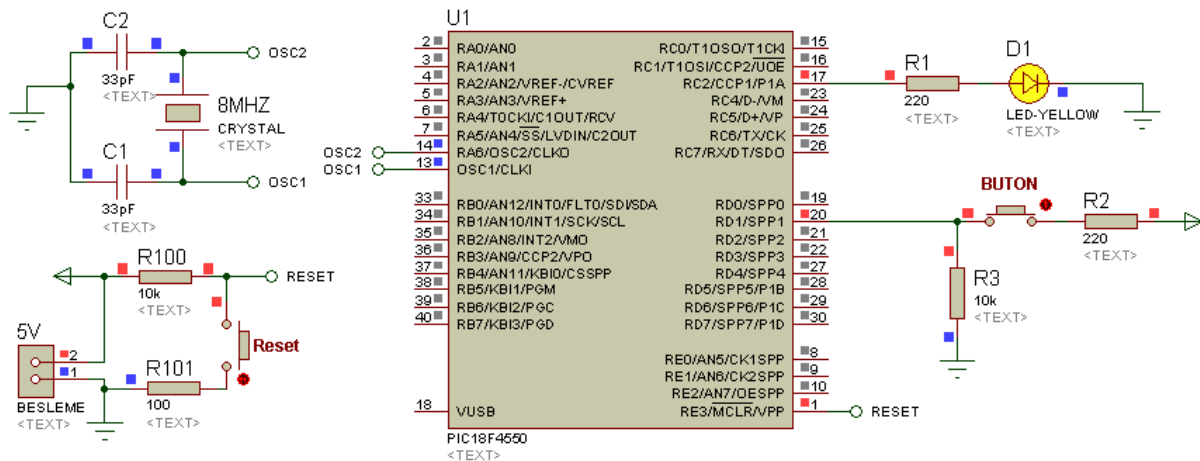
if kontrol yapısı da daha önceki LED uygulamalarında kullanmadığımız bir yapıdır. Buton uygulamalarında sıkça karşılaşılabilecek olan bu kontrol komutu, yazılan şartın sağlanması durumunda iki süslü parantez arasında belirtilen işlemleri yerine getirecektir.

Yukarıdaki program kodları incelendiğinde butona basılıp basılmadığı, **if** kontrol yapısı kullanılarak kontrol edilmiştir. **if (RD1_bit==1)** yazılmak suretiyle program RD1 bitinin 1 olup olmadığını kontrol eder. Aşağıdaki devre şeması incelendiğinde şayet butona basılmışsa RD1 bitine lojik 1 gelecektir. Dolayısıyla şart sağlanmış olduğundan program şartın yazılımından

BUTON UYGULAMALARI

sonra belirtilen iki süslü parantez içerisindeki işlemleri yerine getirecektir. Yani ilgili LED’i yakacaktır.

while(RD1_bit=1); ifadesi ise butona basılıyken programın burada beklemesini sağlar. Bu komut yazılmadığı takdirde program akışı devam edeceğinden sonraki satırda bulunan komut gereği LED sönecektir. Ama sonsuz döngü içerisinde bulunulduğundan daha biz butondan elimizi çekmeden **if** şartı tekrar sağlandığı için LED tekrar yanacaktır. Dolayısıyla gerçek uygulamada LED'in ya tamamen yandığı ya da tamamen söndüğü veya LED'in kararsız çalıştığı görülecektir. O nedenle bu tür buton uygulamalarında butona basıldığında işlem bir kez yapılsın diye böyle bir bekleme komutu kullanılır ve programın bu noktadan öteye gitmesi engellenir.



Şekil 6.2 Butona basılıyken bir LED'in yakılmasına ilişkin ISIS şeması

UYGULAMA_02

Bu uygulamada bir önceki uygulamaya çok benzeyen bir örnek üzerinde durulacaktır. Ancak burada farklı olarak birden fazla LED'in butonla kontrol edilmesi ve butona her basıldığında LED'lerin durumlarının terslenmesi söz konusudur. Yani butona her basıldığında LED'ler sönmükse yanacak, yanıyorsa sönecektir.

Bu uygulamadaki buton da bir önceki örnekte olduğu gibi RD1 pinine bağlanmıştır. Ancak LED'ler B portuna bağlanmıştır. İlgili program kodları aşağıda görülmektedir. Bu uygulamadaki buton kontrolü, butona basılması durumunda ilgili bitin 1 olacağı dikkate alınarak yapılmıştır. Uygulamanın MikroC programındaki ekran görüntüsü ve devre şeması sırasıyla Şekil 6.3 ve Şekil 6.4'te verilmiştir.

PROGRAM KODLARI:

*/*D portunun 1. bitine bağlı butona her basıldığında B portundaki LED'lerin durumunu tersleyen program.*/*

//===== Program başlangıcı =====

```
void main() {           // Ana program başlangıcı
    ADCON1 |= 0x0F;     //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |=7;         //Karşılaştırma (COMPARATOR) modülü iptal edildi.
    TRISD1_bit=1;       //Sadece D portunun 1. biti GİRİŞ yapıldı (BUTON bağlanacak)
    TRISB=0;            //B portunun tamamı ÇIKIŞ yapıldı (LED bağlanacak)
    RD1_bit= 0;         //RD1=0 yapıldı ("Henüz butona basılmadı" yapıldı)
    PORTB=0;            //B portuna bağlı tüm LED'ler yanmıyor olarak ayarlandı
    while (1) {         //Programı sonsuz döngüye sok
        if (RD1_bit==1) //Eğer RD1=1 ise yani butona basılmışsa aşağıdaki işlemleri yap
        {
            PORTB=~PORTB; //B portunun durumunu tersle (LED'ler sönmükse yak, yanıyorsa
                           // söndür)
            while (RD1_bit=1); //RD1=1 iken yani butona basılıyken bekle*/
        }
    }
```

```

    }                                //Sonsuz döngüye ait parantez
}                                    //Ana program sonlandırıldı.
//===== Program sonu =====

```

Butona her basıldığında LED'lerin durumunun terslenmesi **if** kontrol yapısı içerisine yazılan **PORTB=~PORTB;** komutu ile sağlanmıştır. Şart sağlandığında yani butona her basıldığında LED'lerin durumu terslenecektir. Bir önceki örnekte olduğu gibi burada da butona basıldığında sadece bir kez işlem yapılması için **while (RD1_bit=1);** komutu kullanılmıştır.

```

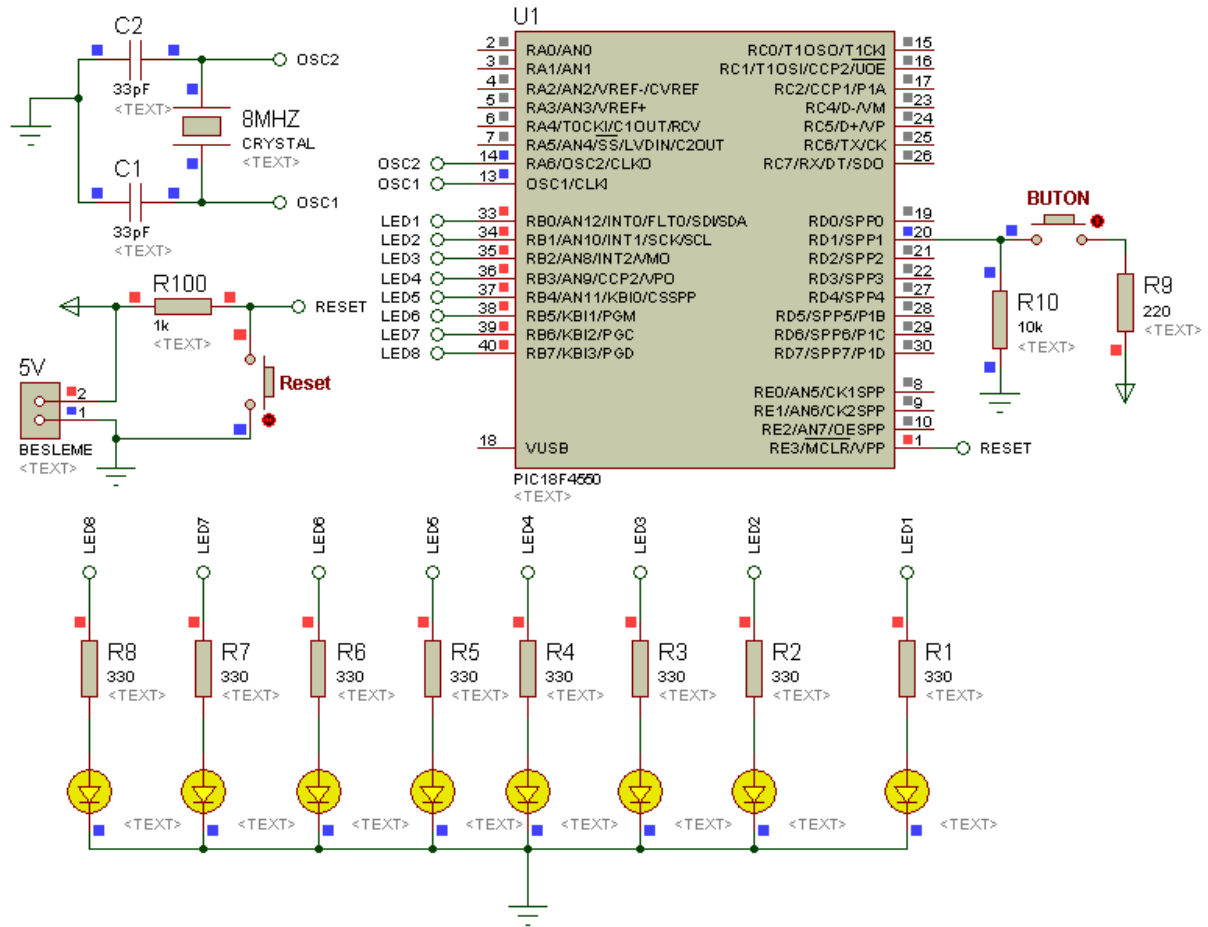
//===== Program başlangıcı =====
void main() {                                // Ana program başlangıcı
    ADCON1 |= 0x0F;                          //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |=7;                              //Karşılaştırma (COMPARATOR) modülü iptal edildi.
    TRISD1_bit=1;                            //Sadece D portunun 1. biti GİRİŞ yapıldı (BUTON bağlanacak)
    TRISB=0;                                 //B portunun tamamı ÇIKIŞ yapıldı (LED bağlanacak)
    RD1_bit= 0;                              //RD1=0 yapıldı ("Henüz butona basılmadı" yapıldı)
    PORTB=0;                                 //B portuna bağlı tüm LED'ler yanmıyor olarak ayarlandı
    while(1){                                //Programı sonsuz döngüye sok
        if (RD1_bit==1)                     //Eğer RD1=1 ise yani butona basılmışsa aşağıdaki işlemleri yap
        {
            PORTB=~PORTB;                  //B portunun durumunu tersle
            while(RD1_bit=1);              //RD1=1 iken yani butona basılıyken bekle*/
        }
        //Sonsuz döngüye ait parantez
    }                                        //Ana program sonlandırıldı.
//===== Program sonu =====

```

Şekil 6.3 Butona her basıldığında LED'lerin durumunu tersleyen programa ilişkin MikroC programının ekran görüntüsü

AD SOYAD:

BUTON UYGULAMALARI



Şekil 6.4 Butona her basıldığında LED'lerin durumunu tersleyen programa ilişkin ISIS şeması

UYGULAMA_03

Bu uygulamada A portunun 3. bitine bağlı butona her basıldığında, B portundaki LED'lerin tamamını belirli aralıklarla 10 kez yakıp söndüren program verilecektir. İlgili program kodları aşağıda sunulmuştur.

PROGRAM KODLARI:

```
//===== Program başlangıcı =====
unsigned int i=0;          //FOR döngüsünde kullanılmak üzere tanımlanan değişken
void main() {              // Ana program başlangıcı
    ADCON1 |= 0x0F;        //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |=7;            //Karşılaştırma (COMPARATOR) modülü iptal edildi.
    TRISA3_bit=1;          //Sadece A portunun 3. biti GİRİŞ yapıldı (BUTON bağlanacak)
    TRISB=0;               //B portunun tamamı ÇIKIŞ yapıldı (LED bağlanacak)
    RA3_bit= 0;            //RA3=0 yapıldı ("Henüz butona basılmadı" yapıldı)
    PORTB=0;               //B portuna bağlı tüm LED'ler yanmıyor olarak ayarlandı
    while(1){              //Programı sonsuz döngüye sok
        if (RA3_bit==1)    //Eğer RA3=1 ise yani butona basılmışsa aşağıdaki işlemleri yap
        {                  //if kontrol şartı gerçekleştiğinde program akışı bu parantezin içine
                            //girecek
            do {            // do-while döngüsüne girildi
                PORTB=~PORTB; //B portunun durumunu tersle
                Delay_ms(200); //LED'lerin mevcut durumunu koruması için 200ms bekle
                i++;          // i'nin değerini 1 artır
            } while(i<20);    // i'nin aldığı değer 20'dan küçükken yukarıdakileri (do'dan
                            //sonrakileri) yap
                i=0;          //Butona tekrar basıldığında sistemin istenildiği gibi tekrar
                            //çalışması için bu
                            // değişkenin sıfırlanması gerekir. Aksi halde 2. ve sonraki butona
                            //basmalarda //LED'ler sadece bir kez terslenir!
            while(RA3_bit=1); //RA3=1 iken yani butona basılıyken bekle*/
    }
```

```

    }                      //if kontrol yapısı için açılan parantez kapatıldı
}                          //Sonsuz döngüye ait parantez
}                          //Ana program sonlandırıldı.
//===== Program sonu =====

```

Program kodları incelendiğinde şimdiye kadarki uygulamaların hiçbirinde kullanmadığımız bir kontrol döngüsünün burada kullanıldığı görülecektir. Bu örnekte **if** kontrol yapısının içerisinde kullandığımız **do-while** kontrol yapısı, diğer **while** döngüsünden farklı olarak program akışının en az bir kez bu döngünün içine girmesini sağlayan bir özelliğe sahiptir. Bu döngünün kullanımında **do** komutundan sonra iki süslü parantez içerisine yazılan komutlar **while** ile belirtilen şarta bakılmaksızın en az bir kez işletilir. Eğer **while** ile belirtilen şart sağlanıyorsa, şart sağlandığı sürece tekrar **do** satırına geri dönülür ve iki süslü parantez arasındaki işlemler şart bozuluncaya kadar tekrarlanır. Döngünün tekrarlanma sayısı için tanımlanan **i** değişkeninin de döngü içerisinde bir arttırılması gerektiği burada unutulmaması gereken önemli bir noktadır.

Yine program kodları incelendiğinde **do-while** döngüsünün tekrarlama şartı olarak **while(i<20);** yazılmıştır. Yani **i** değişkeninin 20'den küçük olması gerektiği belirtilmiştir. Buradaki 20 değeri, B portundaki LED'lerin 10 kez yakılıp söndürülmesi istendiği içindir. Her yakıp söndürme ayrı bir tersleme işlemi ile yapıldığı için LED'lerin 10 kez yanıp sönmesi için B portunun 20 kez terslenmesi gerekir. Dolayısıyla 20 değerine bu şekilde ulaşılmıştır.

do-while döngüsünden çıktıktan sonra **i** değişkeninin **i=0;** yazılarak sıfırlanması gerekir. Aksi halde 2. ve sonraki butona basmalarda sistem istenildiği gibi çalışmayıp LED'ler sadece bir kez terslenecektir.

Uygulamanın MikroC programındaki ekran görüntüsü ve devre şeması sırasıyla Şekil 6.5 ve Şekil 6.6'da verilmiştir.

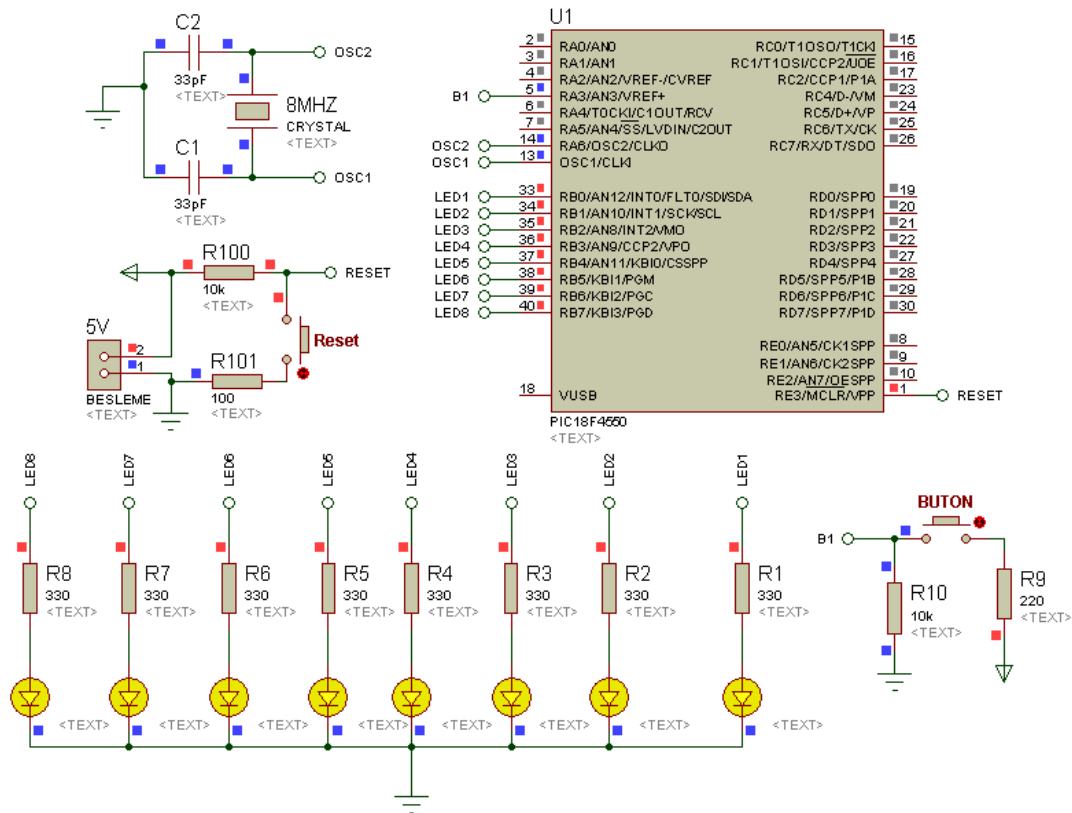
Uygulamayı gerçekleştirip butona bastığınızda, elinizi henüz butondan çekmeden sistemin çalışmaya başladığını göreceksiniz. Burada sistemin butondan elinizi çektikten sonra çalışması için nasıl bir değişiklik yapılması gerektiğini irdelleyiniz...

```

Start Page  Buton_03.c
//===== Program başlangıcı =====
unsigned int i=0; //do-while döngüsünde kullanılmak üzere tanımlanan değişken
void main() { // Ana program başlangıcı
    ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON |=7; //Karşılaştırma (COMPARATOR) modülü iptal edildi.
    TRISA3_bit=1; //Sadece A portunun 3. biti GİRİŞ yapıldı (BUTON bağlanacak)
    TRISB=0; //B portunun tamamı ÇIKIŞ yapıldı (LED bağlanacak)
    RA3_bit= 0; //RA3=0 yapıldı ("Henüz butona basılmadı" yapıldı)
    PORTB=0; //B portuna bağlı tüm LED'ler yanmıyor olarak ayarlandı
    while(1){ //Programı sonsuz döngüye sok
        if (RA3_bit==1) //Eğer RA3=1 ise yani butona basılmışsa aşağıdaki işlemleri yap
        {
            //if kontrol yapısı için açılan parantez
            do{ //do-while döngüsüne girildi
                PORTB=~PORTB; //B portunun durumunu tersle
                Delay_ms(200); //LED'lerin mevcut durumunu koruması için 200ms bekle
                i++; //i'nin değerini 1 artır
            } while(i<20); //i'nin aldığı değer 20'dan küçükken yukarıdakileri
            //i'nin aldığı değeri 20'dan küçükken yukarıdakileri
            //do-while döngüsüne girildi
            i=0; //Butona tekrar basıldığında sistemin istenildiği gibi
            //tekrar çalışması için bu değişkenin sıfırlanması gerekir.
            //Aksi halde 2. ve sonraki butona basmalarda LED'ler sadece
            //bir kez terslenir!
            while(RA3_bit=1); //RA3=1 iken yani butona basılıyken bekle*/
        } //if kontrol yapısı için açılan parantez kapatıldı
        //Sonsuz döngüye ait parantez
        //Ana program sonlandırıldı.
    } //===== Program sonu =====
}

```

Şekil 6.6 Butona her basıldığında LED'leri 10 kez yakıp söndüren programa ilişkin MikroC programının ekran görüntüsü



Şekil 6.7 Butona her basıldığında LED'leri 10 kez yakıp söndüren programa ilişkin ISIS şeması

UYGULAMA_04

Bu uygulamada aynı portun hem giriş hem de çıkış olarak nasıl kullanılacağı, buton kullanılarak yapılan bir animasyon sayesinde gösterilmiştir. Ayrıca şimdiye kadarki uygulamalarımızın hiçbirinde kullanmadığımız **if-else** kontrol yapısı da burada kullanılmıştır. Bu yapı kullanılarak gereksiz kontrol süreleri kısaltılmakta ve böylelikle programın toplamdaki bir döngü süresi kısaltılmaktadır. Ayrıca bu programda, daha önceki programlarımızda kullanmadığımız bir değişken türü olan **char** tipi bir değişken tanımlanmıştır. Bu tür değişkenin alabileceği en büyük değer 255 olabilmektedir.

Uygulamada B portunun 7. bitine buton, diğer bitlerine LED'ler bağlanmıştır. Butona her basıldığında LED'ler sırasıyla önce sola kayar. RB6'ya bağlı LED yandıktan sonra butona basılmaya devam edilirse bu kez LED'lerin hepsi birlikte yanar ve söner.

Butona basmaya devam edildiğinde LED'ler sağa doğru yanarak kayar. 0. bite bağlı LED yandığında butona bir daha basılırsa bu kez LED'ler tekrar hepsi birlikte yanar ve söner. Nihayetinde program tekrar en başa dönmüş olur.

İlgili program kodları aşağıda sunulmuştur. Uygulamanın MikroC programındaki ekran görüntüsü ve devre şeması ise sırasıyla Şekil 6.7 ve Şekil 6.8'de verilmiştir.

PROGRAM KODLARI:

```
//===== Program başlangıcı =====
char m=0;           //Maksimum değeri 255 olabilecek bir değişken tanımlandı
void main() {       //Ana program başlangıcı
    ADCON1 |= 0x0F;  //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |= 7;     //Karşılaştırma modülü iptal edildi.
    TRISB=0b10000000; //BUTON için 7. bit GİRİŞ, LED'ler için diğerleri ÇIKIŞ yapıldı
    PORTB=0;         //Tüm LED'ler sönmük ve butona basılmıyor olarak ayarlandı
    while(1){        //Program sonsuz döngüye sokuldu
        if (RB7_bit==1) m++;    //Eğer RB7=1 ise yani butona basılmışsa m'yi bir artır
        while(RB7_bit=1);      //Butona basılırken burada bekle
```

```

if(m==1) portb=0b00000001;    //Eğer m=1 ise sadece RB0=1 yap ve kontrolü bırak
else if (m==2) portb=0b00000010; //Eğer m=2 ise sadece RB1=1 yap ve kontrolü bırak
else if (m==3) portb=0b00000100; //Eğer m=3 ise sadece RB2=1 yap ve kontrolü bırak
else if (m==4) portb=0b00001000; //Eğer m=4 ise sadece RB3=1 yap ve kontrolü bırak
else if (m==5) portb=0b00010000; //Eğer m=5 ise sadece RB4=1 yap ve kontrolü bırak
else if (m==6) portb=0b00100000; //Eğer m=6 ise sadece RB5=1 yap ve kontrolü bırak
else if (m==7) portb=0b01000000; //Eğer m=7 ise sadece RB6=1 yap ve kontrolü bırak
else if (m==8) portb=127;      //Eğer m=8 ise tüm LED'leri yak ve kontrolü bırak
else if (m==9) portb=0;        //Eğer m=9 ise tüm LED'leri söndür ve kontrolü bırak
else if (m==10) portb=0b01000000; //Eğer m=10 ise sadece RB6=1 yap ve kontrolü bırak
else if (m==11) portb=0b00100000; //Eğer m=11 ise sadece RB5=1 yap ve kontrolü bırak
else if (m==12) portb=0b00010000; //Eğer m=12 ise sadece RB4=1 yap ve kontrolü bırak
else if (m==13) portb=0b00001000; //Eğer m=13 ise sadece RB3=1 yap ve kontrolü bırak
else if (m==14) portb=0b00000100; //Eğer m=14 ise sadece RB2=1 yap ve kontrolü bırak
else if (m==15) portb=0b00000010; //Eğer m=15 ise sadece RB1=1 yap ve kontrolü bırak
else if (m==16) portb=0b00000001; //Eğer m=16 ise sadece RB0=1 yap ve kontrolü bırak
else if (m==17) portb=127;      //Eğer m=17 ise tüm LED'leri yak ve kontrolü bırak
else {portb=0;m=0;}            //Yukarıdaki hiçbir koşul sağlanmıyorsa tüm LED'leri
                                //söndür ve en başa dönmek için m değişkenini sıfırla
}                                //Sonsuz döngüye ait parantez
}                                //Ana program sonlandırıldı.
//===== Program sonu =====

```

Program kodları incelendiğinde **TRISB=0b10000000**; yazmak suretiyle B portunun 7. bitinin giriş, diğer bitlerinin çıkış olarak ayarlandığı görülecektir. Çünkü RB7’de buton, diğer bitlerde ise LED’ler bağlıdır. Hatırlanacağı üzere butonlar PIC’e bağlandığında butonun bağlandığı bit, ilgili TRIS yazmacının ilgili biti **lojik 1** yapılmak suretiyle giriş olarak yönlendirilmişti. LED’lerin bağlı olduğu bitler ise ilgili TRIS yazmacının ilgili bitlerine **lojik 0** vermek suretiyle çıkış şeklinde yönlendirilmekteydi.


```

StartPage  Buton_04.c
char m=0; //Maksimum değeri 255 olabilecek bir değişken tanımlandı
void main() { //Ana program başlangıcı
    ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON |= 7; //Karşılaştırma modülü iptal edildi.
    TRISB=0b10000000; //BUTON için 7. bit GİRİŞ, LED'ler için diğerleri ÇIKIŞ yapıldı
    PORTB=0; //Tüm LED'ler sönmük ve butona basılmıyor olarak ayarlandı
    while(1){ //Program sonsuz döngüye sokuldu
        if (RB7_bit==1) m++; //Eğer RB7=1 ise yani butona basılmışsa m'yi bir artır
        while(RB7_bit=1); //Butona basılırken burada bekle
        if(m==1) portb=0b00000001; //Eğer m=1 ise sadece RB0=1 yap ve kontrolü bırak
        else if (m==2) portb=0b00000010; //Eğer m=2 ise sadece RB1=1 yap ve kontrolü bırak
        else if (m==3) portb=0b00000100; //Eğer m=3 ise sadece RB2=1 yap ve kontrolü bırak
        else if (m==4) portb=0b00001000; //Eğer m=4 ise sadece RB3=1 yap ve kontrolü bırak
        else if (m==5) portb=0b00010000; //Eğer m=5 ise sadece RB4=1 yap ve kontrolü bırak
        else if (m==6) portb=0b00100000; //Eğer m=6 ise sadece RB5=1 yap ve kontrolü bırak
        else if (m==7) portb=0b01000000; //Eğer m=7 ise sadece RB6=1 yap ve kontrolü bırak
        else if (m==8) portb=127; //Eğer m=8 ise tüm LED'leri yak ve kontrolü bırak
        else if (m==9) portb=0; //Eğer m=9 ise tüm LED'leri söndür ve kontrolü bırak
        else if (m==10) portb=0b01000000; //Eğer m=10 ise sadece RB6=1 yap ve kontrolü bırak
        else if (m==11) portb=0b00100000; //Eğer m=11 ise sadece RB5=1 yap ve kontrolü bırak
        else if (m==12) portb=0b00010000; //Eğer m=12 ise sadece RB4=1 yap ve kontrolü bırak
        else if (m==13) portb=0b00001000; //Eğer m=13 ise sadece RB3=1 yap ve kontrolü bırak
        else if (m==14) portb=0b00000100; //Eğer m=14 ise sadece RB2=1 yap ve kontrolü bırak
        else if (m==15) portb=0b00000010; //Eğer m=15 ise sadece RB1=1 yap ve kontrolü bırak
        else if (m==16) portb=0b00000001; //Eğer m=16 ise sadece RB0=1 yap ve kontrolü bırak
        else if (m==17) portb=127; //Eğer m=17 ise tüm LED'leri yak ve kontrolü bırak
        else {portb=0;m=0;} //Yukarıdaki hiçbir koşul sağlanmıyorsa tüm LED'leri
        //söndür ve en başa dönmek için m değişkenini sıfırla
    } //Sonsuz döngüye ait parantez
} //Ana program sonlandırıldı.

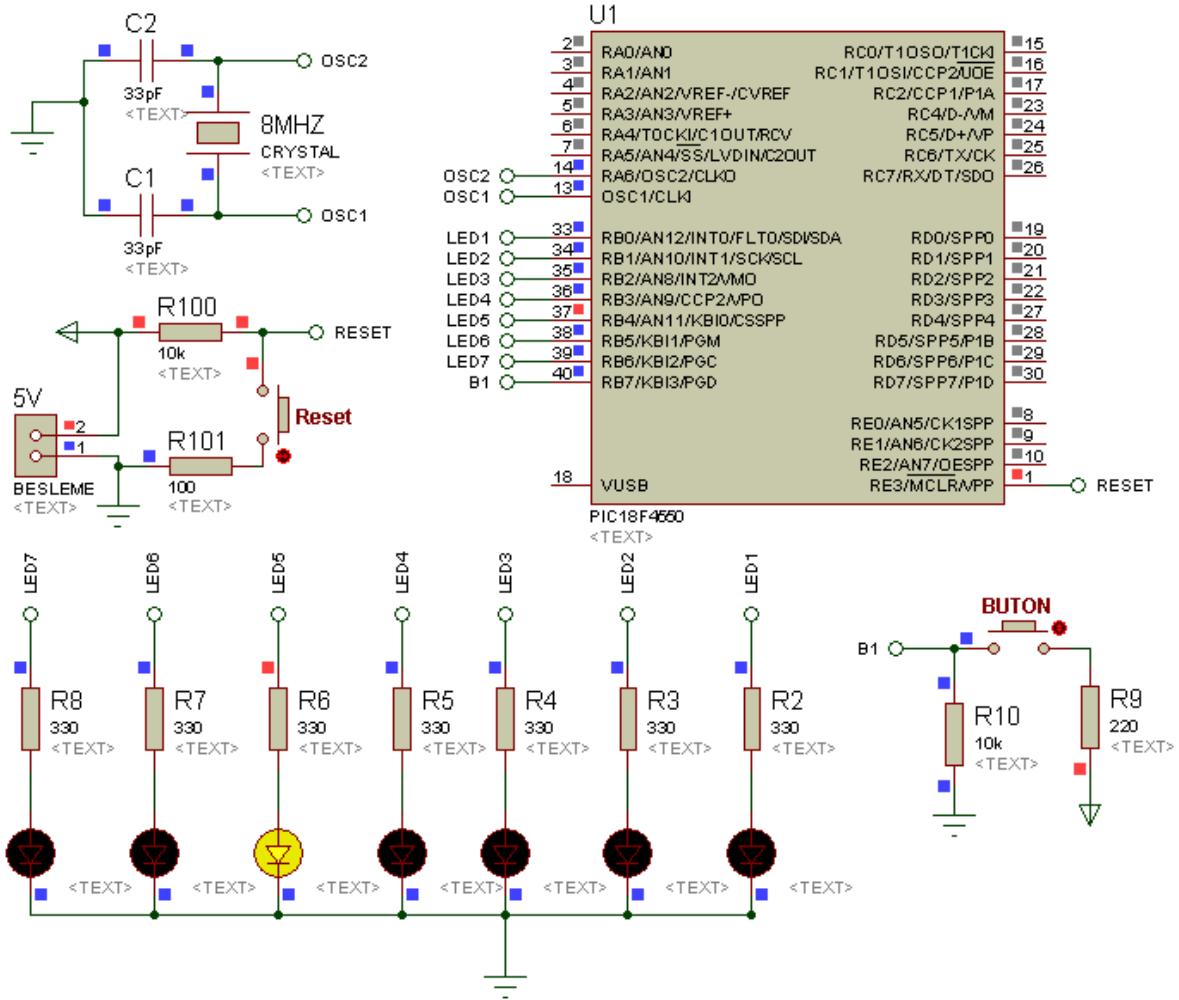
```

Şekil 6.7 Buton kullanılarak yapılan animasyon kodlarına ilişkin MikroC programının ekran görüntüsü

if-else kontrol yapısı bizim programımızda 16. satırda başlayıp 33. satırda bitmiştir. Bu kontrol yapısında bizim programımız için *m* değişkeninin aldığı değer denetlenmiştir. Anlaşılacağı üzere butona her basıldığında *m*'nin değeri bir artmaktadır.

16. satırdaki *if(m==1) portb=0b00000001;* komut dizisinde (*m==1*) ifadesi (yani *m=1* ise) bizim 1. koşulumuz, *portb=0b00000001;* ise bu koşul sağlandığında yapılacak işlemidir. Eğer bu satırdaki koşul sağlanmazsa hemen alt satırdaki yani 17. satırdaki *else if (m==2) portb=0b00000010;* satırı işletilir. Buradaki *else if* ifadesinde koşul olarak *m*'nin 2'ye eşit olması belirtilmiş, şart sağlandığında ise B portunun yeni durumunda sadece RB1'in 1 olacağı belirtilmiştir. Yani yanan LED bir sola kaymıştır. Şayet bu satırdaki şart da sağlanmazsa bu durumda hemen alt satırdaki koşula bakılır. Bu işleyiş bu şekilde alt satırlara doğru devam etmektedir. Ama şart sağlanmışsa, şartın sağlanması durumunda yapılması gereken işlem o

satırda gerçekleştirilir ve program alttaki diğer koşullara bakmadan en sondaki **else** satırının altından devam eder. Dikkat edilirse **else** satırında herhangi bir koşul bulunmamaktadır.



Şekil 6.8 Buton kullanılarak yapılan animasyon kodlarının uygulanacağı ISIS devre şeması

UYGULAMA_05

Bu uygulamada 3 buton kullanılmış olup bunlar A portunun 1, 2 ve 3. bitlerine bağlanmıştır. RA1'e bağlı butona (Buton-1'e) basıldığında B portundaki ilk 2 LED, RA2'ye bağlı butona (Buton-2'ye) basıldığında B portundaki son 2 LED, RA3'e bağlı butona (Buton-3'e) basıldığında ise B portunun tam ortasındaki 4 LED yanacaktır. Eğer 3 butona birden basılmışsa bu durumda D portunun tamamı ile B portundaki LED'lerin tek numaralı bitlerine bağlı olan LED'ler yanacaktır. Fakat hiçbir butona basılmamışsa bu durumda tüm LED'ler sönük kalacaktır.

Bir tane butona basılması durumunda herhangi bir LED veya LED grubunun yakılması, şimdiye kadar yaptığımız örnekler sayesinde kolaylıkla gerçekleştirilebilir. Ancak aynı anda birden fazla butona basılmasına göre farklı bir eylemin gerçekleştirilmesini sağlamak için **switch- case** dışında **if** veya diğer kontrol yapılarını kullanmak çok daha uzun programlar yazmayı gerektirir. O nedenle bu uygulamada aynı anda butona basıldığında yapılması gerekenler eylemler olduğu için, butonlara tek tek veya hep birlikte basılması durumunda gerçekleşecek tüm eylemler **switch-case** yapısı kullanılarak sağlanmıştır.

İlgili program kodları aşağıda sunulmuştur. Uygulamanın MikroC programındaki ekran görüntüsü ve devre şeması ise sırasıyla Şekil 6.9 ve Şekil 6.10'da verilmiştir.

PROGRAM KODLARI:

```
//===== Program başlangıcı =====
void main() { //Ana program başlangıcı
    ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON |= 7; //Karşılaştırma modülü iptal edildi.
    TRISA=0xFF; //Butonlar için A portunun tamamı GİRİŞ yapıldı
    TRISB=0; TRISD=0; //B ve D portları çıkış olarak ayarlandı
    PORTA=0; //Butonlara basılmıyor olarak ayarlandı
    PORTB=0; PORTD=0; //Tüm LED'ler sönük olarak ayarlandı
    while(1){ //Program sonsuz döngüye sokuldu
        switch (porta){ //A portunu aşağıdaki durumlar için kontrol et
```

```

case 0b00000010:    //Sadece Buton-1'e basılmışsa hemen alt satıra geç
    portb=0b00000011; //B portundaki ilk iki LED'i yak
    break;          //Bu döngüdeki kontrolü bırak
case 0b00000100:    //Sadece Buton-2'ye basılmışsa hemen alt satıra geç
    portb=0b11000000; //B portundaki son iki LED'i yak
    break;          //Bu döngüdeki kontrolü bırak
case 0b00001000:    //Sadece Buton-3'e basılmışsa hemen alt satıra geç
    portb=0b00111100; //B portunun tam ortasındaki 4 LED'i yak
    break;          //Bu döngüdeki kontrolü bırak
case 0b00001110:    //Butonların hepsine basılmışsa hemen alt satıra geç
    portb=0xAA;      //B portuna bağlı LED'leri birer boşluk bırakarak yak
    portd=0xFF;      //D portuna bağlı tüm LED'leri yak
    break;          //Bu döngüdeki kontrolü bırak
default:            //Butona basılma durumu yukarıdakilere uymuyorsa alt satıra geç
    portB=0;portD=0; //B ve D portuna bağlı LED'leri söndür
    break;          //Bu döngüdeki kontrolü bırak
}                  //switch yapısı için açılan parantez kapatıldı
}                  //Sonsuz döngüye ait parantez
}                  //Ana program sonlandırıldı.
//===== Program sonu =====

```

switch-case yapısı yukarıdaki program kodlarında da görüleceği üzere sonsuz döngü içerisinde kullanılmıştır. **switch (porta)** yazmak suretiyle A portunun durumlarının dikkate alınacağı belirtilmiştir. A portunun belirlenen sayısal değerlerine karşılık gelen durumları, her bir **case** bölümünde ifade edilmiştir. **case** ile ifade edilen A portunun sayısal değeri gerçekleşmişse, **case** satırının hemen altındaki satırlar **break;** ifadesine kadar işlenmeye başlar. Ardından **break;** komutu sayesinde bu döngüdeki kontrol bırakılır ve program **switch** kodunun bitiminden devam eder. Ancak bizim programımızda sonsuz döngü içerisinde **switch**'den başka bir kod bölümü olmadığından program tekrar **switch** bölümünden döngüye başlar.

```

void main() {
    //Ana program başlangıcı
    ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON |= 7; //Karşılaştırma modülü iptal edildi.
    TRISA=0xFF; //Butonlar için A portunun tamamı GİRİŞ yapıldı
    TRISB=0; TRISD=0; //B ve D portları ÇIKIŞ olarak ayarlandı
    PORTA=0; //Butonlara basılmıyor olarak ayarlandı
    PORTB=0; PORTD=0; //Tüm LED'ler sönmük olarak ayarlandı
    while(1){ //Program sonsuz döngüye sokuldu
        switch (porta){ //A portunu aşağıdaki durumlar için kontrol et
            case 0b00000010: //Sadece Buton-1'e basılmışsa hemen alt satıra geç
                portb=0b00000011; //B portundaki ilk iki LED'i yak
                break; //Bu döngüdeki kontrolü bırak
            case 0b00000100: //Sadece Buton-2'ye basılmışsa hemen alt satıra geç
                portb=0b11000000; //B portundaki son iki LED'i yak
                break; //Bu döngüdeki kontrolü bırak
            case 0b00001000: //Sadece Buton-3'e basılmışsa hemen alt satıra geç
                portb=0b00111100; //B portunun tam ortasındaki 4 LED'i yak
                break; //Bu döngüdeki kontrolü bırak
            case 0b00001110: //Butonların hepsine basılmışsa hemen alt satıra geç
                portb=0xAA; //B portuna bağlı LED'leri birer boşluk bırakarak yak
                portd=0xFF; //D portuna bağlı tüm LED'leri yak
                break; //Bu döngüdeki kontrolü bırak
            default: //Butona basılma durumu yukarıdakilere uymuyorsa alt satıra geç
                portB=0;portD=0; //B ve D portuna bağlı LED'leri söndür
                break; //Bu döngüdeki kontrolü bırak
        }
    }
}

```

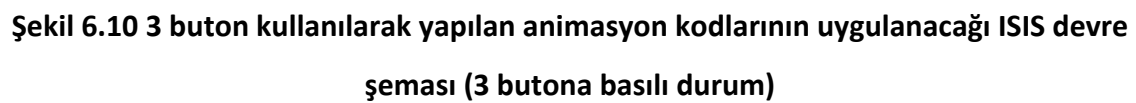
Şekil 6.9 3 buton kullanılarak yapılan animasyon kodlarına ilişkin MikroC programının ekran görüntüsü

Örneğin Şekil 6.9'daki ekran görüntüsünün 14. satırındaki **case 0b00000010:** kodu, “şayet Buton-1’e basılmışsa hemen alt satırdan devam et” demektir. Yani “B portunun ilk 2 bitine bağlı LED’leri yak ve ardından **switch** yapısından çık” demektir.

Yine aynı şekilde, Şekil 6.9'daki ekran görüntüsünün 23. satırındaki **case 0b00001110:** ve hemen akabinde 3 satırlık kod, “şayet 3 butona birden basılmışsa hemen alt satıra geç, D portunun tamamı ile B portundaki LED’lerin tek numaralı bitlerine bağlı olan LED’leri yak ve ardından **switch** yapısından çık” demektir. Bu durum gerçekleştiğinde LED’lerin alacağı görüntü Şekil 10’da görülmektedir.

Eğer hiçbir butona basılmamışsa veya bizim ilgilenmediğimiz bir kombinasyonla butonlara basılmışsa bu durumda tüm LED’ler sönmük kalacaktır. Bu durum **default:** komutundan hemen sonra yazılan **portB=0; portD=0;** komutlarıyla sağlanmıştır.

BUTON UYGULAMALARI



BÖLÜM 7

7. LCD UYGULAMALARI

UYGULAMA_01

Bu programda PIC ile LCD (Liquid Crystal Display-Sıvı Kristal Ekran) ekranına yazı yazdırmaya yönelik çok basit bir uygulama yapılacaktır. Uygulamada 2x16 'lık bir LCD kullanılmıştır. Bilineceği üzere 2x16 ifadesindeki 2 rakamı LCD'deki satır sayısını, 16 rakamı ise sütun sayısını göstermektedir. Yani böyle bir ekranda aynı anda en fazla 32 harf görülebilir. Uygulama kodları aşağıda verilmiştir.

PROGRAM KODLARI:

*/*Bu programda LCD uygulaması yapılacaktır. LCD kullanılacağında aşağıdaki tanımlamalar mutlaka yapılır. Burada yapılan şey; LCD ile PIC'in hangi pinlerinin birbirine bağlanacağını belirtmesidir*/*

//===== Program başlangıcı =====

// LCD modül bağlantıları tanımlanıyor

```
sbit LCD_RS at RB0_bit;
sbit LCD_EN at RB1_bit;
sbit LCD_D4 at RB2_bit;
sbit LCD_D5 at RB3_bit;
sbit LCD_D6 at RB4_bit;
sbit LCD_D7 at RB5_bit;
sbit LCD_RS_Direction at TRISB0_bit;
sbit LCD_EN_Direction at TRISB1_bit;
sbit LCD_D4_Direction at TRISB2_bit;
sbit LCD_D5_Direction at TRISB3_bit;
sbit LCD_D6_Direction at TRISB4_bit;
sbit LCD_D7_Direction at TRISB5_bit;
```

```
// LCD modül bağlantılarının sonu
```

```
void main() {           //Ana program başlangıcı
    ADCON1 |= 0x0F;      //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |= 7;         //Karşılaştırma modülü iptal edildi.
    TRISB7_bit=0;        //LED için RB7 biti çıkış olarak ayarlandı
    RB7_bit=1;           //PIC'e enerji verildiğinde bu bite bağlı LED yanacak
    Lcd_Init();          //LCD kullanıma hazır hale getirildi
    Lcd_Cmd(_LCD_CURSOR_OFF); //LCD ekranındaki yanıp sönen imleç kapatıldı
    delay_ms(200);       //LCD'nin hazırlanması için 200 ms bekleniyor
    Lcd_Cmd(_LCD_CLEAR); //LCD ekranı temizlendi
    Lcd_Out(1,1,"MESLEKi AKADEMi");//LCD'de 1.satır 1.sütundan itibaren MESLEKi AKADEMi yaz
    Lcd_Out(2,6,"KONYA");//2. satır 6. sütundan itibaren KONYA yazdırıldı
}                        //Ana program sonlandırıldı.

//===== Program sonu =====
```

Programın MikroC programındaki görüntüsü ve devre şeması sırasıyla Şekil 7.1 ve Şekil 7.2'de görülmektedir.

PIC'e LCD bağlanması durumunda öncelikle PIC ile LCD'nin hangi pinlerinin birbirine bağlandığının program başında belirtilmesi gerekir. Bu ilişkilendirmeye yönelik yazılan kodlar, bizim programımızda 5. satırdan başlayıp 16. satırda sonlanmaktadır (Şekil 7.1). Yazılan bu kodların içerisinde LCD'nin bağlanacağı pinlerin bitler halinde çıkış olarak yönlendirilmesi de yapılmaktadır. O nedenle ana program içerisinde ayrıca çıkış olarak yönlendirme yapılması gerekmez. Bu tanımlamalar LCD kullanılan her programda yazılmak zorunda olduğundan bunları tek tek yazmak yerine kopyala-yapıştır yapmak daha uygun olacaktır. Ancak bu durumda her zaman LCD'nin PIC'in aynı pinlerine bağlanması gerektiğini unutmayınız.

Uygulamamızda PIC'e enerji verildiğinde doğrudan yanacak bir LED de bağlanmıştır. Bu nedenle ilgili bit çıkış olarak yönlendirilmiştir.

LCD uygulamalarında LCD'nin başlatılması için mutlaka **Lcd_Init();** komutu kullanılmalıdır. Bu komutun kullanımından sonra ise LCD'nin fiziksel olarak başlamasını

beklemek için ortalama 150-200ms'lik gecikmeler konular. Şayet bu gecikme konulmazsa, LCD'ye gönderilen ve çok kısa aralıklarla değişen verilerin daha LCD başlamadan ekrana yazdırılıp kaybolacağı veya diğer verilerin ekranda görüleceği unutulmamalıdır. Dolayısıyla bunu engellemek için bu gecikmelerin konulması daha uygun olacaktır.

Yine LCD uygulamalarında, kullanımının alışkanlık haline getirilmesi gereken bir diğer komut ise LCD başlatıldıktan sonra LCD içerisinde bulunan verinin veya bir diğer ifadeyle ekranının temizlenmesidir. Bunun için kullanılan komut ise ***Lcd_Cmd(_LCD_CLEAR);*** komutudur. Kullanıcını tercihine bağlı olarak kullanılabilecek bir diğer kod ise ***Lcd_Cmd(_LCD_CURSOR_OFF);*** komutudur. Bu komut sayesinde ekranda yanıp sönen imleç kapatılır. Ekrana yazı yazdırmak ise MikroC sayesinde oldukça kolaydır. Yapılması gereken tek şey ***Lcd_Out(a,b,"c");*** komutunu kullanmaktır. Burada "c" ile gösterilen kısımdaki c ifadesi ekrana yazdırılacak harf, kelime veya kelime grubu olup bunun başlangıç koordinatları, **a** ve **b** ile gösterilir. Burada **a** satır numarasını **b** ise sütun numarasını ifade eder. Yazdığımız bu programda ***Lcd_Out(1,1,"MESLEKi AKADEMi");*** yazmak suretiyle LCD'nin 1. satır, 1. sütunundan itibaren **MESLEKi AKADEMi** ifadesi yazdırılmıştır. LCD'nin 2. satır, 6. sütunundan itibaren de **KONYA** ifadesi yazdırılmıştır. Buradaki 6 rakamının, **KONYA** kelimesinin ekranda ortalanarak yazılmasını sağlamak için olduğuna dikkat ediniz.

LCD'de karakterleri yazdırırken İngilizce karakterlerin kullanılması gerektiğine dikkat edilmelidir. Aksi halde ekranda sizin yazmadığınız karakterlerle karşılaşsınız. Bu uygulamada **MESLEKi AKADEMi** kelimeleri yazdırılırken *i* harflerinin küçük harfle yazıldığını unutmayınız. Kodları hatasız yazıp derledikten sonra simülasyonu başlattığınızda, LCD ekranında görülmesini istediğiniz yazının bulunduğunu göreceksiniz (Şekil 7.2).

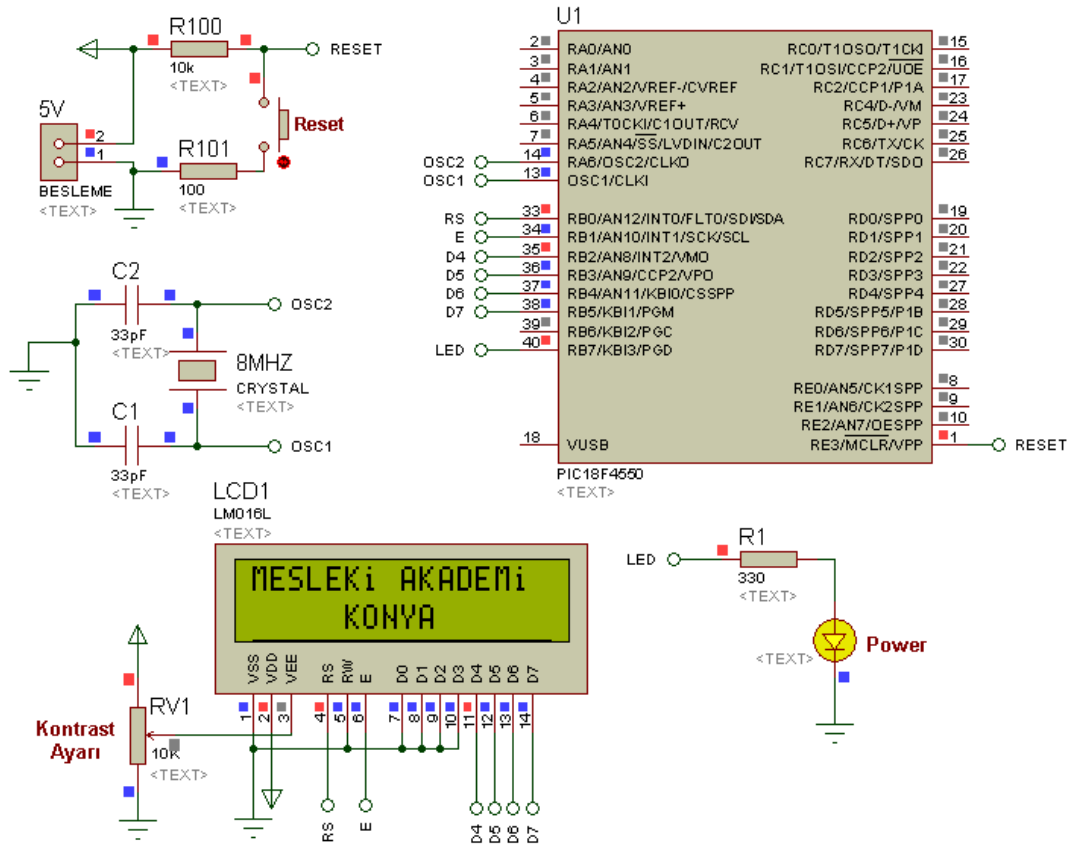
LED ve buton örneklerinde RESET butonuna basıldığı takdirde PIC'deki programın en baştan başladığını, dolayısıyla yanan LED varsa bunların söndüğünü hatırlayınız. Aynısını aşağıdaki devrede uyguladığınızda RB7'ye bağlı LED'in söndüğünü ancak LCD ekranında bulunan verinin RESET butonundan elinizi çekmediğiniz sürece kaybolmadığını, elinizi çektiğinizde ise ekrandaki verinin anlık olarak gidip tekrar geldiğini göreceksiniz. LCD enerjili ancak devredeki RESET butonu basılıyken LCD ekranında bulunan verinin kaybolmama nedenini araştırınız.

```

StartPage LCD_01.c
// LCD modül bağlantıları tanımlanıyor
sbit LCD_RS at RB0_bit;
sbit LCD_EN at RB1_bit;
sbit LCD_D4 at RB2_bit;
sbit LCD_D5 at RB3_bit;
sbit LCD_D6 at RB4_bit;
10 sbit LCD_D7 at RB5_bit;
sbit LCD_RS_Direction at TRISB0_bit;
sbit LCD_EN_Direction at TRISB1_bit;
sbit LCD_D4_Direction at TRISB2_bit;
sbit LCD_D5_Direction at TRISB3_bit;
sbit LCD_D6_Direction at TRISB4_bit;
sbit LCD_D7_Direction at TRISB5_bit;
// LCD modül bağlantılarının sonu
void main() { //Ana program başlangıcı
    ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON |= 7; //Karşılaştırma modülü iptal edildi.
20    TRISB7_bit=0; //LED için RB7 biti çıkış olarak ayarlandı
    RB7_bit=1; //PIC'e enerji verildiğinde bu bite bağlı LED yanacak
    Lcd_Init(); //LCD display kullanıma hazır hale getirildi
    Lcd_Cmd( LCD_CURSOR_OFF); //LCD ekranındaki yanıp sönen imleç kapatıldı
    delay_ms(200); //LCD'nin hazırlanması için 200 ms bekleniyor
    Lcd_Cmd( LCD_CLEAR); //LCD ekranı temizlendi
    Lcd_Out(1,1,"MESLEKİ AKADEMİ"); //LCD'nin 1.satır 1.sütunundan itibaren MESLEKİ
    //AKADEMİ yazdırıldı
    Lcd_Out(2,6,"KONYA"); //2. satır 6. sütundan itibaren KONYA yazdırıldı
30 } //Ana program sonlandırıldı.

```

Şekil 7.1 LCD'ye yazı yazdırılmasına ilişkin MikroC programının ekran görüntüsü



Şekil 7.2 İlk LCD uygulamasında program kodlarının yükleneceği ISIS şeması

UYGULAMA_02

Bu uygulamamızda LCD'ye yazdırılan metnin butonlar ile değiştirilmesine yönelik bir örnek verilecektir. Ayrıca buraya kadarki hiçbir örneğimizde kullanmadığımız yeni bir sonsuz döngü yapısı ve butonlara isim vererek program içerisinde bu isimlerin kullanılması gösterilecektir.

Uygulamaya ilişkin program kodları aşağıda verilmiştir. MikroC programındaki ekran görüntüsü tek seferde alınamadığı için bu uygulamaya ait ekran görüntüsü verilememiştir. Ancak devre şeması Şekil 7.3'te sunulmuştur. Programın içeriğini kısaca özetlemek gerekirse:

Öncelikle ekrana SELCUK UNİVERSİTESİ yazdırıldı. Ardından butonların durumları tek tek kontrol edilerek her butona farklı bir işlev yüklendi. Örneğin;

- **animasyon** isimli butona basıldığında ekrana SELCUK UNİVERSİTESİ yazdırıldı ve ardından bu metin 10 kez yakılıp söndürüldü.
- **sil** isimli butona basıldığında LCD ekranı temizlendi.
- **yeni** isimli butona basıldığında önceki yazı silinip LCD UYGULAMALARI UYGULAMA-2 yazdırıldı.
- **saga** isimli butona basıldığında ekranda hangi metin varsa onun bir karakter sağa kayması sağlandı.
- **sola** isimli butona basıldığında ekranda hangi metin varsa onun bir karakter sola kayması sağlandı.

PROGRAM KODLARI:

```
//===== Program başlangıcı =====  
#define animasyon portD.rD0 //RD0'daki butona animasyon ismi verildi  
#define sil portD.rD1 //RD1'deki butona sil ismi verildi  
#define yeni portD.rD2 //RD2'deki butona yeni ismi verildi  
#define saga portD.RD3 //RD3'deki butona saga ismi verildi  
#define sola portD.RD4 //RD4'deki butona sola ismi verildi  
char i=0; //Döngüde kullanmak için değişken tanımlandı
```

```
//===== LCD modül bağlantıları tanımlanıyor =====
```

```
sbit LCD_RS at RB0_bit;
sbit LCD_EN at RB1_bit;
sbit LCD_D4 at RB2_bit;
sbit LCD_D5 at RB3_bit;
sbit LCD_D6 at RB4_bit;
sbit LCD_D7 at RB5_bit;
sbit LCD_RS_Direction at TRISB0_bit;
sbit LCD_EN_Direction at TRISB1_bit;
sbit LCD_D4_Direction at TRISB2_bit;
sbit LCD_D5_Direction at TRISB3_bit;
sbit LCD_D6_Direction at TRISB4_bit;
sbit LCD_D7_Direction at TRISB5_bit;
```

```
//===== LCD modül bağlantılarının sonu =====
```

```
void main() {           //Ana program başlangıcı
    ADCON1 |= 0x0F;      //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |= 7;         //Karşılaştırma modülü iptal edildi.
    trisd=0xFF;          //D portu butonlar için GİRİŞ olarak ayarlandı
    TRISB7_bit=0;        //Power LED'i için RB7 biti çıkış olarak ayarlandı
    RB7_bit=1;           //PIC'e enerji verildiğinde bu bite bağlı LED yanacak
    Lcd_Init();          //LCD display kullanıma hazır hale getirildi
    Lcd_Cmd(_LCD_CURSOR_OFF); //LCD ekranındaki yanıp sönen imleç kapatıldı
    Lcd_Cmd(_LCD_CLEAR); //LCD ekranı temizlendi
    delay_ms(200);       //LCD'nin hazırlanması için 200 ms bekleniyor
    Lcd_Out(1,6,"SELCUK"); //LCD'nin 1.satır 6.sütunundan itibaren Selcuk yazdırıldı
    Lcd_Out(2,3,"UNİVERSİTESİ"); //2. satır 3. sütundan itibaren Üniversitesi yazdırıldı
    for ( ; ; )          //Program for döngüsünün süslü parantezleri arasında sonsuz döngüye sokuldu
    {
        if (animasyon) { //Animasyon isimli butonuna basılmışsa aşağıdakileri yap
            Lcd_Cmd(_LCD_CLEAR); //LCD ekranı temizle
```

```

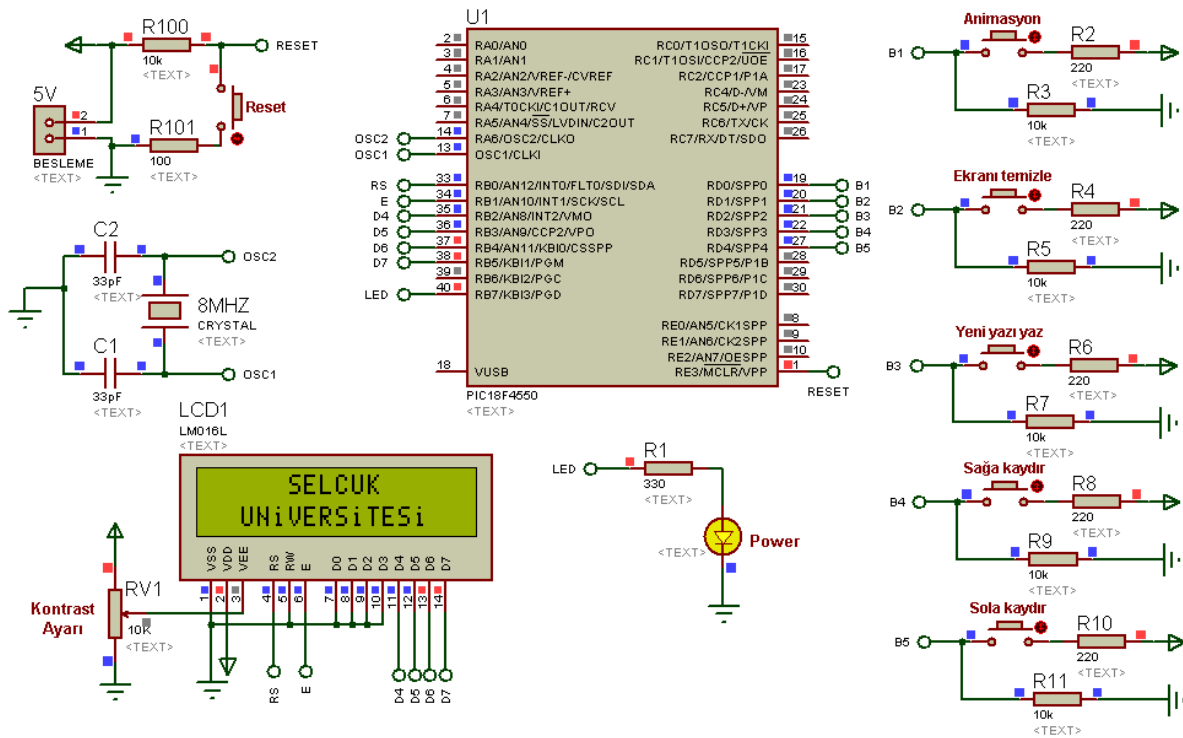
Lcd_Out(1,6,"SELCUK");    //LCD'nin 1.satır 6.sütunundan itibaren SELCUK yazdırıldı
Lcd_Out(2,3,"UNİVERSİTESİ");//2. satır 3. sütundan itibaren UNİVERSİTESİ yazdırıldı
for(i=0;i<10;i++){        //Bu FOR döngüsü içindeki işlemleri 10 kez tekrarla
    Lcd_Cmd(_LCD_TURN_OFF);    //Ekrandaki veriyi gizle
    Delay_ms(250);            //250ms bekle
    Lcd_Cmd(_LCD_TURN_ON);    //Ekrandaki veriyi görünür yap
    Delay_ms(250);            //250ms bekle
}                            //10 kez tekrar yaptıran FOR döngüsünü kapatma parantezi
while (animasyon);          //animasyon butonuna basılıyken bekle
}                            //animasyon isimli butona basıldığında yapılacaklar burada bitti
if (sil) Lcd_Cmd(_LCD_CLEAR); //sil isimli butona basılmışsa sadece ekranı temizle
if (yeni) {                  //yeni isimli butona basılmışsa aşağıdakileri yap
    Lcd_Cmd(_LCD_CLEAR);      //LCD ekranı temizle
    Lcd_Out(1,1,"LCD UYGULAMALARI"); //1. satır 1. sütundan itibaren istenileni yaz
    Lcd_Out(2,4,"UYGULAMA-2"); //2. satır 4. sütundan itibaren istenileni yaz
}                            //yeni isimli butona basıldığında yapılacaklar burada bitti
if (saga){                  //saga isimli butona basılmışsa aşağıdakileri yap
    Lcd_Cmd(_LCD_SHIFT_RIGHT); //Ekrandaki yazıyı 1 karakter sağa kaydır
    while (saga);            //saga isimli buton hâlâ basılıysa bekle
}                            //saga isimli butona basıldığında yapılacaklar burada bitti
if (sola){                  //saga isimli butona basılmışsa aşağıdakileri yap
    Lcd_Cmd(_LCD_SHIFT_LEFT);  //Ekrandaki yazıyı 1 karakter sola kaydır
    while (sola);            //sola isimli buton hâlâ basılıysa bekle
}                            //sola isimli butona basıldığında yapılacaklar burada bitti
}                            //Sonsuz döngü FOR için açılan parantez kapatıldı
}                            //Ana program sonlandırıldı.
//===== Program sonu =====

```

Yukarıda da değinildiği üzere bu programın yazımı sırasında yeni bir tanımlama yapılmış olup uygulamada kullanılacak butonlara isimler verilmiştir. Bu isimler butonların bağlanacağı pinlerle ilişkilendirilmiştir. Örneğin **#define animasyon portD.RD0** denildiğinde

RD0 bitine bağlanacak olan butona **animasyon** ismi verilmiş olur. Dolayısıyla programın bundan sonraki bölümlerinde bu butonun durumu kontrol edilirken **RD0** değil de **animasyon** yazılmıştır. Böyle yapmak suretiyle çok fazla butonun olduğu uygulamalarda hangi butonun hangi bite bağlandığını sürekli aklımızda tutmak yerine, o butonun işlevine yönelik ona verdiğimiz ismi kullanmak çok daha kolay olacaktır. Butonlara isim verilirken İngilizce karakter kullanılması gerektiği unutulmamalıdır.

Yine kodlar incelendiğinde **if** kontrol yapısı içerisinde butonların **lojik 1** kontrolleri yapılırken sadece butonun ismi yazılmıştır. Çünkü **if(animasyon==1)** yazmak ile **if(animasyon)** yazmak aynı şeydir. Dolayısıyla **lojik 1** kontrolü yapılırken **if** komutundan sonra sadece isim veya bit ismini yazmak yeterlidir.



Şekil 7.3 Uygulama-2 için yazılan kodların yükleneceği devreye ilişkin ISIS şeması

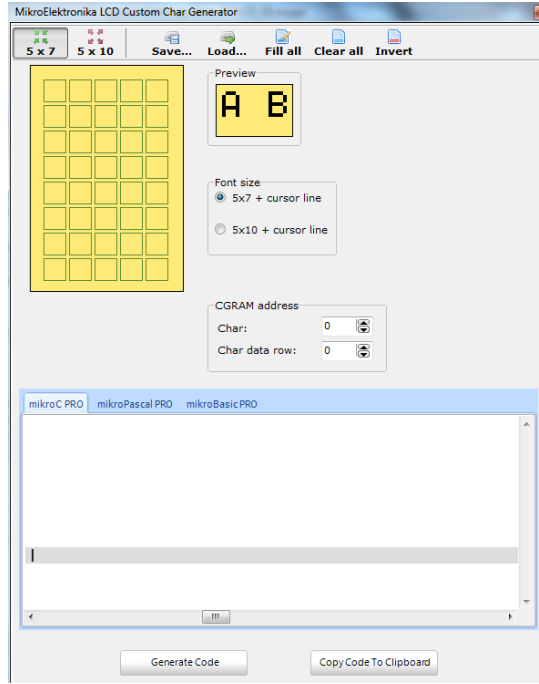
UYGULAMA_03

Önceden de ifade edildiği gibi Türkçeye özgü harf veya karakterler doğrudan LCD'ye yazdırılamazlar. Ancak MikroC programının **Tools** menüsü altında kullanıcıya sunulan **LCD Custom Character** aracı sayesinde bunu başarmak mümkün olmaktadır. Dolayısıyla bu uygulamada LCD'ye Türkçe karakter veya çok daha farklı bir sembol yazdırılırken nasıl bir yol izleneceği gösterilecektir. Dolaylı olarak da fonksiyon oluşturma işlemi anlatılmış olacaktır.

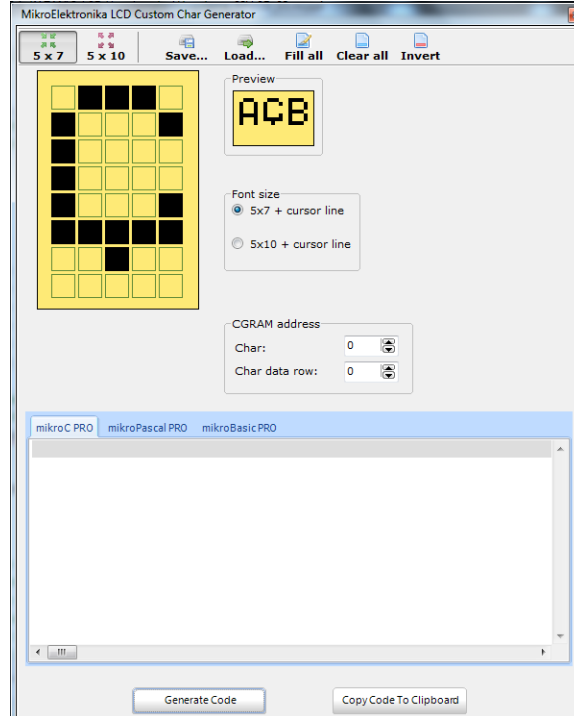
LCD uygulamalarında her zaman yapıldığı gibi öncelikle LCD ile PIC pinleri ilişkilendirilmiştir. Ardından pinlerin dijital amaçlı kullanımı, LCD'nin başlatılması, ekranın temizlenmesi ve istenilen verinin LCD ekranına yazdırılması gibi bundan önceki iki örneğimizde de detaylı olarak anlatılan işlemler yapılmıştır. Uygulamada LCD ekranının 1. satırına **DİKKAT! ELEKTRİK**, 2. satırına ise **ÇARPABİLİR** yazdırılacaktır. İngilizcede büyük **İ** harfi olmadığı için onun yerine küçük **i** harfi yazılarak sanki büyük **i** harfi yazılmış intibayı uyandırılmış ve bu problem çözülmüştür. Ancak **ç** harfinin ne büyüğü ne de küçüğü bu mantıkla yazdırılamaz. İşte bu noktada **LCD Custom Character** aracına başvurulacaktır. **Tools** menüsü altındaki bu araç çalıştırıldığında Şekil 7.4'teki pencere açılacaktır. Açılan pencerede istenilen karakteri ortaya koyacak şekilde ilgili kareler tıklanır. Bu işlemler ve diğer ayarlamalar Şekil 7.5'te görüldüğü gibi yapıldıktan sonra istenilen karakter elde edilir. Bu noktadan sonrası ise tamamen bu araç tarafından yapılacaktır. Programcıya düşen ilk iş, Şekil 7.5'teki pencerede görüleceği üzere **Generate Code** butonunu tıklayıp aracın MikroC kodlarını üretmesini sağlamaktır. Ardından **Copy Code to Clipboard** butonuna tıklayıp ilgili kodların panoya kopyalanmasını sağlamalıdır. Daha sonra ise bu kodları LCD modül bağlantılarının sonlandırıldığı bölümün altına yapıştırmalıdır. Artık geriye kalan tek şey, bu özel karaktere gerek duyulan yerde bu fonksiyon ismini (**CustomChar**) yazıp, karakterin LCD'deki koordinatlarını vermektir. Programcı istediği takdirde **CustomChar** isimli bu fonksiyona kendisi isim verebilir. Ancak bu değişikliği yapmışsa, program içerisinde de aynı isimle bu fonksiyonu çağırması gerektiğini unutmamalıdır.

AD SOYAD:

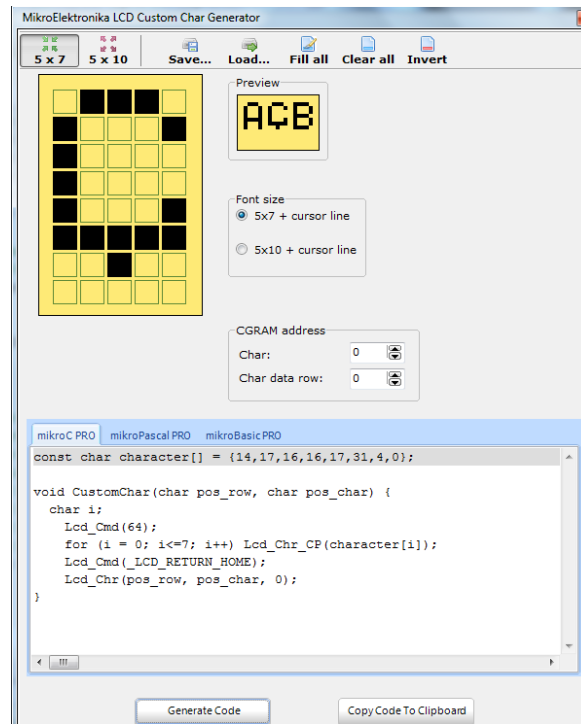
LCD UYGULAMALARI



Şekil 7.4 LCD için özel karakter üretme aracı (Henüz kod üretilmedi)



Şekil 7.5 İlgili karakterlere tıklanarak istenilen karakterin resmedilmesi



Şekil 7.6 Generate Code butonuna tıklanarak ilgili karaktere ait MikroC kodlarının üretilmesi (Ç harfine ilişkin kod üretildi)

Uygulamaya ilişkin program kodları aşağıda sunulmuştur. Program kodlarının MikroC ekranındaki görüntüsü tek seferde alınamadığı için bu uygulamaya ait ekran görüntüsü verilememiştir. Ancak devre şeması Şekil 7.7’de sunulmuştur.

Program hatasız bir şekilde derlendiği takdirde ilgili metnin Türkçe karakter kullanılarak Şekil 7.7’deki gibi ekrana yazdırıldığı görülecektir.

PROGRAM KODLARI:

//Bu programda LCD'ye Türkçe karakter yazdırma uygulaması yapılacaktır.

//===== Program başlangıcı =====

// LCD modül bağlantıları tanımlanıyor

sbit LCD_RS at RB0_bit;

sbit LCD_EN at RB1_bit;

sbit LCD_D4 at RB2_bit;

sbit LCD_D5 at RB3_bit;

sbit LCD_D6 at RB4_bit;

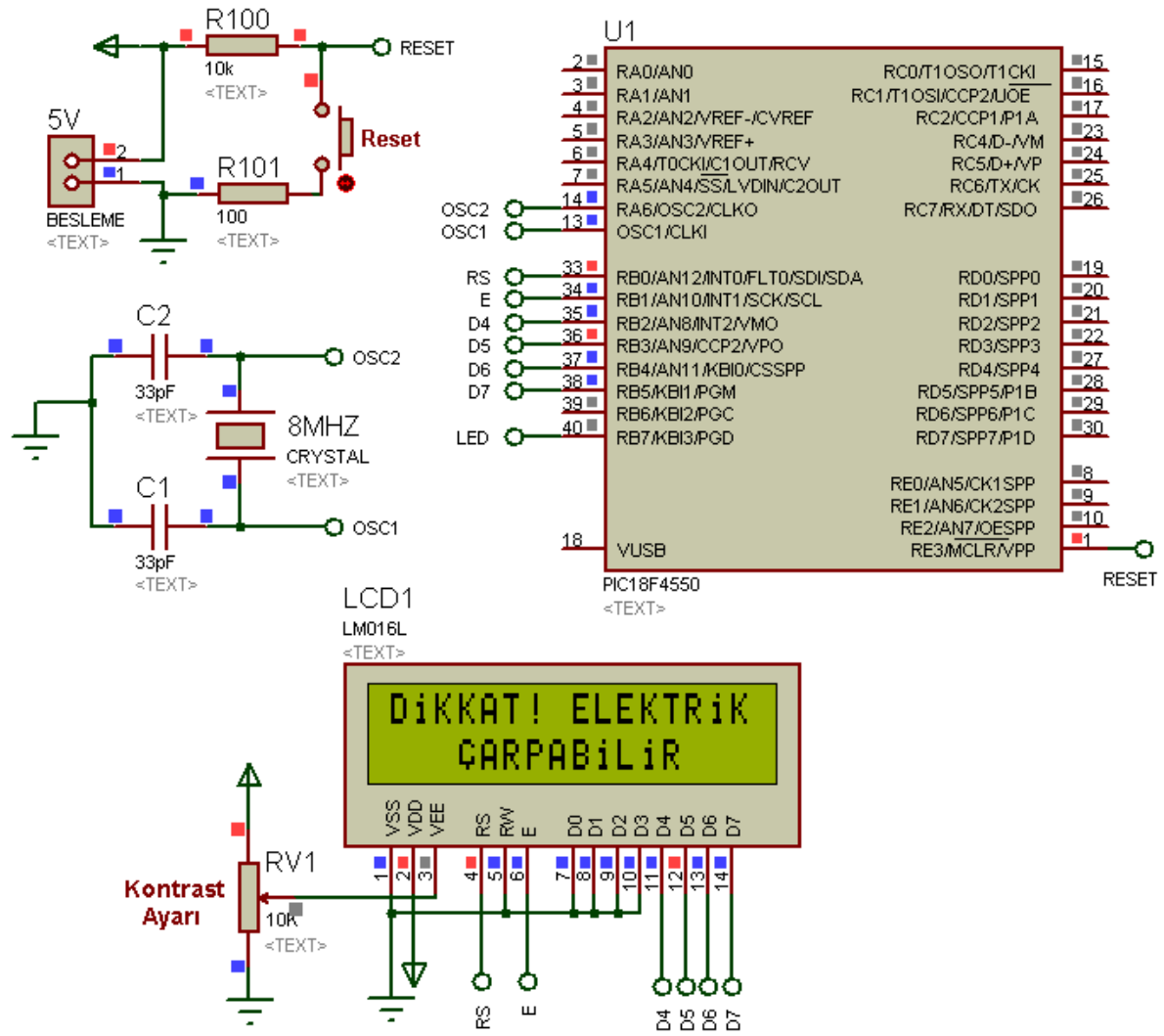
```
sbit LCD_D7 at RB5_bit;
sbit LCD_RS_Direction at TRISB0_bit;
sbit LCD_EN_Direction at TRISB1_bit;
sbit LCD_D4_Direction at TRISB2_bit;
sbit LCD_D5_Direction at TRISB3_bit;
sbit LCD_D6_Direction at TRISB4_bit;
sbit LCD_D7_Direction at TRISB5_bit;
// LCD modül bağlantılarının sonu
// LCD özel karakter (LCD Custom Character) oluşturma aracından kopyalanan bölüm
//buradan başlıyor
const char character[] = {14,17,16,16,17,31,4,0}; //LCD Custom Character tarafından
//oluşturulan kod
void CustomChar(char pos_row, char pos_char) { //LCD Custom Character tarafından
//oluşturulan kod
char i; //LCD Custom Character tarafından
//oluşturulan kod
Lcd_Cmd(64); //LCD Custom Character tarafından
//oluşturulan kod
for (i = 0; i<=7; i++) Lcd_Chr_CP(character[i]); //LCD Custom Character tarafından
//oluşturulan kod
Lcd_Cmd(_LCD_RETURN_HOME); //LCD Custom Character tarafından
//oluşturulan kod
Lcd_Chr(pos_row, pos_char, 0); //LCD Custom Character tarafından
//oluşturulan kod
} //LCD Custom Character tarafından
//oluşturulan kod
// LCD özel karakter (LCD Custom Character) oluşturma aracından kopyalanan bölüm burada
//sonlandı
void main() { //Ana program başlangıcı
ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
```

```
CMCON |=7; //Karşılaştırma modülü iptal edildi.
Lcd_Init(); //LCD display kullanıma hazır hale getirildi
Lcd_Cmd(_LCD_CURSOR_OFF); //LCD ekranındaki yanıp sönen imleç kapatıldı
Lcd_Cmd(_LCD_CLEAR); //LCD ekranı temizlendi
delay_ms(200); //LCD'nin hazırlanması için 200 ms bekleniyor
Lcd_Out(1,1,"DiKKAT! ELEKTRİK "); //1. satır 1. sütundan itibaren DiKKAT! ELEKTRİK yazıldı
CustomChar(2,4); //2. satır 4. sütuna yukarıda oluşturulan karakter (Ç harfi) yazıldı
Lcd_Out_CP("ARPABİLiR"); //İmlecin kaldığı (Ç harfinin bittiği) yerden ARPABİLİR
// yazıldı
} //Ana program sonlandırıldı.
//===== Program sonu =====
```

Özetlenecek olursa; özel karakter oluşturmak için yapılan şey aslında bir fonksiyon oluşturmak ve lazım olduğunda da bu fonksiyonu çağırmaktan ibarettir. Bu uygulamada ilk iki LCD uygulamasından farklı olarak **Lcd_Out_CP** komutu kullanılmıştır. Bu komut, ekrana yazdırılacak metin için koordinat verme gereğini ortadan kaldırır. Yani ekrana en son ne yazdırılmışsa otomatik olarak onun hemen sağındaki koordinattan ilgili verinin yazdırılmasını sağlar.

AD SOYAD:

LCD UYGULAMALARI



Şekil 7.7 LCD'ye Türkçe karakter yazdırılmasına ilişkin ISIS şeması

UYGULAMA_04

Bu uygulamada oldukça basit bir dijital saat programı yazılacak ve simülasyonu gerçekleştirilecektir. Uygulamada DS1307 ve benzeri gerçek zamanlı saat (Real Time Clock) entegresi kullanılmayacaktır. Dolayısıyla bu entegreler kullanıldığında elde edilen yüksek hassasiyet burada elde edilemeyecektir. Ancak PIC ve LCD ile neler yapılabileceğinin görülmesi adına oldukça güzel bir uygulama olacaktır.

Bu uygulamanın bir diğer amacı da değişken olan sayısal bir verinin LCD ekranına nasıl yazdırılacağıın öğrenilmesini sağlamaktır. Normalde ilgili komutlar aracılığıyla istenen metni LCD'ye yazdırmanın mümkün olduğunu bundan önceki uygulamalarımızda görmüştük. Ancak sürekli değişen bir sayısal veriyi barındıran bir değişken, mevcut hâliyle LCD ekranına gönderildiğinde hata ile karşılaşılacaktır. Çünkü değişkenin türü ile LCD ekranına yazdırılması gerek verinin türü farklıdır.

Burada yapılması gereken şey, sürekli değişen sayısal verinin değerini ekrana yazdırmaya çalışmadan önce bunu string formatına dönüştürmektir. Bu dönüşüm sağlandıktan sonra string formatlı yeni veri LCD'ye gönderildiğinde artık hiçbir problemle karşılaşılmayacaktır. Bu uygulamamızda değişken sayısal verilerin barındırılacağı **short unsigned** türündeki **saat**, **dakika** ve **saniye** isimli değişkenlerin yanı sıra, bir de bunların string formatına dönüştürülmesinden sonra elde edilecek yeni verileri saklamak için **char** türünde 7 eleman boyutlu **S_saat**, **S_dakika** ve **S_saniye** isimli diziler tanımlanmıştır. Dolayısıyla ekrana yazdırılacak veriler bu dizilerin içerisinde saklanacaktır.

Uygulamaya ilişkin program kodları aşağıda görülmektedir. Uygulama çalıştırıldığında ekranda önce kısa süreli bir metin görünecektir. Ardından 12:00:00 formatında dijital saat çıkacak ve zaman ilerlemeye başlayacaktır. Saat, dakika ve saniye ayarı E portuna bağlı olan butonlar kullanılarak ayrı ayrı ayarlanabilecektir. Eğer ayarlama yapılmazsa saat 12:00:00'dan itibaren ilerleme devam edecektir.

Bu uygulamada **do-while** döngüsü kullanılarak da sonsuz döngünün oluşturulabileceği gösterilmiştir. Ayrıca LCD modül bağlantıları da bundan öncekilerden farklı ifadeler kullanılarak gerçekleştirilmiştir.

Program kodlarının MikroC ekranındaki görüntüsü tek seferde alınamadığı için bu uygulamaya ait ekran görüntüsü de verilememiştir. Ancak devre şeması çalışır bir şekilde Şekil 7.8’de sunulmuştur. Program hatasız bir şekilde derlenip Şekil 7.8’deki devre kurulduğunda, 24 saat formatlı dijital saatin problemsiz bir şekilde çalıştığı görülecektir.

PROGRAM KODLARI:

```

/*Bu uygulamada basit bir dijital saat programı yazılacaktır. Saat için herhangi bir entegre
kullanılmayıp sadece 8MHz’lik osilatör kullanılacaktır. Uygulama çalıştırıldığında, ekranda
önce kısa süreli bir metin görünecek, ardından 12:00:00 formatında dijital saat çıkacaktır. */
//===== Program başlangıcı =====
// ===== LCD modül bağlantıları tanımlanıyor =====
sbit LCD_D4 at LATB.B0;
sbit LCD_D5 at LATB.B1;
sbit LCD_D6 at LATB.B2;
sbit LCD_D7 at LATB.B3;
sbit LCD_RS at LATB.B4;
sbit LCD_EN at LATB.B5;
sbit LCD_D4_Direction at TRISB.B0;
sbit LCD_D5_Direction at TRISB.B1;
sbit LCD_D6_Direction at TRISB.B2;
sbit LCD_D7_Direction at TRISB.B3;
sbit LCD_RS_Direction at TRISB.B4;
sbit LCD_EN_Direction at TRISB.B5;
// ===== LCD modül bağlantılarının sonu =====
//Global değişkenler tanımlanmaya başlanıyor
short unsigned saat=112;           //saat değişkeni tanımlanıp 12'ye ayarlandı
short unsigned dakika=100;         //dakika değişkeni tanımlanıp 00'a ayarlandı
short unsigned saniye=100;         //saniye değişkeni tanımlanıp 00'a ayarlandı
char S_saat[7];                    //saat string dizisi
char S_dakika[7];                  //dakika string dizisi

```

```

char S_saniye[7];           //saniye string dizisi
//Global değişkenlerin tanımlanması bitti

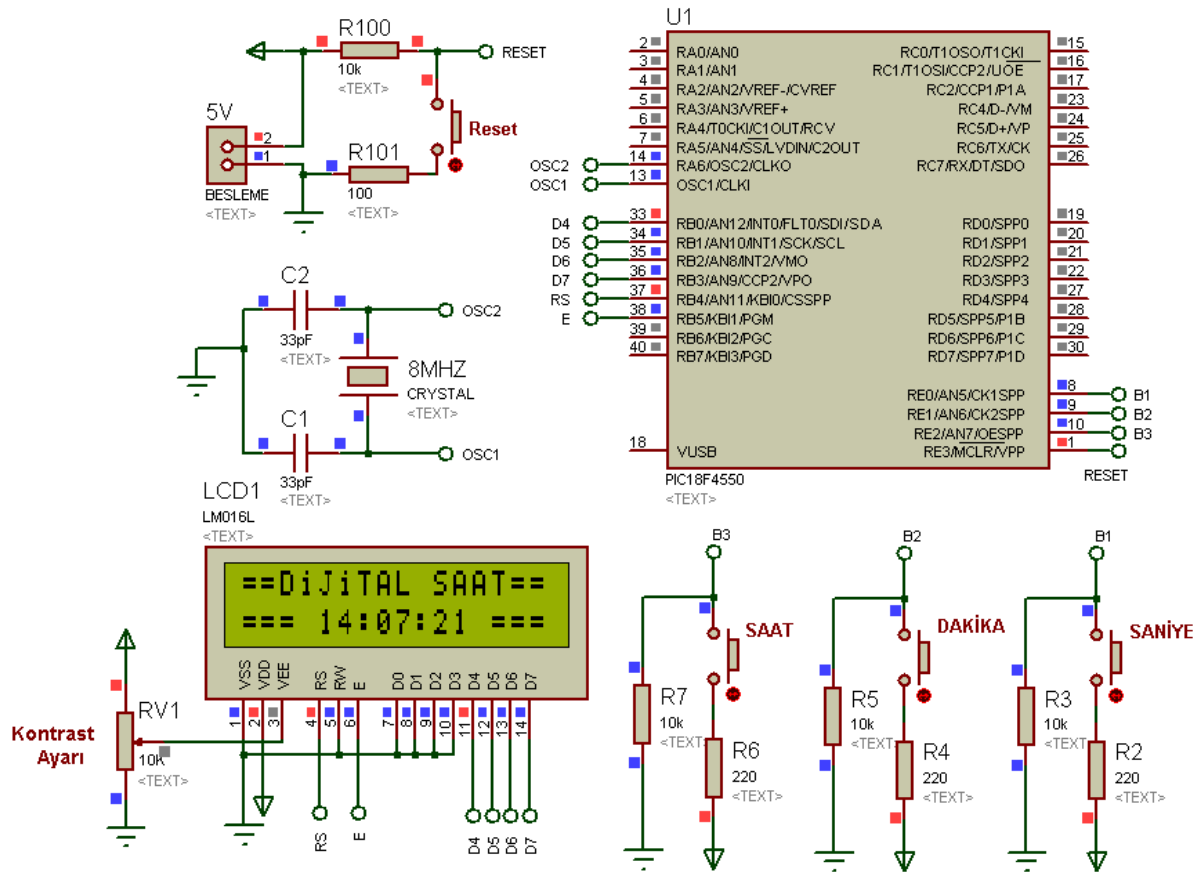
void main() {               //Ana program başlangıcı
    ADCON1 |= 0x0F;         //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |= 7;           //Karşılaştırma modülü iptal edildi.
    TRISE=0xFF;            //E portu GİRİŞ olarak ayarlandı
    LATE=0;                //E portunu temizle
    Lcd_Init();             //LCD display kullanıma hazır hale getirildi
    Lcd_Cmd(_LCD_CLEAR);    //LCD ekranı temizlendi
    Lcd_Cmd(_LCD_CURSOR_OFF); //LCD ekranındaki yanıp sönen imleç kapatıldı
    Lcd_Out(1,2, "BASİT BİR SAAT"); //İlgili koordinatlara ilgili metni yazdır
    Lcd_Out(2,4, "UYGULAMASI");   //İlgili koordinatlara ilgili metni yazdır
    Delay_ms(3000);          //3 saniye bekle
    Lcd_Cmd(_LCD_CLEAR);    //LCD ekranı temizlendi
    Lcd_Out(1,1, "==DİJİTAL SAAT=="); //İlgili koordinatlara metin eklendi
    Lcd_Out(2,1, "===");        //İlgili koordinatlara görsellik eklendi
    Lcd_Out(2,14, "===");       //İlgili koordinatlara görsellik eklendi
    Lcd_Out(2,7, ":");          //Saat ile dakika arasına ayıraç (:) eklendi
    Lcd_Out(2,10,":");          //Dakika ile saniye arasına ayıraç (:) eklendi
    do{                      //do-while sonsuz döngüsü başlatılıyor
        inttostr(saat,S_saat);  //saati ekrana yazdırabilmek için stringe dönüştür
        inttostr(dakika,S_dakika); //dakikayı ekrana yazdırabilmek için stringe dönüştür
        inttostr(saniye,S_saniye); //saniyeyi ekrana yazdırabilmek için stringe dönüştür
        Lcd_Out(2,5,S_saat+4);  //saat bilgisini ekrana yazdır
        Lcd_Out(2,8,S_dakika+4); //dakika bilgisini ekrana yazdır
        Lcd_Out(2,11,S_saniye+4); //saniye bilgisini ekrana yazdır
        delay_ms(978);          //1 saniyelik döngü periyotu elde etmek için eklendi
        delay_us(664); //1 saniyelik döngü periyotunu daha doğru elde etmek için eklendi
        saniye=saniye++;        //saniyeyi 1 artır
        if (PORTE.B2==1){       //saat ayar butonuna basılmışsa

```

AD SOYAD:

LCD UYGULAMALARI

```
    delay_ms(10);                //10 ms bekle
    saat=saat++;                 //saati 1 artır
}
if (PORTE.B1==1){               //dakika ayar butonuna basılmışsa
    delay_ms(10);                //10 ms bekle
    dakika=dakika++;             //dakikayı 1 artır
}
if (PORTE.B0==1){               //saat ayar butonuna basılmışsa
    delay_ms(10);                //10 ms bekle
    saniye=saniye+5;             //saniyeyi 1 artır
}
if (saniye>=160){               //saniye 60'a ulaşmışsa alt satıra geç
    dakika=dakika++;             //dakikayı 1 artır
    saniye=100;                 //saniyeyi sıfırla
}
if (dakika>=160){               //dakika 60'a ulaşmışsa alt satıra geç
    saat=saat++;                 //saati 1 artır
    dakika=100;                 //dakikayı sıfırla
}
if (saat>=124){                 //saat 24'e ulaşmışsa alt satıra geç
    saat=100;                    //saati sıfırla
}
} while(1);                     //do-while sonsuz döngüsünün sonu
}                                //Ana program sonlandırıldı.
//===== Program sonu =====
```

Şekil 7.8 Basit bir dijital saat uygulaması

BÖLÜM 8

8. DISPLAY UYGULAMALARI

UYGULAMA_01

Bilindiği gibi 7 parçalı (seven segment) displayler, içerisinde barındırdığı 7 adet küçük LED'in farklı kombinasyonlarla enerjilenmesi sonucunda farklı rakam veya karakter ortaya koyarlar. Bu displayler, toprak uçlarının (katotlarının) veya kaynak uçlarının (anotlarının) ortak olmasına göre sırasıyla ortak katotlu ve ortak anotlu olmak üzere iki farklı türde üretilirler. Ortak anotlu yapıda ilgili karakteri displayde görmek için yanması istenen LED'lere **lojik 0** verilir. Ortak katotlu yapıda ise bunun tam tersi yapılır, yani ilgili karakteri displayde görmek için yanması istenen LED'lere **lojik 1** verilir.

Uygulamamıza geçmeden önce bu displaylerin çeşitleri, bağlantı uçları, hangi LED'in yanması için hangi uca **lojik 1** veya **0** verilmesi gerektiği hakkında kısa bilgi edinmeniz, bu ve sonraki uygulamaların anlaşılmasını kolaylaştıracaktır.

7 parçalı displaylerle ilgili bu ilk örneğimizde yarım saniye aralıklarla 0'dan 9'a kadar sayan bir yukarı sayıcı gerçekleştirilecektir. Öncelikle, 0-9 arası karakterleri displayde görebilmek için ortak katot yapısına uygun olarak bir dizi oluşturulacaktır. Sabit karakter (**const char**) türünde oluşturulacak bu diziye kurallar çerçevesinde istenilen isim verilebilir. Bu uygulamada "deger" ismi verilecektir. Böyle bir dizi oluşturmanın amacı, program yazımını kolaylaştırmak ve daha kısa program yazmaktır.

Uygulamamızda kullanacağımız ortak katotlu yapıda displaye binary, hex veya decimal formatında hangi veri gönderilirse displayde hangi rakamın görüleceği, aşağıdaki program kodlarında sabit karakter türündeki "deger" isimli değişkenin tanımlanması sırasında belirtilmiştir. Programcı binary, hex veya decimal olarak bu tanımlamayı yapmakta özgürdür. Bu uygulamada displayin **a, b, c, d, e, f, g** isimli uçlarına ne gönderildiğinde hangi rakamın elde edildiğini göstermek için diziye veri yüklenmesi sırasında binary formatı tercih edilmiştir.

Uygulamaya ilişkin program kodları aşağıda sunulmuştur.

PROGRAM KODLARI:

/*===== Program başlangıcı =====*/

Öncelikle 0-9 arası karakterleri displayde görebilmek için ortak katot yapısına uygun olarak “deger” isimli sabit karakter türünde bir dizi oluşturulacaktır.

	g f e d c b a	HEX	Decimal	Görünecek sayı	*/
const char deger[10] = {0b00111111,		//0X3F	63	0	
0b00000110,		//0X06	6	1	
0b01011011,		//0X5B	91	2	
0b01001111,		//0X4F	79	3	
0b01100110,		//0X66	102	4	
0b01101101,		//0X6D	109	5	
0b01111101,		//0X7D	125	6	
0b00000111,		//0X07	7	7	
0b01111111,		//0X7F	127	8	
0b01101111};		//0x6F	111	9	
short b=0;	//ilk değeri 0 olan işaretli bir kısa tam sayı (b) tanımlandı				
void main() {	//Ana program başlangıcı				
ADCON1 = 0x0F;	//PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı				
CMCON = 7;	//Karşılaştırma modülü iptal edildi				
trisD=0;	//Display'in bağlanacağı D portu çıkış olarak atandı				
portD=0;	//D portu temizlendi (Lojik 0 gönderildi)				
while(1){	//Program sonsuz döngüye sokuldu				
portD= deger[b];	//"deger" isimli dizinin b. elemanı olan veri D portuna yüklendi				
b++;	//b'nin değerini 1 artır				
delay_ms(500);	//500ms bekle				
if (b > 9) b = 0;	// b'nin değeri 9'dan büyükse b'yi sıfırla (tekrar 0'dan başlamak için)				
}	//Sonsuz döngünün sınırını gösteren süslü parantez				
}	//Ana program sonlandırıldı.				

//===== Program sonu =====

Program kodları incelendiğinde programın oldukça basit olduğu görülecektir. Çünkü burada yapılan şey, programın başında tanımlanmış olan 10 elemanlı bir dizideki verileri sırayla alıp ilgili porta (D portuna) göndermekten ibarettir.

Dizideki elemanların sırayla alınması, tanımlanan bir değişkenle sağlanır. Bu uygulamamızda **short** (kısaca tamsayı) türünde **b** olarak tanımladığımız değişkene ilk olarak sıfır değeri yüklenir. Çünkü dizideki elemanların sıra numarası sıfırdan başlar. Dizideki elemanların sırayla alınması için de ilk dizi elemanı (sıfırıncı eleman) alındıktan sonra bu değişkenin değeri bir arttırılır (**b++**).

Dizinin son elemanının numarası, dizi boyutunun bir eksiklidir. Dolayısıyla 10 elemanlı olarak tanımladığımız dizimizdeki son elemanın numarası 9'dur. Bu nedenle displye göndereceğimiz verinin tekrar baştan başlamasını istiyorsak (ki bu uygulamada onu istiyoruz) bu durumda dizinin son elemanını ilgili porta gönderdikten sonra dizinin elemanlarını almamızı sağlayan **b** değişkeninin değerini sıfırlamak gerekir. Bunu ise her döngüde "**b'nin değeri 9'dan büyükse b'yi sıfırla**" anlamına gelen **if (b > 9) b = 0;** satırıyla sağlarız.

Uygulamanın MikroC programındaki ekran görüntüsü Şekil 8.1'de verilmiştir. Şayet program hatasız bir şekilde derlenip Şekil 8.2'deki devre şemasında uygulanırsa, **delay_ms(500);** olarak belirlediğimiz yarım saniyelik aralıklarla 0'dan 9'a kadar sayacaktır. Bu işlemi PIC'in enerjisi kesilene veya RESET butonuna basıncaya kadar devam ettirecektir.

Şayet uygulamanın kodları aşağıdaki gibi değiştirilecek olursa, bu durumda sayıcının tam tersi yönde yani 9'dan 0'a doğru saydığı görülecektir.

- **b=0;** yerine **b=9;** yazılmalı
- **b++** yerine **b--;** yazılmalı
- **if (b > 9) b = 0;** yerine **if (b < 0) b =9;** yazılmalı

Aşağı sayma uygulamasında **b** değişkeninin türünü **short** yerine **unsigned short** yapmak bir şeyi değiştirir mi? Şayet değiştiriyorsa nedenini araştırınız.

```

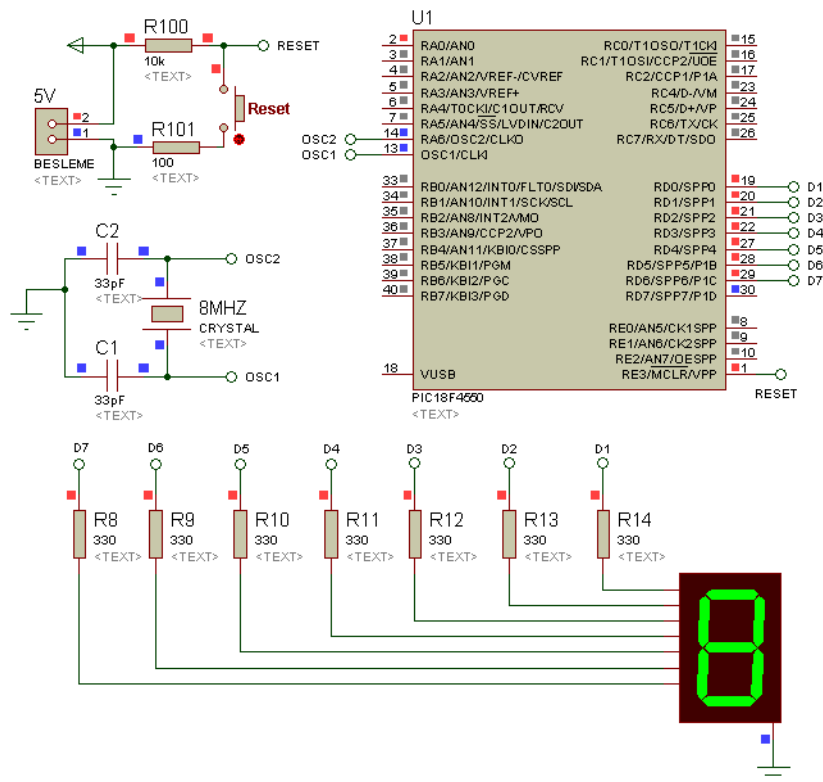
// g f e d c b a      HEX      Decimal      Görünecek sayı
const char deger[10] = {0b00111111, //0X3F      63          0
                        0b00000110, //0X06      6          1
                        0b01011011, //0X5B      91         2
                        0b01001111, //0X4F      79         3
                        0b01100110, //0X66     102         4
                        0b01101101, //0X6D     109         5
                        0b01111101, //0X7D     125         6
                        0b00000111, //0X07      7          7
                        0b01111111, //0X7F     127         8
                        0b01101111}; //0x6F     111         9

short b=0; //İlk değeri 0 olan bir kısa tam sayı (b) tanımlandı

void main() { //Ana program başlangıcı
    ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON |= 7; //Karşılaştırma modülü iptal edildi
    trisD=0; //Display'in bağlanacağı D portu çıkış olarak atandı
    portD=0; //D portu temizlendi (Lojik 0 gönderildi)
    while(1){ //Program sonsuz döngüye sokuldu
        portD= deger[b]; // "deger" isimli dizinin b. elemanı olan veri D portuna
                          //yükledi
        b++; //b'nin değerini 1 artır
        delay_ms(500); //500ms bekle
        if (b > 9) b = 0; //b'nin değeri 9'dan büyükse b'yi sıfırla (tekrar 0'dan
                          //başlamak için)
    } //Sonsuz döngünün sınırını gösteren süslü parantez
    //Ana program sonlandırıldı.
}

```

Şekil 8.1 Displayde sürekli 0'dan 9'a kadar sayan uygulamanın MikroC programındaki ekran görüntüsü



Şekil 8.2 0-9 yukarı sayıcı uygulamasına ilişkin ISIS devre şeması

UYGULAMA_02

Şekil 8.2'deki devrede de görüleceği üzere bir önceki uygulamamızda 7 parçalı displayin PIC'e bağlantısı için D portunun 7 biti kullanılmıştır. Bu bağlantı türü PIC ile bir tek displayin sürülmesinde kullanışlı olabilmektedir. Ancak çok daha fazla sayıda displayin kullanılması veya devreye buton ve LED'ler gibi ekstra bağlantıların yapılması durumunda PIC'in pin sayısı yetersiz kalabilir. Bu durumda ilâve PIC kullanılması gerekebilir. Bu olumsuzluğun önüne geçmek için 7 parçalı displayin sürülmesinde 7448 ya da 4511 gibi bir binary kodlu decimal (BCD) decoder entegresi kullanılır. Bu entegreler, 4 bitlik binary olarak kodlanmış decimal sayıyı uygun şekilde dönüştürerek displaye gönderirler. Yani sizin porta gönderdiğiniz decimal sayı, **dec2bcd** komutuyla önce sizin tarafınızdan 4 bitlik binary sayıya dönüştürülür. Ardından bu entegre, 4 bitlik binary sayıya dönüştürdüğünüz decimal sayıyı display ekranında görmenizi sağlayacak şekilde displayi sürer.

Özetleyecek olursa; ilgili porta gönderdiğiniz ve binary gösterimi 4 biti geçmeyen decimal bir sayı, yukarıda sözü edilen entegrelerden birini kullandığınız takdirde doğrudan displayde görünecektir. Dolayısıyla bir önceki örnekte yaptığımız gibi bir dizi tanımlamamıza gerek kalmayacaktır.

İşte bu uygulamada, ortak katotlu displaylerin sürülmesinde kullanılan ve yukarıda bahsi geçen 7448 kod çözücü entegre kullanılarak 0-9 yukarı/aşağı sayıcı uygulaması gerçekleştirilecektir. Program 500ms aralıklarla önce 0'dan 9'a kadar sayıp 1s bekleyecek, ardından tekrar 0'a doğru geri sayacaktır. 0'a ulaştığında da 1s bekleyip tekrar yukarı doğru saymaya devam edecektir. Bu çalışma şekli PIC'in enerjisi kesilene kadar devam edecektir.

İlgili program kodları aşağıda verilmiştir.

PROGRAM KODLARI:

```
/* Bu programda ortak katotlu displaylerin sürülmesinde kullanılan 7448 entegresi
kullanılarak 7 segment displayli 0-9 yukarı/aşağı sayıcı yapılacaktır*/
/*===== Program başlangıcı =====
short b=0;           //ilk değeri 0 olan işaretli bir kısa tam sayı (b) tanımlandı
```

```

void main() {           //Ana program başlangıcı
    ADCON1 |= 0x0F;      //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |=7;          //Karşılaştırma modülü iptal edildi
    trisD=0;             //Display'in bağlanacağı D portu çıkış olarak atandı
    portD=0;             //D portu temizlendi (Lojik 0 gönderildi)
    while(1){            //Program sonsuz döngüye sokuldu
        do{              //Yukarı saydıracak döngü başlıyor
            portD=dec2bcd(b);    //b'nin decimal değeri binary formatına dönüştürüldü ve D
                                   //portuna yüklendi

            b++;              //b'nin değerini 1 artır
            delay_ms(500);     //500ms bekle
        }while (b<10);        //b'nin değeri 10 oluncaya kadar bu döngünün içinde kal
        delay_ms(1000);       //1 saniye bekle (9 rakamına ulaşıldı)
        do{                  //Aşağı saydıracak döngü başlıyor
            b--;              //b'nin değerini 1 azalt
            portD=dec2bcd(b);    //b'nin decimal değeri binary formatına dönüştürüldü ve D
                                   //portuna yüklendi

            delay_ms(500);     //500ms bekle
        }while (b>0);         //b'nin değeri 0 oluncaya kadar bu döngünün içinde kal
        delay_ms(1000);       //1 saniye bekle (0 rakamına ulaşıldı)
    }                         //Sonsuz döngünün sınırını gösteren süslü parantez
}                             //Ana program sonlandırıldı.

//===== Program sonu =====

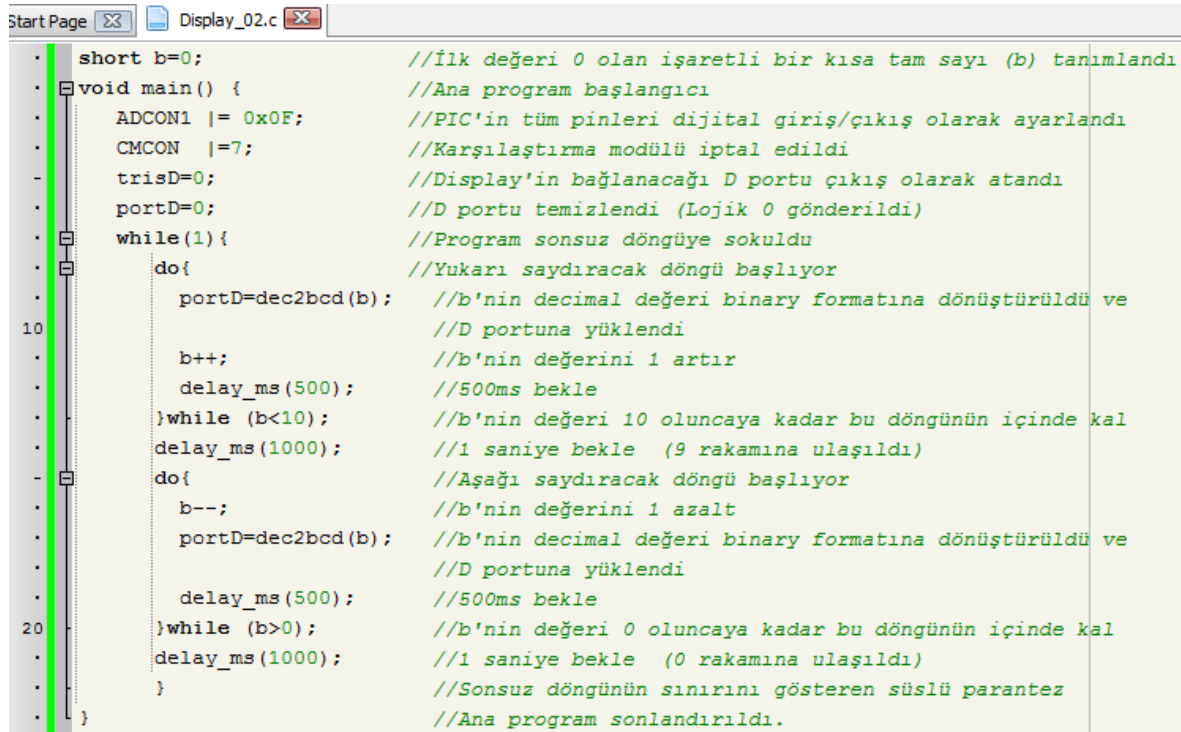
```

Program kodları incelendiğinde problemin **do-while** yapısı kullanılarak çözüldüğü görülecektir. Negatif değerler alabilen bir değişken türü olan ve program başında tanımlanan **short** türündeki **b** değişkeni, ana programa başlanmadan önce tanımlanmış ve ilk değer olarak sıfır atanmıştır. Program sonsuz döngüdeki ilk **do-while** yapısının içine girdiğinde öncelikle decimal olan bu değişkenin **dec2bcd** komutuyla dönüşümü yapıp D portuna gönderilmiştir. Daha sonra bu değişkenin değeri **b++**; koduyla 1 arttırılıp gecikme eklenmiştir.

İlk döngüdeki şart, yani **b**'nin 10'dan küçük olması şartı bozuluncaya kadar bu işlem 10 kez tekrarlanır. Dolayısıyla 0'dan 9'a kadar yukarı sayma işlemi bu döngü sayesinde gerçekleşir.

İlk **do-while** yapısından çıktıktan sonra ikinci **do-while** yapısına girilir. Bu defa displayde görülen mevcut sayı 9 olduğundan öncelikle **b**'nin değerini 1 azaltmak gerekir. O nedenle **b--;** satırı yazılır. Daha sonra, yukarı sayma işleminde yapıldığı gibi gerekli dönüşüm yapılır ve veri D portuna gönderilir. Bu döngüdeki şart **b**'nin 0'dan büyük olmasıdır. Dolayısıyla program akışı, bu şart sağlandıkça bu kontrol yapısı içinde kalmaya devam edecek ve her döngüde **b**'nin değeri 1 azalacaktır. Nihayet **b**'nin değeri sıfır olduğunda artık şart sağlanamayacak ve döngüden çıkılarak tekrar başa dönelecektir.

Uygulamanın MikroC programındaki ekran görüntüsü Şekil 3'de verilmiştir. Şayet program hatasız bir şekilde derlenir ve ardından Şekil 8.4'teki devrede uygulanırsa, yarım saniyelik aralıklarla 0'dan 9'a kadar sayılacak, ardından 1 saniye bekleyip bu kez 9'dan 0'a doğru sayma gerçekleşecektir. 0'a ulaşıldığında da 1 saniye beklendikten sonra tekrar yukarı sayma işlemi başlayacaktır. Bu işlem PIC'in enerjisi kesilene kadar devam edecektir.



```

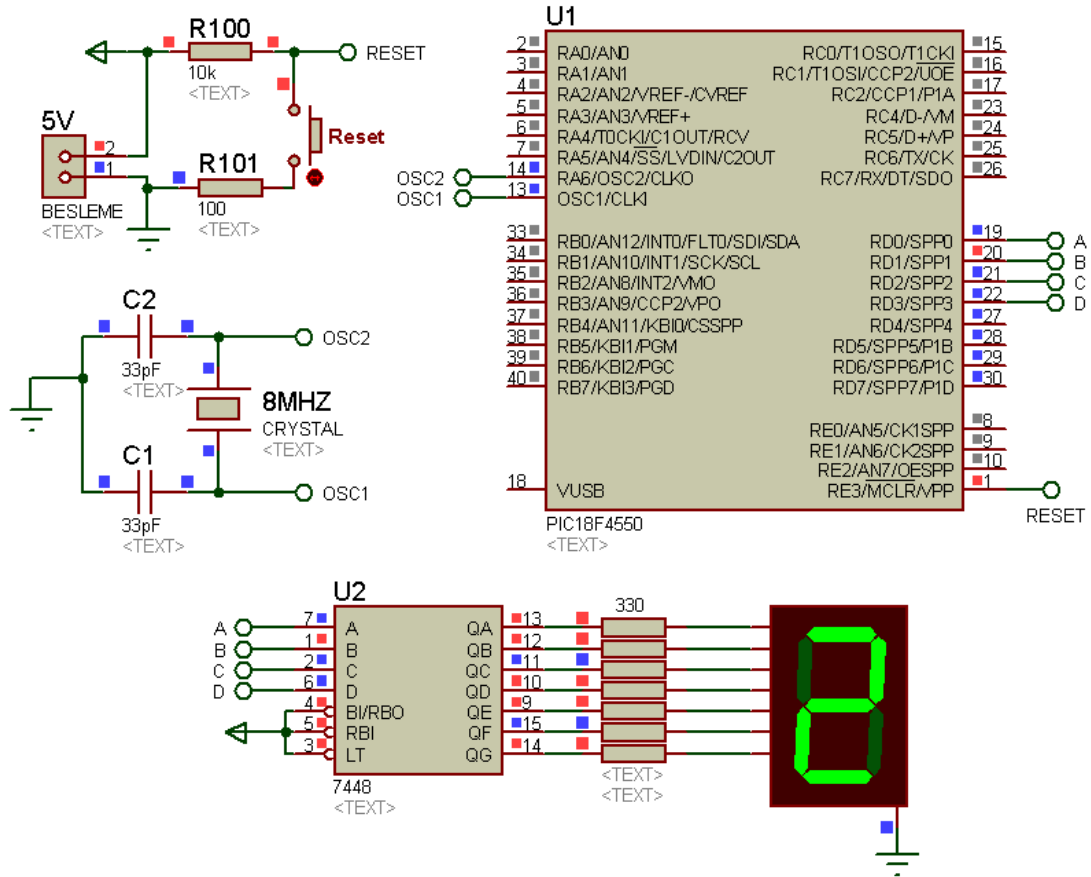
short b=0; //İlk değeri 0 olan işaretli bir kısa tam sayı (b) tanımlandı
void main() { //Ana program başlangıcı
    ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON |= 7; //Karşılaştırma modülü iptal edildi
    trisD=0; //Display'in bağlanacağı D portu çıkış olarak atandı
    portD=0; //D portu temizlendi (Lojik 0 gönderildi)
    while(1){ //Program sonsuz döngüye sokuldu
        do{ //Yukarı saydıracak döngü başlıyor
            portD=dec2bcd(b); //b'nin decimal değeri binary formatına dönüştürüldü ve
            //D portuna yüklendi
            b++; //b'nin değerini 1 artır
            delay_ms(500); //500ms bekle
        }while (b<10); //b'nin değeri 10 oluncaya kadar bu döngünün içinde kal
        delay_ms(1000); //1 saniye bekle (9 rakamına ulaşıldı)
        do{ //Aşağı saydıracak döngü başlıyor
            b--; //b'nin değerini 1 azalt
            portD=dec2bcd(b); //b'nin decimal değeri binary formatına dönüştürüldü ve
            //D portuna yüklendi
            delay_ms(500); //500ms bekle
        }while (b>0); //b'nin değeri 0 oluncaya kadar bu döngünün içinde kal
        delay_ms(1000); //1 saniye bekle (0 rakamına ulaşıldı)
    } //Sonsuz döngünün sınırını gösteren süslü parantez
} //Ana program sonlandırıldı.

```

Şekil 8.3 0-9 yukarı/aşağı sayıcı uygulamasına ilişkin MikroC programı ekran görüntüsü

AD SOYAD:

DISPLAY UYGULAMALARI



Şekil 8.4 0-9 yukarı/aşağı sayıcı uygulamasına ilişkin ISIS şeması

UYGULAMA_03

Bu örnekte 4 adet 7 segment display ile yapılmış bir sayı göstergesi (skorboard) uygulaması üzerinde durulacaktır. 0'dan 99'a kadar sayabilen bu sayı göstergesinde artırma, azaltma ve sıfırlama işlemleri için ikişer tane olmak üzere toplam 6 adet buton kullanılmıştır. Bir önceki uygulamamızda ifade ettiğimiz 7448 kod çözücü entegre bu uygulamada da kullanılmıştır. Bu entegre kullanıldığı için display bağlantılarına ayrılan toplam pin sayısı 16'da kalmıştır. Dolayısıyla butonlarla bu sayı 22'ye ulaşmıştır. Şayet kod çözücü entegre kullanılmasaydı sadece display bağlantılarına ayrılacak pin sayısı 28 olacaktı.

Uygulamaya ilişkin program kodları aşağıda verilmiştir.

PROGRAM KODLARI:

```
/*===== Program başlangıcı =====
/* Uygulamada 6 tane buton olacağından bunlara isim verilecektir. Dolayısıyla program
içerisinde bu butonların durumu kontrol edilirken bağlı oldukları bit değil bu isimler
kullanılacaktır. */
#define ARTIR_1    RC0_bit    //RC0 pini ARTIR_1 olarak tanımlandı
#define AZALT_1    RC1_bit    //RC1 pini AZALT_1 olarak tanımlandı
#define SIFIRLA_1  RC2_bit    //RC2 pini SIFIRLA_1 olarak tanımlandı
#define ARTIR_2    RC4_bit    //RC4 pini ARTIR_2 olarak tanımlandı
#define AZALT_2    RC5_bit    //RC5 pini AZALT_2 olarak tanımlandı
#define SIFIRLA_2  RC6_bit    //RC6 pini SIFIRLA_2 olarak tanımlandı
short deg_1 = 0;    //işaretli kısa tam sayı türünde deg_1 isimli değişken tanımlandı
short deg_2 = 0;    //işaretli kısa tam sayı türünde deg_2 isimli değişken tanımlandı
void main() {        //Ana program başlangıcı
    ADCON1 |= 0x0F;   //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |= 7;      //Karşılaştırma modülü iptal edildi
    TrisC=0xFF;       //C portu giriş olarak atandı (Butonlar için)
    TrisB=0; TrisD=0; //B ve D portu çıkış olarak atandı (1. ve 2. display çiftleri için)
    PortB=0; PortD=0; //B ve D portu temizlendi
```

```
Portc=0;           //C portu sıfırlandı (Butonlara basılmıyor olarak ayarlandı)
while(1){          //Program sonsuz döngüye sokuldu
    if (ARTIR_1){   //ARTIR_1 isimli butona basılmışsa aşağıdakileri yap
        deg_1++;    //deg_1 isimli değişkenin değerini 1 arttır
        while(ARTIR_1); } //ARTIR_1 butonuna basılıyken bekle
    if(AZALT_1){    //AZALT_1 isimli butona basılmışsa aşağıdakileri yap
        deg_1--;    //deg_1 isimli değişkeni 1 azalt
        while(AZALT_1); } //AZALT_1 butonuna basılıyken bekle
    if(SIFIRLA_1){  //SIFIRLA_1 isimli butona basılmışsa aşağıdakileri yap
        deg_1=0;    //deg_1 isimli değişkeni sıfırla
        while(SIFIRLA_1); } //SIFIRLA_1 butonuna basılıyken bekle
    if (ARTIR_2){   //ARTIR_2 isimli butona basılmışsa aşağıdakileri yap
        deg_2++;    //deg_2 isimli değişkeni 1 artır
        while(ARTIR_2); } //ARTIR_2 butonuna basılıyken bekle
    if(AZALT_2){    //AZALT_2 isimli butona basılmışsa aşağıdakileri yap
        deg_2--;    //deg_2 isimli değişkeni 1 azalt
        while(AZALT_2); } //AZALT_2 butonuna basılıyken bekle
    if(SIFIRLA_2){  //SIFIRLA_2 isimli butona basılmışsa aşağıdakileri yap
        deg_2=0;    //deg_2 isimli değişkeni sıfırla
        while(SIFIRLA_2); } //SIFIRLA_2 butonuna basılıyken bekle
    PortB=dec2bcd(deg_1); //deg_1'in decimal değeri binary formatına dönüştürüldü ve
                          //B portuna yüklendi
    PortD=dec2bcd(deg_2); //deg_2'in decimal değeri binary formatına dönüştürüldü ve
                          // D portuna yüklendi

    if (deg_1 > 99) deg_1=0; //deg_1'in değeri 99'dan büyükse deg_1'i sıfırla
    if (deg_1 < 0) deg_1=99; //deg_1'in değeri 0'dan küçükse deg_1'i 99 yap
    if (deg_2 > 99) deg_2=0; //deg_2'nin değeri 99'dan büyükse deg_2'yi sıfırla
    if (deg_2 < 0) deg_2=99; //deg_2'nin değeri 0'dan küçükse deg_2'yi 99 yap
} //Sonsuz döngünün sınırını gösteren süslü parantez
} //Ana program sonlandırıldı.
```

AD SOYAD:

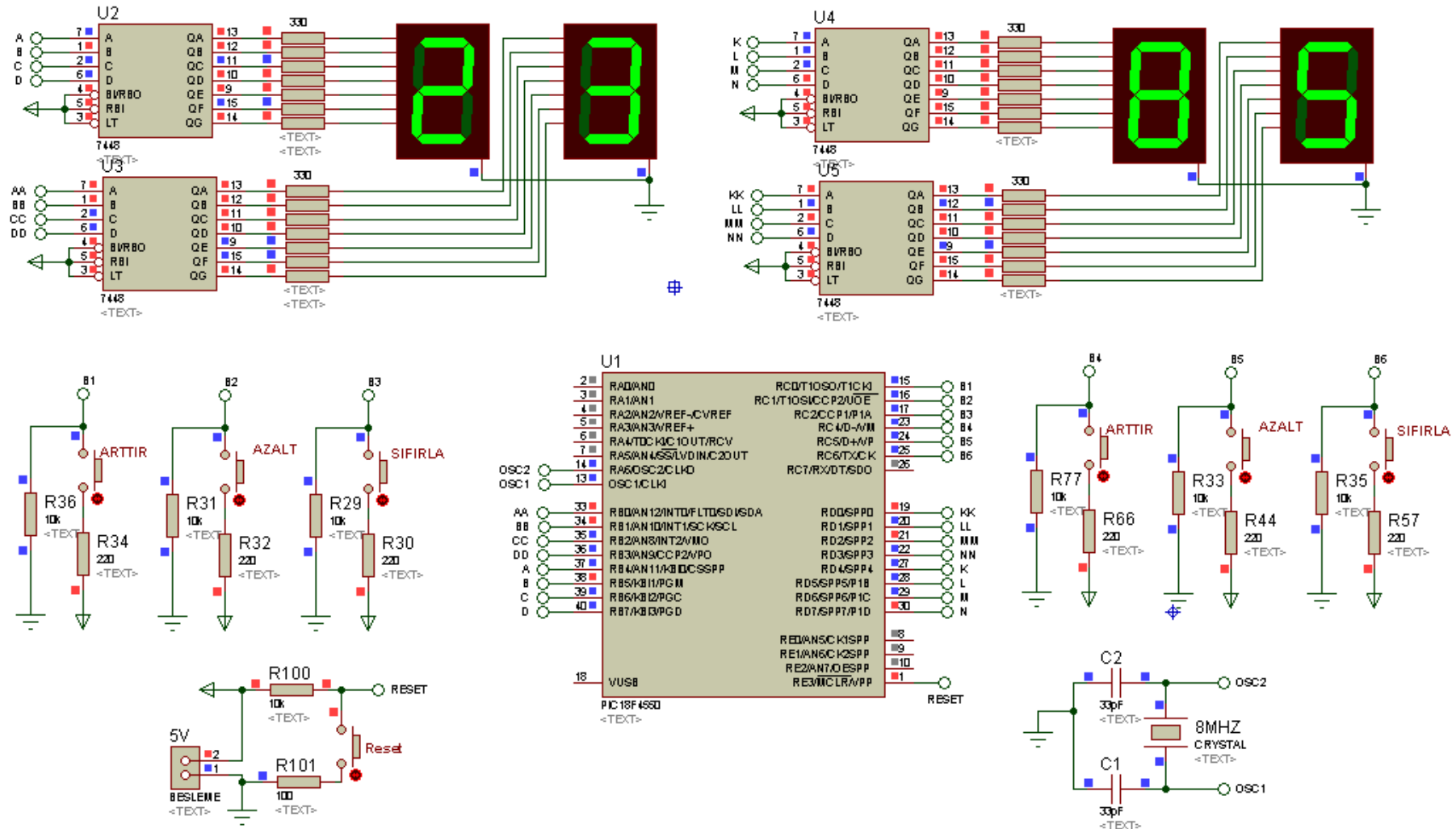
DISPLAY UYGULAMALARI

//===== Program sonu =====

Yazılan programın uzun olması münasebetiyle uygulamanın MikroC programındaki ekran görüntüsü tek seferde alınamamıştır. O nedenle bu uygulamada ekran görüntüsü verilmemiştir. Ancak program hatasız bir şekilde derlenip Şekil 8.5'teki devrede uygulanırsa, tasarlanan sistemin sorunsuz bir şekilde çalıştığı görülecektir.

AD SOYAD:

DISPLAY UYGULAMALARI



Şekil 8.5 Skorboard uygulamasına ilişkin ISIS devresi

BÖLÜM 9

9. TIMER UYGULAMALARI

UYGULAMA_01

Programın yazılımı sırasında başka bir satıra dallanma gibi herhangi bir özel kod yazılmamışsa, mikrodenetleyicilerde kodların işletilmesi ilk satırdan son satıra doğru satır satır gerçekleşir. Dolayısıyla eğer program bir döngü içerisine girmişse, döngü dışındaki bir butonun kontrolü veya PIC'in herhangi bir biti üzerinde işlem yapılamaz. Bu durum, PIC'in aynı anda iki işlemi yapamaması anlamına gelir. Örnek verecek olursak; PIC'e bağlı bir LED'in durumu butonunun durumuna göre değiştirilecekken, diğer yandan da 1 saniye aralıklarla başka bir LED grubu yakılıp söndürülemez.

Ancak bu dezavantaj, PIC'in sahip olduğu harici kesme ve TIMER (taymır diye okunur) donanım özelliği sayesinde aşılabilmektedir. PIC'lerin aynı anda yapabileceği işlem sayısı, kendi bünyelerinde barındırdıkları Timer sayısına bağlı olarak değişir. Timer0, Timer1, Timer2 şeklinde isimlendirilen Timer'lar zamanlayıcı veya sayıcı olarak kullanılabilirler.

Timer'ın zamanlayıcı olarak kullanılması durumunda oluşturulan kesmeler, aslında PIC'in ana programın akışı dışında, belirli süre sonunda rutin olarak yapılması gereken işlemleri yerine getirir. Böylelikle bu rutin işlemler için hem ana programın meşgul edilmesi önlenmiş olur hem de işlemlerin yerine getirilmesi gereken periyot sabit kalmış olur. Örneğin 5 saniyede bir A portuna bağlı LED'ler durum değiştirsin, ancak diğer taraftan da butona basılmışsa D portundaki LED'ler yakılsın gibi...

8 bit ve 16 bit modunda çalışabilen Timer'ların kesme süresi hesabında aşağıdaki formül kullanılır.

$$\text{Kesme zamanı} = \text{Komut işleme süresi} \times \text{Prescaler oranı} \times (A - \text{TMR} \times \text{ön değeri})$$

Kesme zamanı hesabında geçen komut işleme süresi, kullanılan osilatör frekansının dörtte birine ilişkin periyottur. Örneğin 8 MHz'lik osilatör kullanıldığından komut işleme süresi aşağıdaki gibi hesaplanır.

$$\text{Komut işleme süresi} = \frac{1}{\text{Osilatör frekansı (Hz)}/4} = \frac{1}{8\,000\,000/4} = 0,5 \cdot 10^{-6} \text{ s} = 0,5 \mu\text{s}$$

Prescaler oranı, **X** Timer'ındaki ilgili bitlerle ayarlanan bir değerdir. Örneğin Timer0 için bu değer, **TOCON** yazmacındaki ilk 3 bit ile ayarlanır. Timer0'da, **TOCON**'daki bu bitlerin alacağı 7 farklı kombinasyona göre değişen 8 farklı oran vardır. Bu değerler 1:2, 1:4, 1:8, 1:16, 1:32, 1:64, 1:128 ve 1:256'dır. Bu durum diğer Timer'larda değişkenlik arz etmektedir.

Yine kesme zamanı formülünde geçen **A** ifadesi 8 bit modundaki kullanım için $2^8 = 256$ 'dır. 16 bit modunda ise $2^{16} = 65536$ olmaktadır.

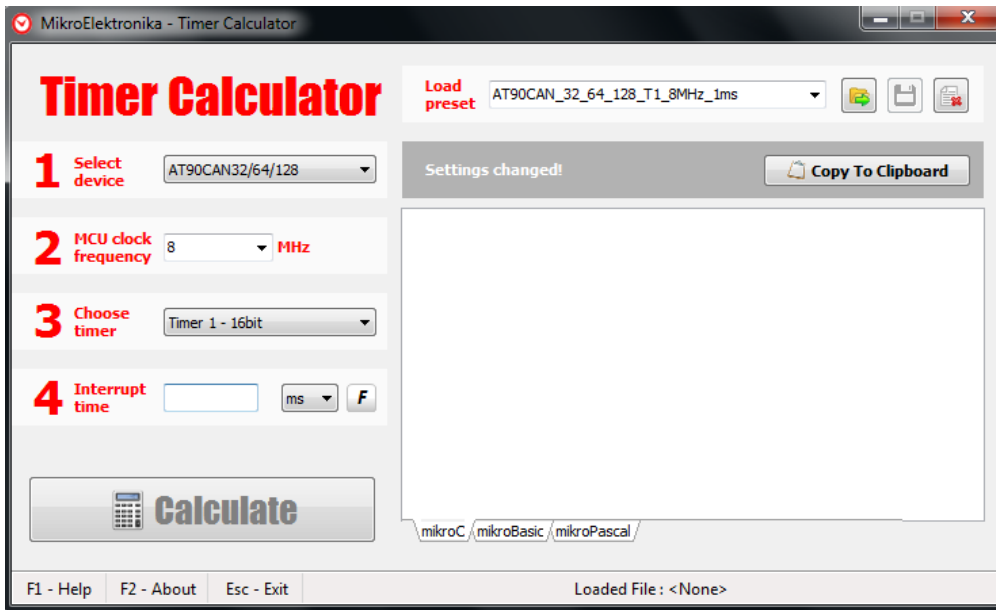
TMRX ön değeri, **X** Timer'ı ile istenen kesme zamanına ulaşmak için girilmesi gereken bir değerdir. Bu değer 8 bit modunda 255'i, 16 bit modunda 65535'i geçemez.

Tüm bu ön bilgilerden sonra, istenen kesme zamanına ulaşmak için programcının kesme süresi hesabında oynayacağı parametrelerin sabit frekans altında *Prescaler oranı* ve **TMRX** ön değeri olduğu ortaya çıkmaktadır. Eğer bu değerler uygun seçilirse, istenen süreler veya bunlara en yakın değerlere birkaç matematiksel hesaplamaadan sonra ulaşılabilir.

Ancak can sıkıcı tüm bu hesaplamalar yerine *MikroElektronika* tarafından kullanıma sunulan ve tamamen ücretsiz olan **Timer Calculator** programı, istenilen kesme sürelerinin hesaplanmasını birçok farklı mikrodenetleyici için kolaylaştırmaktadır. *MikroElektronika*'nın web sitesinden indirilen ve kurulum gerektirmeyen bu program, MikroC'nin yanı sıra MikroBasic ve MikroPascal için de kesme süresi hesabını çok kolay bir şekilde yapmanızı sağlar. Bununla da yetinmeyip ürettiği kodları yazmanız yerine kopyalayıp yapıştırmanıza imkân verir. Eğer istenen kesme süresi tam olarak ayarlanamıyorsa, bu durumda ona en yakın değerdeki süreyi, hata yüzdesiyle birlikte size sunar. Bu programı kullanmanın bir diğer avantajı da, kullandığınız PIC'in Timer donanımına ait detaylı bilgiye sahip olma gereğini ortadan kaldırmasıdır.

Sayılan tüm bu nedenlerden dolayı Timer uygulamalarında klasik hesaplama yöntemini kullanmak yerine **Timer Calculator** programını kullanmak çok daha mantıklıdır. Çünkü yeni başlayanlar bu programı kullandıklarında PIC programlama konusunda hevesleri kırılmadan rahatlıkla çok daha ileri noktalara ulaşabilmektedirler.

Bu kitabın yazımı sırasında 2.7.0 sürümlü **Timer Calculator** programı kullanılmıştır. Belirli aralıklarla yeni sürümleri çıkan bu program çalıştırıldığında Şekil 9.1'deki ekran görüntüsü ile karşılaşılacaktır.

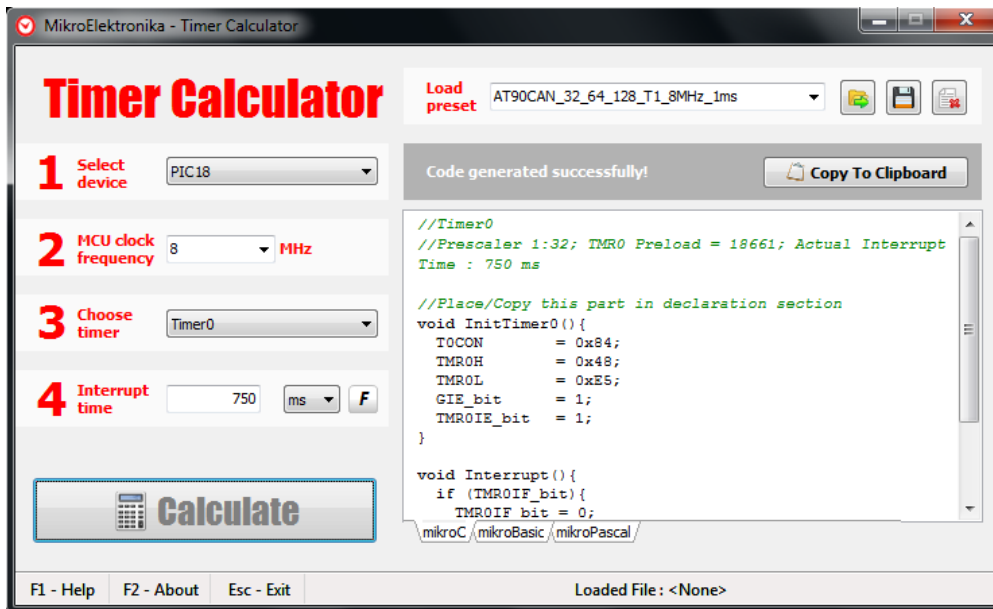


Şekil 9.1 Timer Calculator programı ekran görüntüsü

Şekil 9.1'de de görüleceği üzere programın kullanımı oldukça basittir. Öncelikle 1 numaralı bölümden kullanılan mikrodenetleyicinin serisi seçilir. Kullandığımız mikrodenetleyici 18F4550 olduğu için bizim PIC18'i seçmemiz gerekir. Ardından kullanılan osilatör frekansı 2 numaralı bölümden belirlenir. Kullanılacak Timer'ın seçimi 3 numaralı bölümden yapıldıktan sonra geriye sadece 4 numaralı bölümden istenen kesme süresinin girilmesi kalmaktadır. Bu bölüme nano saniye, mikro saniye, milisaniye ve saniye olmak üzere 4 farklı türden kesme süresi girilebilir. Burada girilecek sayının tam sayı olmasına dikkat edilmelidir. Mesela 1,25 saniyelik bir kesme süresi milisaniye cinsinden 1250 olarak girilmelidir.

Timer'ın zamanlayıcı olarak kullanılmasına yönelik olan bu örnekte Timer0 birimi kullanılacaktır. 8Mz'lik osilatörün kullanılacağı uygulamada Timer0'ın her 0,75 saniyede bir kesme oluşturması ve bu kesme gerçekleştiğinde D portuna bağlı LED'lerin durumlarını terslemesi sağlanacaktır.

Öncelikle Timer0 kullanılarak yapılacak **0,75s = 750ms**'lik kesmeyi, yukarıdaki bilgiler ışığında Şekil 9.2'deki ayarları kullanarak elde edebiliriz. Gerekli bilgiler girilip **Calculate** butonuna tıkladığımızda ihtiyaç duyduğumuz kodlar artık kullanıma hazırdır. Artık yapılacak tek şey, **Copy To Clipboard** butonuna tıklayarak bu kodu panoya kopyalamak ve ardından kod yazımı sırasında MikroC programına yapıştırmaktır.



Şekil 9.2 750ms'lik kesmeye ilişkin ayarlar ve üretilen kodlar

Copy To Clipboard butonuna tıklanarak panoya alınan kodlar, programın tamamı verilmeden önce aşağıda sunulacaktır. Her satırda ne yapıldığına dair bilgi, program kodlarının yazımı sırasında açıklama olarak verileceğinden burada verilmemiştir. Ancak genel hatlarıyla bu kodlar içerisinde;

- TOCON yazmacında gerekli ayarları yapmak için TOCON yazmacına değer yüklenmesi,
- 750ms'yi elde etmek için Timer0 ön değeri yüklenmesi,
- Tüm kesmelerin aktif edilmesini sağlayan bitin lojik 1 yapılması,

- Timer0 kesmesine izin verilmesi,
- Timer0 kesmesinin gerçekleşip gerçekleşmediğinin kontrol edilmesi,
- Kesme meydana geldiğinde tekrar yapılması gereken ayarlar bulunmaktadır.

Yine kodlar incelendiğinde anlaşılaçağı üzere bu kodlar içerisinde aşağıdaki açıklama satırları da bulunmaktadır:

- Kullanılan Timer bilgisi (Timer0),
- Prescaler oranı (1:32),
- Timer0 için yüklenecek ön değeri (Preload = 18661),
- Gerçek kesme zamanı (Actual Interrupt Time : 750 ms)
- Kodunuzu buraya giriniz (Enter your code here)

Timer Calculator programı tarafından üretilen kodların yalın hâli:

```
//Timer0
//Prescaler 1:32; TMR0 Preload = 18661; Actual Interrupt Time : 750 ms
//Place/Copy this part in declaration section
void InitTimer0(){
    TOCON      = 0x84;
    TMR0H      = 0x48;
    TMR0L      = 0xE5;
    GIE_bit     = 1;
    TMR0IE_bit  = 1;
}

void Interrupt(){
    if (TMR0IF_bit){
        TMR0IF_bit = 0;
        TMR0H      = 0x48;
        TMR0L      = 0xE5;
        //Enter your code here
```

```

    }
}

```

Uygulamaya ilişkin program kodlarının tamamı aşağıda görülmektedir.

PROGRAM KODLARI:

```

//===== Program başlangıcı =====
//===Timer Calculator programından kopyalanan kodları aşağıya yapıştırıyoruz===
//Prescaler 1:32; TMR0 Preload = 18661; Actual Interrupt Time : 750 ms
void InitTimer0(){ // Timer0 ayarlarının yapılacağı InitTimer0 isimli fonksiyon oluşturuluyor
    TOCON= 0x84; //Timer0 için Prescaler 1:32
    TMR0H = 0x48; //750ms'yi elde etmek için yüklenecek Timer0 ön değeri (18661'in High
                  //kısmı)
    TMR0L = 0xE5; //750ms'yi elde etmek için yüklenecek Timer0 ön değeri (18661'in Low
                  //kısmı)
    GIE_bit = 1; //Tüm kesmeler aktif edildi
    TMR0IE_bit = 1; //Timer0 kesmesine izin verildi
} // InitTimer0 isimli fonksiyonun sonu
void Interrupt(){ //Interrupt isimli fonksiyon oluşturuluyor
    if (TMR0IF_bit){ //Timer0 kesmesi oluştuğunda 1 olan bayrak (Flag) kontrol ediliyor
        TMR0IF_bit = 0; //Kesme bayrağı bir sonraki kesme için sıfırlandı
        TMR0H = 0x48; //750ms'yi elde etmek için yüklenecek Timer0 ön değeri (18661'in High
                      //kısmı)
        TMR0L = 0xE5; //750ms'yi elde etmek için yüklenecek Timer0 ön değeri (18661'in Low
                      //kısmı)
        //Kesme gerçekleştiğinde yapılması istenilen şey aşağıya yazılıyor
        PORTD = ~PORTD; //D portunu tersle
    } //if kontrol yapısı sonlandırıldı
} //Interrupt isimli fonksiyonun sonu
//===== Timer Calculator programından kopyalanan kodlar burada bitti =====
//===== Ana program yazılıyor =====

```

```

void main() {           //Ana program başlatılıyor
    ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |= 7;      //Karşılaştırma modülü iptal edildi.
    TRISD = 0;        //D portu çıkış olarak ayarlandı
    PORTD = 0xCC;      //D portunda 2,3,6 ve 7. bitlere bağlı LED'ler yakıldı
    InitTimer0();      //InitTimer0 (Timer0 kesmesi) fonksiyonu başlatıldı
    while(1);          //Ana program hiçbir şey yapmadan burada bekletildi
}                       //Ana programın sonu

//===== Program sonu =====

```

Kesme hesabının yapılması dışında kesme gerçekleştiğinde ve sonrasında ilgili yazmaçlarda yapılması gereken şeylerin olduğu, ancak bunların da **Timer Calculator** programı tarafından otomatik olarak yapıldığı kodlar incelendiğinde görülmektedir. Daha önce de ifade edildiği gibi burada programcının yapması gereken tek şey, kesme gerçekleştiğinde ne yapılmasını istiyorsa ona ilişkin kodları yazmaktır. Bizim programımızda sadece D portunun durumu tersleneceğinde buna ilişkin kod yazılmıştır.

Ana program içerisinde yapmamız gereken şey ise, Timer ayarlarının yapılacağı fonksiyonu başlatmaktır. Bu fonksiyonun ismi **Timer Calculator** programı tarafından **InitTimer0** olarak verilmiştir. Eğer kullanıcı isterse bu fonksiyon ismini değiştirebilir. Dolayısıyla ana program içerisinde bu yeni ismi yazması gerekecektir.

Kodların MikroC programındaki ekran görüntüsü ve devre şeması sırasıyla Şekil 9.3 ve Şekil 9.4'te görülmektedir.

```

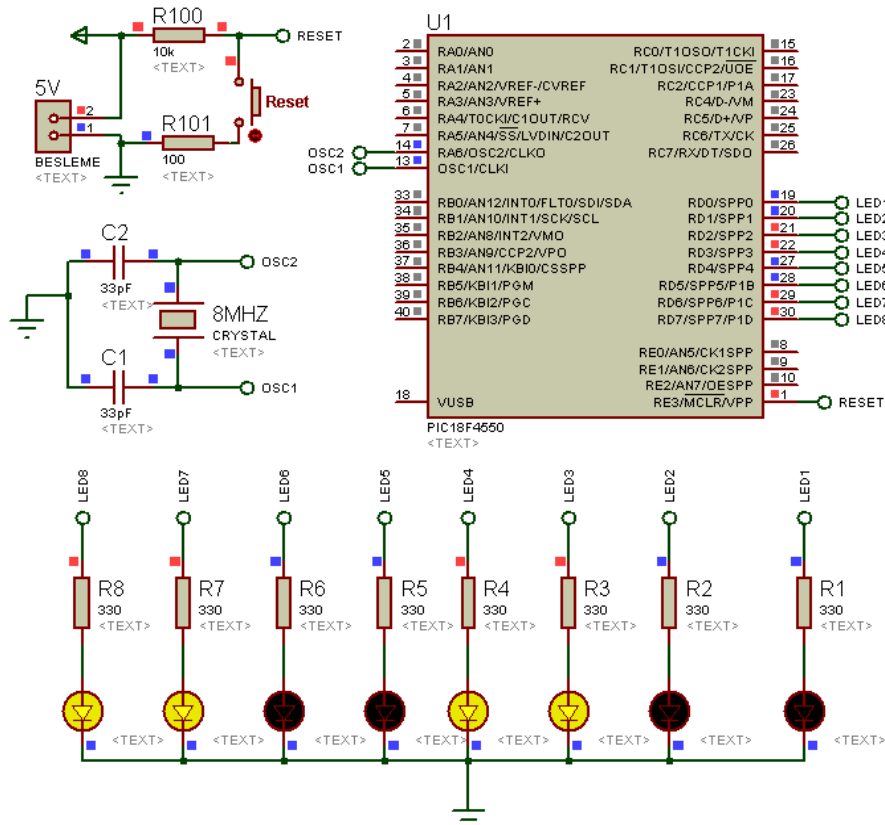
Start Page  Timer_01.c
//====Timer Calculator programından kopyalanan kodları aşağıya yapıştırıyoruz====
//Prescaler 1:32; TMR0 Presload = 18661; Actual Interrupt Time : 750 ms
void InitTimer0(){
    TOCON = 0x84; //Timer0 için Prescaler 1:32
    TMR0H = 0x48; //750ms'yi elde etmek için yüklenecek Timer0 ön değeri (18661'in High kısmı)
    TMR0L = 0xE5; //750ms'yi elde etmek için yüklenecek Timer0 ön değeri (18661'in Low kısmı)
    GIE_bit = 1; //Tüm kesmeler aktif edildi
    TMR0IE_bit = 1; //Timer0 kesmesine izin verildi
}

void Interrupt(){ //Interrupt isimli fonksiyon oluşturuluyor
    if (TMR0IF_bit){ //Timer0 kesmesi oluştuğunda 1 olan bayrak (Flag) kontrol ediliyor
        TMR0IF_bit = 0; //Kesme bayrağı bir sonraki kesme için sıfırlandı
        TMR0H = 0x48; //750ms'yi elde etmek için yüklenecek Timer0 ön değeri (18661'in High kısmı)
        TMR0L = 0xE5; //750ms'yi elde etmek için yüklenecek Timer0 ön değeri (18661'in Low kısmı)
        //Kesme gerçekleştiğinde yapılması istenilen şey aşağıya yazılıyor
        PORTD = ~PORTD; //D portunu tersle
    }
}

//===== Timer Calculator programından kopyalanan kodlar burada bitti =====
//===== Ana program yazılıyor =====
void main() { //Ana program başlatılıyor
    ADCON1 |= 0x0F; //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON |= 7; //Karşılaştırma modülü iptal edildi.
    TRISD = 0; //D portu çıkış olarak ayarlandı
    PORTD = 0xCC; //D portunda 2,3,6 ve 7. bitlere bağlı LED'ler yakıldı
    InitTimer0(); //InitTimer0 (Timer0 kesmesi) fonksiyonu başlatıldı
    while(1); //Ana program hiçbir şey yapmadan burada bekletildi
}

```

Şekil 9.3 Timer0'ın zamanlayıcı olarak kullanılmasına ilişkin MikroC programının ekran görüntüsü

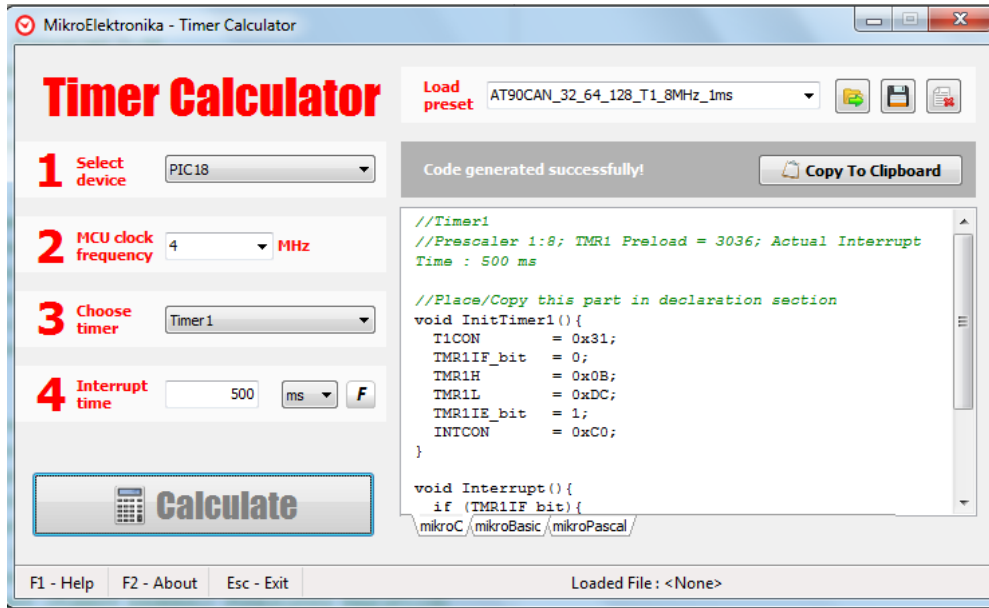


Şekil 9.4 Timer0'ın zamanlayıcı olarak kullanılmasına yönelik oluşturulan ISIS şeması

UYGULAMA_02

Bir önceki uygulamada Timer0'ın zamanlayıcı olarak çalıştırılmasıyla elde edilen sürelerde LED'lerin yanıp sönməsi sağlanmıştı. Ancak bu uygulamada buna ilaveten buton kontrollü LED'de bulunmaktadır. Dolayısıyla Timer donanımının ana program akışından bağımsız olarak nasıl çalıştığını gözlemlemek adına önemli bir örnek olacaktır.

Bu uygulamada da yine D portuna bağılı LED'ler kullanılacaktır. Bu LED'lerin durumu Timer'ın zamanlayıcı olarak kullanılması ile elde edilen kesmeler sonunda değıştirilecektir. Ancak bu kez 4MHz'lik osilatör kullanılmakta olup Timer0 yerine Timer1, 0.75s yerine 0.5s'lik kesme kullanılacaktır. Ayrıca RA1 pinine bağılı butona basılmışsa RB1 pinine bağılı LED'in durumu terslenecektir. Buna ilişkin program kodlarına geçmeden önce, kesme süresinin hesaplanma görüntüsü Şekil 9.5'te verilmiştir.



Şekil 9.5 500ms'lik kesme süresinin hesaplanmasına ilişkin ekran görüntüsü

Timer Calculator programı tarafından üretilen kodlar herhangi bir değışiklik yapılmadan aşağıda sunulmuştur.

```
//Timer1
```

```
//Prescaler 1:8; TMR1 Preload = 3036; Actual Interrupt Time : 500 ms
```

```
//Place/Copy this part in declaration section
```

```

void InitTimer1(){
    T1CON      = 0x31;
    TMR1IF_bit = 0;
    TMR1H      = 0x0B;
    TMR1L      = 0xDC;
    TMR1IE_bit = 1;
    INTCON     = 0xC0;
}

void Interrupt(){
    if (TMR1IF_bit){
        TMR1IF_bit = 0;
        TMR1H      = 0x0B;
        TMR1L      = 0xDC;
        //Enter your code here
    }
}

```

Yukarıda sözü edilen çalışmayı gerçekleştirecek program kodlarının tamamı aşağıda görülmektedir. Ancak **Timer Calculator** programı tarafından üretilen kodlarda **InitTimer1** olarak oluşturulan fonksiyonun ismi bu uygulamada **Alpaslan** olarak değiştirilmiştir. Daha önce de ifade edildiği gibi bu fonksiyonun vazifesi kesme ayarlarını yapmaktır.

Üretilen kodların içeriği, yani yüklenen ön değer ve Prescaler gibi değerler bizim için önemli olmadığından, **Timer Calculator** tarafından üretilen kodların satır satır açıklaması bundan sonra verilmeyecektir. Uygulamaya ait program kodlarının tamamı aşağıda verilmiştir.

PROGRAM KODLARI:

```

//===== Program başlangıcı =====
/* Timer'ın zamanlayıcı olarak kullanılmasına yönelik olan bu örnekte Timer1 birimi kullanılacaktır. 4Mz'lik osilatörün kullanılacağı uygulamada Timer1'in her 500ms bir kesme

```


oluşturması ve bu kesme gerçekleştiğinde D portuna bağlı LED'lerin durumlarını terslemesi sağlanacaktır. Ayrıca RA1 pinine bağlı butona basılmışsa RB1 pinine bağlı LED'in durumu terslenecektir. */

//===Timer Calculator programından kopyalanan kodları aşağıya yapıştırıyoruz=====

//Timer1, Prescaler 1:8; TMR1 Preload = 3036; Actual Interrupt Time : 500 ms

void Alpaslan(){ *//Timer1ayarlarının yapılacağı fonksiyonun ismini Alpaslan olarak değiştirdik*

T1CON = 0x31;

TMR1IF_bit = 0;

TMR1H = 0x0B;

TMR1L = 0xDC;

TMR1IE_bit = 1;

INTCON = 0xC0;

}

void Interrupt(){

if (TMR1IF_bit){

TMR1IF_bit = 0;

TMR1H = 0x0B;

TMR1L = 0xDC;

//Kesme gerçekleştiğinde yapılması istenilen şey aşağıya yazılıyor

PORTD = ~PORTD; *//D portunu tersle*

}

} *// Timer Calculator programından kopyalanan kodlar burada bitti*

//===== Ana program yazılıyor =====

void main() { *//Ana program başlatılıyor*

ADCON1 |= 0x0F; *//PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı*

CMCON |= 7; *//Karşılaştırma modülü iptal edildi.*

TRISA1_bit=1; *//RA1 pini buton için GİRİŞ olarak ayarlandı*

TRISB1_bit=0; *//RB1 pini LED için ÇIKIŞ olarak ayarlandı*

TRISD = 0; *//D portu çıkış olarak ayarlandı*

RA1_bit=0; *//Butona basılmıyor olarak ayarlandı*

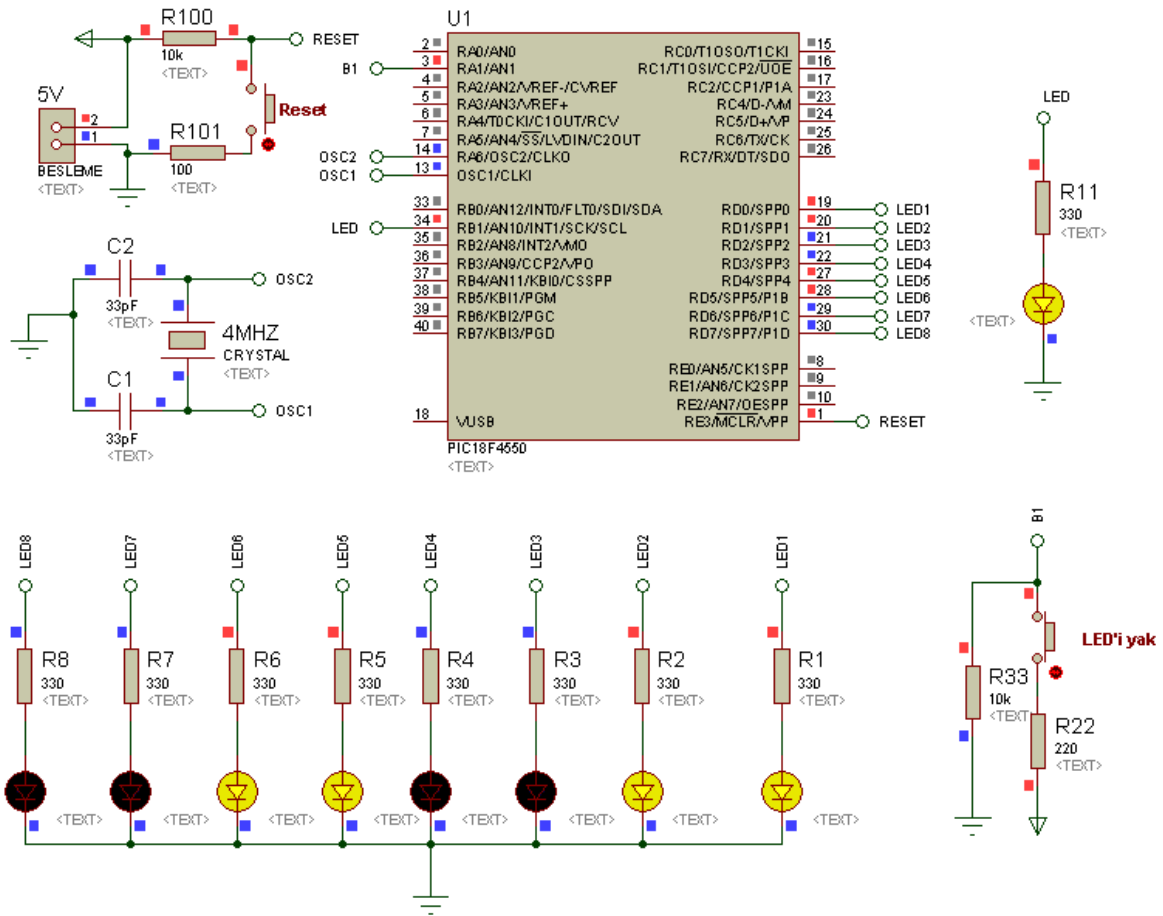
```

RB1_bit=0;           //RB1 pinine bağlı LED sönmük olarak ayarlandı
PORTD = 0xCC;        //D portunda 2,3,6 ve 7. bitlere bağlı LED'ler yakıldı
Alpaslan();          //Alpaslan (Timer1 kesmesi) fonksiyonu başlatıldı
while(1){            //Sonsuz döngüye girildi
    if(RA1_bit){      //Eğer butona basılmışsa süslü parantez içindekileri yap
        RB1_bit=~RB1_bit; //RB1' bağlı LED'i tersle
        while(RA1_bit);  //Butona basılıyken bekle
    }                  //if kontrol yapısının sonu
}                      //Sonsuz döngünün sonu
}                      //Ana programın sonu
//===== Program sonu =====

```

Program kodları incelendiğinde ana program içerisindeki sonsuz döngüde sadece buton kontrolünün yapıldığı görülmektedir. Şayet butona basılmışsa ilgili LED'in durumu terslenmiş ve butona basılıyken programın orada kalması sağlanmıştır. Buton uygulamalarımız hatırlanacak olursa *butona basılıyken bekle* komutu eklendiğinde program akışı orada durur. Şayet buton serbest bırakılmışsa program devam eder. Aynı mantık burada da uygulanmıştır. Ancak buton basılıyken D portuna bağlı LED'lerin belirlenen aralıklarla yanıp sönmeye devam ettiği görülmektedir. Bu durum, Timer kesmesi kullanılarak gerçekleştirilen işlemlerin, ana programdan bağımsız olarak işletildiğini gösterir. Aksi halde buton basılıyken D portuna bağlı LED'lerin durumlarının kesinlikle değişmemesi gerekirdi.

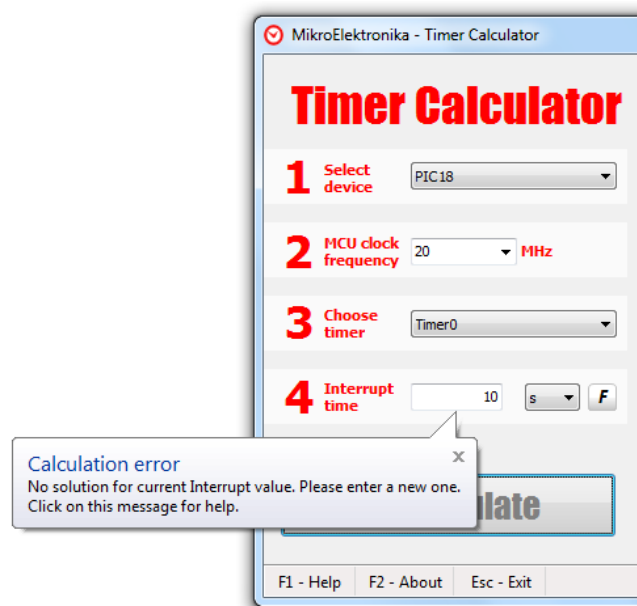
Bu uygulamaya ait MikroC ekran görüntüsü tek seferde alınamadığı için verilememiştir. Ancak ISIS şeması Şekil 9.6'da verilmiştir. Yukarıda sözü edilen hususları butona basılı tutmak suretiyle gözlemlemeniz, Timer kesmesi mantığını daha iyi kavramanız açısından önemli olacaktır.



Şekil 9.6 Timer1'in zamanlayıcı olarak kullanılmasına yönelik oluşturulan ISIS şeması

UYGULAMA_03

Timer'lar belirli değerlere kadar sayabilirler. Dolayısıyla istenilen her kesme süresi değerine doğrudan Timer ile ulaşılamaz. Örneğin 20MHz'lik osilatör frekansında Timer0 ile 10s'lik bir kesme süresi istenildiğinde bunun mümkün olamayacağı, Şekil 9.7'deki ekran görüntüsünden de anlaşılacaktır.



Şekil 9.7 Timer'lardaki kesme sürelerinde bir sınırlama olduğuna ilişkin ekran görüntüsü

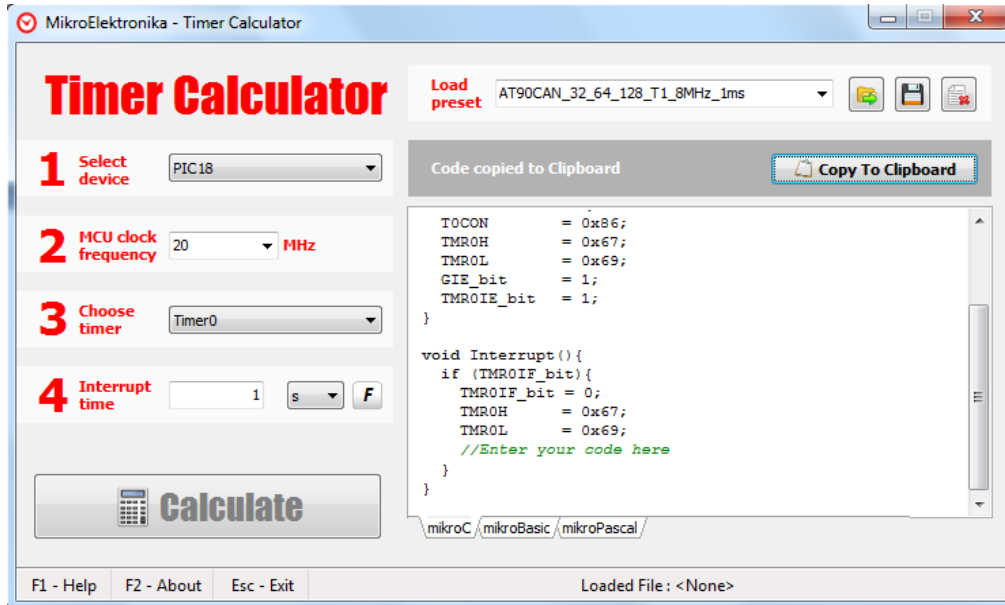
Ancak bu problemi aşmak da oldukça kolaydır. Burada yapılması gereken tek şey bir değişken tanımlayıp bu değişkenin değerini her kesme gerçekleştiğinde bir artırmak ve istenilen değere (yani zaman gecikmesine) ulaşıldığında işlemleri gerçekleştirmektir. O nedenle bu uygulamada buna ilişkin bir örnek uygulama yapılacaktır.

Uygulamada 20MHz'lik bir osilatör ve Timer0 birimi kullanılacak olup her 10s'de bir D portuna bağlı LED'lerin durumu terslenecektir. 10s'lik kesme süresine bu şartlarda Timer0 ile ulaşılamayacağından önce 1s'lik kesme gerçekleştirilecek, ardından tanımlanan değişkenin değeri her kesmede 1 arttırılacaktır. Değişken değeri 10'a ulaştığında (yani 10s'lik gecikmeye ulaşıldığında) ise D portu terslenecektir. 10s'lik gecikmeye tekrar baştan başlamak için son olarak değişken sıfırlanacaktır.

AD SOYAD:

TIMER UYGULAMALARI

Şekil 9.8'de 1s'lik kesme süresi için **Timer Calculator** programındaki ayarlar görülmektedir.



Şekil 9.8 1s'lik Timer0 kesmesine ait ekran görüntüsü

Program tarafından üretilen kodlar yalın haliyle aşağıda verilmiştir. Uygulamaya ait tüm kodlar ise Program Kodları başlığı altında sunulmuştur.

```
//Timer0
```

```
//Prescaler 1:128; TMR0 Preload = 26473; Actual Interrupt Time : 1.0000128 s
```

```
//Place/Copy this part in declaration section
```

```
void InitTimer0(){
```

```
    TOCON      = 0x86;
```

```
    TMR0H      = 0x67;
```

```
    TMR0L      = 0x69;
```

```
    GIE_bit    = 1;
```

```
    TMR0IE_bit = 1;
```

```
}
```

```
void Interrupt(){
```

```

    if (TMROIF_bit){
        TMROIF_bit = 0;
        TMROH      = 0x67;
        TMR0L      = 0x69;
        //Enter your code here
    }
}

```

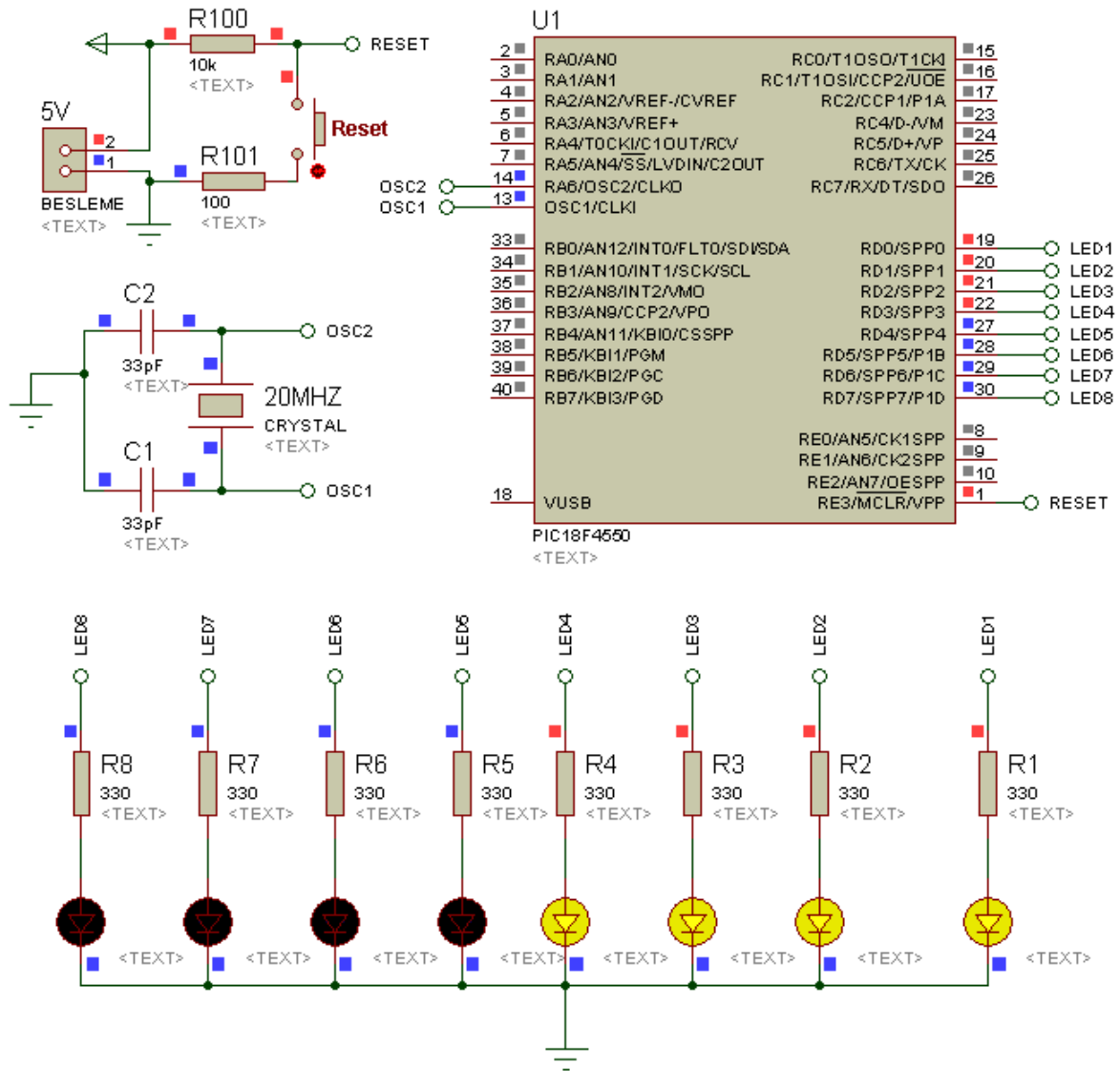
PROGRAM KODLARI:

```

//===== Program başlangıcı =====
short sayac =0;  //sayac isimli bir kısa tam sayı tanımladık
//=== Timer Calculator programından kopyalanan kodları aşağıya yapıştırıyoruz ===
//Timer0, Prescaler 1:128; TMR0 Preload = 26473; Actual Interrupt Time: 1.0000128 s
void Hakan(){ //Timer0 ayarlarının yapılacağı fonksiyonun ismini Hakan olarak değiştirdik
    TOCON = 0x86;
    TMROH = 0x67;
    TMR0L = 0x69;
    GIE_bit = 1;
    TMROIE_bit = 1;
}
void Interrupt(){
    if (TMROIF_bit){
        TMROIF_bit = 0;
        TMROH = 0x67;
        TMR0L = 0x69;
        //Kesme gerçekleştiğinde yapılması istenilen şeyler aşağıya yazılıyor
        sayac++;           //sayac isimli değişkenin değerini bir arttır
        if(sayac==10){     //sayacın değeri 10 olduysa aşağıdakileri yap
            PORTD = ~PORTD; //D portunu tersle
            sayac=0;        // sayacı sıfırla
        }
    }
}

```

```
    }  
}  
}  
  
//===== Timer Calculator programından kopyalanan kodlar burada bitti =====  
//===== Ana program yazılıyor =====  
void main() {           //Ana program başlatılıyor  
    ADCON1 |= 0x0F;      //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı  
    CMCON  |= 7;         //Karşılaştırma modülü iptal edildi.  
    TRISD = 0;           //D portu çıkış olarak ayarlandı  
    PORTD = 240;         //D portunda 4,5,6 ve 7. bitlere bağlı LED'ler yakıldı  
    Hakan();             //Hakan (Timer0 kesmesi) fonksiyonu başlatıldı  
    while(1){           //Sonsuz döngüye girildi  
                           //Sonsuz döngü içinde hiçbirsey yapmadığımıza dikkat ediniz  
    }                   //Sonsuz döngünün sonu  
}                       //Ana programın sonu  
  
//===== Program sonu =====  
  
Uygulamaya ait kodların MikroC programındaki ekran görüntüsü tek seferde  
alınamadığından burada ona yer verilememiştir. Ancak ISIS devre şeması Şekil 9.9'da  
görölmektedir.
```

Şekil 9.9 Timer uygulaması

UYGULAMA_04

Timer'ların zamanlayıcı ve sayıcı olarak kullanılabileceğine daha önce değinmiştik. Buraya kadar yaptığımız Timer uygulamalarında da Timer'ları hep zamanlayıcı olarak kullandık. Ancak bu uygulamada Timer'ın sayıcı olarak nasıl kullanılabileceğine ilişkin bir örnek yapılacaktır. Timer sayıcı olarak kullanıldığında, PIC'in ilgili bitine gelen harici sinyalleri düşen veya yükselen kenarda sayar ve programcının kullanımına sunar. Programcı da bu bilgiyi ilgili yerlerde kullanır. Mesela motorların devir sayısını ölçmek için kullanılan enkoderlerden gelen sinyal bu yöntemle sayılarak bir motorun devir sayısı ölçülebilir. Timer'ın sayıcı olarak kullanılması durumunda ana program, Timer'ların zamanlayıcı olarak kullanılmasıdaki gibi başka görevleri yerine getirebilir. Dolayısıyla harici sinyallerin sayılması işlemi ana program içerisinde yapılmadığından programın işleyişi veya harici sinyallerin sayılması işlemi sekteye uğramaz.

Timer'ın zamanlayıcı modunda kullanımı durumunda kesme sürelerinin hesabı *Timer Calculator* programı tarafından kolayca üretiliyordu. Ancak sayıcı durumunda böyle bir kolaylık olmadığını üzülerek ifade etmek gerekir. Fakat kesme oluşma sayılarının hesabı, kesme sürelerinin hesabından daha kolaydır. Çünkü kesme sayısı hesabı, kullanılan osilatör frekansından bağımsızdır. Bu da işlem sayısını azaltmakta ve frekansa bağlı farklı kombinasyonları deneme zarureti ortadan kaldırmaktadır.

Timer kullanılarak oluşturulacak kesme sayısı hesabı aşağıda verilmiştir. Formüldeki üç parametreyle oynayarak arzulanan kesme oluşma sayısına ulaşılabilir.

$$\text{Kesme oluşması zamanı} = \text{Prescaler oranı} \times (A - \text{Timer} \times \text{ön değeri})$$

Formülde geçen *Prescaler değeri*, **X** Timer'ındaki ilgili bitlerle ayarlanan bir değer olup buna ilişkin detaylı bilgi ilk Timer uygulamasında verilmişti. O nedenle burada tekrar değinilmeyecektir.

A ifadesi 8 bit modundaki kullanım için $2^8 = 256$ 'dır. 16 bit modunda kullanılması durumunda ise $2^{16} = 65536$ değerini alır. Dolayısıyla çok daha büyük değerlere ulaşmak için

programcının 16 bit modunu kullanması gerekir. Bu uygulamada da 16 bit modu tercih edilecektir.

TMRX ön değeri, istenen kesme sayısına **X** Timer'ını kullanarak ulaşmak için girilmesi gereken bir değerdir. Bu değer 8 bit modunda 255'i, 16 bit modunda 65535'i geçemez.

Anlatılanlara ilişkin yapacağımız bu uygulamada 20MHz'lik bir osilatör ve Timer0 birimi kullanılacaktır. Sayıcı olarak Timer0 kullanılacağından harici sinyal, bu Timer'a ait harici sinyal girişi olan **TOCKI** pinine bağlanacaktır. Bağlanacak olan sinyal bizim uygulamamızda bir butondan ibarettir. Dolayısıyla butona her basıldığında **TOCKI** pinine **lojik 0** gelecektir. Düşen kenar ayarlamasına göre de değer bir artacaktır. Arzulanan sayıya ulaşıldığında ise (ki uygulamamızda bu değer 6'dır) B portundaki ilgili LED'lerin durumu terslenecektir.

Timer'ın zamanlayıcı olarak kullanıldığı uygulamalarda olduğu gibi bu uygulamada da sayıcı ayarı bir fonksiyon olarak tanımlanacaktır. Dolayısıyla ana program içerisinde sadece sayıcı fonksiyonunu başlatmak yeterli olacaktır. Bunun dışında ana programda başka hiçbir işlem yaptırılmayacaktır.

Uygulamada kullanılacak kesme oluşma sayısı 6'dır. Bu değere **Prescaler değeri 2**, **TMRX ön değeri** 65533 alındığında ulaşılabilmektedir. Yukarıdaki formülün kullanıldığı buna ait hesaplama aşağıda verilmiştir.

$$\text{Kesme oluşma sayısı} = 2 \times (65536 - 65533) = 6$$

65533 olan **TMRX ön değeri**, 16 bit modunda kullanım nedeniyle TMR0L ve TMR0H yazmacına 8 bitlik iki parça olarak yüklenir. Hexadecimal karşılığı FFFD olan 65533 sayısının ilk 8 biti TMR0L yazmacına, son 8 biti de TMR0H yazmacına yüklenir. Buna ilişkin yazılan kodlar TMR0H = 0xFF; ve TMR0L = 0xFD; şeklinde olmaktadır.

GIE biti tüm kesmeleri aktif etme bitidir. **PEIE** ise çevresel kesmelerin aktif edildiği bittir. Timer'ın sayıcı olarak kullanıldığı bu tür uygulamalarda bu bitlerin **lojik 1** yapılması gerekir. Bu bitler müstakil olarak **lojik 1** yapılabileceği gibi, kesmelerin kontrol edildiği **INTCON** yazmacına doğrudan ilgili değer yüklenerek de çözülebilir. Aşağıda da görüleceği üzere bu uygulamada **INTCON** yazmacına doğrudan ilgili değer yüklenerek ilgili kesmelere

izin verme işlemi gerçekleştirilmiştir. Bu ve diğer yazmaçlarla ilgili detaylı bilgiye 18F4550'nin veri sayfasından ulaşılabilir.

Uygulamaya ilişkin program kodları aşağıda verilmiştir.

PROGRAM KODLARI:

```
//===== Program başlangıcı =====

//Sayıcı ayarı için Serkan isimli fonksiyon tanımlanıyor
void Serkan(){ //Timer0 ayarlarının yapılacağı fonksiyonu Serkan olarak ayarladık
    TMR0L = 0xFD;           // TMR0L ön değeri yüklendi (65533'ün Low kısmı)
    TMR0H = 0xFF;           // TMR0H ön değeri yüklendi (65533'ün High kısmı)
    TOCON = 0xF0;           // TMR0 aktif, 16 bit modu, giriş olarak T0CKI seçili, düşen kenar
                             // tetiklemesi aktif, prescaler 000 = 1:2 olarak atandı
    INTCON = 0xC0;           // GIE, PEIE bitleri set edildi (Rutin işlem)
    TMR0IE_bit = 1;          // TMRO kesmesi aktif edildi
}

//Kesme gerçekleştiğinde yapılacaklar ve yapılması gerekenler yazılıyor
void interrupt() {
    if (TMR0IF_bit) {        // Timer0 kesme bayrağı biti 1 ise (YAPILMASI GEREK)
        TMR0L = 0xFD;        // TMR0L ön değeri yüklendi (YAPILMASI GEREK)
        TMR0H = 0xFF;        // TMR0H ön değeri yüklendi (YAPILMASI GEREK)
        TMR0IF_bit = 0;      // Timer0 kesme bayrağı biti sıfırlandı (YAPILMASI GEREK)
        PORTD = ~PORTD;      // D portunu tersle (YAPILMASINI İSTEDİĞİNİZ ŞEY)
    }
}

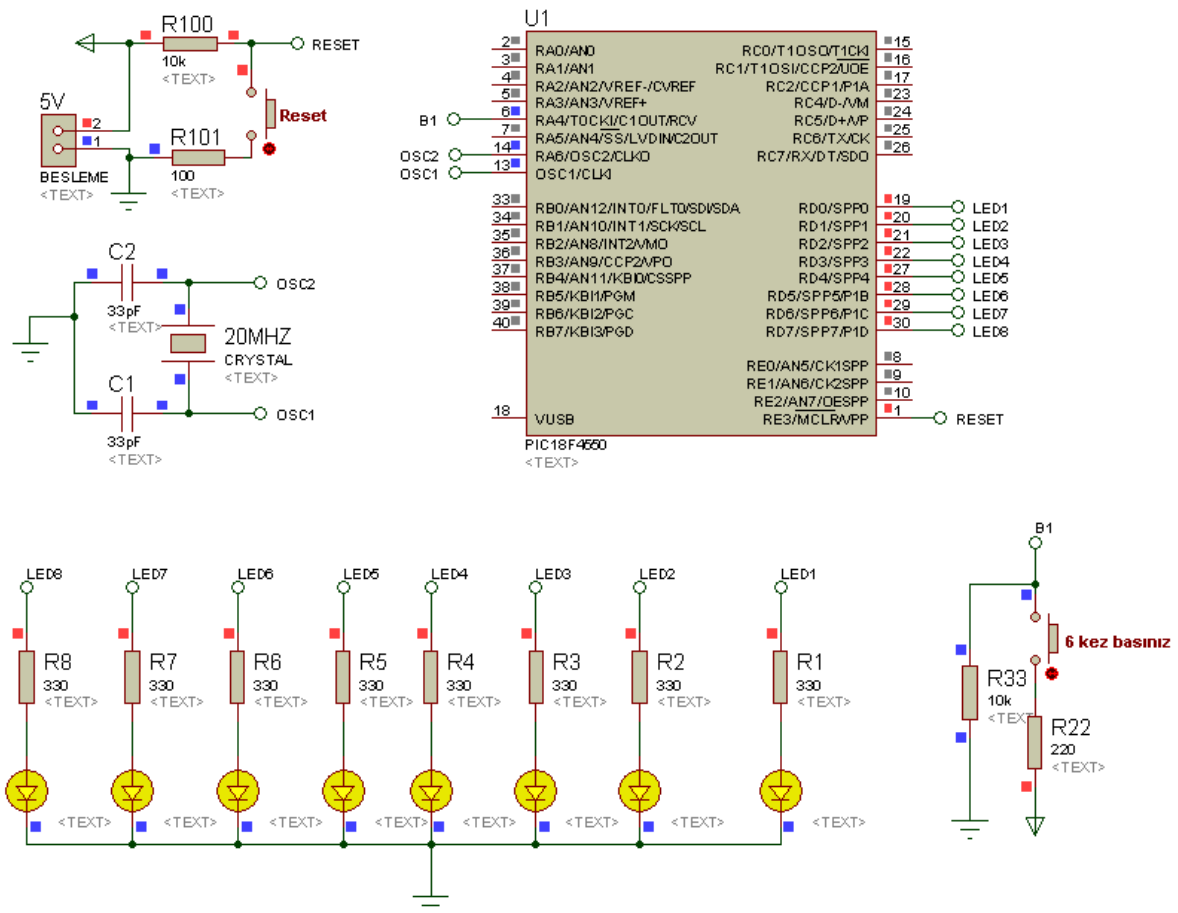
void main() {                // Ana program başlatılıyor
    ADCON1 |= 0x0F;          // PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON |= 7;              // Karşılaştırma modülü iptal edildi.
    TRISD = 0;               // D portu çıkış olarak ayarlandı
    PORTD = 0xFF;            // D portuna bağlı LED'ler yakıldı
    Serkan();                // Sayıcı ayarının yapıldığı Serkan isimli fonksiyon çağrıldı
}
```

AD SOYAD:

TIMER UYGULAMALARI

```
while(1)                // Program sonsuz döngüye sokuldu, ama döngünün içi boş!  
{                       //Sonsuz döngünün başlangıcı  
    }                   // Sonsuz döngünün sonu  
}                       //Ana programın sonu  
//===== Program sonu =====
```

Uygulamanın gerçekleştirileceği devreye ilişkin ISIS şeması Şekil 9.10'da görülmektedir. Kodları derleyip uygulama devresine yükledikten sonra, butona 6 kez basıp çıktığınızda D portuna bağlı LED'lerin tersleneceğini gözlemleyiniz. Ancak düşen kenar ayarlaması nedeniyle sayma işleminin butondan elinizi çektikten sonra gerçekleşeceğini de unutmayınız. Bunu son (6.) kez butona basıp elinizi çektiğiniz anda LED'lerin terslendiğini görerek fark edebilirsiniz.



Şekil 9.10 Timer'ın sayıcı olarak kullanılmasına ilişkin devre

BÖLÜM 10

10. ADC UYGULAMALARI

UYGULAMA_01

Buraya kadar yaptığımız tüm uygulamalarda mikrodenetleyiciye gelen sinyalin her zaman **lojik 0** veya **lojik 1** olduğunu varsayarak işlem yaptık. Mesela **lojik 1** kontrollü bir butona basılmışsa PIC'e **lojik 1** gelir, basılmamışsa **lojik 0** gelir. Ya da **lojik 0** kontrollü bir butona basılmışsa PIC'e **lojik 0** gelir, basılmamışsa **lojik 1** gelir gibi... Dolayısıyla sürekli ya 0 volt ya da 5 volt düzeyinde işlem yapmış olduk. Ancak endüstriyel uygulamalarda PIC'e her zaman lojik sinyaller gelmeyip analog sinyaller de gelecektir. Yani 2.8V, 0.5v, 4.7V, 0.0358V gibi. Bu durumda PIC, 0 ya da 5V düzeyinde olmayan ve sürekli değişen bu değerlerle nasıl karar verecektir veya nasıl işlem yapacaktır? En basit örnekle; sıcaklık artışına göre değişen bir analog sinyal üreten devre elemanları ile sıcaklık değeri nasıl ölçülecektir?

İşte bu noktada yapılması gereken şey, sürekli olan analog sinyalleri PIC'in anlayacağı ayrık sayısal sisteme dönüştürmektir. Bu amaçla kullanılan 10 bit, 12 bit, 16 bit gibi farklı çözünürlüklere sahip farklı ADC (Analog to Digital Converter) entegreleri bulunmaktadır. Ancak bu dönüştürme işlemi 18F4550'in kendi bünyesinde bulunan 13 kanallı (**AN0-AN12**) 10 bit çözünürlüklü ADC donanımı sayesinde, başka bir entegreye gerek kalmadan kolaylıkla yapılabilmektedir.

18F4550'deki referans gerilim, PIC'in beslenmesinde kullanılan ve maksimum değeri 5V olan besleme gerilimidir. Referans gerilimi 5V olduğundan PIC'in ADC donanımı, sahip olduğu 10 bitlik çözünürlük sayesinde referans gerilimini $2^{10}=1024$ parçaya böler ve 1024 değerinin her birine bu gerilimi eşit olarak dağıtır. Yani 0-5V arasında değer alan analog sinyal, ADC modülü tarafından okunduğunda, okunan değer 0V 0'a, 5V ise 1023 değerine denk gelir. Dolayısıyla her bir bit değerinin alacağı gerilim değeri $5/1024=0.00488V$ olur. Bu durumda ADC modülünün okuduğu değer 0.00488 ile çarpılarak ADC girişindeki analog sinyalin değeri hesaplanır.

Mesela ADC modülle okuma yapıldığında bu birimden 348 değerinin alındığını varsayalım. Bu durumda ADC girişindeki gerilimin değeri aşağıdaki gibi hesaplanır:

$$348 \times 0,00488 = 1,698 \text{ V}$$

18F4550 ile analog sinyal okunacağı zaman PIC’de yapılması gereken bazı ayarlamalar olacaktır. Bu ayarlamalar 18F4550’deki **ADCON0**, **ADCON1** ve **ADCON2** yazmaçları üzerinde yapılır. Bu noktaya kadar yaptığımız tüm uygulamalarda, **ADCON1** yazmacına yüklenen değer sayesinde PIC’e ait tüm uçların dijital giriş çıkış olarak kullanılacağını deklare etmiştik. Ancak bu uygulamamızda analog veri okuma özelliğine sahip pinlerin birisinden (**RA0/AN0** pininden) analog veri okunacağı için **ADCON1** yazmacına yüklenen değer değişecektir. Analog veri okuma özelliğine sahip tüm bu kanalların analog/dijital yönlendirmesi aşağıdaki Tablo 10.1’de görülmektedir. Tablodaki **A** harfi analog kullanımı, **D** harfi ise dijital amaçlı kullanımı ifade etmektedir. Bu tablodan da anlaşılacağı üzere **ADCON1** yazmacının ilk 4 biti (**PCFG3:PCFG0**) üzerinde değişiklik yapmak suretiyle istenilen analog pinlerin analog veri okumak amacıyla yapılandırılabilceği görülmektedir.

Analog veri okuma sırasında yapılması gereken bir diğer ayar da, kullanılacak analog pinlerin seçilmesi ve ADC biriminin aktif edilmesidir. Bu işlem **ADCON0** yazmacı üzerinde yapılır. ADC biriminin aktif edilmesi için **ADCON0** yazmacının 0. bitini 1 yapmak gerekir. Kanal seçimi ise 2, 3, 4 ve 5. bitlere yüklenecek veri ile yapılır. Bu 4 bitin alacağı değere göre yapılan kanal seçimi Tablo 10.2’de görülmektedir. Ancak MikroC’de bu işlemin yapılmasına gerek yoktur. Çünkü analog okuma komutu kullanıldığında okumanın hangi kanaldan yapılacağı vurgulanmakta, dolayısıyla ilgili analog kanal aktif duruma geçmektedir. Fakat kullanımın görülmesi adına ilk uygulamada bu yazmaç üzerinde de işlem yapılacaktır.

Tablo 10.1 ADCON1 yazmacının ilk 4 bitine göre pinlerin analog veri okuma ilişkin yönlendirilmesi

PCFG3:PCFG0	AN12	AN11	AN10	AN9	AN8	AN7	AN6	AN5	AN4	AN3	AN2	AN1	AN0
0000	A	A	A	A	A	A	A	A	A	A	A	A	A
0001	A	A	A	A	A	A	A	A	A	A	A	A	A
0010	A	A	A	A	A	A	A	A	A	A	A	A	A
0011	D	A	A	A	A	A	A	A	A	A	A	A	A
0100	D	D	A	A	A	A	A	A	A	A	A	A	A
0101	D	D	D	A	A	A	A	A	A	A	A	A	A
0110	D	D	D	D	A	A	A	A	A	A	A	A	A
0111	D	D	D	D	D	A	A	A	A	A	A	A	A
1000	D	D	D	D	D	D	A	A	A	A	A	A	A
1001	D	D	D	D	D	D	D	A	A	A	A	A	A
1010	D	D	D	D	D	D	D	D	A	A	A	A	A
1011	D	D	D	D	D	D	D	D	D	A	A	A	A
1100	D	D	D	D	D	D	D	D	D	D	A	A	A
1101	D	D	D	D	D	D	D	D	D	D	D	A	A
1110	D	D	D	D	D	D	D	D	D	D	D	D	A
1111	D	D	D	D	D	D	D	D	D	D	D	D	D

Tablo 10.2 ADCON0 yazmacındaki analog kanal seçim bitleri ve bitlerin durumuna göre seçilen kanallar

Analog kanal seçme bitleri (CHS3:CHS0)	Seçilen kanal	Pin adı
0000	Kanal 0	AN0
0001	Kanal 1	AN1
0010	Kanal 2	AN2
0011	Kanal 3	AN3
0100	Kanal 4	AN4
0101	Kanal 5	AN5
0110	Kanal 6	AN6
0111	Kanal 7	AN7
1000	Kanal 8	AN8
1001	Kanal 9	AN9
1010	Kanal 10	AN10
1011	Kanal 11	AN11
1100	Kanal 12	AN12

Verilen tüm bu ön bilgilerden sonra artık uygulamamıza geçebiliriz. Bu uygulamada PIC'in **AN0** kanalına (**RA0/AN0** pinine) bağlı bir potansiyometre sayesinde 0-5V arasında bir analog bilgi girişi yapılacaktır. Bu girişten okunan değere karşılık gelen dijital veri, LCD ekranının 1. satırında sıfırlı 2. satırında sıfırsız olmak üzere 2 farklı şekilde gösterilecektir. **AN0** kanalının analog kullanıma, diğer tüm pinlerin ise dijital kullanıma ayrılması ADCON1 |= 0x0E; komutuyla sağlanmıştır. **AN0** kanalının analog kullanım için seçilmesi ise ADCON0=1; yazılarak yapılmıştır.

Uygulamaya ilişkin program kodları aşağıda verilmiştir.

PROGRAM KODLARI:

```
//===== Program başlangıcı =====  
//===== LCD modül bağlantıları tanımlanıyor =====  
  
sbit LCD_RS at RB0_bit;  
sbit LCD_EN at RB1_bit;  
sbit LCD_D4 at RB2_bit;  
sbit LCD_D5 at RB3_bit;  
sbit LCD_D6 at RB4_bit;  
sbit LCD_D7 at RB5_bit;  
  
sbit LCD_RS_Direction at TRISB0_bit;  
sbit LCD_EN_Direction at TRISB1_bit;  
sbit LCD_D4_Direction at TRISB2_bit;  
sbit LCD_D5_Direction at TRISB3_bit;  
sbit LCD_D6_Direction at TRISB4_bit;  
sbit LCD_D7_Direction at TRISB5_bit;  
  
//===== LCD modül bağlantılarının sonu =====  
  
long int OkunanDeger;           //long int türünde değişken tanımlanıyor  
char ADC_Degeri[12];           //12 karakterlik ADC_Degeri isimli değişken tanımlandı  
  
void main() {                   //Ana program yazılıyor  
    ADCON1 |= 0x0E; //PIC'in sadece AN0 pini analog, diğerleri dijital giriş/çıkış olarak ayarlandı  
    ADCON0=1;         //AN0 kanalı analog ölçüm için açıldı  
    CMCON |=7;        //Karşılaştırma modülü iptal edildi.  
    TRISB=0;          //B portu LCD için ÇIKIŞ olarak ayarlandı  
    Lcd_Init();        //LCD'yi başlat  
    Lcd_Cmd(_LCD_CURSOR_OFF); //İmleci kapat  
    Lcd_Cmd(_LCD_CLEAR); //LCD ekranını temizle  
    Delay_ms(200);      //LCD'nin hazırlanması için 200ms bekle  
    while(1) {          //Sonsuz döngüye gir  
        Lcd_Cmd(_LCD_CLEAR); //LCD ekranını temizle  
        //ANO kanalından okunan analog değer karşılığını değişkene aktar
```

```

OkunanDeger = ADC_Read(0);

/*Int türünde sayısal bir veri olan OkunanDeger isimli değişkeni LCD'ye yazdırabilmek için
string formatına SIFIRLI olarak dönüştür ve char türündeki ADC_Degeri isimli değişkene
yükle */

LongIntToStrWithZeros(OkunanDeger,ADC_Degeri);
//LCD ekranın 1.satır 1.sütununa bunu yazdır (0-1023 aralığında değer gösterir)
Lcd_Out(1,1,ADC_Degeri);

/*Int türünde sayısal bir veri olan OkunanDeger isimli değişkeni LCD'ye yazdırabilmek için
string formatına SIFIRSIZ olarak dönüştür ve char türündeki ADC_Degeri isimli değişkene
yükle */

LongToStr(OkunanDeger,ADC_Degeri);
//ADC_Degeri isimli değişkenin değerini LCD 2. satır 1. sütuna yazdır
Lcd_out(2,1,ADC_Degeri);
Delay_ms(500);    //Bir sonraki ölçüm için 500ms bekle
}                //Sonsuz döngünün sınırı
}                //Ana programın sonu
//===== Program sonu =====

```

PIC'in ADC birimi tarafından analog veri okuması, yukarıdaki kodlardan da anlaşılacağı üzere **ADC_Read()**; komutuyla sağlanır. Biz burada 0. kanaldan (**AN0**'dan) okuma yaptığımız için **ADC_Read(0)**; yazdık. Şayet 7. kanaldan (**AN7**'den) okuma yapsaydık bu defa komutumuz **ADC_Read(7)**; olarak değişirdi.

Yukarıda verilen kodlar MikroC'de yazıldığında Şekil 10.1'deki ekran görüntüsü elde edilecektir. Ancak LCD bağlantılarına ilişkin ayar standart olarak yapıldığı için bu ekran görüntüsüne LCD bağlantı bölümü dâhil edilmemiştir.

Derlenen programın hex kodları Şekil 10.2'deki devreye yüklenip simülasyon başlatılacak olursa, analog veri girişinin sağlandığı potansiyometre ile oynanması durumunda LCD ekranındaki değer değiştiği görülecektir.

Potansiyometre %0'a getirildiğinde PIC'e gelen analog değerin maksimum olduğunu ve dolayısıyla buna karşılık 1023 sayısının ekranda görüldüğünü, potansiyometre %100'e getirildiğinde ise PIC'e gelen analog değerin minimum olduğunu ve bu nedenle buna karşılık 0 sayısının ekranda görüldüğünü mutlaka gözlemleyiniz.

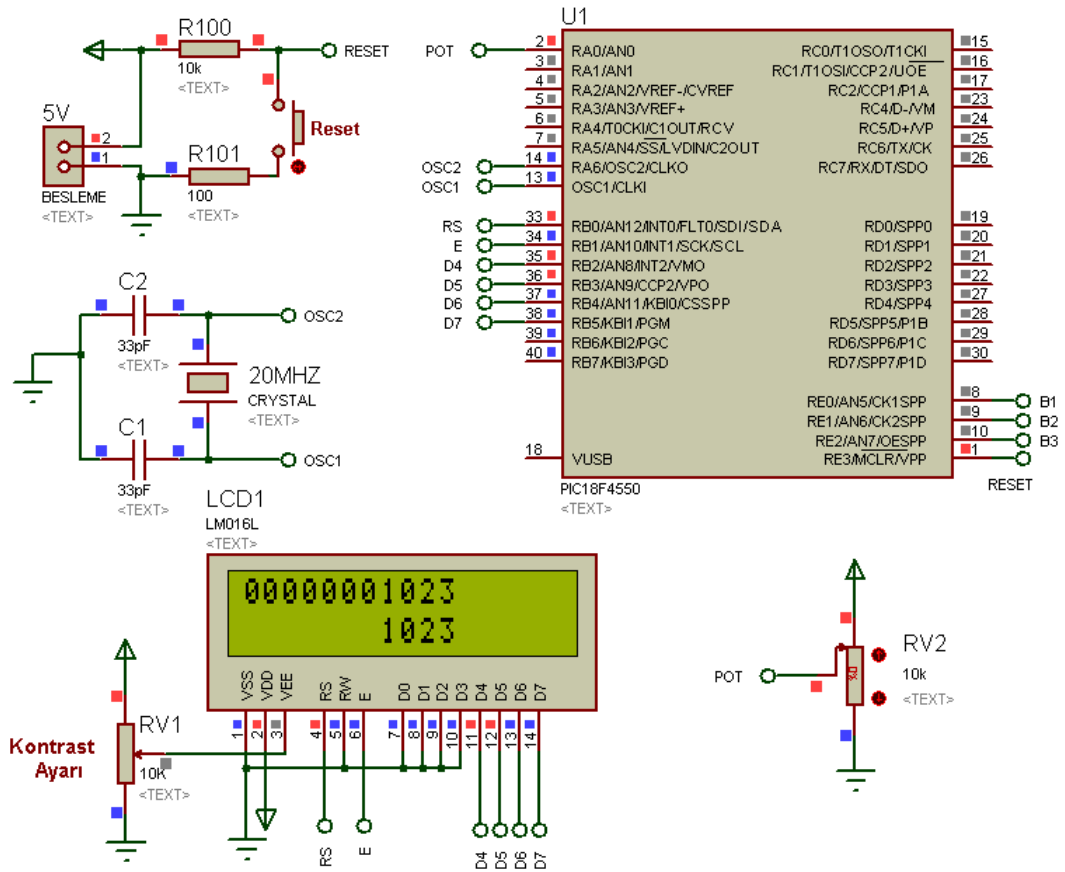
Gerçek uygulamalarda bu işlem yapıldığında şayet 5V değerine ulaşamıyorsa bunun referans geriliminden yani besleme geriliminden kaynaklanacağını unutmayınız. O nedenle besleme geriliminin iyi regüle edilmesini ve tam olarak 5V olmasını sağlayınız.

```

tart Page  ADC_01.c
• long int OkunanDeger; //long int türünde değişken tanımlanıyor
• char ADC_Degeri[12]; //12 karakterlik ADC_Degeri isimli değişken tanımlandı
• void main() { //Ana program yazılıyor
•     ADCON1 |= 0x0E; //PIC'in sadece AN0 pini analog, diğerleri dijital giriş/çıkış olarak ayarlandı
20 •     ADCON0=1; //ANO kanalı analog ölçüm için açıldı
•     CMCON |=7; //Karşılaştırma modülü iptal edildi.
•     TRISB=0; //B portu LCD için ÇIKIŞ olarak ayarlandı
•     Lcd_Init(); //LCD'yi başlat
•     Lcd_Cmd(_LCD_CURSOR_OFF); //İmleci kapat
•     Lcd_Cmd(_LCD_CLEAR); //LCD ekranını temizle
•     Delay_ms(200); //LCD'nin hazırlanması için 200ms bekle
•     while(1) { //Sonsuz döngüye gir
•         Lcd_Cmd(_LCD_CLEAR); //LCD ekranını temizle
30 •         OkunanDeger = ADC_Read(0);
•         /*Int türünde sayısal bir veri olan OkunanDeger isimli değişkeni LCD'ye yazdırabilmek için
•         string formatına SIFIRLI olarak dönüştür ve char türündeki ADC_Degeri isimli değişkene yükle */
•         LongIntToStrWithZeros(OkunanDeger,ADC_Degeri);
•         //LCD ekranın 1.satır 1.sütununa bunu yazdır (0-1023 aralığında değer gösterir)
•         Lcd_Out(1,1,ADC_Degeri);
•         /*Int türünde sayısal bir veri olan OkunanDeger isimli değişkeni LCD'ye yazdırabilmek için
•         string formatına SIFIRLI olarak dönüştür ve char türündeki ADC_Degeri isimli değişkene yükle */
•         LongToStr(OkunanDeger,ADC_Degeri);
•         //ADC_Degeri isimli değişkenin değerini LCD 2. satır 1. sütuna yazdır
40 •         Lcd_out(2,1,ADC_Degeri);
•         Delay_ms(500); //Bir sonraki ölçüm için 500ms bekle
•     }
• }

```

Şekil 10.1 ADC uygulamasına ilişkin MikroC ekran görüntüsü



Şekil 10.2 ADC'ye yönelik yapılan birinci uygulamanın ISIS devre şeması

UYGULAMA_02

Önceki ADC uygulamamız, analog kanaldan okunan verinin dijital karşılığını LCD ekranda görüntülemeye yönelikti. Ancak bu uygulamada, 0-5V arasında değişen analog verinin mV cinsinden anlık ölçülen değeri de LCD ekranda gösterilecektir. Dolayısıyla dijital ve analog verinin kıyaslanması aynı ekranda rahatlıkla yapılabilecek ve çözünürlük kavramı pekiştirilebilecektir.

Uygulamaya ilişkin program kodları aşağıda sunulmuştur. Değerlerin LCD ekrana yazdırılması sırasında 1. satırda analog verinin dijital karşılığı olduğunu vurgulamak için DJTL kısaltması kullanılmıştır. 2. satırdaki veri ise mV cinsinden analog değeri ifade ettiğinden mV kısaltması eklenmiştir.

PROGRAM KODLARI:

```
//===== Program başlangıcı =====  
//===== LCD modül bağlantıları tanımlanıyor =====  
sbit LCD_RS at RB0_bit;  
sbit LCD_EN at RB1_bit;  
sbit LCD_D4 at RB2_bit;  
sbit LCD_D5 at RB3_bit;  
sbit LCD_D6 at RB4_bit;  
sbit LCD_D7 at RB5_bit;  
sbit LCD_RS_Direction at TRISB0_bit;  
sbit LCD_EN_Direction at TRISB1_bit;  
sbit LCD_D4_Direction at TRISB2_bit;  
sbit LCD_D5_Direction at TRISB3_bit;  
sbit LCD_D6_Direction at TRISB4_bit;  
sbit LCD_D7_Direction at TRISB5_bit;  
//===== LCD modül bağlantılarının sonu =====  
long int OkunanDeger;      //long int türünde değişken tanımlanıyor  
char ADC_Degeri[12];      //12 karakterlik ADC_Degeri isimli değişken tanımlandı
```

```
float GerilimDegeri;           //float türünde GerilimDegeri isimli değişken tanımlanıyor
//VoltajaDonustur isimli alt program yazılıyor (0-5 Volt aralığında okuma yapar)
float VoltajaDonustur(float x)
{
    return(x*5000/1023);       //milivolt olarak göstermesi için 5000 ile çarptık
}                               //10 bit çözünürlük nedeniyle 1023'e böldük (0-1023)
void main() {                  //Ana program yazılıyor
    ADCON1 |= 0x0E; //PIC'in sadece AN0 pini analog, diğerleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |=7;      //Karşılaştırma modülü iptal edildi.
    TRISB=0;         //B portu LCD için ÇIKIŞ olarak ayarlandı
    Lcd_Init();       //LCD'yi başlat
    Lcd_Cmd(_LCD_CURSOR_OFF); //İmleci kapat
    Lcd_Cmd(_LCD_CLEAR); //LCD ekranını temizle
    Delay_ms(200);    //LCD'nin hazırlanması için 200ms bekle
    while(1) {        //Sonsuz döngüye gir
        Lcd_Cmd(_LCD_CLEAR); //LCD ekranını temizle
        //AN0 kanalından okunan analog değer karşılığını değişkene aktar
        OkunanDeger = ADC_Read(0);
        /*Int türünde sayısal bir veri olan OkunanDeger isimli değişkeni LCD'ye yazdırabilmek için
        string formatına SIFIRSIZ olarak dönüştür ve char türündeki ADC_Degeri isimli değişkene
        yükle */
        LongToStr(OkunanDeger,ADC_Degeri);
        //ADC_Degeri isimli değişkeni LCD 1. satır 1. sütuna yazdır
        Lcd_out(1,1,ADC_Degeri);
        //LCD ekranında var olan görüntünün hemen sonuna Dijital için kullanılan DJTL
        //kısaltmasını ilave et
        Lcd_Out_Cp(" DJTL");
        //===== Okunan gerilimin ekrana yazdırılmasına başlanılıyor =====
        /*AN0 kanalından okunan analog değeri (OkunanDeger), 0-5000 milivolt arası değere
        dönüştür ve GerilimDegeri isimli değişkene yükle*/
```



```

GerilimDegeri=VoltajaDonustur(OkunanDeger);
/*Float türünde sayısal bir veri olan GerilimDegeri isimli değişkeni LCD'ye yazdırabilmek
için string formatına dönüştür ve char türündeki ADC_Degeri isimli değişkene yükle */
FloatToStr(GerilimDegeri, ADC_Degeri);
/*İçerisinde okunan gerilim değerini barındıran char türündeki ADC_Degeri isimli
değişkeni LCD 2. satır 4. sütuna yazdır*/
Lcd_out(2,4,ADC_Degeri);
//LCD ekranında var olan görüntünün hemen sonuna mV yazısını ilave et
Lcd_Out_Cp(" mV");
delay_ms(500);           //Bir sonraki ölçüm için 500ms bekle
}                         //Sonsuz döngünün sınırı
}                         //Ana programın sonu
//===== Program sonu =====

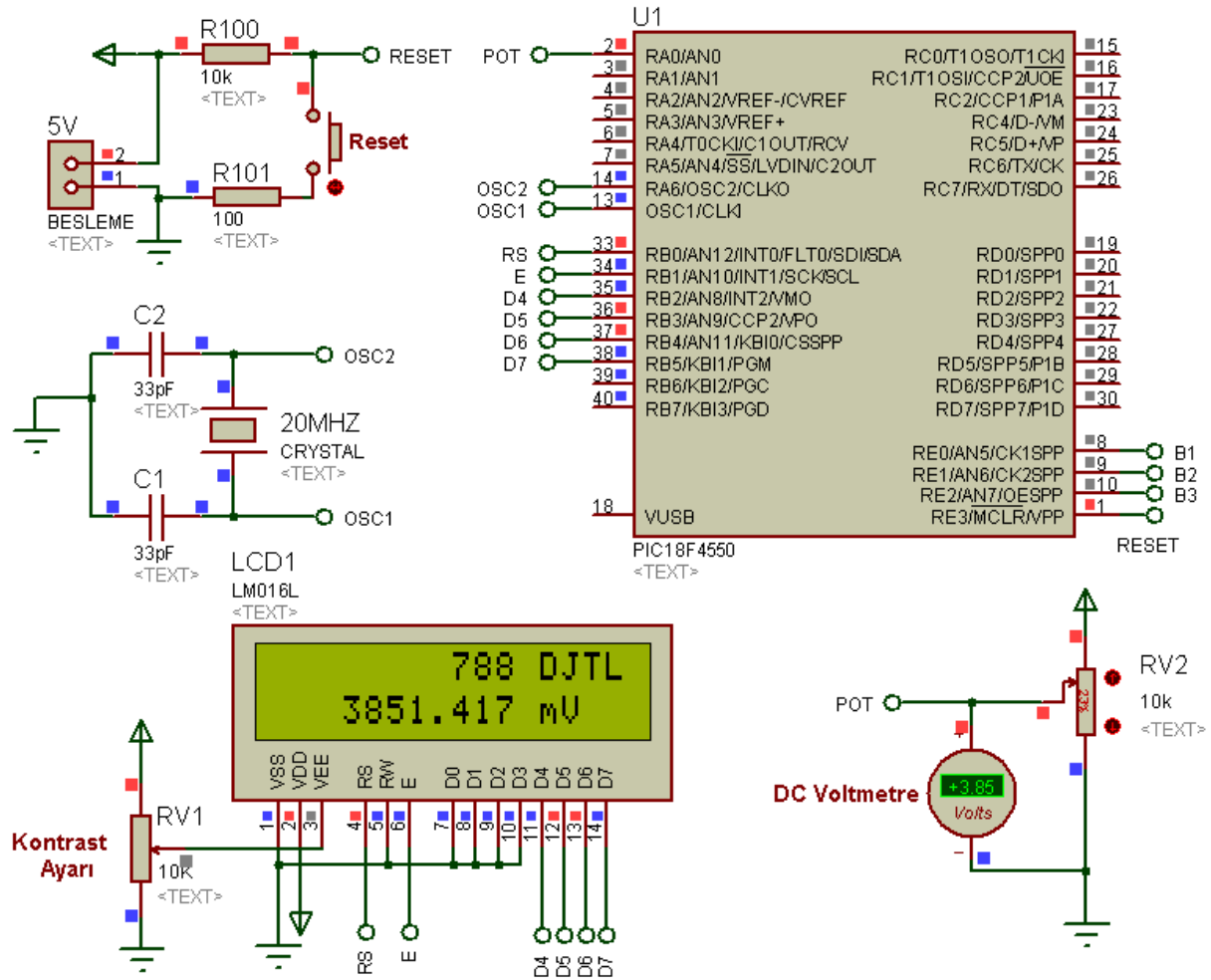
```

Uygulamanın MikroC programındaki ekran görüntüsü tek seferde alınamadığı için burada verilememiştir. Ancak ISIS şeması Şekil 10.3'te görülmektedir. Şekil 10.3'ten de anlaşılacağı üzere uygulamanın doğruluğunu kontrol etmek için analog giriş ile şase arasına bir DC voltmetre bağlanmıştır. Potansiyometre ile oynandığında bu DC voltmetreden okunan değer ile LCD'den okunan değer her zaman eşit olduğu görülecektir. Ancak LCD'deki değer mV, voltmetredeki değer V cinsinden olduğu hatırdan çıkarılmamalıdır.

Potansiyometre 0'a getirildiğinde PIC'e gelen analog değer dijital karşılığı olarak 1. satırda 1023 sayısının görüldüğünü, 2. satırdaki gerilimin ise 5000mV olduğunu, DC voltmetreden de +5.00 volt değerinin okunduğunu gözlemleyiniz.

Gerçek uygulamalarda bu işlem yapıldığında şayet 5V değerine ulaşamıyorsa bunun referans geriliminden, yani besleme gerilimindeki maksimum değer 5V olmamasından kaynaklanacağını unutmayınız. O nedenle besleme geriliminin iyi regüle edilmesini ve maksimum değerinin tam olarak 5V olmasını sağlayınız.

Potansiyometre %100'e getirildiğinde ise PIC'e gelen analog değerin dijital karşılığı olarak 1. satırda 0 sayısının görüldüğünü, 2. satırdaki gerilimin ise 0mV olduğunu, DC voltmetreden de 0.00 değerinin okunduğunu gözlemleyiniz.



Şekil 10.3 ADC birimi sayesinde okunan analog gerilimin LCD ekranına yazdırılmasına yönelik ISIS devresi

UYGULAMA_03

Bu uygulamada ise **RA0**, **RA1** ve **RA2** kanalları kullanılarak 3 farklı kanaldan veri okuma işlemi yapılacaktır. Buna ilişkin program kodları aşağıda verilmiş olup uygulamaya ilişkin ISIS şeması Şekil 10.4'te görülmektedir. Ancak program kodlarının MikroC programındaki ekran görüntüsü tek seferde alınamadığı için burada verilememiştir.

PROGRAM KODLARI:

```
//===== Program başlangıcı =====
//===== LCD modül bağlantıları tanımlanıyor =====

sbit LCD_RS at RB0_bit;
sbit LCD_EN at RB1_bit;
sbit LCD_D4 at RB2_bit;
sbit LCD_D5 at RB3_bit;
sbit LCD_D6 at RB4_bit;
sbit LCD_D7 at RB5_bit;

sbit LCD_RS_Direction at TRISB0_bit;
sbit LCD_EN_Direction at TRISB1_bit;
sbit LCD_D4_Direction at TRISB2_bit;
sbit LCD_D5_Direction at TRISB3_bit;
sbit LCD_D6_Direction at TRISB4_bit;
sbit LCD_D7_Direction at TRISB5_bit;

////===== LCD modül bağlantılarının sonu =====

long int OkunanDeger1;    //long int türünde OkunanDeger1 isimli değişken tanımlanıyor
long int OkunanDeger2;    //long int türünde OkunanDeger2 isimli değişken tanımlanıyor
long int OkunanDeger3;    //long int türünde OkunanDeger3 isimli değişken tanımlanıyor
char ADC_Degeri1[12];     //12 karakterlik ADC_Degeri1 isimli değişken tanımlandı
char ADC_Degeri2[12];     //12 karakterlik ADC_Degeri2 isimli değişken tanımlandı
char ADC_Degeri3[12];     //12 karakterlik ADC_Degeri3 isimli değişken tanımlandı
float GerilimDegeri1;     //float türünde GerilimDegeri1 isimli değişken tanımlanıyor
float GerilimDegeri2;     //float türünde GerilimDegeri2 isimli değişken tanımlanıyor
```

```

float GerilimDegeri3;      //float türünde GerilimDegeri3 isimli değişken tanımlanıyor
//VoltajaDonustur isimli alt program yazılıyor (0-5 Volt aralığında okuma yapar)
float VoltajaDonustur(float x)
{
    return(x*5000/1023);    //mV olarak göstermesi için 5000 ile çarptık, 10 bit çözünürlük
}                          // nedeniyle 1023'e böldük (0-1023)
void main() {              //Ana program yazılıyor
    ADCON1 |= 0x0C;        //PIC'in sadece AN0-AN1-AN2 pinleri analog, diğerleri dijital//
    giriş/çıkış olarak ayarlandı
    CMCON  |=7;            //Karşılaştırma modülü iptal edildi.
    TRISB=0;               //B portu LCD için ÇIKIŞ olarak ayarlandı
    Lcd_Init();            //LCD'yi başlat
    Lcd_Cmd(_LCD_CURSOR_OFF); //İmleci kapat
    Lcd_Cmd(_LCD_CLEAR);   //LCD ekranını temizle
    Delay_ms(200);         //LCD'nin hazırlanması için 200ms bekle
    while(1) {             // Sonsuz döngüye gir
        //AN0-AN1-AN2 kanallarından okunan analog değer karşılıklarını değişkenlere aktar
        OkunanDeger1 = ADC_Read(0);
        OkunanDeger2 = ADC_Read(1);
        OkunanDeger3 = ADC_Read(2);
        /*AN0-AN1-AN2 kanallarından okunan analog değerleri, 0-5000 milivolt arası değere
        dönüştür ve GerilimDegeri isimli değişkenlere yükle*/
        GerilimDegeri1=VoltajaDonustur(OkunanDeger1);
        GerilimDegeri2=VoltajaDonustur(OkunanDeger2);
        GerilimDegeri3=VoltajaDonustur(OkunanDeger3);
        /*Float türünde sayısal bir veri olan GerilimDegeri isimli değişkenleri LCD'ye yazdırabilmek
        için string formatına dönüştür ve char türündeki ADC_Degeri isimli değişkenlere yükle */
        FloatToStr(GerilimDegeri1, ADC_Degeri1);
        FloatToStr(GerilimDegeri2, ADC_Degeri2);
        FloatToStr(GerilimDegeri3, ADC_Degeri3);
    }
}

```

AD SOYAD:

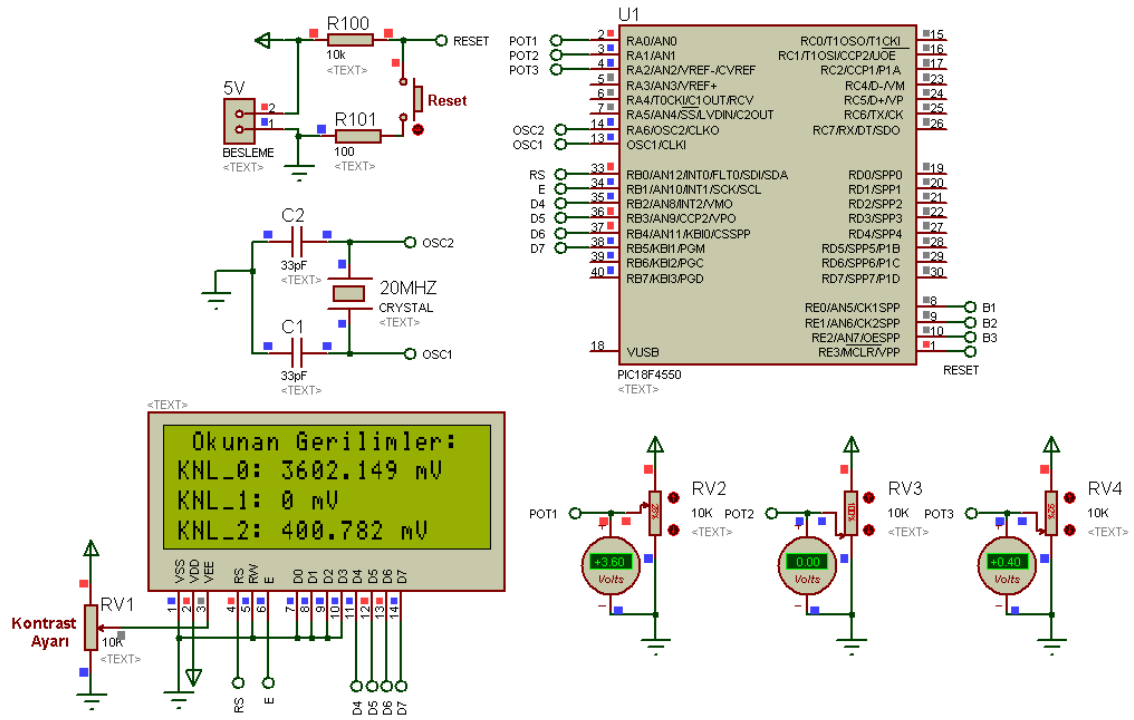
ADC UYGULAMALARI

```
/*Her kanaldan okunan gerilim değerleri LCD'nin 2, 3 ve 4. sütunlarına ayrı ayrı yazdırılıyor.*/
```

```
Lcd_Cmd(_LCD_CLEAR);          //Yeni veriler için LCD ekranını temizle
Lcd_Out(1,2,"Okunan Gerilimler: "); //Belirtilen yazıyı belirtilen adrese yaz
Lcd_Out(2,1,"KNL_0: ");        //KNL_0: yazısını belirtilen adrese yaz
Lcd_Out_Cp(ADC_Degeri1);       //KNL_0: yazısının hemen sonuna ADC_Degeri1'in
                                //değerini ekle
Lcd_Out_Cp(" mV");             //ADC_Degeri1'in hemen sonuna mV yazısını ekle
Lcd_Out(3,1,"KNL_1: ");        //KNL_1: yazısını belirtilen adrese yaz
Lcd_Out_Cp(ADC_Degeri2);       //KNL_1: yazısının hemen sonuna ADC_Degeri2'nin
                                //değerini ekle
Lcd_Out_Cp(" mV");             //ADC_Degeri2'nin hemen sonuna mV yazısını ekle
Lcd_Out(4,1,"KNL_2: ");        //KNL_2: yazısını belirtilen adrese yaz
Lcd_Out_Cp(ADC_Degeri3);       //KNL_2: yazısının hemen sonuna ADC_Degeri3'ün
                                //değerini ekle
Lcd_Out_Cp(" mV");             //ADC_Degeri3'ün hemen sonuna mV yazısını ekle
delay_ms(500);                 //Bir sonraki ölçüm için 500ms bekle
}                               //Sonsuz döngünün sınırı
}                               //Ana programın sonu
//===== Program sonu =====
```

AD SOYAD:

ADC UYGULAMALARI



Şekil 10.4 ADC uygulama sorusuna ilişkin ISIS devresi

BÖLÜM11

11. PWM UYGULAMALARI

UYGULAMA_01

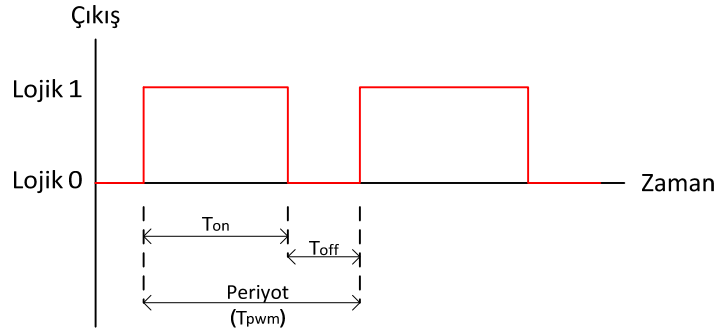
Bilindiği gibi **PWM** (**P**ulse **W**idth **M**odulation) Sinyal Genişlik Modülasyonu anlamına gelir. Temelde cihaza verilecek elektrik gücünü kontrol altında tutmak için kullanılır. Birçok kullanım alanına sahip olmakla birlikte kullanım alanlarını aşağıdaki gibi özetlemek mümkündür:

- Elektrik makinelerinde güç kontrolü ve aktarılan güç miktarının ayarlanmasında,
- Elektrik motorlarında devir sayısının kontrolünde,
- İnvertörlerde,
- Robot uygulamalarında,
- Haberleşme sistemlerinde,
- Anahtarlamalı güç kaynaklarında,
- Ses efektlerinde.

PWM yönteminin temelinde, genişliği değiştirilebilen kare dalga üretimi yatar. Bu kare dalgaların genişlikleri kontrol edilerek, çıkışta istenen analog elektriksel değer veya sinyal elde edilebilir. Bir diğer ifadeyle; DC gerilimin tepe değeri belirli bir seviyede sabit tutulup, belirli aralıklarla çıkış **lojik 1** ve **lojik 0** yapılarak çıkış sinyalinin genişliği ayarlanır. Dolayısıyla çıkıştaki gerilimin ortalama değeri ayarlanmış olur.

Güç veya hız kontrolü için kullanıldığında ise PWM sinyali üreten kaynağın çıkışına bağlanan yarı iletken anahtarların iletim ve kesim süreleri değiştirilmiş olur. Böylelikle cihazın gücü veya hızı kontrol altında tutulabilir.

PWM yönteminde "*Duty Cycle*" (görev döngüsü) olarak ifade edilen bir kavram vardır. Bu kavram, çıkışın **lojik 1** kaldığı sürenin (T_{on}) tüm periyota (T_{pwm}) oranını ifade eder. Bu durum Şekil 11.1'de açıkça görülmektedir.



Şekil 11.1 Bir PWM sinyalinde duty cycle ve periyot ilişkisi

Duty Cycle oranı 1'e eşit olduğunda PWM çıkışından 5V alınır ki bu maksimum değerdir. Eğer bu oran sıfır olursa bu durumda çıkışta alınan gerilim 0V olur. Şayet *Duty Cycle* oranı 0 ile 1 arasında bir değer alırsa, bu durumda çıkış geriliminin ortalaması 0 ile 5V arasındaki bir değerdir.

Giriş gerilimi (V_{in}), çıkış gerilimi (V_{out}) ve *Duty Cycle* (D) oranı arasındaki ilişkiyi izah etmeden önce giriş ve çıkış gerilimleri ifadelerini izah etmek faydalı olacaktır. Buradaki giriş gerilimi PIC'in beslendiği gerilim değeri olup 18F4550'de bu değer 5V'tur. Çıkış gerilimi ise PWM çıkışından alınacak sinyalin ortalama değeridir. PIC'in giriş gerilimi sabit olduğundan PWM çıkış geriliminin değeri *Duty Cycle* oranı ile değişmektedir. Bu durum aşağıdaki eşitlikle ifade edilebilir.

$$V_{out} = D \cdot V_{in}$$

Duty Cycle oranı ise aşağıdaki gibi ifade edilir:

$$D = \frac{T_{on}}{T_{pwm}} = \frac{T_{on}}{T_{on} + T_{off}}$$

Şekil 11.1'deki PWM sinyalinin frekansı (f_{pwm}) ile PWM sinyalinin periyodu (T_{pwm}) arasındaki ilişki ise şu şekildedir:

$$T_{PWM} = \frac{1}{f_{PWM}}$$

Güç kontrolü sırasında eğer düşük değerli bir periyot tercih edilmişse aktarılan gücün değeri düşük olur. Ama yüksek değerli periyotta durum bunun tam tersidir.

PWM ile ilgili tüm bu ön bilgilerden sonra kullanacağımız PIC'in üreteceği PWM sinyali ve buna ilişkin ayrıntılara girebiliriz. 18F4550 bünyesinde iki adet PWM çıkışı vardır. Bunlar **RC2** ve **RC1** ile aynı pinleri kullanan **CCP1** ve **CCP2** pinleridir. Bu iki pinden birbirinden bağımsız olarak iki farklı PWM sinyali alınabilir.

PIC ile PWM sinyali üretilirken belirtilmesi gereken iki parametre vardır. Bunlardan ilki PWM sinyalinin frekansı, diğeri de *Duty Cycle* oranıdır.

Frekans değeri, her iki PWM çıkışı için sırasıyla **PWM1_Init(X);** ve **PWM2_Init(X);** komutlarındaki **X** yerine girilen değer ile belirtilir. Ancak buraya yazılacak frekans değerinin Hz cinsinden olması gerektiği unutulmamalıdır. Örneğin 1KHz'lik bir PWM sinyali üretilecekse buraya 1000 yazılmalıdır.

Duty Cycle oranının girilmesinde kullanılan komut ise yine her iki PWM çıkışı için sırasıyla **PWM1_Set_Duty(D);** ve **PWM2_Set_Duty(D);** komutlarıdır. Bu komutlarda geçen **D** ifadesi, üretilecek olan PWM sinyalinin *Duty Cycle* oranıdır. Normalde bu oran, yukarıda da izah edildiği gibi 0 ile 1 arasında bir değerdir. Ancak MikroC programı yazılırken bu değer 0 ile 255 arasında olması gerektiğine dikkat edilmelidir. O nedenle buna ilişkin bir değişken tanımlaması yapılacaksa, bu değişkenin türünü **unsigned short** yapmak faydalı olacaktır. Çünkü böyle yapılırsa, *Duty Cycle* oranının kullanıcı tarafından arttırılıp azaltılması sırasında bu değer 0 ile 255 arasında kalacaktır. Eğer 255 aşılsa 0'a, 0'ın altına inilirse tekrar 255'e geri döner. Hatırlanacağı üzere **unsigned short** türündeki bir değişkenin alabileceği değer 0 ile 255 arasındadır.

Belirlenen tüm bu değerlerden sonra yapılması gereken tek şey, artık PWM sinyalinin üretilmesini başlatmaktır. Bu amaçla kullandığımız komutlar ise 1. ve 2. PWM çıkışları için sırasıyla **PWM1_Start();** ve **PWM2_Start();** komutlarıdır. Bu komutlar verildikten sonra artık PWM sinyali otomatik olarak üretilmeye başlanacak ve kesintisiz devam edecektir.

Ana program içerisinde başka işlemlerin yapılması, bu sinyallerin üretilmesini sekteye uğratmaz. Eğer PWM sinyalinin üretilmesini durdurmak gerekirse bu durumda PWM1 ve PWM2 için **PWM1_Stop();** ve **PWM2_Stop();** komutları kullanılmalıdır.

Verilen tüm bu ön bilgilerden sonra artık uygulamamıza geçebiliriz. Bu uygulamada PIC'in CCP1 pininden 1KHz'lik PWM sinyali alınacaktır. Üretilen sinyalin 0 ile 255 arasında değişen *Duty Cycle* oranı butonla artırılıp azaltılabilecektir. Ancak herhangi bir butona basmadan önce *Duty Cycle* oranının %50 olması için bu değer 128 olarak atanacaktır. Üretilen bu PWM sinyalinin periyodu ve *Duty Cycle* oranındaki değişime göre sinyaldeki değişim, CCP1 pinine bağlı osilaskop ile izlenebilecektir.

Uygulamaya ilişkin program kodları aşağıda verilmiştir.

PROGRAM KODLARI:

```
//Bu uygulamada üretilen PWM sinyalinin Duty Cycle değeri RD0 ve RD1'e bağlı butonlara
//basılarak artırılıp azaltılıyor. Sinyalin frekansı ve Duty Cycle değeri LCD ekrana yazdırılıyor.
//===== Program başlangıcı =====
//===== LCD modül bağlantıları tanımlanıyor =====

sbit LCD_RS at LATB0_bit;
sbit LCD_EN at LATB1_bit;
sbit LCD_D4 at LATB2_bit;
sbit LCD_D5 at LATB3_bit;
sbit LCD_D6 at LATB4_bit;
sbit LCD_D7 at LATB5_bit;
sbit LCD_RS_Direction at TRISB0_bit;
sbit LCD_EN_Direction at TRISB1_bit;
sbit LCD_D4_Direction at TRISB2_bit;
sbit LCD_D5_Direction at TRISB3_bit;
sbit LCD_D6_Direction at TRISB4_bit;
sbit LCD_D7_Direction at TRISB5_bit;
//===== LCD modül bağlantılarının sonu =====
```

```

#define artir RD0_bit    //RD0'daki butona artir ismi verildi
#define azalt RD1_bit    //RD1'deki butona azalt ismi verildi
unsigned short Periyot; //Duty Cycle oranını girmek için Periyot isimli değişken tanımlandı
char Periyot_S[12];     //Periyot_S isimli char türü değişken tanımlandı
void main() {           //Ana program başlangıcı
    ADCON1 |= 0x0F;      //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |=7;          //Karşılaştırma modülü iptal edildi.
    TRISD=0xFF;          //Butonlar bağlanacağından D portu çıkış olarak ayarlandı
    TRISC=0;             //PWM sinyali alınacağından C portu çıkış olarak ayarlandı,
    PORTD=0;             //D portuna bağlı butonlar basılmıyor olarak ayarlandı
    PORTC=0;             //C portu sıfırlandı

    //----- PWM1 ayarları -----
    PWM1_Init(1000);     //PWM1 kanalından 1KHz'lik sinyal istendiği deklare edildi
    Periyot=127;         //Mevcut Duty Cycle değeri 127 olarak Periyot değişkenine atandı
    PWM1_Start();        //PWM1 başlatıldı
    PWM1_Set_Duty(Periyot); //PWM1'in Duty Cycle değeri olarak Periyot (127) verildi

    //----- LCD ayarları -----
    Lcd_Init();           //LCD'yi başlat
    LCD_CMD(_LCD_CURSOR_OFF); //Ekranı yanıp sönen imleci kapat
    LCD_CMD(_LCD_CLEAR);  //LCD ekranını temizle
    LCD_OUT(1,1,"MEVCUT PWM AYARLARI "); //Sabit olarak kalacak bu metni belirtilen
                                         //adrese ekle

    LCD_OUT(2,1,"Frekans: 1000 Hz"); //PWM frekansını bunun yanına yazdıracağız
    LCD_OUT(3,1,"D.Cycle: "); //Değişken olan Duty Cycle değerini bunun yanına yazdıracağız
    DELAY_MS(250);        //LCD'nin hazır olması için bekle

    //----- Mevcut Duty Cycle değeri LCD ekrana yazdırılıyor-----
    LongToStr(Periyot, Periyot_S); //unsigned short türünde olan periyotu string formatına
                                     //dönüştür ve Periyot_S isimli değişkene yükle
    Lcd_Out(3,10,Periyot_S); //string türündeki Periyot_S isimli değişkenin içeriğini
                              //belirtilen adrese yaz

```

```

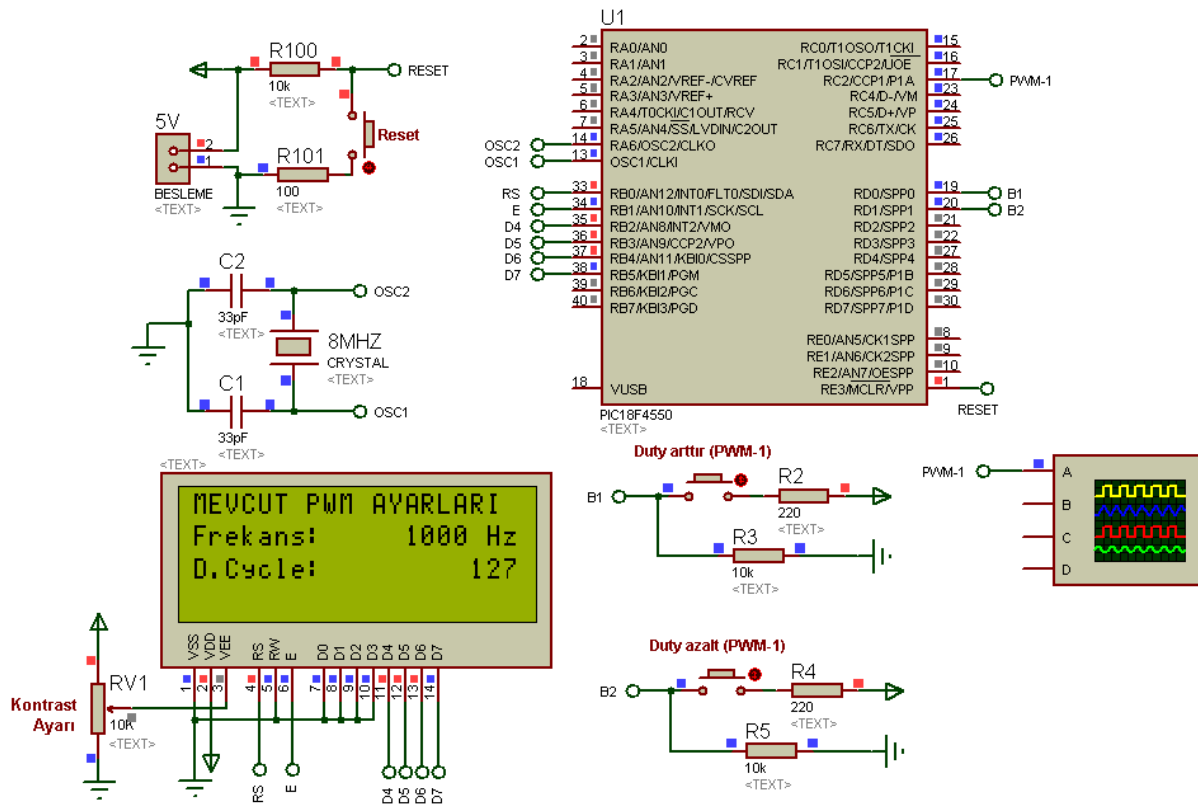
while(1){                                     //Sonsuz döngüyü başlat
    if(artir)                                 //artir butonuna basılmışsa aşağıdakileri yap
    {
        delay_ms(50);                        //50ms bekle
        Periyot++;                           //Periyot değerini 1 arttır
        PWM1_Set_Duty(Periyot);              //PWM1'in yeni Duty Cycle değeri, Periyot olarak
                                                //tekrar verildi
        LongToStr(Periyot, Periyot_S);       //unsigned short türünde olan periyotu string
                                                //formatına dönüştür ve Periyot_S isimli değişkene yükle
        Lcd_Out(3,10,Periyot_S)              //string türündeki Periyot_S isimli değişkenin içeriğini
                                                //belirtilen adrese yaz
    }
    if(azalt)                                 //azalt butonuna basılmışsa aşağıdakileri yap
    {
        delay_ms(50);                        //50ms bekle
        Periyot--;                           //Periyot değerini 1 azalt
        PWM1_Set_Duty(Periyot);              //PWM1'in yeni Duty Cycle değeri, Periyot olarak
                                                //tekrar verildi
        LongToStr(Periyot, Periyot_S);       //unsigned short türünde olan periyotu string formatına
                                                // dönüştür ve Periyot_S isimli değişkene yükle
        Lcd_Out(3,10,Periyot_S);              //string türündeki Periyot_S isimli değişkenin içeriğini
                                                // belirtilen adrese yaz
    }
    delay_ms(30);                             //Bir sonraki buton kontrolü için 30ms bekle
}                                               //Sonsuz döngünün sınırı
}                                               //Ana programın sonu
//===== Program sonu =====

```

Derlenen programın hex kodları Şekil 11.2'deki devreye yüklenip simülasyon başlatılacak olursa, ilk yüklenen *Duty Cycle* oranı olan 127 değeri ile PWM sinyalin frekansı olan 1000Hz değeri LCD ekranda görülecektir.

AD SOYAD:

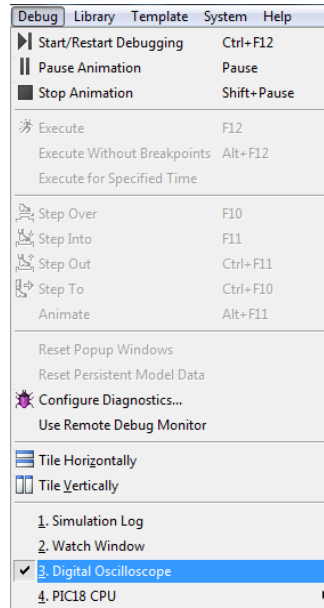
PWM UYGULAMALARI



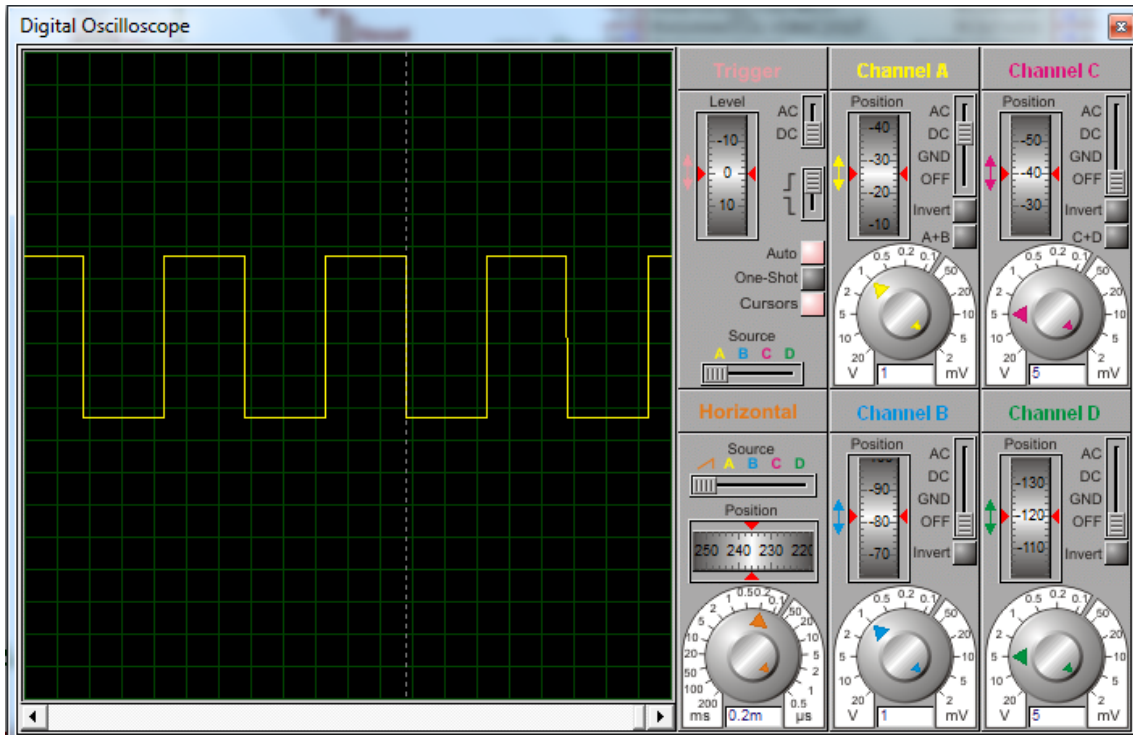
Şekil 11.2 PWM uygulamasına yönelik ISIS şeması.

Simülasyonun başlatılmasıyla birlikte osilaskop ekranı da açılacaktır. Şayet açılmıyorsa, ISIS menülerinden Debug sekmesini tıklayıp açılan bölümden dijital osilaskop işaretlemesini Şekil 11.3'teki gibi yapınız.

Bu işlem yapıldıktan sonra Şekil 11.4'teki gibi bir görüntü elde edilecektir. Buradaki gerekli ayarlamaları yaparak sinyalin rahatça görülmesini sağlayabilirsiniz.



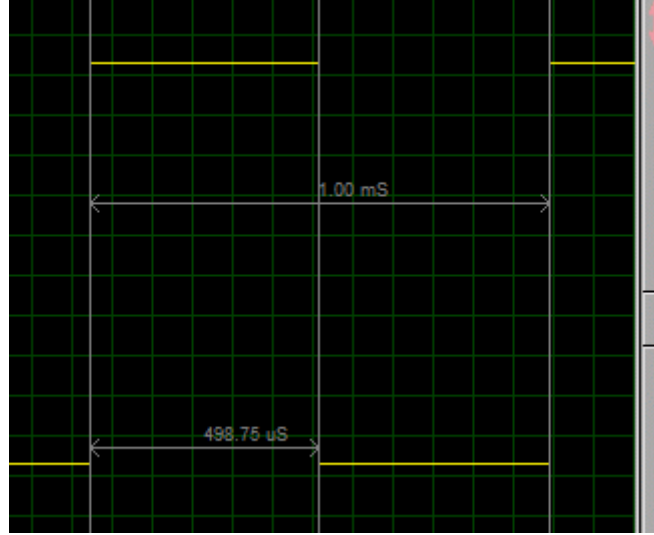
Şekil 11.3 ISIS programında simülasyon sırasında osilaskop ekranının görünür yapılması.



Şekil 11.4 PWM sinyalinin gözleneceği osilaskop ekranı.

Osilaskop ayarlarını yukarıdaki gibi yaptıktan sonra, üretilen PWM sinyalin şeklini **Duty arttır** ve **Duty azalt** butonlarına basarak değiştirebiliriz. Butonlara her basıldığında LCD ekrandaki **Duty Cycle** oranıyla birlikte PWM sinyalindeki değişikliği gözlemleyiniz. Her değişiklik durumunda osilaskobun özelliğini kullanarak T_{on} süresindeki artma veya

azalmayı da inceleyiniz. İlk yüklenen *Duty Cycle* oranı için T_{on} ve periyot değerleri Şekil 11.5'teki osilaskop ekranı görüntüsünden okunabilir (Şekildeki T_{on} süresi $498,75\mu s$, T_{pwm} süresi ise $1ms$ 'dir).



Şekil 11.5 1KHz'lik PWM sinyalinde Duty Cycle değerinin 127 verilmesi durumunda elde edilecek Ton değeri.

UYGULAMA_02

Önceki PWM uygulamasından farklı olarak bu uygulamada;

1. Her iki PWM kanalı da kullanılacaktır.
2. *Duty Cycle* oranının üretilen PWM sinyali üzerindeki etkisi osilaskop ekranından izlenebileceği gibi LED'lerin parlaklıklarının değişmesi suretiyle de izlenebilecektir.
3. *Duty Cycle* oranı 0-255 arasındaki bir değer olarak değil de % cinsinden LCD ekranına yazdırılacaktır.

İlgili program kodları aşağıda verilmiştir.

PROGRAM KODLARI:

```
//===== Program başlangıcı =====  
//===== LCD modül bağlantıları tanımlanıyor =====  
sbit LCD_RS at LATB0_bit;  
sbit LCD_EN at LATB1_bit;  
sbit LCD_D4 at LATB2_bit;  
sbit LCD_D5 at LATB3_bit;  
sbit LCD_D6 at LATB4_bit;  
sbit LCD_D7 at LATB5_bit;  
sbit LCD_RS_Direction at TRISB0_bit;  
sbit LCD_EN_Direction at TRISB1_bit;  
sbit LCD_D4_Direction at TRISB2_bit;  
sbit LCD_D5_Direction at TRISB3_bit;  
sbit LCD_D6_Direction at TRISB4_bit;  
sbit LCD_D7_Direction at TRISB5_bit;  
//===== LCD modül bağlantılarının sonu =====  
#define LED1_artir RD0_bit           //RD0'daki butona LED1_artir ismi verildi  
#define LED1_azalt RD1_bit          //RD1'deki butona LED1_azalt ismi verildi  
#define LED2_artir RD2_bit          //RD2'deki butona LED2_artir ismi verildi  
#define LED2_azalt RD3_bit          //RD3'deki butona LED2_azalt ismi verildi
```

```

unsigned short Duty1;           //Duty1 isimli deęişken tanımlandı
unsigned short Duty2;           //Duty2 isimli deęişken tanımlandı
char Duty1_S[12];              //Duty1_S isimli char türü deęişken tanımlandı
char Duty2_S[12];              //Duty2_S isimli char türü deęişken tanımlandı
void main() {                   //Ana program başlangıcı
    ADCON1 |= 0x0F;             //PIC'in tüm pinleri dijital giriş/çıkış olarak ayarlandı
    CMCON  |=7;                 //Karşılaştırma modülü iptal edildi.
    TRISD=0xFF;                 //Butonlar bağlanacağından D portu çıkış olarak ayarlandı
    TRISC=0;                    //PWM sinyali alınacağından C portu çıkış olarak ayarlandı,
                                //aynı zamanda PWM1 ucuna LED bağlayıp parlaklığın
                                //deęiştğini göreceğiz
    PORTD=0;                    //D portuna bağlı butonlar basılmıyor olarak ayarlandı
    PORTC=0;                    //C portu sıfırlandı (LED'ler de sönük olarak ayarlandı)
    //----- PWM1 ayarları -----
    PWM1_Init(1000);            //PWM1 kanalından 1KHz'lik sinyal istendięi deklare edildi
    PWM2_Init(1000);            //PWM1 kanalından 1KHz'lik sinyal istendięi deklare edildi
    Duty1=51;                   //PWM1 kanalı için Mevcut Duty Cycle deęeri 51 olarak (%20'ye denk gelir)
                                //Duty1 deęişkenine atandı
    Duty2=128;                  //PWM2 kanalı için Mevcut Duty Cycle deęeri 128 olarak (%50'ye denk gelir)
                                //Duty1 deęişkenine atandı
    PWM1_Start();               //PWM1 başlatıldı
    PWM2_Start();               //PWM2 başlatıldı
    PWM1_Set_Duty(Duty1);       //PWM1'in Duty Cycle deęeri olarak Duty1 verildi
    PWM2_Set_Duty(Duty2);       //PWM2'nin Duty Cycle deęeri olarak Duty2 verildi
    //----- LCD ayarları -----
    Lcd_Init();                 //LCD'yi başlat
    LCD_CMD(_LCD_CURSOR_OFF);   //Ekranda yanıp sönen imleci kapat
    LCD_CMD(_LCD_CLEAR);        //LCD ekranı temizle
    LCD_OUT(1,1,"MEVCUT PWM AYARLARI "); //Sabit olarak kalacak
    LCD_OUT(2,1,"F1=F2: 1000 Hz"); //PWM frekanslarını yazdırdık

```

AD SOYAD:

PWM UYGULAMALARI

```
LCD_OUT(3,1,"D.CYCLE1:"); //Değişken olan Duty1 değerini % olarak bunun
                           //yanına yazdıracağız
LCD_OUT(4,1,"D.CYCLE2:"); //Değişken olan Duty2 değerini % olarak bunun
                           //yanına yazdıracağız
DELAY_MS(250);           //250 ms bekle
LongToStr((Duty1/2.55), Duty1_S); //unsigned short türünde olan Duty1 değerini
                                   // yüzde olarak yazdırmak için 2.55'e bölüp string
                                   // formatına dönüştür ve Duty1_S isimli
                                   //değişkene yükle
LongToStr((Duty2/2.55), Duty2_S); //unsigned short türünde olan Duty2 değerini
                                   // yüzde olarak yazdırmak için 2.55'e bölüp string
                                   // formatına dönüştür ve Duty2_S isimli
                                   //değişkene yükle
Lcd_Out(3,10,Duty1_S);      //string türündeki Duty1_S isimli değişkenin içeriğini
                             //belirtilen adrese yaz
Lcd_Out(4,10,Duty2_S);      //string türündeki Duty2_S isimli değişkenin içeriğini
                             //belirtilen adrese
while(1){                   //Sonsuz döngüye gir
    if(LED1_artir)          //LED1_artir butonuna basılmışsa aşağıdakileri yap
    {
        delay_ms(40);       //40ms bekle
        Duty1++;            //Duty1 değerini 1 arttır
        PWM1_Set_Duty(Duty1); //PWM1'in yeni Duty Cycle değeri Duty1 olarak tekrar verildi
        LongToStr((Duty1/2.55), Duty1_S); //unsigned short türünde olan Duty1 değerini
                                           // yüzde olarak yazdırmak için 2.55'e bölüp string
                                           // formatına dönüştür ve Duty1_S isimli
                                           //değişkene yükle
        Lcd_Out(3,10,Duty1_S); //string türündeki Duty1_S isimli değişkenin içeriğini
                                //belirtilen adrese yaz
    }
}
```

```
if(LED1_azalt)                //LED1_azalt butonuna basılmışsa aşağıdakileri yap
{
    delay_ms(40);              //40ms bekle
    Duty1--;                   //Duty1 değerini 1 azalt
    PWM1_Set_Duty(Duty1);      //PWM1'in yeni Duty Cycle değeri, Duty1 olarak tekrar verildi
    LongToStr((Duty1/2.55), Duty1_S); //unsigned short türünde olan Duty1 değerini
                                    // yüzde olarak yazdırmak için 2.55'e bölüp string
                                    // formatına dönüştür ve Duty1_S isimli
                                    // değişkene yükle
    Lcd_Out(3,10,Duty1_S);     //string türündeki Duty1_S isimli değişkenin içeriğini
                                    // belirtilen adrese yaz
}
if(LED2_artir)                //LED2_artir butonuna basılmışsa aşağıdakileri yap
{
    delay_ms(40);              //40ms bekle
    Duty2++;                   //Duty2 değerini 1 arttır
    PWM2_Set_Duty(Duty2);      //PWM2'nin yeni Duty Cycle değeri, Duty2 olarak tekrar
verildi
    LongToStr((Duty2/2.55), Duty2_S); //unsigned short türünde olan Duty2 değerini
                                    // yüzde olarak yazdırmak için 2.55'e bölüp string
                                    // formatına dönüştür ve Duty2_S isimli
                                    // değişkene yükle
    Lcd_Out(4,10,Duty2_S);     //string türündeki Duty2_S isimli değişkenin içeriğini
                                    // belirtilen adrese yaz
}
if(LED2_azalt)                //LED1_azalt butonuna basılmışsa aşağıdakileri yap
{
    delay_ms(40);              //40ms bekle
    Duty2--;                   //Duty2 değerini 1 azalt
    PWM2_Set_Duty(Duty2);      //PWM2'nin yeni Duty Cycle değeri, Duty2 olarak tekrar verildi
```

```

LongToStr((Duty2/2.55), Duty2_S);    //unsigned short türünde olan Duty2 değerini
                                     // yüzde olarak yazdırmak için 2.55'e bölüp string
                                     // formatına dönüştür ve Duty2_S isimli
                                     //değişkene yükle

Lcd_Out(4,10,Duty2_S);               //string türündeki Duty2_S isimli değişkenin içeriğini
                                     //belirtilen adrese yaz

}

delay_ms(20);                        //Bir sonraki buton kontrolü için 20ms bekle
}                                    //Sonsuz döngünün sınırı
}                                    //Ana programın sonu
//===== Program sonu =====

```

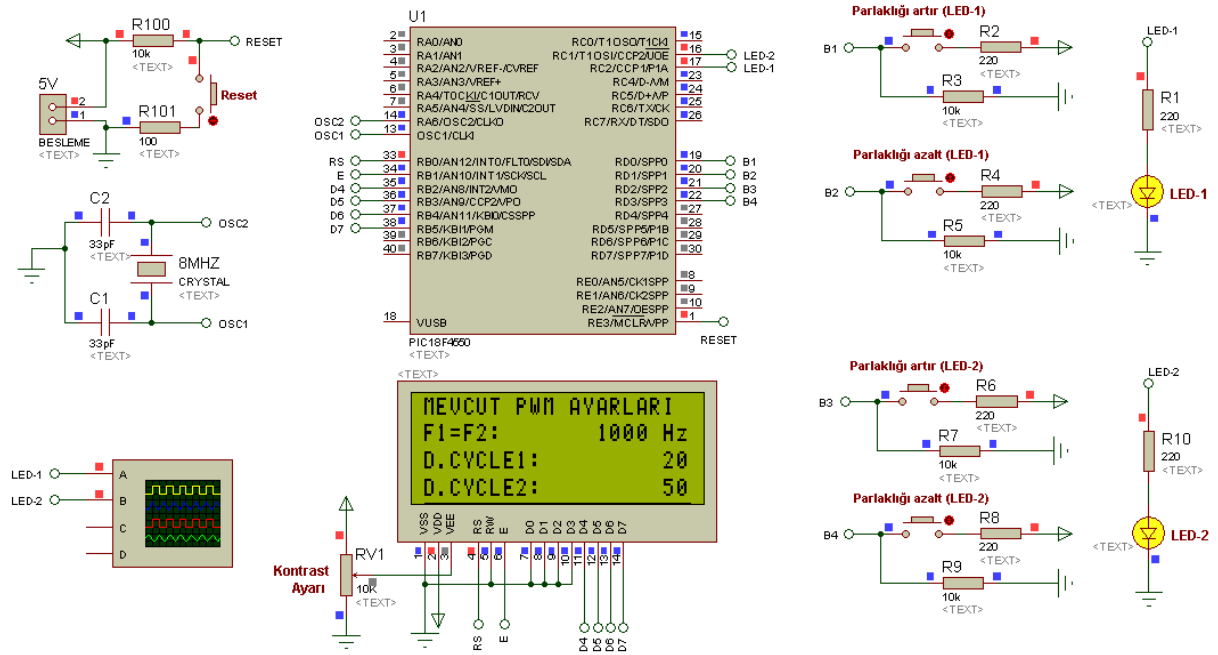
Kodlar yazılıp derlendikten sonra elde edilen hex kodları Şekil 11.6'daki devreye yüklenip simülasyon başlatılacak olursa, ilk yüklenen değerler sayesinde PWM1'de %20'lik bir *Duty Cycle* oranı görülecektir. PWM2'de ise %50'lik bir *Duty Cycle* oranı okunacaktır. Bu değerlere denk ifadeler, programın yazımı sırasında verilen açıklama satırlarından anlaşılabilir. **Parlaklığı artır** veya **Parlaklığı azalt** butonlarına basıldığında ilgili LED'in parlaklığında yavaş yavaş bir değişme olduğu gözlenecektir. Ancak bu değişiklik simülasyonda anlaşılmayıp gerçek ortamda kurulan devreyle kolayca gözlenebilir.

Mevcut başlangıç ayarlarıyla elde edilen osilaskop görüntüsü ise Şekil 11.7'de verilmiştir. PWM2'de başlangıç ayarında verilen %50'lik *Duty Cycle* oranının PWM sinyali üzerindeki etkisi bu ekrandan kolayca anlaşılabilir.

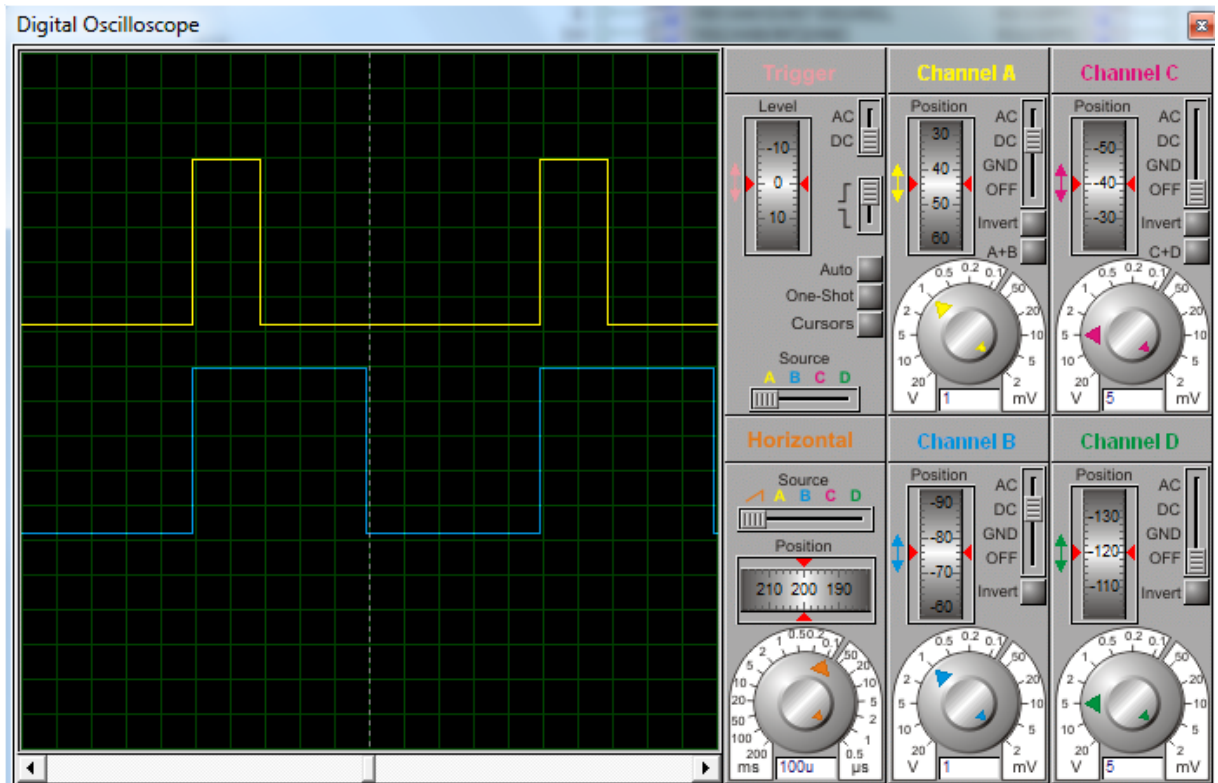
Uygulamanın başında da söylendiği gibi bu uygulamadaki *Duty Cycle* değişiminin LCD'ye yansıtılması 0 il 255 arasında olmayıp % cinsindendir. Dolayısıyla *Duty Cycle* oranı 0 ile 100 arasında bir der alır. Bu değer yüzde cinsinden hesaplanması, normal değeri 0 ile 255 arasında değişen *Duty Cycle* değerinin 2.55'e bölünmesiyle elde edilmiştir. 2.55 değeri 255'in %100'e, 0'ın ise %0'a karşılık gelmesinden ortaya çıkmaktadır.

AD SOYAD:

PWM UYGULAMALARI



Şekil 11.6 Çift kanallı PWM uygulamasına yönelik ISIS şeması.



Şekil 11.7 Çift kanallı PWM uygulamasındaki osilaskop görüntüsü.

BÖLÜM 12

12. DEVİR SAYACI (FREKANS SAYICI) UYGULAMASI

Bu uygulamada mikrodenetleyicinin girişine uygulanan saat darbesini sayan bir uygulama yapacağız. Uygulamanın temel mantığı **Timer** ve **CCP** modülünün yakalama modunu beraber kullanmasından ileri gelmektedir. PIC18F4550’de iki adet CCP modülü bulunur. Bunlar **CCP1** ve **CCP2** dir. Yakalama modunda, aşağıda verilen sinyal durumlarından herhangi biri seçildiği zaman, CCPx pininden gelen sinyal, seçilen şartlardan birini gerçekleştirdiği zaman Timer1 veya Timer3’ün içeriği CCPRxH ve CCPRxL kaydedicisinin içerisine aktarılır ve kesme oluşur. Aktarılacak olan diğer 16 bitlik bir değer olduğu için iki kaydedici kullanılır. İkinci bir kesme olayında Timer1 ve Timer3’ün değeri tekrar aynı kaydedicilere aktarılır ve önceki değerleri silinir.

CCPx pininde oluşabilecek sinyal durumları aşağıda verilmiştir. Bu durumlar **CCPxM3:CCPxM0** bitleri ayarlanarak elde edilebilir.

- Sinyalin her düşen kenarında
- Sinyalin her yükselen kenarında
- Sinyalin her 4. yükselen kenarında
- Sinyalin her 16. yükselen kenarında

CCPxCON: STANDARD CCPx CONTROL REGISTER

U-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
— ⁽¹⁾	— ⁽¹⁾	DCxB1	DCxB0	CCPxM3	CCPxM2	CCPxM1	CCPxM0
bit 7							bit 0

Legend:

R = Readable bit

W = Writable bit

U = Unimplemented bit, read as '0'

-n = Value at POR

'1' = Bit is set

'0' = Bit is cleared

x = Bit is unknown

bit 7-6 **Unimplemented:** Read as '0'⁽¹⁾bit 5-4 **DCxB1:DCxB0:** PWM Duty Cycle Bit 1 and Bit 0 for CCPx ModuleCapture mode:

Unused.

Compare mode:

Unused.

PWM mode:

These bits are the two LSBs (bit 1 and bit 0) of the 10-bit PWM duty cycle. The eight MSBs of the duty cycle are found in CCPR1L.

bit 3-0 **CCPxM3:CCPxM0:** CCPx Module Mode Select bits

0000 = Capture/Compare/PWM disabled (resets CCPx module)

0001 = Reserved

0010 = Compare mode: toggle output on match (CCPxIF bit is set)

0011 = Reserved

0100 = Capture mode: every falling edge

0101 = Capture mode: every rising edge

0110 = Capture mode: every 4th rising edge

0111 = Capture mode: every 16th rising edge

1000 = Compare mode: initialize CCPx pin low; on compare match, force CCPx pin high (CCPxIF bit is set)

1001 = Compare mode: initialize CCPx pin high; on compare match, force CCPx pin low (CCPxIF bit is set)

1010 = Compare mode: generate software interrupt on compare match (CCPxIF bit is set, CCPx pin reflects I/O state)

1011 = Compare mode: trigger special event, reset timer, start A/D conversion on CCPx match (CCPxIF bit is set)

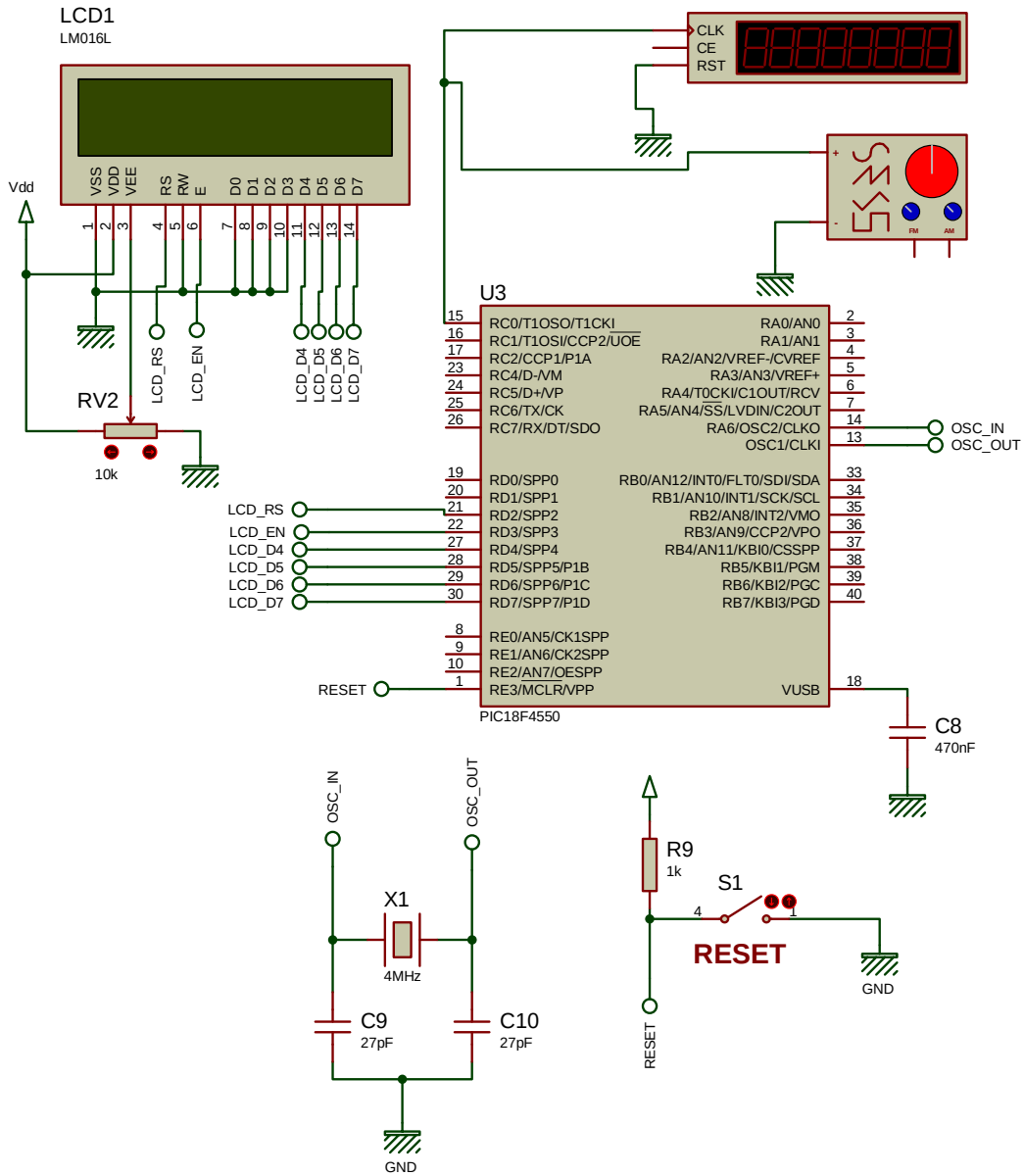
11xx = PWM mode

Note 1: These bits are not implemented on 28-pin devices and are read as '0'.

Şekil 12.1'de devre şeması görülmektedir.

AD SOYAD:

DEVİR SAYACI UYGULAMALARI



Şekil 12.1 Devir sayacı uygulamasına ilişkin ISIS devresi

Timer1 bu devrede sayıcı olarak kullanılmakta, T1CKI girişine uygulanan sinyalin her yükselen kenarında Timer1 sayacı artmaktadır. Ayrıca devrede Timer0, 1000 ms lik zamanlayıcı olarak kullanılmış ve her 1000 ms de bir kesme oluşması sağlanmıştır. Böylelikle frekansımız her 1000 ms de bir ölçülmüş olacaktır.

MikroC yazılımına ait kodlar aşağıdaki gibidir:

PROGRAM KODLARI:

```

// LCD modül bağlantıları
sbit LCD_RS at RD2_bit;
sbit LCD_EN at RD3_bit;
sbit LCD_D4 at RD4_bit;
sbit LCD_D5 at RD5_bit;
sbit LCD_D6 at RD6_bit;
sbit LCD_D7 at RD7_bit;

sbit LCD_RS_Direction at TRISD2_bit;
sbit LCD_EN_Direction at TRISD3_bit;
sbit LCD_D4_Direction at TRISD4_bit;
sbit LCD_D5_Direction at TRISD5_bit;
sbit LCD_D6_Direction at TRISD6_bit;
sbit LCD_D7_Direction at TRISD7_bit;

unsigned long FRQ1_Value;
char *FRQ1 = "00000";

void interrupt()
{
    if (CCP1IF);           // CCP1 yakalama oluştu
                           // TMR1'in içeriği otomatik olarak CCPR1 içeriğine
                           // aktarılır.
    CCP1IF_bit = 0;        // CCP1IF bayrağı temizlendi

    //timer0 1000 ms lik interrupta ulaştı ise
    if (TMR0IF_bit)
    {
        TMR0IF_bit = 0;
        TMR0H      = 0x0B;
        TMR0L      = 0xDC;

        //CCP1'i sıfır yaparak RPM değerini hesaplar
        RC2_bit = 0;           //CCP1 pinini pasifleştir ve yüklenen değere göre hesapla
        TMR1IE_bit = 0;
        CCP1IE_bit = 0;
        FRQ1_Value = (256*TMR1H + TMR1L); // *2; // *60;
        //RPM_Value = (256*CCPR1H + CCPR1L)*120;

        //Timer 1 ön değerlerini sıfırla
        TMR1L = 0x00;          // TMR1L ve TMR1H kaydedici çifti sıfırlandı
        TMR1H = 0x00;
        T1CON = 0x03;          // TMR1 1:1 prescaler ve T13CKI girişinden gelen
                                // her 1. yükselen darbe kenarında 1 artmaya ayarlandı
        TRISC.RC0 = 1;          // RC0 (T1CKI) pini giriş yapıldı
        CCPR1L = 0x00;          // CCPR1L ve CCPR1H kaydedici içeriği temizlendi
        CCPR1H = 0x00;
        CCP1CON = 0x04;         // CCP1 donanımı sinyalin her düşen kenarını
                                // yakalar. CCP1 donanımı aktif edildi.
        CCP1IE_bit = 1;

        RC2_bit = 1;           //CCP1 pinini aktifleştir
    }
}

```

```

void CCP1_Init()
{
    //----- CCP1 girişı için kesmeleri ayarla -----
    //-----
    TMR1L = 0x00;           // TMR1L ve TMR1H kaydedici çifti sıfırlandı
    TMR1H = 0x00;
    T1CON = 0x03;           // TMR1 1:1 prescaler ve T13CKI girişinden gelen
                           // her 1. yükselen darbe kenarında 1 artmaya ayarlandı
    TRISC.RC0 = 1;          // RC0 (T1CKI) pini giriş yapıldı

    CCPR1L = 0x00;          // CCPR1L ve CCPR1H kaydedici içeriği temizlendi
    CCPR1H = 0x00;
    CCP1CON = 0x04;         // CCP1 donanımı sinyalin her düşen kenarını
    CCP1IE_bit = 1;         // yakalar. CCP1 donanımı aktif edildi.

    RC2_bit = 1;            //CCP1 pinini aktifleştir

    INTCON = 0xC0;          // GIE, PEIE bitleri set edildi

    //Timer0
    //Prescaler 1:16; TMR0 Preload = 3036; Actual Interrupt Time : 1 sn
    T0CON = 0x83;
    TMR0H = 0x0B;
    TMR0L = 0xDC;
    GIE_bit = 1;
    TMR0IE_bit = 1;
}

void main() {
    Lcd_Init();
    Lcd_Cmd(_LCD_CLEAR);           // LCD yi temizle
    Lcd_Cmd(_LCD_CURSOR_OFF);     //imleci kapat
    delay_ms(200);

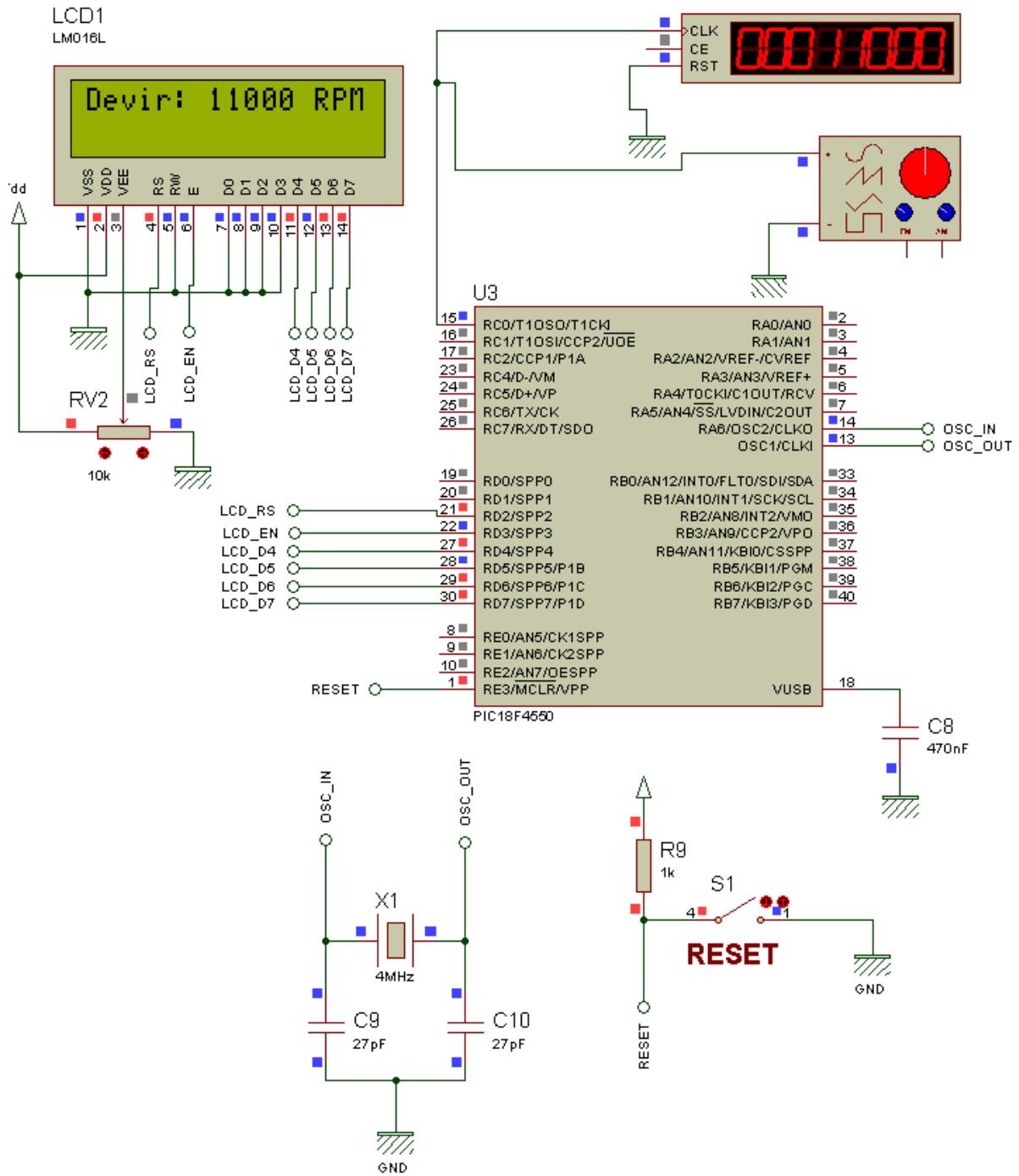
    CCP1_Init();
    while(1)
    {
        Lcd_Cmd(_LCD_CLEAR);
        sprintf(FRQ1, "%05u", FRQ1_Value);
        Lcd_Out(1,1,"Devir: ");
        Lcd_Out_CP(FRQ1);
        Lcd_Out_CP(" RPM");
        delay_ms(500);
    }
}

```

Proje çalıştırılarak farklı frekansta sinyaller uygulanmış ve ekran görüntüleri Şekil 12.2 ve Şekil 12.3'teki gibi olmuştur.

AD SOYAD:

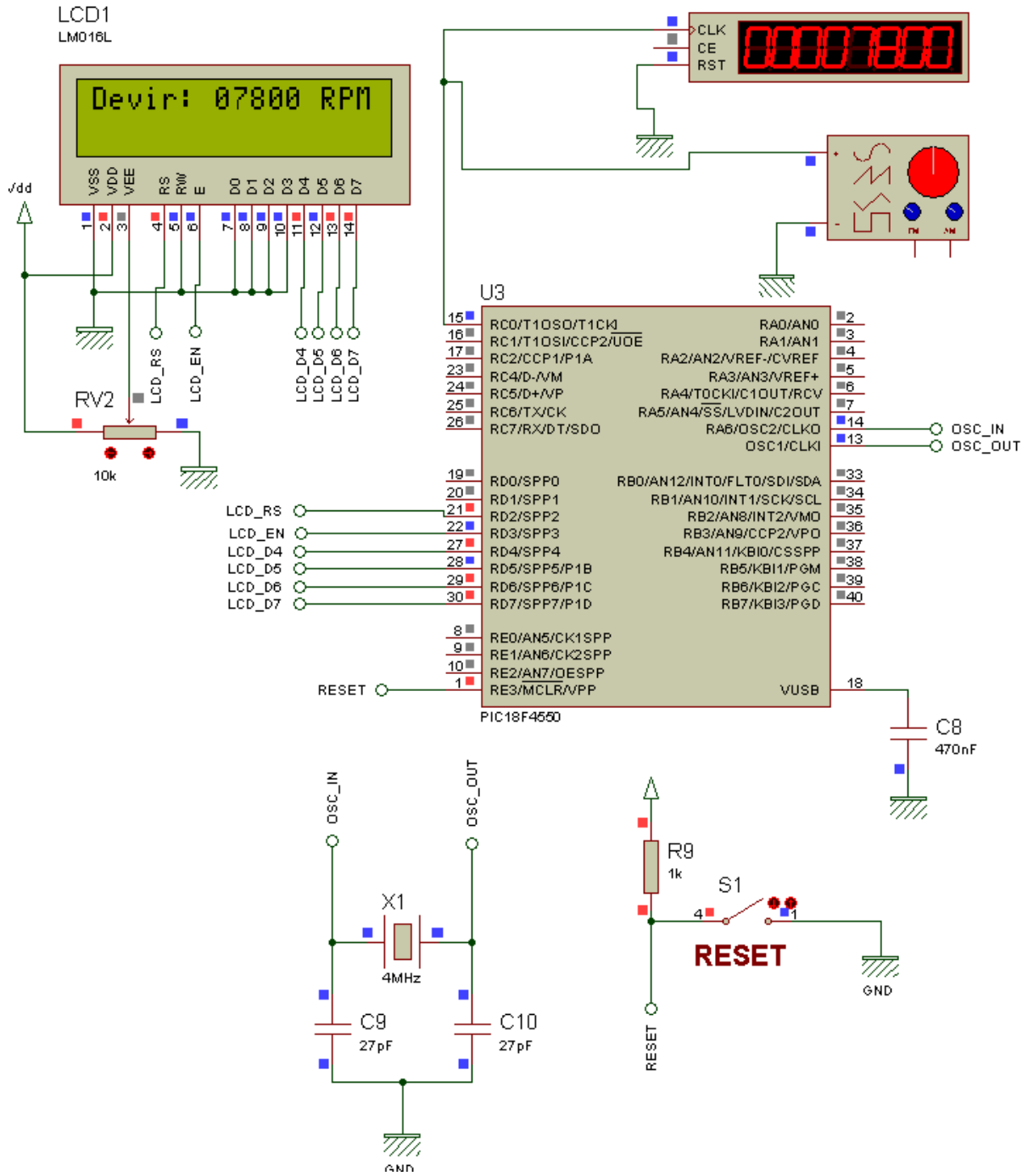
DEVİR SAYACI UYGULAMALARI



Şekil 12.2 Devir sayacı uygulamasına ilişkin (11000rpm) ISIS devresi

AD SOYAD:

DEVİR SAYACI UYGULAMALARI



Şekil 12.3 Devir sayacı uygulamasına ilişkin (7800rpm) ISIS devresi

BÖLÜM 13

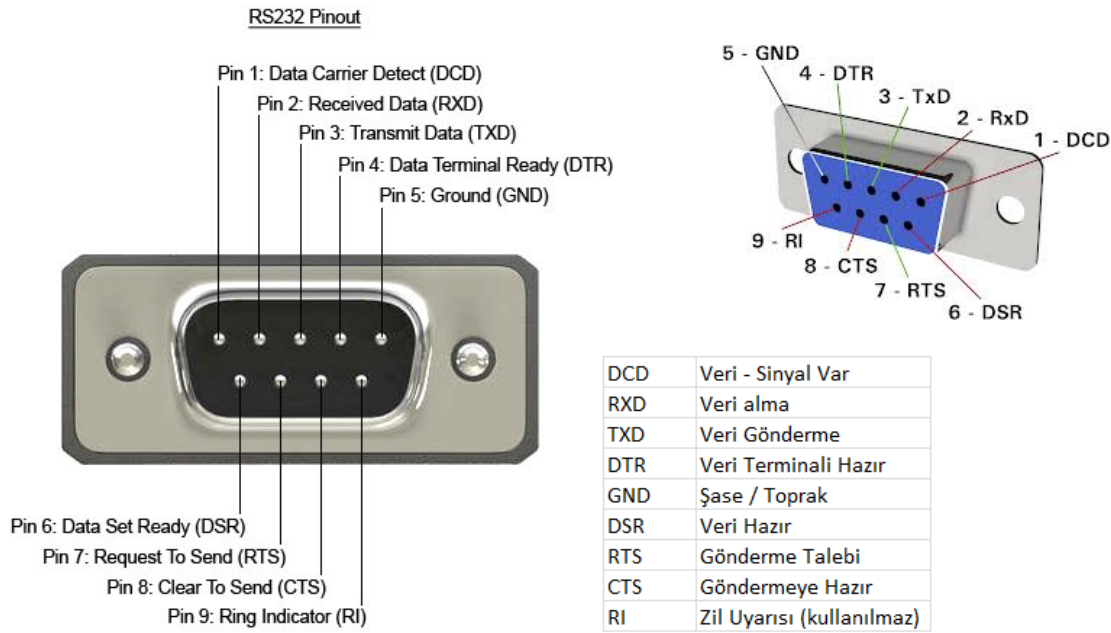
13. SERİ HABERLEŞME (EUSART)

Seri iletişim en basit anlamıyla dijital bilginin yani 1 ve 0'ların tek bir hat üzerinden peş peşe iletilmesi anlamına gelmektedir. PIC18F4550'nin sahip olduğu Enhanced Universal Synchronous Asynchronous Receiver Transmitter (EUSART) modülü, çevresel seri haberleşme arabirimi olarak kullanılmaktadır. Genellikle USART olarak bilinir. Bu modül, bağımsız olarak seri haberleşme komutlarını işleyebilecek şekilde kendi içerisinde saat darbesi üretene, kaydırma yazmaçlarına ve veri tampon belleğine sahiptir. Bu modül senkron ve asenkron veri iletişimi destekler.

EUSART arabirimi şu özelliklere sahiptir:

- Çift yönlü asenkron haberleşme
- Yüksek yoğunlukta baud rate üretebilme
- 8 bit veya 9 bit veri transferi
- Ayarlanabilir stop biti sayısı (1-2 bit)
- Senkron haberleşme için sinyal çıkışı
- Tek hat üzerinden tek yönlü haberleşebilme
- 4 adet hata tespit bayrağı (overrun, noise, frame, parity)
- 2 adet kesme vektörü (RXI, TXI)
- Azaltılmış güç tüketim modu
- Çoklu işlemci haberleşme

Seri haberleşme arabirimi, RS232 olarak da bilinir. Bilgisayarlarımızda bulunan ve cihazlarla bağlantı kurulan seri haberleşme portları Şekil 13.1'deki gibidir.

**Şekil 13.1 Seri Haberleşme Portu**

Seri iletişimde belirli bir format kullanılmaktadır. Bu formatta veriler baytlar halinde iletilir. Her bir bayt için belirli bir başlangıç ve bitiş bitleri de ayrıca yer alır. En çok kullanılan 1 bit start, 8 bit veri ve 1 bit stop biti formatıdır. Yani 1 bayt veriyi iletmek için 10 bit gönderilir. Seri veri iletiminde, bir kerede bir karakterin sadece bir biti iletilir. Alıcı makine, doğru haberleşme için karakter uzunluğunu, başla-bitir (start-stop) bitlerini ve iletim hızını bilmek zorundadır. Paralel veri iletiminde, bir karakterin tüm bitleri aynı anda iletildiği için başla-bitir bitlerine ihtiyaç yoktur. Dolayısıyla doğruluğu daha yüksektir. Paralel veri iletimi, bilginin tüm bitlerinin aynı anda iletimi sebebiyle çok hızlıdır.

Seri Haberleşmede kullanılan bazı kavramlardan bahsedelim:

Bit: Elektriksel bir işarete bit(b) denir. Genelde elektrik olduğunu sayısal 1 ile elektrik olmadığını sayısal 0 ile ifade edilir. Bu durumda bir bilgi işaretinde bulunan her bir 1 ve 0 bilgisi bir bitte karşılık gelmektedir. 8 bit bir byte (B) eder. Örneğin, 1011001011111010 şeklindeki bir işaret 16b(bit) veya 2B(byte)'dır.

Bps (Bit Per Second): Saniyede iletilen bit sayısıdır.

Baud: Genellikle modem benzeri cihazların seri haberleşme hızlarını ifade etmekte kullanılır.

Buad Rate (Baud oranı): Baud oranı verinin 1 bitin ne kadar sürede gönderileceğini ve alınacağını belirleyen Bps birimi ile ölçülür. Baudrate oranları 1200, 2400, 4800, 19200 gibi değerlerdir. Seri iletişimde alıcı ve verici cihazların baud rate değerlerinin aynı olması gerekir ki veriler doğru bir şekilde okunabilsin. Aksi takdirde veri alışverişi doğru bir şekilde gerçekleşmez.

Seri İletişim; hızlı veri iletiminin gerekmediği durumlarda ve çevre biriminin bilgisayardan uzakta bulunduğu durumlarda tercih edilmektedir. İki türlü seri iletişim vardır. Bunlar;

1. Senkron seri iletişim
2. Asenkron seri iletişim

13.1. SENKRON SERİ İLETİŞİM

Senkron iletişim alıcı ve vericinin eş zamanlı çalışması anlamına gelir. Senkron bilgi transferinde bilgi ile saat frekansı da transfer edilir. Bu durum start ve stop bitlerinin gereğini ortadan kaldırır. Aynı zamanda senkron iletişim karakter blokları bazında olduğu için asenkrona göre daha hızlıdır. Ancak daha karmaşık devreler içerir ve daha pahalıdır.

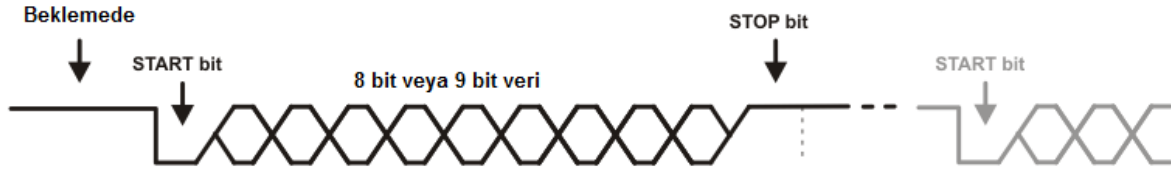
Senkron iletişim alıcı ve vericinin eş zamanlı çalışması anlamına gelir. Saat darbesi ihtiyacı da bu durumdan kaynaklanır. İletim için önce gönderici taraf belirli bir karakter gönderir. Bu her iki tarafça bilinen iletişime başlama karakteridir. Alıcı taraf bu karakteri okursa iletişim kurulur. Verici bilgileri gönderir. Transfer işlemi veri bloğu tamamlanana ya da alıcı verici arasındaki eşleme kayboluncaya kadar devam eder.

13.2. ASENKRON SERİ İLETİŞİM

Herhangi bir zamanda veri gönderilebilir. Transferler karakter bazında yapılır. Veri gönderilmediği zaman hat boşta kalır. Senkron seri iletişimden daha yavaştır. Her veri grubu ayrı olarak gönderilir. Gönderilen veri bir anda bir karakter olacak şekilde hatta bırakılır.

Karakterin başına başlangıç ve sonunda hata sezmek için başka bir bit eklenir. Başlangıç için başla biti (0), veri iletişimini sonlandırmak için ise dur biti (1) kullanılır.

Bir asenkron karakter start biti, parity biti, veri bitleri ve stop bitlerinden oluşur.



Hat boştayken lojik olarak 1 dir. Her veri iletimi 0 olan Start biti ile başlar. Her veri 8 veya 9 bit olarak en düşük değerlikli bittten (LSB) en yüksek değerlikli bite (MSB) doğru gönderilir. Veri gönderimi tamamlandığını bildirmek için stop biti gönderilir.

Start biti: Bir karakterin gönderilmeye başlandığını bildirmek için kullanılır. Her zaman transferin ilk biti olarak gönderilir.

Data bitleri: Veri bitlerini oluşturan gruplar bütün karakterlerden ve klavyedeki diğer tuşlardan oluşur.

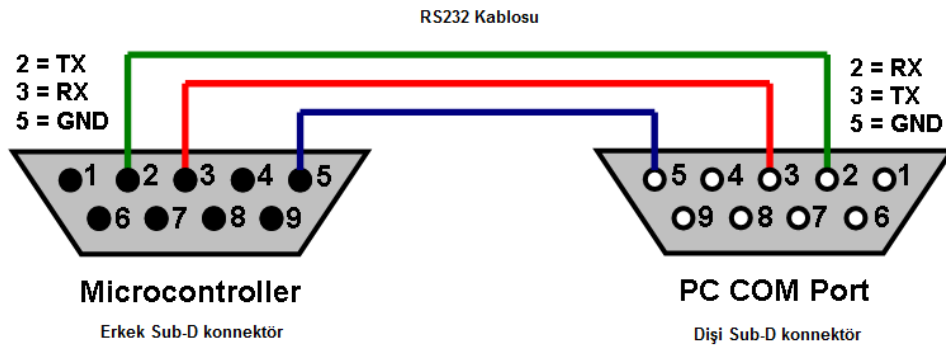
Parity biti: Transfer edilen karakterlerin karşı tarafa doğru gönderilip gönderilmediğini kontrol etmek için kullanılan bittir. Eğer alıcı, alınan parity biti ile hesaplanan parity bitinin eşit olmadığını tespit ederse, hata verir ve o andaki karakteri kabul etmez.

Stop biti: Karakterin bittiğini gösterir. Karakterler arasında boş ya da ölü zamanlar sağlar. Stop biti gönderildikten sonra istenildiği zaman yeni bilgi gönderilebilir.

Seri iletişimde karşılıklı olarak kullanılabilecek en temel bağlantı şekli Şekil 13.2'deki gibidir.

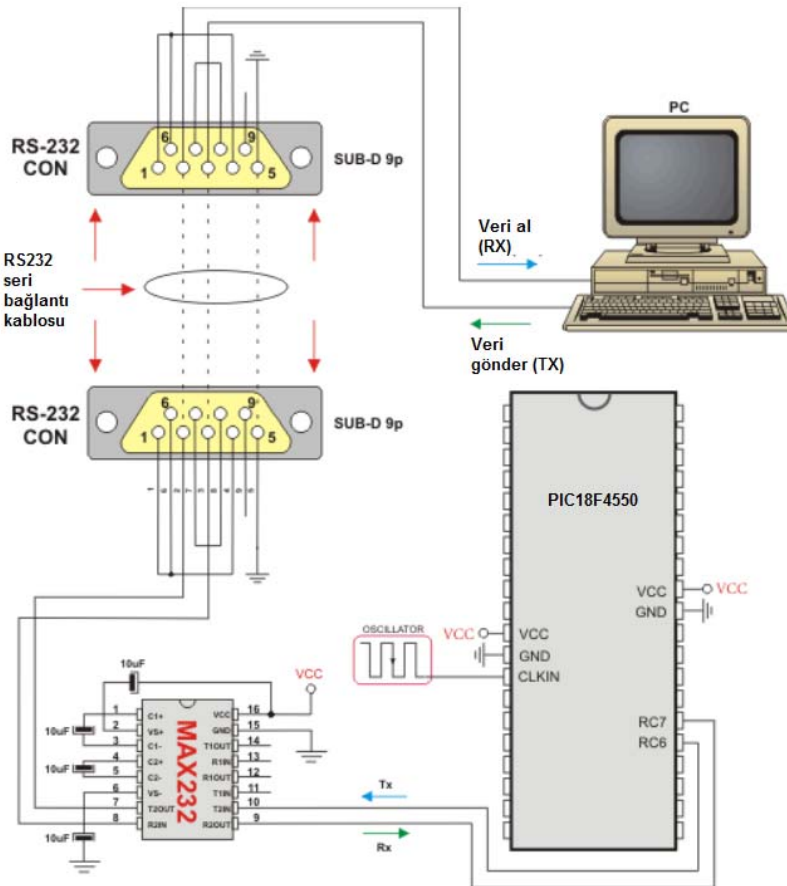
AD SOYAD:

SERİ HABERLEŞME



Şekil 13.2 Seri iletişimde kullanılabilecek en temel bağlantı

Bilgisayarla mikrodnetleyici arasında kullanılan bağlantı şeması ise Şekil 13.3'teki gibidir. Bu devrede seri bağlantı portu ile Mikrodnetleyicinin seri modülü arasına bir adet MAX232 entegre yerleştirilmiştir. Bunun sebebi, bilgisayarın seri portunun geriliminin +/-12V olması, mikrodnetleyicilerin ise 5V ile çalışmasıdır. MAX232 entegresi seri port çıkışındaki bu 12 voltluk gerilimi Mikrodnetleyicinin çalışabileceği gerilim seviyesine düşürür.



Şekil 13.3 PC ile mikrodnetleyici arasındaki seri bağlantı

Seri iletişim aslında karmaşık bir yapıya sahip olmasına rağmen, MikroC derleyicisi bize bu konuda kolaylık sağlamaktadır. MikroC'nin Uart kütüphanesinde aşağıdaki komutlar mevcuttur ve bu komutlar seri iletişimi rahatlıkla yapabilmemizi sağlar.

- Uartx_Init
- Uartx_Data_Ready
- Uartx_Tx_idle
- Uartx_Read
- Uartx_Read_Text
- Uartx_Write
- Uartx_Write_Text
- Uartx_Set_Active

Bazı mikrodenetleyicilerde seri haberleşme modülü birden fazla olabilir. Yukarıdaki Uartx diye başlayan komutlardaki x, modül numarasını ifade etmektedir. Birden fazla modül barındıran mikrodenetleyiciler için komut yazılırken 1. mi yoksa 2. modülün mü kullanılacağına karar verilerek Uart1_..., Uart2_... şeklinde yazılır. 18F4550'de sadece bir modül bulunduğu için komutlarımız sadece Uart1_... şeklinde olacaktır.

Komutların genel özellikleri şunlardır:

Uartx_Init: Seri haberleşmeyi belirtilen baud rate değeri ile başlatır.

Örnek kullanımı:

```
Uart1_Init(9600); //seri haberleşme modülünü 9600 baud rate ile başlat
```

UARTx_Data_Ready: Seri haberleşme modülüne herhangi bir veri gelmiş mi? Geldiyse 1, gelmediyse 0 sonucunu verir.

Örnek kullanımı:

```
if (UART1_Data_Ready() == 1) //seri haberleşme modülüne veri geldi mi
{
    okunan = UART1_Read(); //veriyi oku ve değişkene aktar
}
```

UARTx_Tx_Idle: Seri haberleşme modülünde veri gönderme kaydedicisinin boş olup olmadığını kontrol eder. Gönderildiyse 1, hata varsa 0 sonucunu verir.

Örnek kullanımı:

```
if (UART1_Tx_Idle() == 1) //önceki veri gönderildi mi
{
    UART1_Write(_veri); //veri gönderime hazır. Sonraki veriyi gönder.
}
```

UARTx_Read: Seri haberleşme modülüne gelen bir byte'lık veriyi okur.

Örnek kullanımı:

```
if (UART1_data_Ready() == 1) //modüle veri geldi mi
{
    okunan = UART1_Read(); //veriyi oku ve değişkene aktar
}
```

UARTx_Read_Text: Seri haberleşme modülüne gelen karakter dizisini beklenen bir karakter ya da karakter dizisi gelene kadar okumaya devam eder. Beklenen veri geldiğinde içindeki verileri değişkene aktarır.

Örnek kullanımı:

```
if (UART1_data_Ready() == 1) //modüle veri geldi mi
{
    Uart1_Read_Text(okunan, "OK", 10); // "OK" gelene kadar gelen 10 karakteri oku
    // ve okunan değişkenine aktar
}
```

UARTx_Write: Bir byte'lık veriyi gönderir.

Örnek kullanımı:

```
Unsigned char _veri = 0x1E;

Uart1_Write(_veri); // 0x1E değerini seri porttan gönder
```

UARTx_Write_Text: Karakter dizisini seri porttan gönderir.

Örnek kullanımı:

```
Unsigned char veriler[10] = "merhaba";  
  
Uart1_Write_Text(veriler); // seri porttan "merhaba" karakterlerini gönder
```

UART_Set_Active: Birden fazla UART modülü bulunan mikrodenetleyicilerde istenilen modülün o an için aktif edilmesini sağlar. Bu komut kullanıldığında Uartx ile başlayan komutların yerine doğrudan Uart_Init, Uart_Read şeklinde komutlar kullanılabilir.

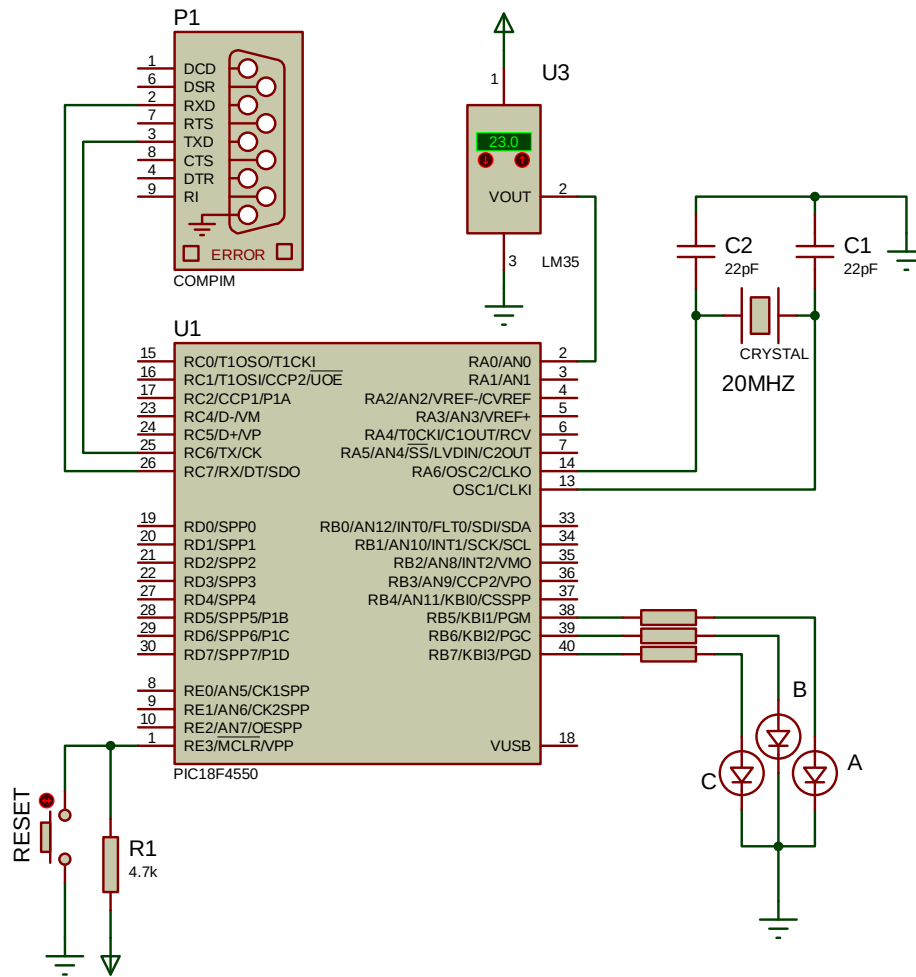
Örnek kullanımı:

```
Uart1_Init(9600); //Uart1 modülünü başlat  
Uart2_Init(9600); //Uart2 modülünü başlat  
  
// 1. Seri haberleşme modülünü aktif et.  
Uart_Set_Active(&Uart1_Read, &Uart1_Write, &Uart1_Data_Ready, &Uart1_Tx_Idle);  
Uart_Write(veri); //aktif olan modülden veriyi gönder
```

ÖRNEK UYGULAMA

Bu uygulamada hem mikrodnetleyici için MikroC ile bir yazılım yapılacaktır, hem de Microsoft Visual C# ile PC arayüz yazılımı tasarlanacaktır. Mikrodnetleyici, LM35 sıcaklık sensörü ile oda sıcaklığını ölçerek seri haberleşme portu aracılığıyla arayüz yazılımına gönderecek, arayüz yazılımından da evdeki A, B ve C odalarındaki lambalar kontrol edilebilecektir. (Lambaları LED'ler temsil edecektir.) Böylelikle çift taraflı bir seri haberleşme işlemi yapılacaktır.

Mikrodnetleyici devresi Şekil 13.4'teki gibi Proteus'ta tasarlanmıştır. Devrede seri port bağlantısını sağlayabilmek için COMPIM isimli bileşen kullanılmıştır. (Gerçek uygulamada MAX232 gerilim seviyesi ayarlayıcı entegre de bağlanır.)



Şekil 13.4 Seri haberleşme uygulamasına ilişkin ISIS devresi

AD SOYAD:

SERİ HABERLEŞME

MikroC yazılımı yapılırken proje ayarlarından mikrodenetleyicinin saat frekansı 20 MHz olacak şekilde ayarlanmıştır.

MikroC program kodları aşağıda verilmiştir. LM35 sıcaklık sensörü, analog çıkış veren bir sensördür. Bu yazılımda sıcaklık sensöründen gelen bilgi analog dijital dönüştürücü modülü aracılığıyla 10 bitlik veri halinde yani 0-1023 aralığında okunmuştur. LM35 sıcaklık sensörü her bir derece için 10 mV gerilim üretmektedir. Dolayısıyla 150 derecede vereceği maksimum gerilim 1500 mV yani 1.5 V tur. Okunan değerin (0-1023 aralığı) derece cinsine (0 - 150 derece) dönüşümü yapılmış ve sıcaklık bilgisi bu şekilde seri porttan gönderilmiştir.

```
long int OkunanDeger;
char ADC_Degeri[12];
char RS232_okunan[2];
void main() {
    TRISB = 0;          //B portundaki tüm pinleri çıkış yap.
    LATB = 0;           //B portundaki bütün LED'leri söndür
    ADC_init();          //Analog ve dijital converter modülünü etkinleştir.
    Uart1_init(9600);    //seri haberleşme modülünü 9600 bps baud rate ile başlat

    while(1)
    {
        //AN0 dan okunan analog değer karşılığını değişkene aktar
        OkunanDeger = ADC_Read(0);
        OkunanDeger = OkunanDeger * 500 / 1023; //LM35 için dönüşüm yapıldı
        //seri porttan gönderilen bilgi string tipinde olmalıdır
        LongToStr(OkunanDeger, ADC_Degeri);

        UART1_Write_Text(ADC_Degeri);    //ADC değerini seri haberleşme ile gönder
        UART1_Write_Text("@");           //gönderilen bilginin sonunda @ işareti olsun.
                                           //visual C# programı doğru veriyi
                                           //@ işaretinden tanıyacak
        uart1_write(13);                  //satır başına dön
        uart1_write(10);                  //satır boşluğu gönder

        if (UART1_data_Ready() == 1) //modüle veri geldi mi
        {
            RS232_okunan[0] = UART1_Read(); //gelen verinin ilk karakterini oku
            RS232_okunan[1] = UART1_Read(); //gelen verinin ikinci karakterini oku
            RS232_okunan[2] = 0;

            //şimdi gelen bilgiyi incele
            switch (RS232_okunan[0])
            {
                case 'A':
                    if (RS232_okunan[1] == '1')
                        portb.f5 = 1; // 1. ledi yak
                    else if (RS232_okunan[1] == '0')
                        portb.f5 = 0; // 1. ledi söndür
                    break;
            }
        }
    }
}
```



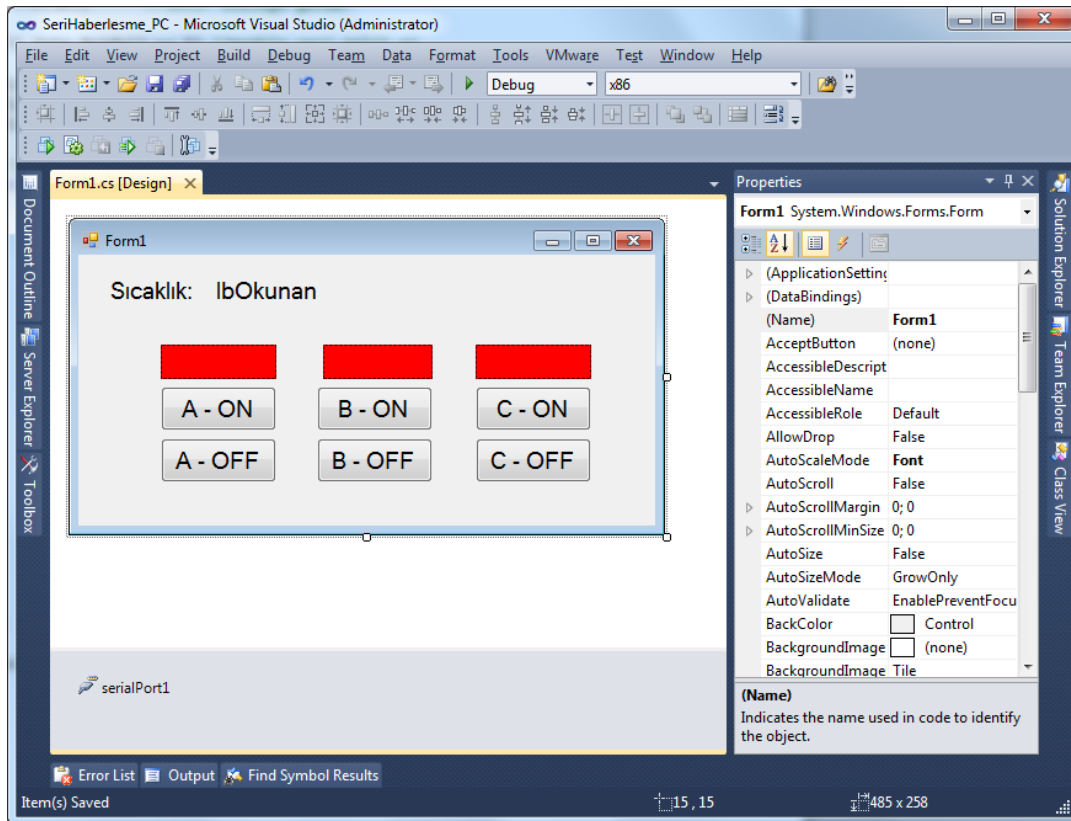
```

case 'B':
    if (RS232_okunan[1] == '1')
        portb.f6 = 1; // 2. ledi yak
    else if (RS232_okunan[1] == '0')
        portb.f6 = 0; // 2. ledi söndür
    break;

case 'C':
    if (RS232_okunan[1] == '1')
        portb.f7 = 1; // 3. ledi yak
    else if (RS232_okunan[1] == '0')
        portb.f7 = 0; // 3. ledi söndür
    break;
}
}
delay_ms(500);

```

Arayüz yazılımı Microsoft Visual Studio 2010 kullanılarak C# ortamında gerçekleştirilmiştir. Arayüz ekranı Şekil 13.5'teki gibi olup sıcaklık bilgisinin gösterimi için iki adet Label kontrolü, lambaların durumunu göstermek için 3 adet Panel kontrolü (backcolor – red yapıldı), lambaların her birini açıp kapatmak için de 6 adet buton kontrolü yerleştirilmiştir. Seri haberleşmenin yapılabilmesi için de serialPort1 komponenti eklendi.



Şekil 13.5 Seri haberleşme uygulaması için Arayüz ekranı

Formun üzerine çift tıklayarak Form1_Load olayı için aşağıdaki kodları yazıyoruz.

```
private void Form1_Load(object sender, EventArgs e)
{
    serialPort1.PortName = "COM5"; //COM5 isimli porttan bağlan
    serialPort1.BaudRate = 9600;    //baudrate değeri 9600 (PIC'deki ile aynı)
    serialPort1.DataBits = 8;
    serialPort1.Parity = Parity.None;
    serialPort1.StopBits = StopBits.One;

    //seri porttan veri geldiğinde sp_DataReceived olayını çalıştır.
    //Bu olay aşağıda tanımlanmıştır
    serialPort1.DataReceived += new SerialDataReceivedEventHandler(sp_DataReceived);

    if (serialPort1.IsOpen == false)
        serialPort1.Open();
}
```

Bu kodların alt tarafına seri porttan veri geldiğinde yapılacak işlemlere ait kodları yazıyoruz.

```
private void si_DataReceived(string veri)
{
    //okunan veri ile ilgili işlemler bu bölümde yapılacak
    string[] Dizi;
    int ADC_Degeri;

    Dizi = veri.Split('@');

    if (int.TryParse(Dizi[0], out ADC_Degeri))
    {
        lbOkunan.Text = ADC_Degeri.ToString();
    }
}

private delegate void SetTextDeleg(string text);

void sp_DataReceived(object sender, SerialDataReceivedEventArgs e)
{
    string veri = serialPort1.ReadLine();
    this.BeginInvoke(new SetTextDeleg(si_DataReceived), new object[] { veri });
}
```

Lambaların yanıp sönmesini sağlamak için PC arayüzünden PIC yazılımına komut göndermek gerekiyor. Bunlara ait kodlar da aşağıdaki gibidir.

```
private void button1_Click(object sender, EventArgs e)
{
    serialPort1.Write("A1");
    panel1.BackColor = Color.Green;
}

private void button2_Click(object sender, EventArgs e)
{
    serialPort1.Write("A0");
    panel1.BackColor = Color.Red;
}

private void button3_Click(object sender, EventArgs e)
{
    serialPort1.Write("B1");
    panel2.BackColor = Color.Green;
}

private void button4_Click(object sender, EventArgs e)
{
    serialPort1.Write("B0");
    panel2.BackColor = Color.Red;
}

private void button5_Click(object sender, EventArgs e)
{
    serialPort1.Write("C1");
    panel3.BackColor = Color.Green;
}

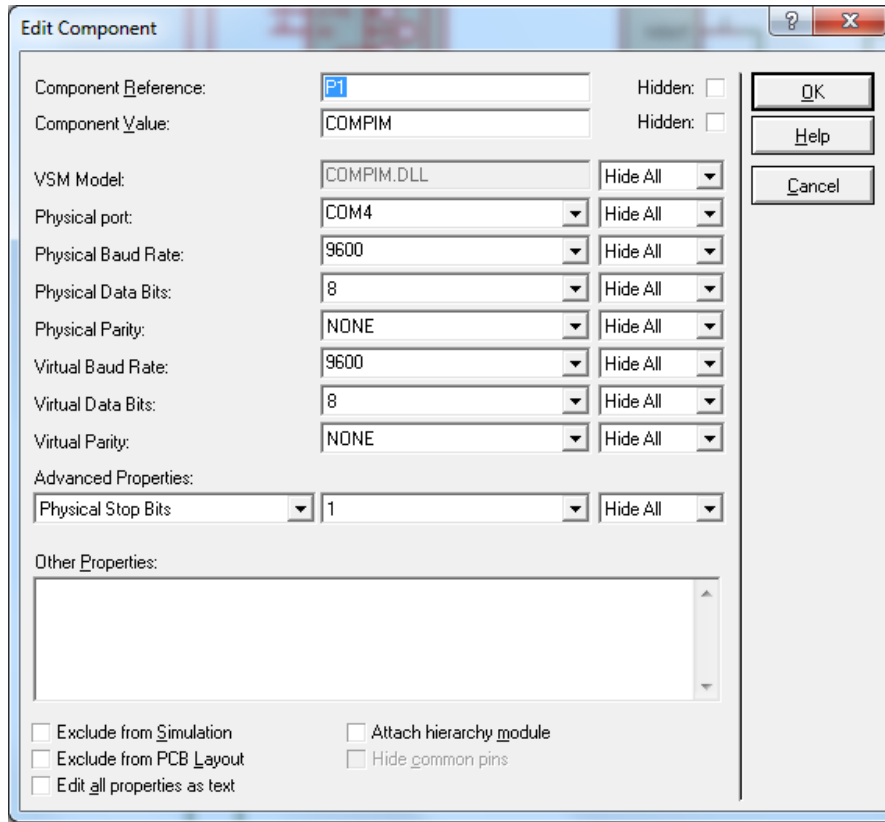
private void button6_Click(object sender, EventArgs e)
{
    serialPort1.Write("C0");
    panel3.BackColor = Color.Red;
}
```

Uygulamaları gerçek ortamda test edebilirsiniz. Ancak iş bilgisayarda Proteus ve Visual Studio arasında test etmeye gelince, burada ayrı bir sıkıntı başlıyor. Bu da Proteus'taki COMPIM komponentinin bilgisayardaki fiziksel port ismine ihtiyaç duymasıdır. Bu nedenle COMPIM ayarlarının Şekil 13.6'daki gibi yapılması gerekir.

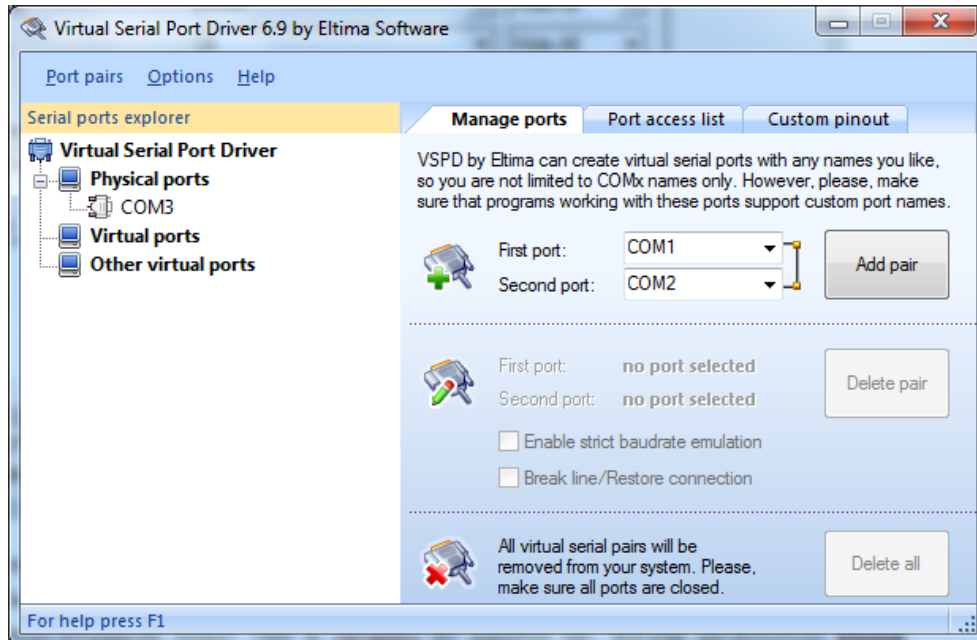
Yine Visual C# yazılımımızda bağlanılacak port ismi olarak COM5 verdik. Peki nasıl olacak da biz sanal olarak bu portları kablo olmadan birbirine bağlayacağız? Yazılımı sadece bilgisayarda test etmek istersek bunu başaramayacak mıyız? Tabi ki herşeyin bir çözümü var. Burada yardımımıza **Eltima Software** firmasının **Virtual Serial Port Driver** programı yetişiyor. (Piyasada buna benzer başka yazılımlar da bulunabilir) 14 günlük bir deneme sürümüyle gelen program açıldığı zaman, Şekil 13.7'deki ekranın solundaki panelde Physical Ports kısmında bilgisayardaki fiziksel seri portları göstermektedir.

AD SOYAD:

SERİ HABERLEŞME



Şekil 13.6 Seri haberleşme uygulamasına ilişkin Proteus'taki COMPIM ayarları

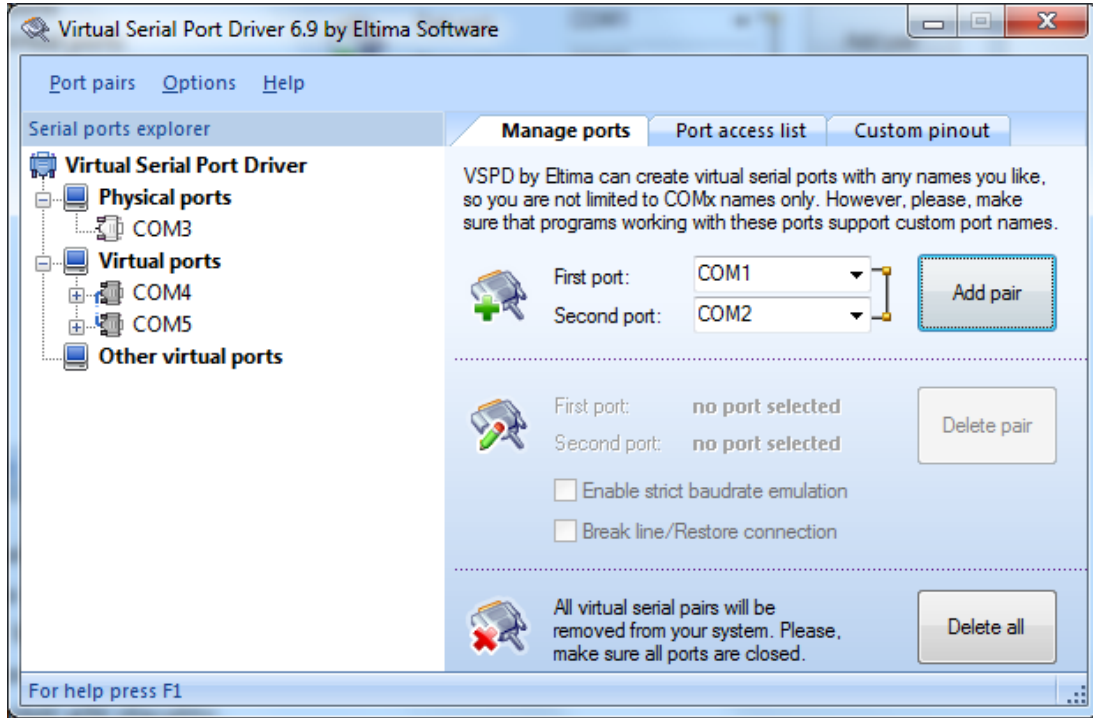


Şekil 13.7 Seri haberleşme uygulamasına ilişkin fiziksel seri port seçim ekranı

AD SOYAD:

SERİ HABERLEŞME

Proteus ile Visual C# yazılımımızı test edebilmemiz için gerçekte birbirine kablo ile bağlanmış gibi iki adet sanal porta ihtiyacımız vardır. Bu sanal portları COM4 ve COM5 olarak seçip Add pair düğmesine tıklıyoruz. Eğer bu isimde portlar zaten bilgisayarınızda fiziksel olarak varsa, farklı isimlerde portlar seçerek sanal port oluşturun ve yazılımlarda bu isimleri düzeltin. Yapılan işlemin ardından program görüntüsü Şekil 13.8'deki gibi olacaktır.

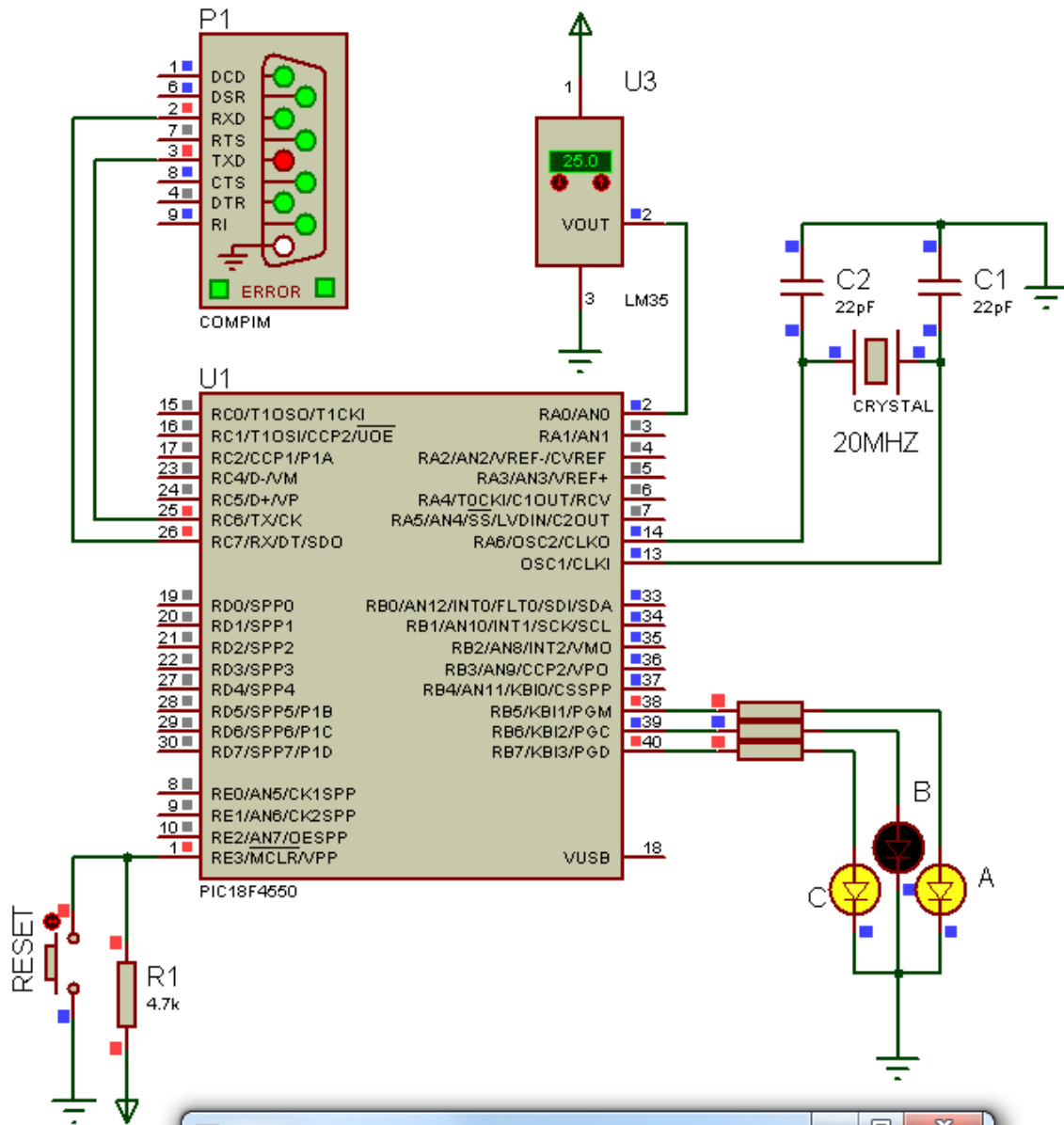


Şekil 13.8 Seri haberleşme uygulamasına ilişkin iki adet sanal port seçim ekranı

Şimdi Proteus ve PIC yazılımımızı çalıştırarak denemeler yapabiliriz. Her iki programın da karşılıklı çalışma şekli Şekil 13.9'da görüldüğü gibidir.

AD SOYAD:

SERİ HABERLEŞME



Form1

Sıcaklık: 25

☐	☐	☐
A - ON	B - ON	C - ON
A - OFF	B - OFF	C - OFF

Şekil 13.9 Seri haberleşme uygulamasına ilişkin ISIS devresinin ledlerle çalıştırılması

BÖLÜM 14

14. USB HABERLEŐME

USB, İngilizce "Universal Serial Bus" kelimesinin kısaltmasıdır. USB'nin Türkçesi "Evrensel Seri Veriyolu"dur. USB dış donanımların bilgisayar ile bağlantı kurabilmesini sağlayan seri yapılı bir bağlantı biçimidir. Son sürümü 3.0'dır. 600 MByte/sn'lik aktarım hızı vardır.

Standart bir usb 2.0 veriyolu 5.00 volt, 500ma çıkış verirken usb 3.0 veriyolu 900ma çıkış değerine sahiptir. Tak Çalıştır (plug and play) özelliğinden dolayı birçok cihazın bağlantısında kullanılmaktadır.

Evrensel seri veriyolu, çevre birimlerinin bilgisayara takıldıkları anda tanınıp otomatik çalışmalarını sağlamaktadır. Yani PnP'dir. Bu yolla 127 tür cihazı çalıştırma imkânı vardır, ek aparatlarla tek porta birden fazla cihaz bağlanabilir.

USB basit bir dört telli bağlantıdır. Veri kodlamasına NRZI (Non-return to Zero Inverted) denir. Modern anakartlarda en az 4 USB portu bulunmaktadır.

14.1. USB HIZLARI

- USB 1.0 ve 1.1 : Hız 12 Mbit/sn (1.5 MByte/sn) (fullspeed)
- USB 2.0 : Hız 480 Mbit/sn (60 MByte/sn) (highspeed)
- USB 3.0 : Hız 4,80 Gbit/sn (600 MByte/sn) (superspeed)[1]
- USB 3.1 : Hız 10 Gbit/sn (1,22 GByte/sn) (superspeed 10 Gbps)

USB 2.0, 480 Mbps bant genişliği sunabilen USB sürümüdür. USB 1.1'de sunulabilen bant genişliği 12 Mbps ile sınırlıdır. USB 2.0, USB 1.1'in 40 katı kadar bant genişliği ile yüksek hız sağlar. Her iki sürümde de kablo yapısı ve bağlantı uçları aynıdır. USB 2.0, USB 1.1 ile uyumludur. Yüksek bağlantı hızı gerektiren harici CD/DVD yazıcı gibi cihazlar USB 2.0 standartını kullanırlar.

USB 3.0 yapılan testler sonucunda en fazla 1320 MBit ile sınırlı kaldığından 5 Gbit hıza ulaşmak zaman alacak gibi görünüyor. Yinede saniyede 1320 MBit hız bile insanı heyecanlandırmaya yetecek bir yazma hızı.

Nitekim aynı şey 2001 yılında olmuştu. USB 2.0 için belirlenen hıza ulaşamamış, en yüksek hız olarak 250 Mbit hıza ulaşılmıştı. Çalışmalar ilerledikçe USB 2.0 gerçekten 480 Mbit hızı gördü ve kullanılmaya başlandı.

Günümüzde ise en son teknoloji USB 3.0'dır. USB 3.0 olup beklenen gibi 5 Gb hız değerine ulaşmıştır. Yeni üretilen anakartların neredeyse hepsinde USB 3.0 desteği vardır. Intel ve AMD'nin yeni yonga setlerinin yaygınlaşmasıyla USB 3.0 gittikçe yüksek hız gerektiren harici HDDler ve Flash Belleklerde kullanılmaya başlanmıştır. Harici HDDlerde büyük çoğunluk USB 3.0 desteğine sahip olmasına rağmen Flash Belleklerde henüz USB 3.0 desteği çok azdır ve pahalıdır.

USB 3.1 31 Temmuz 2013' te duyurulmuştur.

14.2. USB'NİN SAĞLADIĞI AVANTAJLAR:

Kullanım Kolaylığı: Çeşitli cihazlarla kullanım için bir tane arabirim kullanılmaktadır. Böylece donanım her cihaz için ayrı destekler içermemekte, ayrı bir konektör ihtiyacı oluşturmamaktadır. USB cihazlarda kullanıcı ayar yapmaz. Port adresleri, kesme isteği kanalları (IRQ) ile uğraşmaz.

USB'nin en önemli özelliği olan tam Pnp desteği ile bağlanan aygıtlar hiçbir ayar gereksinimi olmadan ve sistemin kapatılmasına gerek kalmadan kullanılabilir. 127 aygıta kadar sunulan birim desteği aynı sistemde birçok farklı ünitenin çoklu kullanımını mümkün kılmaktadır.

Hız: USB 4 hızı destekler. Süper yüksek hız (4.80 Gigabit), Yüksek hız (saniyede 480 Megabit), tam hız (saniyede 12 Megabit) ve düşük hız (saniyede 1,5 Megabit). USB seçeneği olan her bilgisayar düşük ve tam hızı destekler.

Söz konusu hızlar bus tarafından desteklenen bit hızlarıdır. Bir cihaz için beklenen transfer hızı bunlardan daha düşüktür. Veri dışında bus'a statü, kontrol ve hata-kontrol sinyalleri de nakledilir. Bir transferin teorik maksimum hızı yüksek ve tam hızlarda sırasıyla 53 ve 1,2 megabayt/saniye; düşük hızda 800 bayt/saniyedir.

Güvenilirlik: USB'nin güvenilirliği donanım tasarımından ve transfer protokollerinden kaynaklanır. USB düşürücüleri, alıcıları ve kablolarına yönelik donanım spesifikasyonları veri hatalarına yol açan gürültünün büyük kısmını elimine etmektedir. USB protokolü veri hatalarının tespitini mümkün kılmakta ve vericiyi yeniden gönderim yapacağı konusunda bilgilendirmektedir.

Maliyet: Önceki arabirimlerden karmaşık olmakla birlikte, USB'nin devre elemanları pahalı sayılmazlar. USB arabirimli bir cihazın maliyeti en fazla eskilerin maliyeti kadardır. Çok ucuz çevrebirimleri düşünülürse, düşük hız seçeneğine başvurulmak kaydıyla son derece düşük maliyetlerle çalışmak mümkündür.

Güç Sarfiyatı Düşük: Tasarruflu devreler ve kod sayesinde, kullanılmayan USB cihazının gücü kesilir. Ancak bu durumda bile cihazlar gerektiğinde yanıt vermeye hazırdırlar.

14.3. PIC18F4550 USB MODÜLÜ

PIC18F4550, USB host ile PIC cihaz arasında hızlı bir veri iletişimi sağlamak amacıyla, düşük-hız (low speed) ve yüksek-hız (full speed) uyumlu USB Seri Arabirim Motoruna (Serial Interface Engine, SIE) sahiptir. SIE arabirimi, USB veri transferini sağlayan donanım kısmıdır. Aynı zamanda 3,3 V'luk dâhili regülatör, 5 V'luk uygulamalarda pull-up direnci üzerinden fazladan kaynak kullanımını önler.

USB modülünün çalışması, üç kontrol kaydedicisi tarafından yapılandırılır ve yönetilir. Ayrıca, USB işlemleri için toplamda 22 kaydedici kullanılmaktadır. Bu kaydediciler şunlardır:

- USB Kontrol kaydedicisi (UCON)
- USB Konfigürasyon kaydedicisi (UCFG)
- USB Durum kaydedicisi (USTAT)
- USB Cihaz Adres kaydedicisi (UADDR)

- Çerçeve Numarası kaydedicisi (UFRMH:UFRML)
- Son Uç etkinleştirme kaydedicileri 0'dan 15'e kadar (UEPn)

14.3.1. USB DURUM KAYDEDİCİSİ (UCON)

U-0	R/W-0	R-x	R/C-0	R/W-0	R/W-0	R/W-0	U-0
—	PPBRST	SE0	PKTDIS	USBEN	RESUME	SUSPND	—
bit 7							bit 0

Legend:	C = Clearable bit	
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared x = Bit is unknown

bit 7	Unimplemented: Read as '0'
bit 6	PPBRST: Ping-Pong Buffers Reset bit 1 = Reset all Ping-Pong Buffer Pointers to the Even Buffer Descriptor (BD) banks 0 = Ping-Pong Buffer Pointers not being reset
bit 5	SE0: Live Single-Ended Zero Flag bit 1 = Single-ended zero active on the USB bus 0 = No single-ended zero detected
bit 4	PKTDIS: Packet Transfer Disable bit 1 = SIE token and packet processing disabled, automatically set when a SETUP token is received 0 = SIE token and packet processing enabled
bit 3	USBEN: USB Module Enable bit 1 = USB module and supporting circuitry enabled (device attached) 0 = USB module and supporting circuitry disabled (device detached)
bit 2	RESUME: Resume Signaling Enable bit 1 = Resume signaling activated 0 = Resume signaling disabled
bit 1	SUSPND: Suspend USB bit 1 = USB module and supporting circuitry in Power Conserve mode, SIE clock inactive 0 = USB module and supporting circuitry in normal operation, SIE clock clocked at the configured rate
bit 0	Unimplemented: Read as '0'

USB Kontrol kaydedicisi, transfer işlemleri sırasında modülün davranışını kontrol eden bit'leri içerir. Kaydedicinin içerdği konfigürasyon bitleri şunları kontrol eder:

- Ana USB Çevresel Etkinleştirme
- Pinpon Tampon İşaretleyici Sıfırlaması
- Askı (duraklatma) modu kontrolü
- Paket transfer Etkinleştirme

Aynı zamanda USB Kontrol kaydedicisi, veri yolunda oluşabilecek tekli-sonlanmış sıfırı işaret etmek için kullanılan durum biti içerir, SEO (UCON<5>). USB modül etkinleştğinde, farklı veri hatlarından gelen tekli sonlanmış sıfır durumu olup olmadığını belirlemek için bu bit izlenmelidir. Bu bit aynı zamanda USB sıfırlama sinyalinin başlangıç güç durumunu ayırt etmeye yardımcı olur.

USB modülünün tüm işlemleri USBEN bit'i (UCON<3>) tarafından kontrol edilir. Bu bit'in set edilmesi ile modül aktif hale gelir. Bu bit aynı zamanda yonga üzerindeki voltaj regülatörünü aktif hale getirir ve eğer aktif edildiyse, dahili pull-up dirençlerini devreye alır. Kısaca bu bit, USB modülünün sisteme takılıp çıkarılmasını sağlar ve bit değerinin '0' olması durumunda tüm durum ve konfigürasyon bitleri ihmal edilir.

PPBRST biti (UCON<6>), Çiftli-Tamponlama modunda (pinpon modu) çalışılırken sıfırlama durumunu kontrol eder. PPBRST biti set edildiğinde tüm pinpon tampon işaretçileri çift tampon olarak set edilir. PPBRST bitinin yazılımda sıfırlanması gerekir.

PKTDIS biti (UCON<4>) SIE'nin paket iletimi ve alımı için kullanılamaz durumda olduğunu işaret eden bayraktır. Bu bit kurulum işlemine izin verilmesi amacıyla SIE tarafından set edilir, mikrodenetleyici tarafından set edilemez, sadece sıfırlanabilir. Bu bitin sıfırlanması SIE'nin veri gönderimine veya alımına devam etmesine izin verir.

RESUME biti (UCON<2>), USB çevrebirimin devam etme sinyalinin uzaktan kumanda edilerek uyandırma (wake-up) işlemini gerçekleştirebilmesi için kullanılır. Geçerli bir uzaktan uyandırma işlemi için, kodlama RESUME bitini 10ms için set etmeli ve sonra sıfırlamalıdır.

SUSPND biti (UCON<1>), USB modülün ve desteklediği devrenin (örneğin voltaj regülatör devresi) düşük-güç modunda çalışmasını sağlar. Aynı zamanda SIE giriş saat darbesi devre dışı bırakılır. SUSPND biti yazılımla set edilip, mikrodenetleyici kodlaması ile sıfırlanması gerekir. Bu bit aktif edildiğinde, cihaz yola bağlı kalır ancak alıcı/verici çıkışları halen boşta (Idle Modu). VUSB bacağındaki gerilim bu bitin değerine bağlı olarak değişir.

14.3.2. USB KONFIGÜRASYON KAYDEDİCİSİ (UCFG)

R/W-0	R/W-0	U-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
UTEYE	UOEMON ⁽¹⁾	—	UPUEN ^(2,3)	UTRDIS ⁽²⁾	FSEN ⁽²⁾	PPB1	PPB0
bit 7							bit 0

Legend:

R = Readable bit

W = Writable bit

U = Unimplemented bit, read as '0'

-n = Value at POR

'1' = Bit is set

'0' = Bit is cleared

x = Bit is unknown

bit 7	UTEYE: USB Eye Pattern Test Enable bit 1 = Eye pattern test enabled 0 = Eye pattern test disabled
bit 6	UOEMON: USB $\overline{OE}$ Monitor Enable bit ⁽¹⁾ 1 = $\overline{UOE}$ signal active; it indicates intervals during which the D+/D- lines are driving 0 = $\overline{UOE}$ signal inactive
bit 5	Unimplemented: Read as '0'
bit 4	UPUEN: USB On-Chip Pull-up Enable bit ^(2,3) 1 = On-chip pull-up enabled (pull-up on D+ with FSEN = 1 or D- with FSEN = 0) 0 = On-chip pull-up disabled
bit 3	UTRDIS: On-Chip Transceiver Disable bit ⁽²⁾ 1 = On-chip transceiver disabled; digital transceiver interface enabled 0 = On-chip transceiver active
bit 2	FSEN: Full-Speed Enable bit ⁽²⁾ 1 = Full-speed device: controls transceiver edge rates; requires input clock at 48 MHz 0 = Low-speed device: controls transceiver edge rates; requires input clock at 6 MHz
bit 1-0	PPB1:PPB0: Ping-Pong Buffers Configuration bits 11 = Even/Odd ping-pong buffers enabled for Endpoints 1 to 15 10 = Even/Odd ping-pong buffers enabled for all endpoints 01 = Even/Odd ping-pong buffer enabled for OUT Endpoint 0 00 = Even/Odd ping-pong buffers disabled

USB üzerinden haberleşmenin önceliğine göre USB modülünün ilişkili olduğu dâhili ve/veya harici donanımın yapılandırılması gerekir. Yapılandırma işleminin büyük kısmı UCFG kaydedicisi ile gerçekleştirilmektedir. Aynı USB voltaj regülatörü konfigürasyon kaydedicileri tarafından kontrol edilmektedir.

UCFG kaydedicisi, USB modülünün sistem seviye davranışını kontrol eden birçok bit içermektedir. Bu durumlar:

- Yol (bus) hızı (yüksek-hıza karşılık düşük-hız)
- Dahili çekme (pull-up) dirençlerinin etkinleştirilmesi
- Dahili alıcı/vericinin etkinleştirilmesi
- Pinpon tamponunun kullanımı (PPB1:PPB0)

14.3.3. USB DURUM KAYDEDİCİSİ (USTAT)

U-0	R-x	R-x	R-x	R-x	R-x	R-x	U-0
—	ENDP3	ENDP2	ENDP1	ENDP0	DIR	PPBI ⁽¹⁾	—
bit 7							bit 0

Legend:

R = Readable bit

W = Writable bit

U = Unimplemented bit, read as '0'

-n = Value at POR

'1' = Bit is set

'0' = Bit is cleared

x = Bit is unknown

bit 7 **Unimplemented:** Read as '0'bit 6-3 **ENDP3:ENDP0:** Encoded Number of Last Endpoint Activity bits
(represents the number of the BDT updated by the last USB transfer)

1111 = Endpoint 15

1110 = Endpoint 14

....

0001 = Endpoint 1

0000 = Endpoint 0

bit 2 **DIR:** Last BD Direction Indicator bit

1 = The last transaction was an IN token

0 = The last transaction was an OUT or SETUP token

bit 1 **PPBI:** Ping-Pong BD Pointer Indicator bit⁽¹⁾

1 = The last transaction was to the Odd BD bank

0 = The last transaction was to the Even BD bank

bit 0 **Unimplemented:** Read as '0'**Note 1:** This bit is only valid for endpoints with available Even and Odd BD registers.

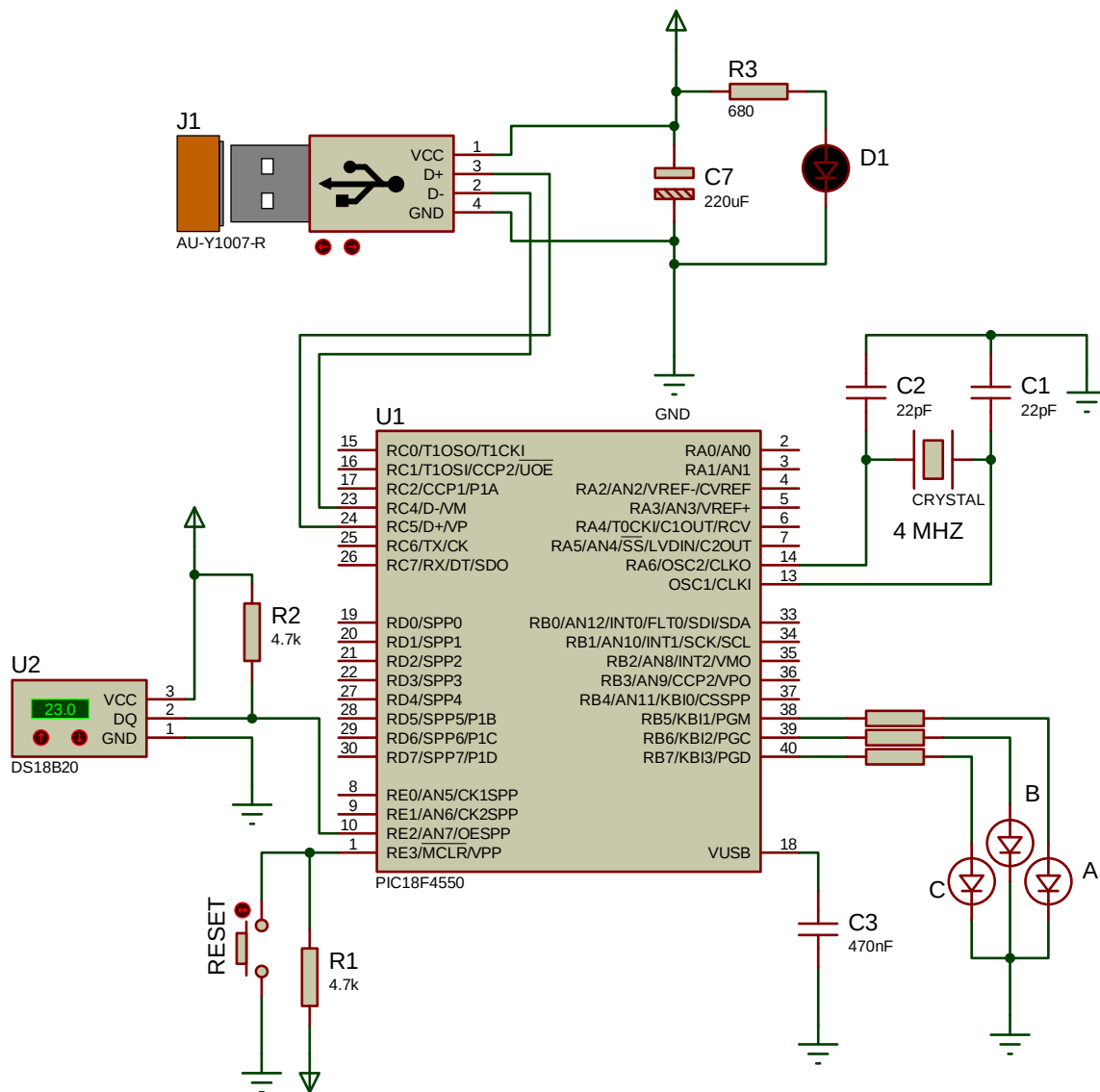
USB durum kaydedicisi SIE içindeki işlem durumunu raporlar. SIE, USB transferinin tamamlandığına dair bir kesme bildirirse, transferin durumunu belirlemek için USTAT kaydedicisinin okunması gerekir. USTAT, transfer son nokta sayısını, yönünü ve pinpon tampon işaretçi değerini içerir (eğer kullanıldıysa).

USTAT kaydedicisi aslında, SIE tarafından sağlanan dört-byte lık FIFO durumuna bir penceredir. O, SIE ek uç noktaları işlerken, mikrodenetleyicinin bir transferi işlemesine izin verir. SIE veri okuma ve yazma için tamponu kullanmayı tamamladığında, USTAT kaydedicisi güncellenir. Servis edilen kesmeye transfer işlemi tamamlanmadan önce başka bir USB transferi yapılırsa, SIE diğer transferin durumunu FIFO durumu içinde saklayacaktır. Transfer tamamlanma bayrak bitinin (TRNIF) temizlenmesi, SIE'nin FIFO yu ilerletmesine neden olur. Kaydı tutan FIFO içindeki veri geçerli ise, SIE TRNIF bitini temizleyerek 5 TCY süresi içinde kesmeyi yeniden oluşturacaktır. Hiçbir ek veri yoksa TRNIF biti temiz kalacak, USTAT verileri güvenilir olmayacaktır.

USB UYGULAMASI

USB arabirimi kendi içerisinde veri iletim yöntemleri olarak çok karmaşık bir yapıya sahipken, MikroC ve Visual C# yazılımları bize bu konuda yardımcı olmaktadır. USB cihazımızın kodları MikroC sayesinde HID (Human Interface Device) sınıfında oluşturulmaktadır. HID sınıfı Windows işletim sistemi tarafından desteklenen bir USB sınıfı olması sebebiyle, işletim sistemi içerisinde ön tanımlı bir sınıf olduğu için herhangi bir ek sürücüyü (driver) ihtiyaç duymaksızın otomatik olarak cihazımız tanınacaktır.

USB devremiz Proteus'ta Şekil 14.1'deki gibi tasarlanmıştır.



Şekil 14.1 USB uygulamasına ilişkin ISIS devre şeması

MikroC’de USBHaberlesme isminde yeni bir proje oluşturarak, saat frekansını 4 MHz olarak ayarlıyoruz.

MikroC programında Tools menüsünden HID Terminal aracını çalıştırıyoruz. Açılan araçta, üst sekmeden Descriptor kısmına geliyoruz. Bu kısımda, cihazımıza ait tanımlamaları yapacağız ve MikroC bize, kendi projemizde kullanabileceğimiz, USB ile ilgili kesme ve alt programların olduğu USBdsc.c isimli bir dosya oluşturacaktır.

mikroElektronika USB (HID) Terminal

Terminal Descriptor

VID and PID
VID: 05F4
PID: 3689

Report Length
Input: 64
Output: 64

Bus power
Bus powered: ☒
50 x2 mA

Endpoints pooling int.
Input: 1 mSec.
Output: 1 mSec.

Strings
Vendor Name: MeslekiAkademi
Product Name: USB HID Uygulama Karti

☒ mikroC ☐ mikroPascal ☐ mikroBasic

Save descriptor

Şekil 14.2 USB uygulamasına ilişkin HID Terminal aracı ekranı

Ayarlar, Şekil 14.2’de görüldüğü gibi yapılır. Burada unutulmaması gereken, cihazın doğru tanınmasını sağlayacak en önemli iki kısım Vendor ID (VID) ile Product ID (PID) kısımlarıdır. Buraya yazacağımız her iki numara da C# programında kullanılacaktır. Altteki bölümden de MikroC seçili haldeyken Save Descriptor düğmesine tıklıyoruz. Program bize

USBdsc.c dosyasını nereye kaydedeceğimiz sorduğunda bu dosyayı proje klasörü içerisinde kaydediyoruz.

Şimdi bu dosyayı projemize dahil etmemiz gerekiyor. Bunu da aşağıdaki gibi kod yazarak kütüphanenin projemize tanıtılmasını sağlıyoruz.

```
#include "USBdsc.c"
```

Daha sonra USB haberleşmesi için gerekli olan kesme alt programları da tanımlanmıştır.

```
unsigned char readbuff[64] absolute 0x500;  
unsigned char writebuff[64] absolute 0x540;  
  
void interrupt()  
{  
    USB_Interrupt_Proc();  
}
```

USB üzerinden veri yazma ve okuma işlemleri paketler halinde yapılmaktadır. MikroC'deki USB HID Tool aracı ile de bu paketlerin 64'er byte uzunlukta olacağını belirtmiş ve dosyamızı buna göre oluşturmuştuk. Bu nedenle MikroC'de okuma için (readbuffer) ve yazma için (writebuffer) kullanılacak tampon değişkenleri 64 byte uzunluğunda tanımlamış olduk.

Programın ana kısmında yazmış olduğumuz aşağıdaki kod, USB'nin devreye girmesini sağlar.

```
HID_Enable(&readbuff, &writebuff);
```

USB üzerinden veri gönderirken writebuffer değişkeninin içi istediğimiz verilerle doldurulduktan sonra bu tampon bellekteki veriler aşağıdaki gibi kullanacağımız kod ile USB üzerinden gönderilir.

```
while(!HID_Write(&writebuff,64));
```

MikroC projemizde, DS18B20 sensörü kullanılmıştır. Bu sensör, sıcaklık ölçümünde kullanılmakta olup, One-Wire diye tabir edilen tek telden iletişim yoluyla veri

AD SOYAD:

USB HABERLEŞME

göndermektedir. Bu projemizde, ölçülen sıcaklık, USB üzerinden Visual C# ta yazılacak programa gönderilecek ve Visual C#'ta yazılmış program aracılığıyla da mikrodenetleyiciye bağlı 3 adet LED'in yanıp sönmesi sağlanacaktır.

DS18B20, 12 bitlik data şeklinde bilgi üretir. Üretilen değerlere göre örnek bir tablo aşağıda verilmiştir.

TEMPERATURE (°C)	DIGITAL OUTPUT (BINARY)	DIGITAL OUTPUT (HEX)
+125	0000 0111 1101 0000	07D0h
+85*	0000 0101 0101 0000	0550h
+25.0625	0000 0001 1001 0001	0191h
+10.125	0000 0000 1010 0010	00A2h
+0.5	0000 0000 0000 1000	0008h
0	0000 0000 0000 0000	0000h
-0.5	1111 1111 1111 1000	FFF8h
-10.125	1111 1111 0101 1110	FF5Eh
-25.0625	1111 1110 0110 1111	FE6Fh
-55	1111 1100 1001 0000	FC90h

Tablodan görülebileceği gibi, negatif değerlerde en yüksek değerlikli 4 adet biti 1 değerini almaktadır. Diğer 12 bitlik bilgi sıcaklık bilgisini içerir. Bu 12 bitin son 4 biti, ondalıklı bilgiyi içerir.

Daha ayrıntılı açıklamak gerekirse;

b15	b14	b13	b12	b11	b10	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0
← Okunan sıcaklığın tamsayı kısmı →												← Okunan sıcaklığın ondalık kısmı →			
Temperature weight of each bit (°C):															
Sign bits			64	32	16	8	4	2	1	0.5	0.25	0.125	0.0625		

Yukarıdaki tablo baz alındığında, 0000 0001 1001 0001 değerine karşılık gelen sıcaklık değeri; $16 + 8 + 1 + 0.0625 = 25.0625$ °C. Olur.

Sıcaklık verisini DS18B20 sensöründen okuyan alt program DS18B20_Oku isimli alt program olup, buradan alınan bilgi Sicaklik isimli bir değişkene atanmıştır. Daha sonra

```
void Sicaklik_Donustur(unsigned int temp2write)
```

alt programı ile sıcaklık değeri yukarıdaki tablolar baz alınarak anlaşılır bir sıcaklık değerine dönüştürülmüştür.

MikroC'de yazılan programın kodları (tamamı) aşağıdaki gibidir:

```
#include "USBdsc.c"

unsigned char readbuff[64] absolute 0x500;
unsigned char writebuff[64] absolute 0x540;
int i;

// TEMP_RESOLUTION DS18x20 sensorün bit sayısıdır:
// 18S20: 9
// 18B20: 12
const unsigned short TEMP_RESOLUTION = 12;
char Sicaklik_String[12];

unsigned Sicaklik;

void DS18B20_Oku()
{
    // Isı DS18B20 sensöründen okunuyor
    Ow_Reset(&PORTE, 2);
    Ow_Write(&PORTE, 2, 0xCC);
    Ow_Write(&PORTE, 2, 0x44);
    Delay_us(120);

    Ow_Reset(&PORTE, 2);
    Ow_Write(&PORTE, 2, 0xCC);
    Ow_Write(&PORTE, 2, 0xBE);

    Sicaklik = Ow_Read(&PORTE, 2);
    Sicaklik = (Ow_Read(&PORTE, 2) << 8) + Sicaklik;
}
```

```

void Sicaklik_Donustur(unsigned int temp2write) {
    const unsigned short RES_SHIFT = TEMP_RESOLUTION - 8;
    char temp_whole;
    unsigned int temp_fraction;

    // sıcaklık negatif mi?
    if (temp2write & 0x8000)
    {
        Sicaklik_String[0] = '-';
        temp2write = ~temp2write + 1;
    }
    else
        Sicaklik_String[0] = ' ';

    // tam değeri elde et
    temp_whole = temp2write >> RES_SHIFT ;

    // tam değeri karaktere dönüştür
    if (temp_whole/100)
        Sicaklik_String[1] = temp_whole/100 + 48;
    else
        Sicaklik_String[1] = '0';

    Sicaklik_String[2] = (temp_whole/10)%10 + 48;           // Extract tens digit
    Sicaklik_String[3] = temp_whole%10 + 48;               // Extract ones digit

    // kesirli değeri elde ederek onu tamsayıya dönüştür
    temp_fraction = temp2write << (4-RES_SHIFT);
    temp_fraction &= 0x000F;
    temp_fraction *= 625;

    Sicaklik_String[4] = '.';

    // kesirli kısmı karaktere dönüştür
    Sicaklik_String[5] = temp_fraction/1000 + 48;          // Extract thousands di-
git
    Sicaklik_String[6] = (temp_fraction/100)%10 + 48;      // Extract hundreds digit
    Sicaklik_String[7] = (temp_fraction/10)%10 + 48;       // Extract tens digit
    Sicaklik_String[8] = temp_fraction%10 + 48;            // Extract ones digit
}

```

```
void interrupt()
{
    USB_Interrupt_Proc();
}

void USB_Gonder()
{
    int __j=0;
    int __k=0;

    writebuff[__j++] = '*';
    writebuff[__j++] = 'M';
    writebuff[__j++] = 'A';
    writebuff[__j++] = '@';

    __k=0;
    while (Sicaklik_String[__k] != 0)
    {
        writebuff[__j++] = Sicaklik_String[__k++];
    }

    writebuff[__j++] = '@';

    // bu kısımdan itibaren
    //başka veriler de gönderilebilir

    // her gönderilen veriyi belirli bir karakterle ayırmak gerekir
    //biz burada @ karakteri ile ayırdık

    writebuff[__j++] = 0;
    writebuff[__j++] = 10;
    writebuff[__j++] = 13;

    while(!HID_Write(&writebuff,64));
}
```

```

void main() {

    ADCON0 = 0;
    ADCON1 |= 0x0F;
    ADCON2 = 0;
    CMCON  |= 7;

    TRISE = 0xff;
    LATE = 0;
    PORTE = 0;

    TRISB = 0;          //B portundaki tüm pinleri çıkış yap.
    LATB = 0;          //B portundaki bütün LED'leri söndür

    HID_Enable(&readbuff, &writebuff);

    while(1)
    {
        while(!HID_Read())
        {
            DS18B20_Oku();
            //okunan sıcaklık verisini anlamlı hale getir.
            Sicaklik_Donustur(Sicaklik);
            USB_Gonder();
            delay_ms(200);
        }

        //şimdi gelen bilgiyi incele
        switch (readbuff[0])
        {
            case 'A':
                if (readbuff[1] == '1')
                    portb.f5 = 1; // 1. ledi yak
                else if (readbuff[1] == '0')
                    portb.f5 = 0; // 1. ledi söndür
                break;

            case 'B':
                if (readbuff[1] == '1')
                    portb.f6 = 1; // 2. ledi yak
                else if (readbuff[1] == '0')
                    portb.f6 = 0; // 2. ledi söndür
                break;

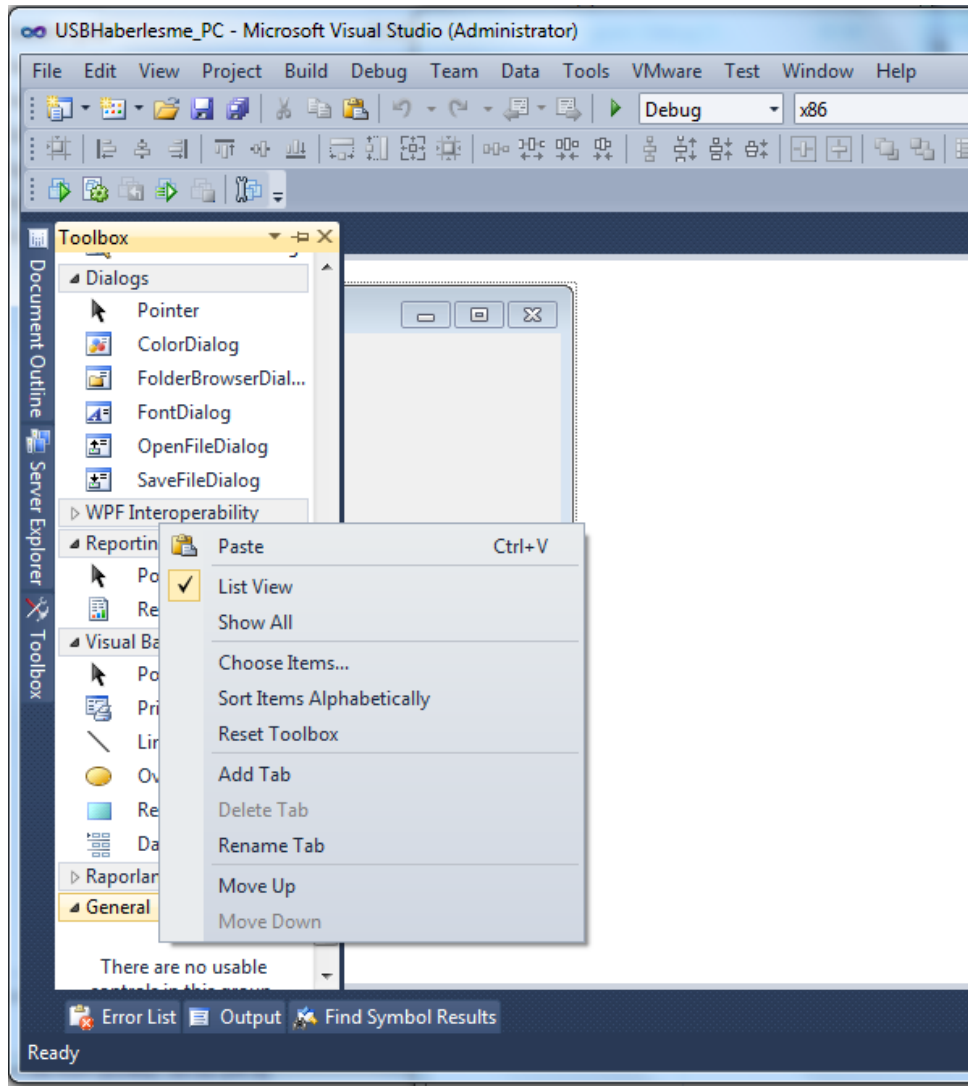
            case 'C':
                if (readbuff[1] == '1')
                    portb.f7 = 1; // 3. ledi yak
                else if (readbuff[1] == '0')
                    portb.f7 = 0; // 3. ledi söndür
                break;
        }

        if(readbuff[0] != 0)
        {
            for(i=0;i<64;i++)
                readbuff[i] = 0;
        }
    }
}

```

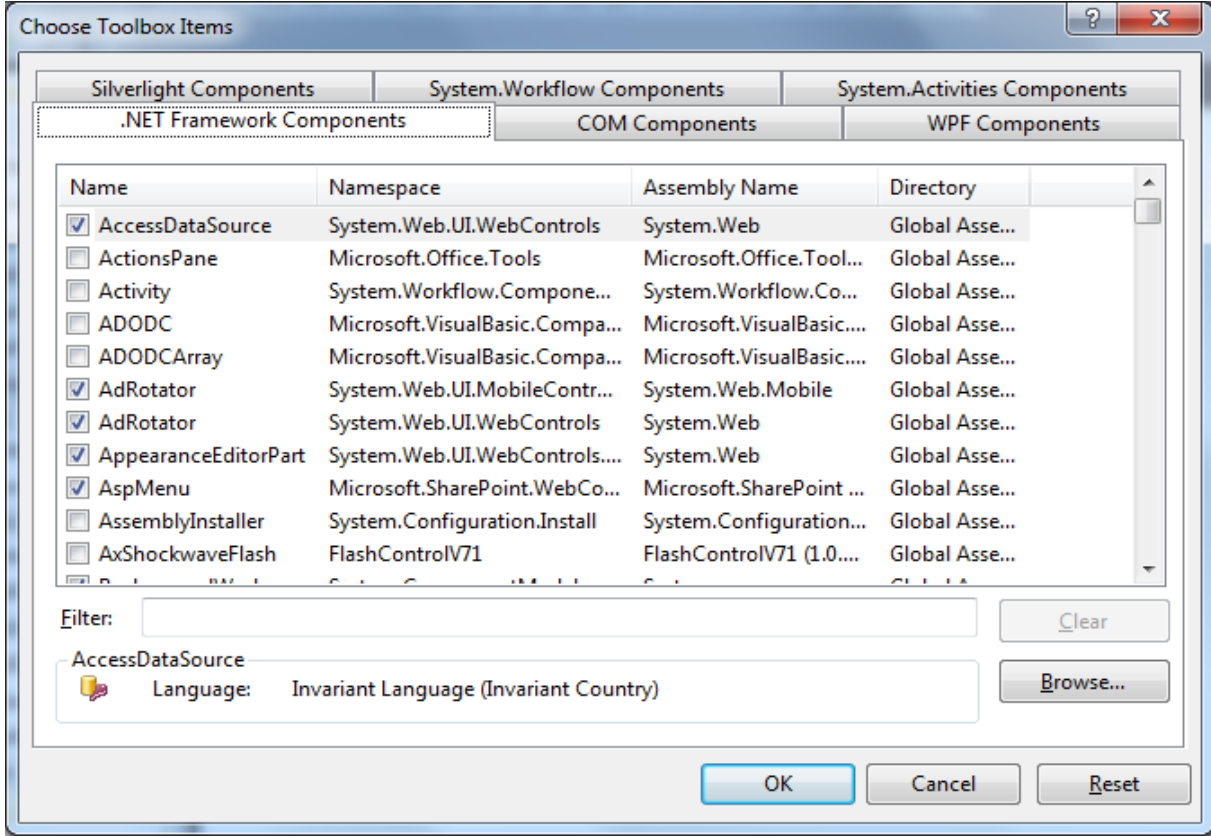
Visual C#'teki yazılım için ücretsiz bir USB komponentinin bilgisayarınıza indirilmesi gerekmektedir.

<http://www.codeproject.com/Articles/18099/A-USB-HID-Component-for-C> adresine girerek buradaki dosyayı indirilmelidir. İndirilen dosya Winzip ya da Winrar yardımıyla sıkıştırılmış dosyayı çıkartılmalıdır. Visual Studio'da yeni bir Windows Forms Application uygulaması açarak bu USB komponentini Şekil 14.3'deki gibi dahil edilmelidir.



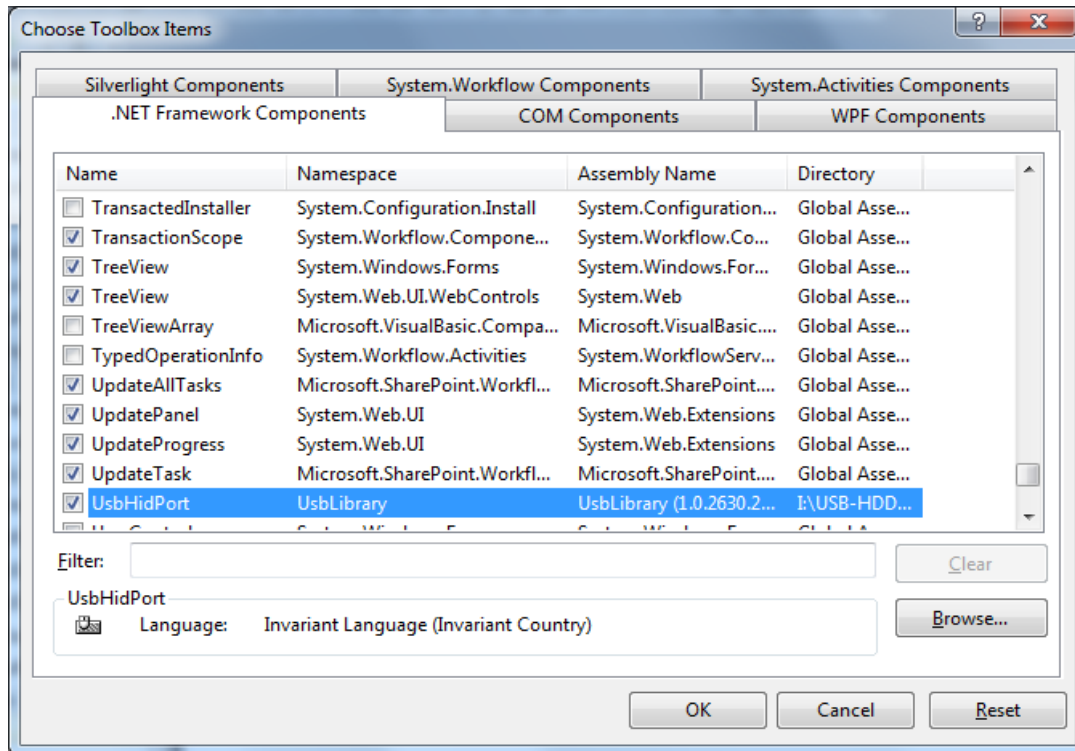
Şekil 14.3 Windows Forms Application uygulaması açarak USB komponentini eklemek

Şekil 14.3'deki çıkan ekranda görüldüğü gibi Toolbox'ta boş bir alana sağ tıklayarak Choose Items... düğmesine tıklayınca Şekil 14.4'deki ekran karşımıza gelmektedir.



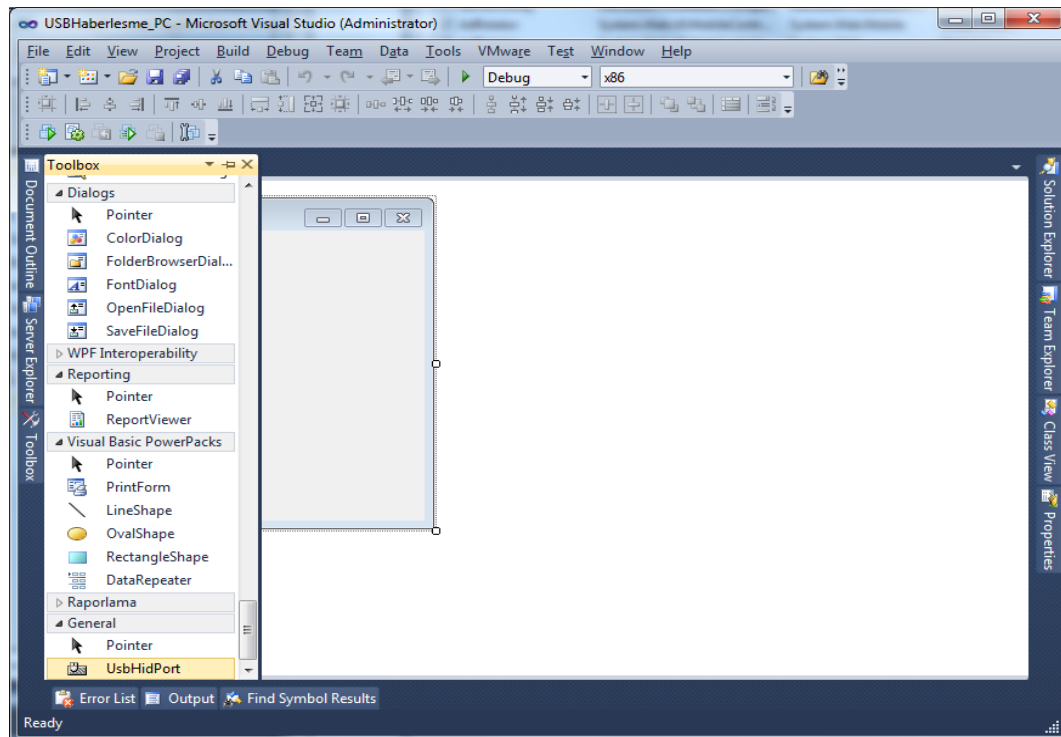
Şekil 14.4 Windows Forms Application uygulaması açarak USB komponentini eklemek

Şekil 14.4'deki ekrandan .Net Framework components kısmından Browse düğmesi yardımıyla USBLibrary.dll dosyasını Şekil 14.5'deki gibi gösteriyoruz.



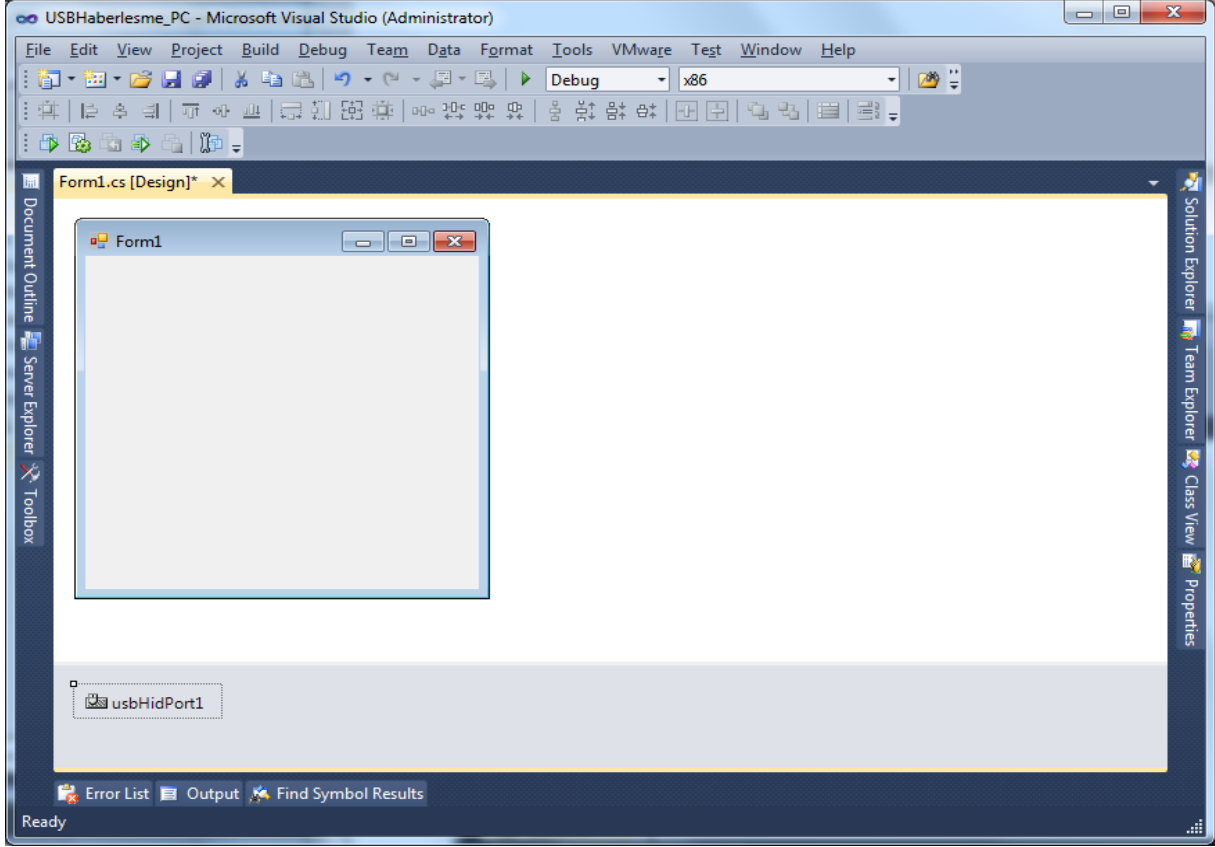
Şekil 14.5 Windows Forms Application uygulaması açarak USB komponentini eklemek

Şekil 14.5'deki ekranda OK düğmesine tıkladıktan sonra Şekil 14.6'da görüldüğü gibi UsbHidPort komponenti ekranımıza gelir.



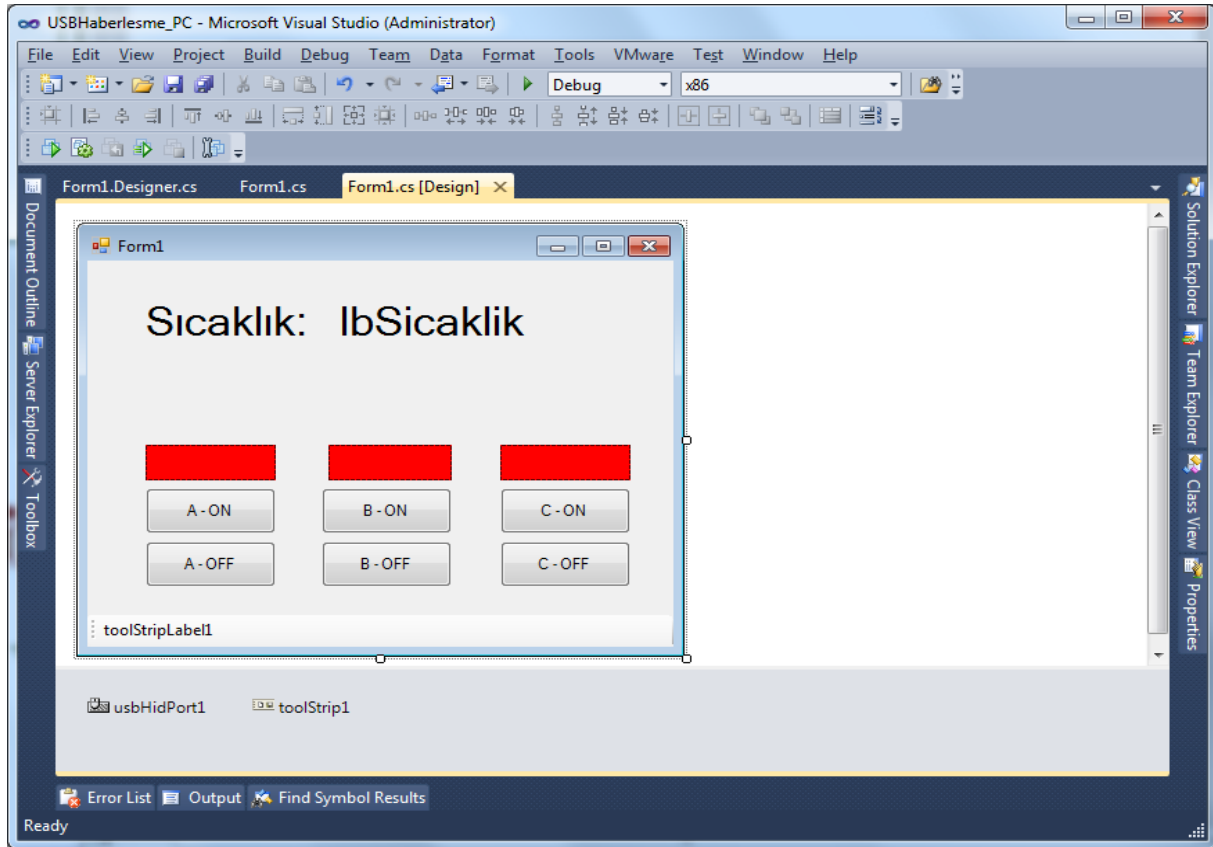
Şekil 14.6 Windows Forms Application uygulaması açarak USB komponentini eklemek

Şekil 14.6'daki gibi komponenti formun üzerine sürükleyerek bıraktığımızda Şekil 14.7'de görüldüğü gibi USB komponenti eklemiş oluruz.



Şekil 14.7 Windows Forms Application uygulaması açarak USB komponentini eklemek

Formun tasarımını ekrandaki gibi yapıyoruz. İki adet label, 3 adet panel (LED'leri temsil eden) 6 adet buton ve 1 adet ToolStrip kontrolünü form üzerine yerleştirip Şekil 14.8'deki gibi etiketlendiriyoruz.



Şekil 14.8 Windows Forms Application uygulaması

Formun kod kısmını açarak aşağıdaki kütüphaneyi de dahil ediyoruz.

```
using UsbLibrary;
```

Daha sonra USB veri akış işlemlerini rahatlıkla yapabilmemiz için aşağıdaki OnHandleCreated ve WndProc isimli iki alt programı da public partial class Form1 bölümün hemen altına köşeli parantezden sonra yazıyoruz. VendorID ve ProductID değişkenlerini de tanımlayıp MikroC’de verdiğimiz değerleri atıyoruz.

USBHidPort1 komponenti üzerindeyken Proeperties penceresini açtığınızda Events (Olaylar) (yıldırım işareti gibi olan düğme) düğmesine tıkladığınızda açılacak olan kısımda OnDataReceived, OnDeviceArrived, OnDeviceRemoved, OnSpecifiedDeviceArrived, OnSpecifiedDeviceRemoved kısımlarının her birine ikişer kez tıklayarak her birine ait olayların kod penceresinde oluşmasını sağlayınız.

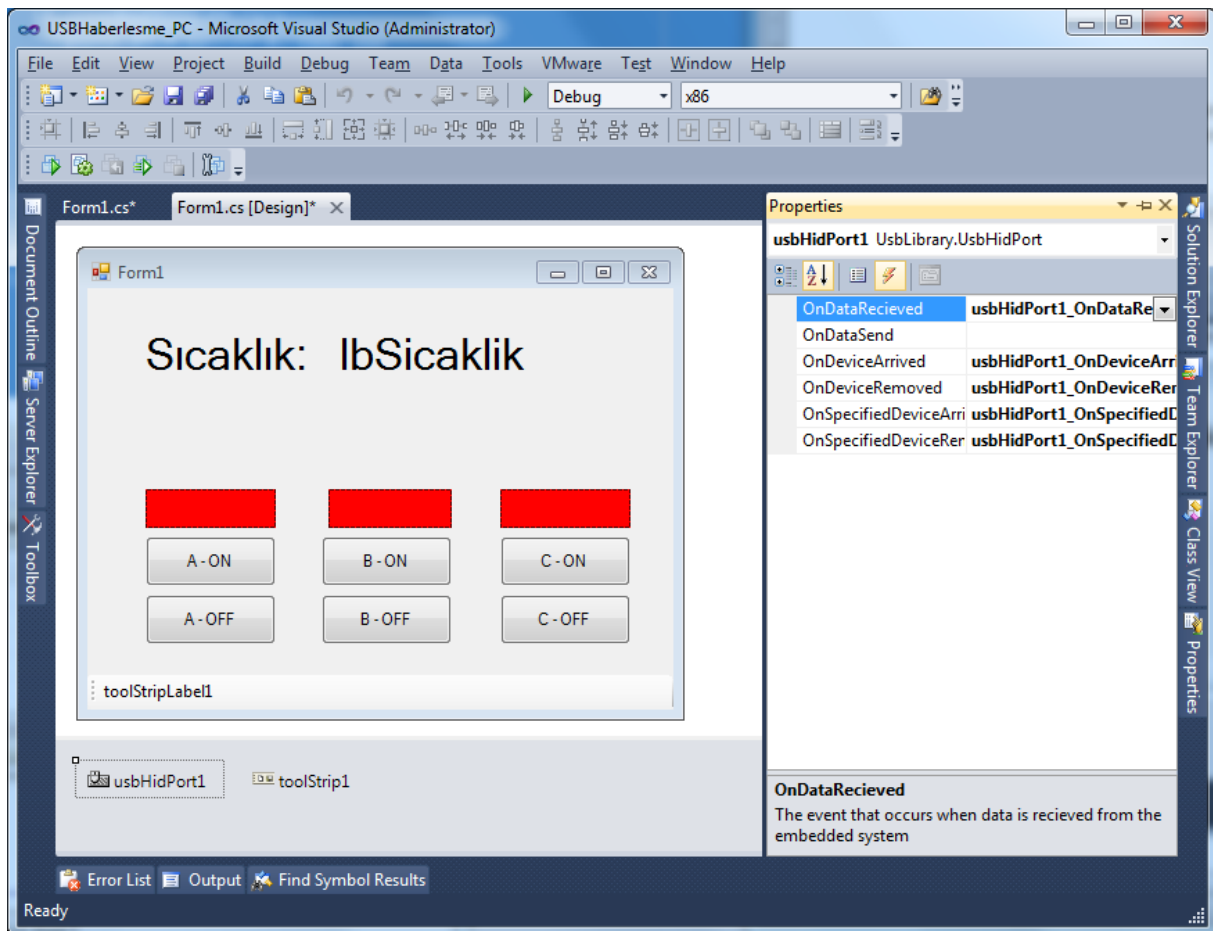
```

public partial class Form1 : Form
{
    int VendorID = 0x05F4;
    int ProductID = 0x3689;

    protected override void OnHandleCreated(EventArgs e)
    {
        base.OnHandleCreated(e);
        usbHidPort1.RegisterHandle(Handle);
    }

    protected override void WndProc(ref Message m)
    {
        usbHidPort1.ParseMessages(ref m);
        base.WndProc(ref m);
    }
}

```



Şekil 14.9 Windows Forms Application uygulaması

Buradaki olaylar kısaca aşağıdaki işlemleri yapar.

- **OnDataReceived:** USB üzerinden herhangi bir veri geldiğinde yapılacak işlemleri
- **OnDeviceArrived:** Bilgisayara herhangi bir USB aygıtı bağlandığında yapılacak işlemleri
- **OnDeviceRemoved:** Bilgisayardan herhangi bir USB aygıtının bağlantısı kesildiğinde yapılacak işlemleri
- **OnSpecifiedDeviceArrived:** Bilgisayara bizim belirlediğimiz VendorID ve ProductID ye sahip bir USB aygıtı bağlandığında yapılacak işlemleri
- **OnSpecifiedDeviceRemoved:** Bilgisayardan bizim belirlediğimiz VendorID ve ProductID ye sahip bir USB aygıtı bağlantısı kesildiğinde yapılacak işlemleri

Visual C# programına ait yazılımın tüm kodları aşağıdaki gibidir:

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using UsbLibrary;

namespace USBHaberlesme_PC
{
    public partial class Form1 : Form
    {
        int VendorID = 0x05F4;
        int ProductID = 0x3689;

        protected override void OnHandleCreated(EventArgs e)
        {
            base.OnHandleCreated(e);
            usbHidPort1.RegisterHandle(Handle);
        }

        protected override void WndProc(ref Message m)
        {
            usbHidPort1.ParseMessages(ref m);
            base.WndProc(ref m);
        }

        public Form1()
        {
            InitializeComponent();
        }

        private void Form1_Load(object sender, EventArgs e)
        {
            toolStripLabel1.Text = "USB Cihaz bağlı değil...";

            usbHidPort1.VendorId = VendorID;
            usbHidPort1.ProductId = ProductID;
            usbHidPort1.CheckDevicePresent();
        }

        private void usbHidPort1_OnSpecifiedDeviceArrived(object sender, EventArgs e)
        {
            toolStripLabel1.Text = " Mesleki Akademi USB HID Cihazı bağlandı!";
        }

        private void usbHidPort1_OnSpecifiedDeviceRemoved(object sender, EventArgs e)
        {
            toolStripLabel1.Text = " Mesleki Akademi USB HID Cihazı çıkarıldı!";
        }
    }
}
```

```

private void usbHidPort1_OnDeviceArrived(object sender, EventArgs e)
{
    usbHidPort1.CheckDevicePresent();
}

private void usbHidPort1_OnDeviceRemoved(object sender, EventArgs e)
{
    usbHidPort1.CheckDevicePresent();
}

private void usbHidPort1_OnDataRecieved(object sender, UsbLib-
rary.DataRecievedEventArgs args)
{
    if (InvokeRequired)
    {
        try
        {
            Invoke(new DataRecievedEventHandler(usbHidPort1_OnDataRecieved),
new[] { sender, args });
        }
        catch (Exception ex)
        {
            //Console.WriteLine(ex.ToString());
            MessageBox.Show(ex.ToString());
        }
    }
    else
    {
        var usb_gelen = new
char[usbHidPort1.SpecifiedDevice.OutputReportLength + 1];
        byte i = 0;
        string str;
        foreach (char myData in args.data)
        {
            usb_gelen[i++] = myData;
        }

        str = new string(usb_gelen);

        int basl = str.IndexOf("MA@");

        if (basl >= 0) //doğru veri geliyse.....
        {
            string[] Dizi;
            Dizi = str.Split('@');

            lbSicaklik.Text = Dizi[1] + (char)176 + "C";
        }
    }
}

```

```
private void button1_Click(object sender, EventArgs e)
{
    if (usbHidPort1.SpecifiedDevice != null)
    {
        byte[] dizi = new byte[usbHidPort1.SpecifiedDevice.OutputReportLength + 1];
        dizi[0] = 0;
        dizi[1] = Convert.ToByte('A');
        dizi[2] = Convert.ToByte('1');
        for (int i = 3; i < 64; i++)
            dizi[i] = 0xFF;

        usbHidPort1.SpecifiedDevice.SendData(dizi);
        panel1.BackColor = Color.Green;
    }
}

private void button3_Click(object sender, EventArgs e)
{
    if (usbHidPort1.SpecifiedDevice != null)
    {
        byte[] dizi = new byte[usbHidPort1.SpecifiedDevice.OutputReportLength + 1];
        dizi[0] = 0;
        dizi[1] = Convert.ToByte('B');
        dizi[2] = Convert.ToByte('1');
        for (int i = 3; i < 64; i++)
            dizi[i] = 0xFF;

        usbHidPort1.SpecifiedDevice.SendData(dizi);
        panel2.BackColor = Color.Green;
    }
}

private void button5_Click(object sender, EventArgs e)
{
    if (usbHidPort1.SpecifiedDevice != null)
    {
        byte[] dizi = new byte[usbHidPort1.SpecifiedDevice.OutputReportLength + 1];
        dizi[0] = 0;
        dizi[1] = Convert.ToByte('C');
        dizi[2] = Convert.ToByte('1');
        for (int i = 3; i < 64; i++)
            dizi[i] = 0xFF;

        usbHidPort1.SpecifiedDevice.SendData(dizi);
        panel3.BackColor = Color.Green;
    }
}
```



```
private void button2_Click(object sender, EventArgs e)
{
    if (usbHidPort1.SpecifiedDevice != null)
    {
        byte[] dizi = new byte[usbHidPort1.SpecifiedDevice.OutputReportLength + 1];
        dizi[0] = 0;
        dizi[1] = Convert.ToByte('A');
        dizi[2] = Convert.ToByte('0');
        for (int i = 3; i < 64; i++)
            dizi[i] = 0xFF;

        usbHidPort1.SpecifiedDevice.SendData(dizi);
        panel1.BackColor = Color.Red;
    }
}

private void button4_Click(object sender, EventArgs e)
{
    if (usbHidPort1.SpecifiedDevice != null)
    {
        byte[] dizi = new byte[usbHidPort1.SpecifiedDevice.OutputReportLength + 1];
        dizi[0] = 0;
        dizi[1] = Convert.ToByte('B');
        dizi[2] = Convert.ToByte('0');
        for (int i = 3; i < 64; i++)
            dizi[i] = 0xFF;

        usbHidPort1.SpecifiedDevice.SendData(dizi);
        panel2.BackColor = Color.Red;
    }
}

private void button6_Click(object sender, EventArgs e)
{
    if (usbHidPort1.SpecifiedDevice != null)
    {
        byte[] dizi = new byte[usbHidPort1.SpecifiedDevice.OutputReportLength + 1];
        dizi[0] = 0;
        dizi[1] = Convert.ToByte('C');
        dizi[2] = Convert.ToByte('0');
        for (int i = 3; i < 64; i++)
            dizi[i] = 0xFF;

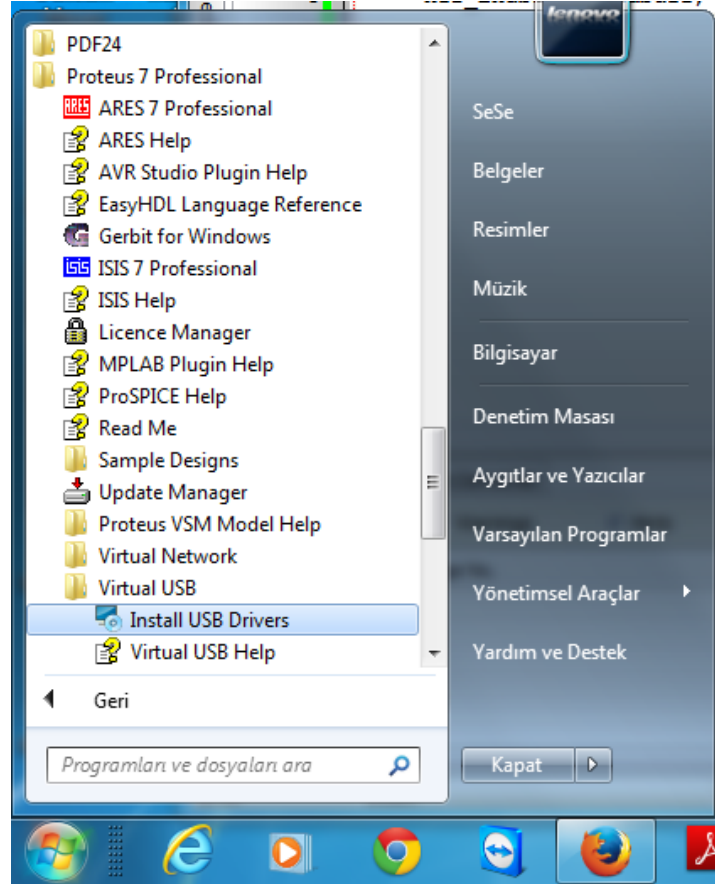
        usbHidPort1.SpecifiedDevice.SendData(dizi);
        panel3.BackColor = Color.Red;
    }
}

}

}
```

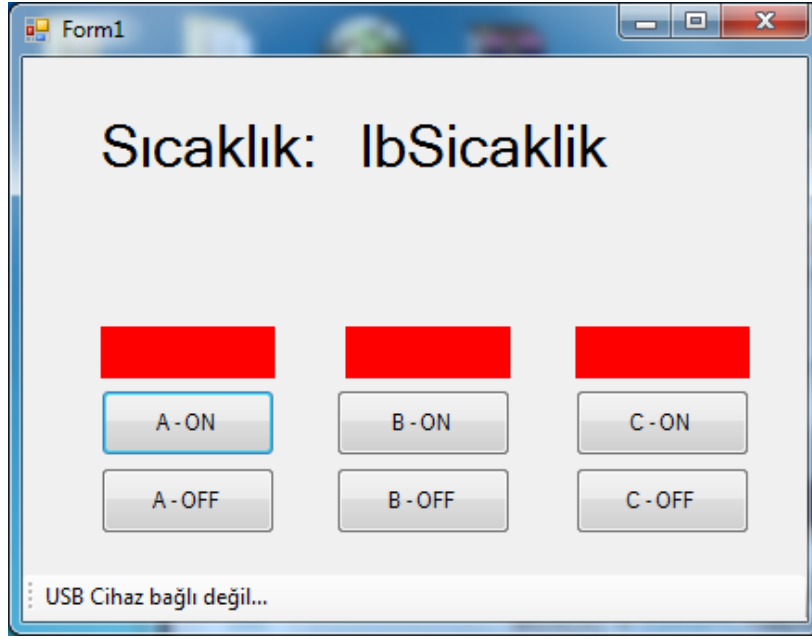
Projemizi Proteus ve Visual C# taraflı test edebilmemiz için öncelikle Proteus'la birlikte gelen sanal USB sürücülerinin yüklenmesi gerekir.

USB sürücülerini yüklemek için Şekil 14.10'da görüldüğü gibi Başlat menüsünden Programlar → Proteus 7 (veya 8) → Virtual USB → Install USB Drivers yolunu izleyerek sürücülerin yüklenmesini sağlıyoruz.



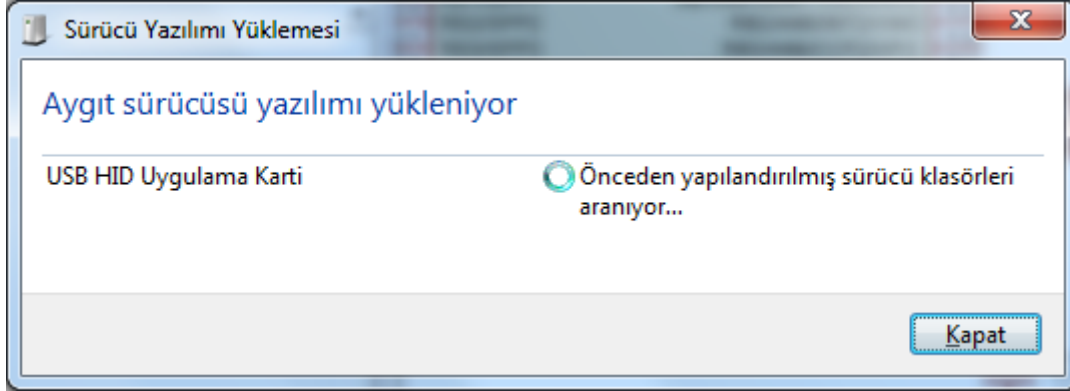
Şekil 14.10 USB sürücüsünün yüklenmesi

Visual C#'ta uygulamamızı çalıştırdığımızda USB aygıtımızın bağlı olmadığını görürüz.



Şekil 14.11 Windows Forms Application uygulaması

Proteus'ta projemizi ilk kez çalıştırdığımızda öncelikle USB HID sürücümüz Şekil 14.12'de görüldüğü gibi bilgisayarımızda tanımlanır.

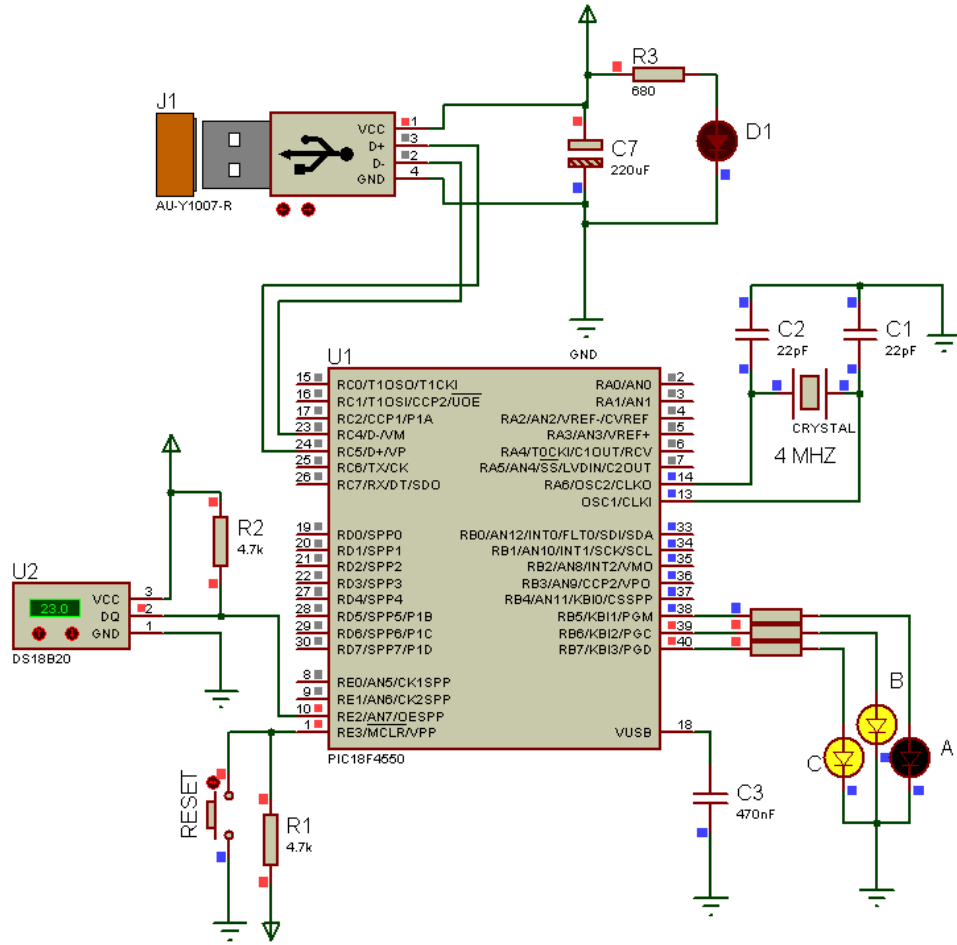


Şekil 14.12 USB HID sürücüsünün tanımlanması

Daha sonra C# uygulamamızda alt kısımda USB cihazımızın bağlandığı görülür ve sıcaklık bilgisi de gelmiştir. Projemizi butonlara tıklayarak test edebiliriz.

AD SOYAD:

USB HABERLEŞME



Şekil 14.13 USB haberleşmesi Proteus uygulaması

The screenshot shows a Windows Forms application window titled "Form1". The main display area shows "Sıcaklık: 023.0000°C". Below this, there are three columns of buttons, each with a colored header (red, green, and blue respectively). The first column has "A - ON" and "A - OFF" buttons. The second column has "B - ON" and "B - OFF" buttons. The third column has "C - ON" and "C - OFF" buttons. At the bottom of the window, a status bar displays the message "Mesleki Akademi USB HID Cihazı bağlandı!".

Şekil 14.14 Windows Forms Application uygulaması

KAYNAKLAR

1. PIC18F2455/2550/4455/4550 Data Sheet, DS39632D, Microchip Technology Inc., 2007
2. Milan VERLE, *PIC Microcontrollers - Programming In C*, Mikroelektronika, 2009
3. <http://www.mikroe.com/products/view/285/book-pic-microcontrollers-programming-in-c/> (Eriřim tarihi: 01.09.2014)
4. <http://www.libstock.com/> (Eriřim tarihi: 01.09.2014)
5. MikroElektronika, <http://www.mikroe.com/> (Eriřim tarihi: 01.09.2014)
6. MikroC Pro for PIC Help, Help version: 2014/06/04
7. Hikmet řAHİN, K. Serkan DEDEOđLU, *Yeni Bařlayanlar İin MikroC ile PIC Programlama PIC16F628A / PIC16F648A*, Altař Yayıncılık, 2012
8. Hikmet řAHİN, K. Serkan DEDEOđLU, *MikroC ile PIC 18F4550*, Altař Yayıncılık, 2013
9. Orhan ALTINBAřAK, *Mikrodenetleyiciler Ve PIC Programlama*, Altař Yayıncılık, 2005
10. Feyzi AKAR, Mustafa YAđIMLI, *PIC 16F877A Proje Tasarımı*, Beta Basım Yayın, 2007
11. Feyzi AKAR, Mustafa YAđIMLI, *PIC Mikrodenetleyiciler 16F84A & 16F628A*, Beta Basım Yayın, 2007
12. Fevzi ENGİN, Musa řANLI, Ođuzhan URHAN, M. Kemal GÜLLÜ, *C Dili ile PIC Uygulamaları*, Birsen Yayınevi, 2006
13. Adil Fatih KİREMİTCİ "PIC18F4550 Mikrodenetleyicisi İle Usb-Pc Veri Aktarım Arabirimi Gereklenmesi" Y.Lisans Tezi, Elektrik ve Elektronik Mühendisliđi Anabilim Dalı, Atatürk Üniversitesi, Fen Bilimleri Enstitüsü, 2007
14. T.C. Millî Eđitim Bakanlığı Elektrik-Elektronik Teknolojisi Mikro İřlemci Ve Mikrodenetleyiciler, Ankara, 2012
15. Hüseyin otuk "PIC Mikrodenetleyiciler İin Gerek Zamanlı İřletim Sistemi" Yüksek Lisans Tezi, Bilgisayar Mühendisliđi Anabilim Dalı, TOBB Ekonomi Ve Teknoloji Üniversitesi, Fen Bilimleri Enstitüsü, Mart 2008, Ankara